MIDNIGHT

The Seven-Layer Magic Trick

A Complete Guide to
Zero-Knowledge Proof Systems

FIRST EDITION

Charles Hoskinson

March 2026

MIDNIGHT NETWORK

Copyright © 2026 Charles Hoskinson.

This work is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0).

You are free to share and adapt this material for any purpose, including commercial use, as long as you give appropriate credit, provide a link to the license, and indicate if changes were made.

Full license text:
https://creativecommons.org/licenses/by/4.0/legalcode

First edition, March 2026.

Published by Midnight Network.

The Seven-Layer Magic Trick: A Complete Guide to Zero-Knowledge Proof Systems.

Table of Contents

Glossary of Key Terms 3

Chapter 1: The Promise of Provable and Programmable Secrets 9

1.1 The Trick . . . 9
1.2 The Proof at the Door . . . 10
1.3 The Phenomenon . . . 12
1.4 Three Converging Forces . . . 13
1.5 The Seven Layers at a Glance . . . 14
1.6 The Deepest Question . . . 16
1.7 How to Read This Guide . . . 17
1.8 The First Decision . . . 18

Chapter 2: Layer 1 – Building the Stage 19

2.1 The Fair Shuffle Problem . . . 19
2.2 The Structured Reference String . . . 20
2.3 Two Ways to Build a Stage . . . 21
2.4 The Capex/Opex Framework . . . 28
2.5 The 141,416-Person Question . . . 29
2.6 The Bug That Was Not a Ceremony Failure . . . 32
2.7 Universal versus Circuit-Specific Setups . . . 33
2.8 The Quantum Shelf Life . . . 34
2.9 BN254's Eroding Security Margin . . . 35
2.10 The ADOPT Framework . . . 36
2.11 Midnight's BLS12-381 Stage . . . 37
2.12 Option-Value Analysis . . . 38
2.13 The Setup Tradeoff . . . 38

Chapter 3: Choreographing the Act 42

3.1 RISC-V Won. Why Taxonomy Still Matters. 42
3.2 From Circuits to Virtual Machines: A Brief Evolution 43
3.3 The Four Philosophies 45
3.4 The Polygon zkEVM Cautionary Tale 54
3.5 The Developer's Actual Experience 55
3.6 Under-Constrained Circuits: The Dominant Failure Mode 57
3.7 Compact's Disclosure Analysis 59
3.8 Midnight: Compiler, IR, Circuit 61

Chapter 4: The Secret Performance 63

4.1 The Hidden Bottleneck 63
4.2 Execution Traces 64
4.3 Witness Generation Costs 66
4.4 Memory: The Binding Constraint 70
4.5 Side-Channel Attacks: When the Walls Leak 74
4.6 Witness-Constraint Divergence 79
4.7 The `disclose()` Boundary: Midnight's Witness Architecture 80
4.8 The Witness as a Multi-Dimensional Problem 83

Chapter 5: Encoding the Performance 86

5.1 Layer 4 – Arithmetization 87
5.2 The Spreadsheet Metaphor (And Where It Works) 88
5.3 The Constraint System Evolution: R1CS, AIR, PLONKish 91
5.4 CCS: The Rosetta Stone 104
5.5 The Sumcheck Protocol: The Hidden Foundation 108
5.6 Lookup Arguments 111
5.7 The Overhead Tax: 10,000x to 50,000x 120
5.8 Midnight's ZKIR: A Concrete Layer 4 130
5.9 Where the Layers Collapse 133
5.10 Where the Analogies Break 135

Chapter 6: Layer 5 – The Sealed Certificate 139

6.1 Sealing the Certificate 139
6.2 The Three Families 141
6.3 The Hybrid Pipeline 145
6.4 Recursion vs. Folding: Russian Dolls and Snowballs 147
6.5 The Folding Genealogy 150
6.6 Nightstream: What a Folding Engine Looks Like From the Inside 154
6.7 Circle STARKs and Stwo: A Generational Leap 160
6.8 Real-Time Ethereum Proving 164
6.9 The Proof Core: Why Layers 4, 5, and 6 Are Inseparable 165

6.10 Fiat-Shamir Vulnerabilities . . . 166
6.11 Case Study: Midnight's Sealed Certificate . . . 167
6.12 SNARK Recursion vs. Folding: The Full Picture . . . 170
6.13 The Post-Quantum Horizon . . . 171
6.14 From Speed Race to Security Race . . . 172
6.15 The Sealed Certificate . . . 173

Chapter 7: Layer 6 – The Deep Craft 175

7.1 The Laws That Break . . . 175
7.2 Three Hardness Assumptions, Three Worlds . . . 176
7.3 Four Families of Commitment Schemes . . . 178
7.4 The Trilemma – And Its Dissolution . . . 186
7.5 Small Fields . . . 188
7.6 The Quantum Threat Horizon . . . 190
7.7 Lattice-Based Proving . . . 192
7.8 Case Study: Midnight . . . 195
7.9 The Cascade Effect . . . 197
7.10 Algebraic vs. Traditional Hash Functions . . . 199
7.11 The Structural Advantage of Lattices . . . 201
7.12 Maturity and Readiness . . . 202
7.13 The One-Way Door . . . 202

Chapter 8: Layer 7 – The Verdict 205

8.1 The Social Layer . . . 205
8.2 The Price of a Verdict . . . 206
8.3 When the Transcript Lies: Fiat-Shamir Vulnerabilities . . . 210
8.4 Governance: The Achilles Heel . . . 213
8.5 Proof Aggregation: The Missing Layer . . . 219
8.6 Case Study: Midnight and the Three-Token Architecture . . . 220
8.7 The Deepest Symmetry . . . 223
8.8 Pricing Attacks . . . 224
8.9 Who Verifies the Verifier? . . . 227
8.10 On-Chain Verification in 2026 . . . 229

Chapter 9: Privacy-Enhancing Technologies 231

9.1 The Four Pillars . . . 232
9.2 Three Kinds of Security . . . 238
9.3 Composability: When One PET Is Not Enough . . . 241
9.4 Real-World Deployments: Five Case Studies . . . 243
9.5 Privacy Architectures for Smart Contracts: Kachina and Zexe . . . 246
9.6 The Regulatory Intersection . . . 248
9.7 The Decision Matrix . . . 250

9.8 Open Problems . . . 251
9.9 The Incomplete Stack . . . 252

Chapter 10: The Synthesis – Three Paths, Not Two 255

10.1 The Binary That Broke . . . 255
10.2 The Map Redrawn . . . 256
10.3 Path One: The Hybrid STARK-to-SNARK Pipeline . . . 257
10.4 Path Two: Pure Transparent . . . 258
10.5 Path Three: Post-Quantum Folding . . . 259
10.6 The Three-Path Table . . . 260
10.7 The Causal Web: Why It Is a DAG, Not a Stack . . . 261
10.8 Trust Decomposition: Seven Weaker Assumptions . . . 263
10.9 "Trustless" versus "Trust-Minimized" . . . 269

Chapter 11: zkVMs – The Universal Stage 271

11.1 zkVMs Across the Stack . . . 271
11.2 The Landscape Table (March 2026) . . . 272
11.3 Three zkVMs Through Seven Layers . . . 274
11.4 The "Proof Core" Triad . . . 280
11.5 Performance: The Cost Collapse . . . 282
11.6 RISC-V Convergence . . . 283

Chapter 12: Midnight – The Privacy Theater 285

12.1 Midnight as Test Case . . . 286
12.2 Midnight at a Glance . . . 286
12.3 Full Seven-Layer Mapping . . . 287
12.4 Where Midnight Validates the Model . . . 291
12.5 Where Midnight Challenges the Model . . . 291
12.6 The Privacy Theater Analogy . . . 293
12.7 Five Lessons for ZK System Design . . . 294

Chapter 13: The Market Landscape 297

13.1 ZK Rollups: The Proving Grounds (Production) . . . 298
13.2 ZK Coprocessors: Off-Chain Computation, On-Chain Verification (Growth) . . . 299
13.3 ZKML: Provable Machine Learning (Research) . . . 300
13.4 ZK Identity (Growth / Regulatory Mandate) . . . 302
13.5 Proving-as-a-Service: The Prover Market (Production) . . . 303
13.6 Enterprise Pilots (Pilot) . . . 305
13.7 Market Sizing . . . 308

Chapter 14: Open Questions and the Road Ahead 311

14.1 The Seven Questions That Remain Open . . . 311

14.2 The Three Frontiers 324
14.3 Convergence 325
14.4 Coda 326

Complete Bibliography **328**

title: “The Seven-Layer Magic Trick” subtitle: “A Complete Guide to Zero-Knowledge Proof Systems” author: “Charles Hoskinson” edition: “First Edition” date: “March 2026” —

Glossary of Key Terms

AIR (Algebraic Intermediate Representation) – A way of encoding computation as a table of numbers governed by polynomial rules that must hold between consecutive rows. Used by STARK proof systems.

Arithmetization – The conversion of a computation into a system of polynomial equations that can be verified mathematically.

BabyBear – A 31-bit prime field ($2^{31} - 2^{27} + 1$) used by SP1 and Plonky3. Enables SIMD-friendly arithmetic on modern CPUs.

BLS12-381 – A specific elliptic curve providing approximately 128-bit security. Used by Zcash, Ethereum's blob commitments, and Midnight. Vulnerable to quantum computers.

BN254 (alt_bn128) – An older elliptic curve hardcoded into Ethereum's built-in operations. Its estimated security has eroded from 128 bits to approximately 100 bits.

CCS (Customizable Constraint Systems) – A unified mathematical framework that includes R1CS, AIR, and PLONKish as special cases.

Circle STARKs – A STARK variant operating over the circle group of a Mersenne prime field, enabling efficient FFTs without requiring smooth-order multiplicative subgroups.

Circuit – A representation of a computation as a network of mathematical operations over a finite field. Not an electrical circuit – a mathematical one.

Commitment scheme – A cryptographic method that lets you "seal" a value in an envelope, later proving properties about it without opening it. The four main families are KZG, FRI, IPA, and lattice-based (Ajtai).

CycleFold – An engineering optimization that delegates the non-native scalar multiplication in folding schemes to a co-processor circuit on a secondary curve, reducing overhead.

Data availability (DA) – The guarantee that transaction data is published and accessible so that anyone can reconstruct state and verify proofs independently.

Discrete logarithm problem (DLP) – Given points P and Q on an elliptic curve where $Q = nP$, finding the integer n. Believed hard classically; broken by Shor's algorithm on a quantum computer.

Elliptic curve – A mathematical curve whose point-addition operation is easy to perform but hard to reverse. The foundation of most modern public-key cryptography.

Execution trace – The complete record of every step in a computation: register values, memory accesses, intermediate results. The raw material from which a witness is derived.

Extension field – A larger field constructed from a base field by adjoining roots of an irreducible polynomial, used to achieve sufficient security when working with small base fields.

FHE (Fully Homomorphic Encryption) – Encryption that allows computation on encrypted data without decrypting it first. Currently 10,000x to 1,000,000x slower than plaintext computation.

Fiat-Shamir transform – A technique for converting an interactive proof into a non-interactive one using a hash function. Every blockchain-based zero-knowledge proof uses this.

Finite field – A set of numbers where arithmetic "wraps around" like a clock. All arithmetic in zero-knowledge proofs happens in finite fields.

Flash loan – A DeFi mechanism allowing uncollateralized borrowing within a single transaction. Used in governance attacks when combined with token-weighted voting.

Folding – A technique for combining two mathematical claims into one using random challenges. Dramatically reduces the cost of proving long computations.

FRI (Fast Reed-Solomon Interactive Oracle Proof of Proximity) – A method for verifying polynomial proximity using only hash functions. The commitment scheme inside every STARK. Transparent and plausibly post-quantum.

Gas – The unit of computational cost on Ethereum. A simple transfer costs roughly 21,000 gas; verifying a Groth16 proof costs roughly 250,000 gas.

Goldilocks field – A specific 64-bit prime ($2^{64} - 2^{32} + 1$). Called "Goldilocks" because it fits in a single machine word for fast arithmetic.

Groth16 – The most compact zero-knowledge proof system: proofs are exactly 192 bytes. Requires a per-circuit trusted setup ceremony.

Halo 2 – A PLONK-family proof system developed by the Electric Coin Company. The original Halo used IPA commitments (no ceremony); Halo 2 as deployed typically uses KZG commitments.

HyperNova – A folding scheme by Kothapalli and Setty that generalizes Nova from R1CS to CCS using the sumcheck protocol, enabling folding of any constraint system.

ISA (Instruction Set Architecture) – The fundamental language a processor understands. RISC-V is the dominant ISA for zero-knowledge virtual machines.

IVC (Incrementally Verifiable Computation) – A framework for proving that a sequence of computation steps was performed correctly, where each step's proof implicitly covers all previous steps.

KZG (Kate-Zaverucha-Goldberg) – A polynomial commitment scheme producing constant-size proofs (~48 bytes) using elliptic curve pairings. Requires a trusted setup. Not quantum-resistant.

Lattice – A regular grid of points in high-dimensional space. Finding a short vector in such a lattice is believed to be hard even for quantum computers.

M31 / Mersenne-31 – A 31-bit prime field ($2^{31} - 1$) used by Circle STARKs and Stwo. Its circle group has order 2^{31}, enabling efficient radix-2 FFTs.

Merkle tree – A pyramid-shaped data structure where a single hash at the top summarizes an entire collection, and membership can be proven with a short path of hashes.

Module-LWE (Module Learning With Errors) – A lattice problem believed hard even for quantum computers. The foundation for NIST's post-quantum encryption standard (FIPS 203) alongside Module-SIS.

Module-SIS (Module Short Integer Solution) – A mathematical problem believed hard even for quantum computers. The foundation for lattice-based ZK proof systems and NIST's post-quantum standards.

MPC (Secure Multi-Party Computation) – Protocols that let multiple parties jointly compute a function without any party revealing its input to the others.

MSM (Multi-Scalar Multiplication) – A computation where many elliptic curve points are multiplied by different scalars and summed. Highly parallelizable on GPUs.

Neo – A lattice-based folding scheme by Nguyen and Setty that adapts HyperNova's CCS folding to small fields (Goldilocks) with Ajtai commitments, introducing pay-per-bit commitment costs.

Nova – The first practical folding scheme, by Kothapalli, Setty, and Tzialla (2022). Introduced relaxed R1CS and reduced the per-step overhead of IVC by orders of magnitude.

NTT (Number Theoretic Transform) – The finite-field version of the Fast Fourier Transform. A critical bottleneck in modern proof systems.

Nullifier – A cryptographic value derived from a secret key and a commitment, used to prevent double-spending in privacy-preserving systems without revealing the spender's identity.

Pedersen commitment – A cryptographic commitment using elliptic curve arithmetic. Not quantum-resistant.

PLONK – A universal zero-knowledge proof system (2019) that works with any circuit up to a fixed size using a single trusted setup ceremony.

PLONKish – The arithmetization format used by PLONK and variants. Uses selector columns for different gate types plus copy constraints for wiring.

Post-quantum (PQ) – Resistant to attacks by quantum computers. Hash-based and lattice-based cryptography are believed to be PQ. Elliptic curve cryptography is not.

Proof aggregation – Combining multiple proofs into a single proof that is cheaper to verify than checking each proof individually. A key technique for amortizing on-chain verification costs.

Prover – The entity that generates a zero-knowledge proof. In the magician metaphor, the prover is the magician who performs the trick.

R1CS (Rank-1 Constraint System) – The simplest constraint system format. The "assembly language" of arithmetization. Used natively by Groth16.

Relaxed R1CS – A generalization of R1CS ($\mathbf{Az} \circ \mathbf{Bz} = u \cdot \mathbf{Cz} + \mathbf{E}$) that introduces a scalar u and error vector $\mathbf{E}$, enabling two instances to be combined via random linear combination. The key enabler of folding.

RISC-V – An open, royalty-free instruction set architecture. Eight of ten major zkVMs use RISC-V as their target.

Rollup – A blockchain scaling technique that executes transactions off the main chain, posting a compact proof back for verification.

Shor's algorithm – A quantum algorithm (1994) that breaks all elliptic curve cryptography in polynomial time.

SNARK (Succinct Non-interactive ARgument of Knowledge) – A proof system producing compact proofs that can be verified quickly without interaction. Groth16, PLONK, and Marlin are SNARKs.

Soundness – The guarantee that a cheating prover cannot create a valid proof for a false statement, except with negligible probability.

SRS (Structured Reference String) – The public mathematical parameters from a trusted setup ceremony. The stage on which the magician performs.

STARK (Scalable Transparent ARgument of Knowledge) – A proof system using hash-based commitments (FRI) with no trusted setup. Transparent, plausibly post-quantum, but with larger proofs than SNARKs.

Stwo – StarkWare's production implementation of Circle STARKs, deployed on Starknet mainnet. Achieved 940x throughput improvement over the previous Stone prover.

Sumcheck protocol – An interactive proof protocol (1992) that reduces verifying a sum over an exponentially large domain to checking a single evaluation. The backbone of modern ZK proof systems.

TEE (Trusted Execution Environment) – A hardware-isolated secure enclave (e.g., Intel SGX, ARM TrustZone) that processes data in a protected region even the hardware operator cannot inspect.

UTXO (Unspent Transaction Output) – A model where each digital coin is a discrete object created by one transaction and consumed by another, like physical bills in a wallet.

Verifier – The entity that checks a zero-knowledge proof. In the magician metaphor, the verifier is the audience that renders the verdict.

Witness – The private data and execution trace that the prover uses to generate a proof. The verifier never sees the witness.

Zero-knowledge – The property that a proof reveals nothing beyond the truth of the statement being proved.

ZKIR – Midnight's Zero-Knowledge Intermediate Representation. A 24-instruction typed bytecode that the Compact compiler produces, which a backend lowers to PLONKish constraints.

zkVM (Zero-Knowledge Virtual Machine) – A virtual processor that executes programs and automatically generates a zero-knowledge proof of correct execution.

Part I: The Invitation

Chapter 1: The Promise of Provable and Programmable Secrets

> *"Any sufficiently advanced technology is indistinguishable from magic."* – Arthur C. Clarke, *Profiles of the Future* (1962)

1.1 The Trick

Every civilization that has ever existed has faced the same problem. You know something. I need to verify it. And the only method anyone has ever devised for verification is *disclosure* – you open the books, you show the document, you reveal the source code, you hand over the key. Knowledge flows from the prover to the checker, and some of it inevitably spills.

For ten thousand years, this was not a problem. It was a law of nature, as inescapable as gravity. To prove is to show. To show is to reveal. To reveal is to lose control.

Then, in 1985, three researchers at MIT wrote a paper and broke the law.

Shafi Goldwasser, Silvio Micali, and Charles Rackoff demonstrated something that sounds, on first hearing, like a contradiction: it is possible to prove a statement is true while revealing *nothing* about *why* it is true. Not approximately nothing. Not mostly nothing. Nothing – in a mathematically rigorous

sense that can be stated as a theorem and verified by anyone who cares to check. The proof convinces. It does not inform. The audience sees the trick succeed and learns nothing about how it was performed.

Clarke was half right. This technology is indistinguishable from magic – until you understand it. Then it is more astonishing than magic, because magic relies on deception while this relies on its opposite. The magician hides the mechanism. The zero-knowledge proof hides the data *and lets you verify the mechanism is honest*. That distinction will occupy us for the rest of this book.

Here is the thesis, stated plainly so you can argue with it for the next three hundred pages.

Zero-knowledge proofs do not eliminate trust. They *decompose* it. They take one monolithic act of faith – trust the bank, trust the platform, trust the government – and shatter it into seven independent, weaker assumptions. Each one is testable. Each one is replaceable. What remains after the mathematics has done its work is not zero trust but *less* trust, distributed across more points of failure, each auditable on its own terms.

The word "trustless" is marketing. The accurate word is "trust-minimized." This book will show you exactly how much trust remains, where it lives, and what breaks when each assumption fails.

There are only two characters in this story. One is the magician. The other is the audience. In the technical literature, they are called the *prover* and the *verifier*. The prover knows the secret and wants to convince someone of a fact without revealing private information. The verifier checks the proof and renders a verdict: accept or reject. Every zero-knowledge system ever built – every billion-dollar rollup, every privacy protocol, every identity credential – reduces to this elemental exchange.

We will use the magician-and-audience framing throughout, not because it is charming but because it is precise. When the analogy works, we will ride it. When it breaks – and it will, most severely at Layer 4, where the mathematics of arithmetization overwhelms any theatrical metaphor – we will say so. A strained metaphor is worse than no metaphor at all. By Chapter 10, we will replace the seven-layer stack entirely with a directed graph. We will tell you when each transition happens.

1.2 The Proof at the Door

Before naming the properties that make this possible, let us watch the trick at the scale of a single human interaction.

You are twenty years old. You walk into a bar. The bouncer asks for your ID. You reach into your wallet and hand over a government-issued card containing your full legal name, your date of birth, your home address, your photograph, your driver's license number, and – depending on your jurisdiction – your

organ donor status and corrective lens prescription. The bouncer glances at the birth year, confirms you are of legal age, and hands it back.

Pause on this. In a three-second interaction, you have disclosed six pieces of personally identifiable information to solve a problem that requires exactly one bit of answer: *are you twenty-one or older? Yes or no.*

The card was designed in an era when the only way to prove a fact about yourself was to reveal the document containing that fact. We have been living with this design for so long that it feels inevitable. It is not.

Now imagine a different exchange. You carry a digital credential on your phone – issued by the same government authority, carrying the same legal weight. The bouncer's terminal displays a challenge: *prove you are 21 or older.* Your phone computes for a fraction of a second and transmits a proof. The green light appears.

What did the bouncer learn? That you are old enough. Nothing else. Not your name. Not your address. Not your exact age. Not even which government issued the credential. The proof is a single bit of verified truth, wrapped in mathematics.

Three properties make this work. They are worth naming now because they recur at every layer of the story – and because, if you watch closely, you have already seen all three in action.

The first: *the honest succeed*. If you genuinely are twenty-one, the proof will always verify. No glitch, no false rejection, no edge case where valid credentials fail. Cryptographers call this **completeness**. Without it, honest people get turned away, and the technology dies on contact with reality.

The second: *the dishonest fail*. A nineteen-year-old cannot forge this proof. They cannot borrow your credential and make it work – the proof is bound to a secret key only you possess. They cannot manipulate the computation to make nineteen appear as twenty-one – the polynomial constraints will not verify. The mathematics enforces the rule on the bouncer's behalf. Cryptographers call this **soundness**, and the word "cannot" is doing real work: the probability of a successful forgery is roughly one in 2^{128}, a number with thirty-nine digits. The sun will burn out first.

The third: *nothing leaks*. Not a partial hint about your birth month. Not a statistical correlation that narrows your age range. An adversary who intercepts the proof learns exactly what the bouncer learned – you are old enough – and nothing more. The proof is, in a precise and formal sense, *simulatable*: anyone could generate something that looks identical to it without knowing your date of birth at all. Cryptographers call this **zero-knowledge**. It is the property that makes the trick feel impossible, and the one this book exists to explain.

1.3 The Phenomenon

Completeness, soundness, zero-knowledge. These three properties were the subject of Goldwasser, Micali, and Rackoff's 1985 paper, "The Knowledge Complexity of Interactive Proof Systems" – one of the most consequential in the history of computer science, and the work that earned Goldwasser and Micali the Turing Award. For roughly twenty years after its publication, zero-knowledge proofs remained almost entirely theoretical. Beautiful mathematics, waiting for the world to catch up.

The world caught up.

What does a proof actually look like? Not a page of equations. Not an argument in English. A Groth16 proof – named after the Danish cryptographer Jens Groth, who published the scheme in 2016 – is exactly three groups of numbers, each about 48 bytes long. The entire proof fits in 192 bytes. That is smaller than a tweet, shorter than this paragraph's first sentence. Three numbers that took a GPU cluster seconds to compute, that a smart contract on Ethereum can verify for roughly 250,000 gas, and that reveal absolutely nothing about the secret they certify.

A STARK proof is larger – typically 50 to 200 kilobytes – but still small compared to the computation it certifies. And unlike Groth16, a STARK requires no trusted setup ceremony: its security rests on hash functions alone. We will spend all of Chapter 2 on the difference, because the choice between these two families is the first real decision any system designer faces, and its consequences cascade through every subsequent layer.

The bar scenario is vivid, but it understates the stakes. The real power emerges when you chain zero-knowledge proofs together.

A bank must prove to a regulator that it holds sufficient reserves to cover all deposits. Today, this requires opening the books – revealing individual account balances, transaction histories, counterparty relationships. The bank's customers never consented to this exposure. The regulator receives more information than necessary. With a zero-knowledge proof, the bank proves a single statement: "The sum of all deposits is less than or equal to the sum of all reserves." The regulator verifies in milliseconds. The proof reveals nothing about any individual account. The regulator is convinced – not because they trust the bank, but because the mathematics makes it impossible for the bank to have produced a valid proof of a false statement.

Or take the supply chain. A pharmaceutical company must prove to the FDA that a batch of medication was manufactured in a GMP-certified facility, that the active ingredient was sourced from an approved supplier, and that the cold chain was maintained throughout transit. Today, this audit requires revealing proprietary manufacturing processes, supplier contracts, and logistics routes – competitive intelligence that companies spend millions to protect. With zero-knowledge proofs, each attestation becomes a

sealed certificate: the proof says the constraints are satisfied, the FDA verifies, and the proprietary process stays proprietary.

Or take the scenario that scales furthest. You prove you are over twenty-one *and* that you hold a valid medical prescription *and* that your insurance covers the medication – all without revealing your name, your diagnosis, or your policy number to anyone except the parties who strictly need each fact. Each proof is independent. Each reveals only its single bit of truth. The pharmacy fills your prescription. Your insurer processes the claim. Your medical history remains yours.

The pattern is always the same. Someone must verify a fact. Verification has always required disclosure. Disclosure leaks information irrelevant to the fact being verified. Zero-knowledge proofs sever the coupling between verification and disclosure. They let you prove the fact and *only* the fact.

This is privacy as an engineering constraint – enforced by polynomial equations, verified in milliseconds, falsifiable by anyone who cares to check.

1.4 Three Converging Forces

For twenty years after Goldwasser, Micali, and Rackoff, zero-knowledge proofs were a theoretical marvel and a practical impossibility. Three forces converged to change that.

The privacy crisis became quantifiable. Proving you are over eighteen requires revealing your full date of birth. Proving your creditworthiness requires revealing your financial history. Proving your vaccination status requires revealing your medical records. Each is a case where a zero-knowledge proof could replace full disclosure with a single verified bit: yes or no, nothing more. The European Union's eIDAS 2.0 regulation, taking effect in 2026, mandates that digital identity wallets for 450 million EU citizens support selective disclosure – revealing only the attributes necessary for a given interaction. That mandate, at continental scale, requires zero-knowledge infrastructure. The regulatory pull is no longer hypothetical. It is law.

The scaling problem acquired a price tag. Ethereum, the world's largest smart-contract platform, processes roughly 15 transactions per second on its base layer. ZK rollups – Layer 2 systems that batch hundreds or thousands of transactions into a single proof verified on-chain – process orders of magnitude more. The rollup inherits the security of the main chain without requiring the main chain to re-execute every transaction. The proof is sufficient. By early 2026, the total value locked in ZK rollups exceeded $20 billion. These are not research projects. They are financial infrastructure securing real capital, and every dollar locked is a dollar betting that the proof system works.

The cost curve broke loose from history. In December 2023, generating a single zero-knowledge proof of a meaningful computation cost approximately $80. By December 2025, it cost $0.04. A 2,000-fold reduction in twenty-four months. To appreciate how unusual this is: solar energy costs dropped 99% over *four decades*. DNA sequencing outran Moore's Law, but it took a decade to achieve its steepest plunge. Zero-knowledge proving achieved a comparable collapse in two years. Nothing in recent technological history matches this pace.

What drove it? Four independent teams – SP1 Hypercube, RISC Zero, Stwo, and the Ethereum Foundation's zkEVM effort – achieved real-time proving of Ethereum blocks by late 2025. SP1 proved a block in 6.9 seconds on 16 GPUs, handling 99.7% of Ethereum L1 blocks in under 12 seconds. The improvements compounded across the entire stack: smaller fields reduced arithmetic cost, Circle STARKs enabled efficient proving over those fields, better arithmetization reduced constraint counts, GPU parallelization accelerated the proving pipeline, and aggregation services amortized verification. The seven-layer model we will build in this book is not merely an analytical framework. It is a map of where the cost reductions came from.

The speed race reached its finish line. In December 2025, the Ethereum Foundation declared it won and pivoted the competition axis to provable security: 128-bit security proofs by end of 2026. The performance frontier has been crossed. The security frontier is now.

At $0.04 per proof, the economics undergo a phase transition – the way water does not gradually become steam but shifts states at a threshold. At $80, only the most valuable financial settlements justify the expense. At four cents, you can prove *everything*: identity checks, game-state transitions, AI model inferences, supply-chain attestations, compliance audits. Projections for 2027 suggest sub-cent proofs for computations exceeding one billion cycles. The question stops being "can we afford to prove this?" and becomes "why would we *not* prove everything?"

These three forces – privacy demand, scaling need, and cost collapse – form a flywheel. Cheaper proofs make more applications viable. More applications create more demand. More demand funds more engineering. More engineering produces cheaper proofs. The cycle is self-reinforcing.

1.5 The Seven Layers at a Glance

Every zero-knowledge system, from the simplest proof to the most complex rollup, decomposes into seven layers. These are not independent floors in a building. They are organs in a body – deeply interdependent, each shaping what the others can do.

Here they are, one sentence each, in the language of our two characters.

Layer 1 – The Setup (Building the Stage). Before the magician can perform, someone must construct the mathematical parameters that make proving and verifying possible – and who builds them, and whether you must trust them, determines the system's deepest guarantees.

Layer 2 – The Language (Choreographing the Act). The magician needs a script: a programming language in which to express the computation she wants to prove, where her choice of language determines what bugs she can make and what assurances the compiler can enforce.

Layer 3 – The Witness (The Secret Backstage Preparation). The magician goes backstage to run the computation with her private data, recording every step in an execution trace that no one else will ever see – and this preparation, not the proof itself, is the most underestimated bottleneck in the entire stack.

Layer 4 – The Arithmetization (Encoding the Performance). The backstage recording is transformed into a system of polynomial equations: a mathematical puzzle that can be checked in seconds, even though solving it took hours.

Layer 5 – The Proof System (Sealing the Certificate). The magician compresses the entire puzzle into a compact, tamper-proof certificate – the proof – using cryptographic machinery that guarantees soundness and zero-knowledge simultaneously.

Layer 6 – The Primitives (The Deep Craft). Beneath the proof system lie the fundamental cryptographic building blocks: elliptic curves, hash functions, and polynomial commitment schemes, whose mathematical hardness assumptions are the bedrock on which everything above rests.

Layer 7 – The Verification (The Audience's Verdict). The audience checks the proof on a public stage – a blockchain, a smart contract, a verifier endpoint – and the economics, governance, and data availability of that stage determine whether the trick actually matters in the world outside mathematics.

A warning. We present these layers in order, from 1 to 7, because a book must be linear. The dependencies do not share this constraint. The choice of cryptographic primitive (Layer 6) determines which proof systems are available (Layer 5). That determines which arithmetizations work (Layer 4). That shapes which languages are efficient (Layer 2). That constrains the setup (Layer 1). A single decision – say, choosing the Goldilocks field over BLS12-381 – can cascade through all seven layers and reshape the entire architecture.

In practice, some layers fuse. Jolt merges witness generation and arithmetization into a single lookup step. Cairo co-designs its language around its constraint system. The proof core – Layers 4, 5, and 6 – behaves as one inseparable design unit in every production system. We will flag each collapse when we reach it. By Chapter 10, we will redraw the model in its honest final form: a directed acyclic graph with fourteen causal edges, not seven tidy floors. The map is provisional. The territory is more tangled. We will be honest about where the map fails.

You do not need to memorize this list. We will build each layer one at a time, and by the end, this overview will feel like a map you drew yourself.

1.6 The Deepest Question

Each of the seven layers introduces a trust assumption – a bet you are making that something is true, without the ability to verify it from first principles in real time.

At Layer 1, you trust that the setup ceremony was conducted honestly, or you choose a transparent setup that requires no ceremony at all, accepting a cost in proof size or verification time. The largest ceremony ever conducted – Ethereum's KZG Summoning – drew 141,416 participants. Among them, there is almost certainly someone whose operational security was impeccable. Almost certainly. Chapter 2 will make that "almost" precise.

At Layer 2, you trust that the program was written correctly. This is where the deepest irony of the field lives: 67% of real-world zero-knowledge vulnerabilities are under-constrained circuits – bugs in the *program*, not in the cryptography [Chaliasos et al., USENIX Security 2024]. The most sophisticated proof system in the world will faithfully prove a false statement if the circuit asks it to. One character – literally, a single = where <== was needed – caused a complete soundness break in Tornado Cash. Chapter 3 tells that story.

At Layer 3, you trust that the hardware generating the witness does not leak secrets through side channels. The proof is zero-knowledge, but the *process of generating the proof* may not be. Privacy, it turns out, is partly a luxury good: the architecture that maximizes it (client-side proving) demands hardware most people cannot afford. Chapter 4 confronts this uncomfortable truth.

At Layers 4 and 5, you trust that the arithmetization faithfully encodes the computation and that the proof system's security reduction is tight. The overhead tax for this encoding is staggering – 10,000x to 50,000x over native execution, falling to 1,000x-5,000x with current optimizations. You are paying for verifiability in compute. Chapters 5 and 6 show where that tax comes from and how the field is reducing it.

At Layer 6, you trust that the underlying mathematical problems – discrete logarithms, hash collision resistance, lattice problems – are genuinely hard. These are conjectures, not theorems. BN254, a widely deployed elliptic curve, has already seen its security erode from an estimated 128 bits to roughly 100. NIST targets 2035 for retiring all pre-quantum algorithms. If you are building infrastructure meant to last decades, this layer's assumption has an expiration date. Chapter 7 explains why, and what comes after.

At Layer 7, you trust the governance of the verification layer. Most ZK rollups today operate at Stage 0 or Stage 1 on the L2Beat maturity framework, meaning a small group holding multisig keys can replace the verifier contract. Six layers of mathematical elegance, and the seventh is a committee. The Beanstalk protocol lost $182 million in 13 seconds when an attacker used a flash loan to capture governance power and drain the treasury. The governance mechanism worked exactly as designed. The system was not broken. It was *used*. Chapter 8 tells that story, and it is the emotional nadir of the book.

No single layer requires trusting one entity with everything. Each assumption is independently falsifiable, independently auditable, independently replaceable – in principle. In practice, the coupling between layers makes this harder than it sounds, and that tension between principle and practice is the spine of the next thirteen chapters.

There is a deep symmetry worth noting early. Layer 1 (setup) and Layer 7 (verification) are both *social* trust – human decisions about who to trust with power. The mathematical layers (2 through 6) are sandwiched between them. The system converts social trust into mathematical certainty and then converts mathematical certainty back into social trust. The cryptography is a bridge between two shores of human judgment. Understanding this symmetry – and the vulnerability it creates at both ends – is the difference between understanding zero-knowledge proofs as mathematics and understanding them as systems that operate in the real world.

1.7 How to Read This Guide

This book has three audiences.

The Executive Path (~45 minutes). Chapter 1, the opening of Chapter 2, the opening of Chapter 8, the Chapter 11 landscape table, and Chapter 14. You will emerge able to evaluate vendor claims, explain ZK proofs to a board, and ask the right questions about governance and quantum risk.

The Engineer Path (~2 hours). Chapters 1-2 in full, Chapters 3-5 through their core sections, Chapter 6 through "Folding: The Snowball," Chapter 7 through "Four Families of Commitment Schemes," Chapter 8 through "Governance: The Achilles Heel," and Chapters 10, 11, and 13.

The Researcher Path (~4+ hours). Read everything. Then work through the seven open questions in Chapter 14. The active research fronts are the folding genealogy (Chapter 6), the lattice revolution (Chapter 7), the CCS unification (Section 5.3), and the proof core triad (Chapters 6, 10, and 11).

Regardless of path: do not skip Chapter 2 or the trust decomposition in Chapter 10. The first is where the deepest trust decisions live. The second is the book's thesis in its most complete form – seven assump-

tions, fourteen causal edges, three architectural paths, each with a different failure profile. Everything between here and there is building the case.

The field is crossing three sequential frontiers. *Performance* (2023-2025) is largely crossed: real-time proving is achieved, costs are sub-cent. *Security* (2026-2028) is the current frontier: formal verification, 128-bit provable security, post-quantum readiness. *Privacy* (2027+) is approaching: compiler-enforced disclosure boundaries, constant-time implementations, metadata protection. Where you enter this story depends on which frontier matters most to your work.

1.8 The First Decision

We have described the trick. We have named its three properties. We have sketched the seven layers, previewed the trust assumptions hiding inside each one, and measured the cost trajectory that is making all of this economically inevitable. A running example – a 4x4 Sudoku puzzle – will ground every abstraction from Chapter 3 onward. It is the computation we prove at each layer: first as a program, then as a witness, then as 72 polynomial constraints, then as a sealed certificate, and finally as a verified proof. The same small puzzle, viewed through seven different lenses.

Now we need to build the stage.

Every magic show requires one. In zero-knowledge proof systems, "building the stage" means creating the mathematical parameters that both characters will use – the prover to generate proofs, the verifier to check them. This step happens before any proof is ever created. And the most important question about the stage is not how it is built but *who builds it, and whether you have to trust them.*

In 2016, the Zcash ceremony involved six participants, each generating a share of the secret parameters and then destroying their share. If even one of the six was honest, the system is secure. In 2023, Ethereum's KZG Summoning Ceremony drew 141,416 participants. The growth from six to 141,416 tells you something about how seriously the field takes this problem. The alternative – transparent setups that require no ceremony at all – tells you something about how seriously the field wishes it could avoid the problem entirely.

That question – trusted setup or transparent setup, ceremony or glass stage, hidden trapdoors or none – is the subject of Chapter 2. It is the first fork in the road. Its consequences echo through every layer that follows.

The stage is where the trust story begins. Let us build it.

Chapter 2: Layer 1 – Building the Stage

"If the toxic waste is destroyed, how does anyone know?" – A question that sounds naive until you realize nobody has a clean answer.

2.1 The Fair Shuffle Problem

Here is a puzzle that has nothing to do with cryptography – at first.

You and a group of friends want to play a card game. Nobody trusts anyone to shuffle the deck fairly. The dealer might stack the cards. She might glimpse the order. She might memorize a few key positions and exploit them later. So you need a method – a protocol – that produces a provably fair shuffle, even though someone must do the physical shuffling.

One solution: get 141,416 people to each add a bit of randomness to the shuffle, one after another. Each person takes the deck, makes some unpredictable change to the ordering, and passes it on. Then each person destroys their memory of what they did. At the end, the deck is fair as long as *even one* of those 141,416 people acted honestly – genuinely random and genuinely forgetful.

That number – 141,416 – is not hypothetical. It is the exact count of participants in the Ethereum KZG Summoning Ceremony, completed in 2023. The largest cryptographic ceremony in history. And the thing they were "shuffling" was not a deck of cards but a set of mathematical parameters called a Structured Reference String, or SRS – the stage on which every proof in the system would subsequently be performed.

Now consider a second solution: skip the dealer entirely. Design a game where no shuffling is needed at all. Derive every rule from publicly known mathematics – hash functions, no secrets, no waste. Anyone can verify the rules are fair by reading them. No trust required. But the game runs slower, and the scorecards take up more space.

These two solutions – the ceremony and the transparent alternative – define the fundamental choice at Layer 1. Every zero-knowledge system ever deployed sits on one side of this divide or the other. And increasingly, the most sophisticated systems use both: a transparent inner proof wrapped in a ceremony-derived outer shell.

Understanding this choice, and its consequences, is where the trust story begins.

2.2 The Structured Reference String

Before we discuss who builds the stage, we need to understand what it is.

A *Structured Reference String* (SRS) is a list of specially constructed numbers. These numbers are points on an elliptic curve – a mathematical curve with a remarkable property. You can "add" points on the curve together easily. Given the result, however, it is extremely hard to figure out how many additions were performed. We will explain elliptic curves more fully in Chapter 7. For now, think of the SRS as a ruler with very precise markings that everyone uses to measure, but that no one can reverse-engineer to discover how the markings were made. The markings let you measure certain things (verify polynomial evaluations). They do not let you reconstruct the manufacturing process (recover the secret value from the markings). The scheme that makes this possible was invented by Kate, Zaverucha, and Goldberg in 2010 [Kate, Zaverucha, Goldberg, 2010] and is universally known as KZG, after the authors' initials.

The secret value used to generate the SRS is called the *trapdoor*. In the zero-knowledge community, it has acquired a more evocative name: *toxic waste*. The metaphor is precise. Like radioactive material, the trapdoor is dangerous if it persists and must be destroyed after it has served its purpose. If anyone – anyone at all – retains knowledge of the trapdoor, they can forge proofs of false statements. They can prove that $2 + 2 = 5$, that an empty bank account holds a billion dollars, that an invalid transaction is valid. No one would be able to tell the forged proof from a real one.

The SRS is the stage. The toxic waste is the spare key to the stage's trapdoor. And the ceremony is the manufacturing process designed to ensure that the spare key is destroyed.

2.3 Two Ways to Build a Stage

2.3.1 The Trusted Stage: Ceremonies and the 1-of-N Model

The first generation of zero-knowledge systems relied on what is called a *trusted setup*. One person, or a small group, would generate the SRS using secret randomness, produce the public parameters, and then – hopefully – destroy the secret.

The problem is obvious: you have to trust them. The original Zcash Sprout ceremony in 2016 involved six pre-selected participants [Bowe, Gabizon, Miers, 2017]. (Note: the BGM17 paper describes the Sapling ceremony protocol, not Sprout. The Sprout ceremony used the earlier BCTV14 protocol.) They used air-gapped computers – machines deliberately disconnected from the internet. After generating their contributions, they physically destroyed the hardware. One participant incinerated their computer. Another dissolved theirs in acid. The ceremony took two days. It was as much a ritual as a protocol.

Six people. Six chances for a mistake, a compromise, or a lie. The Zcash team needed the community to trust that at least one of these six acted honestly. This is the *1-of-N trust model*: security holds if even one participant out of N genuinely destroyed their contribution.

Why, then, would you want six-figure participation instead of six? The mathematics says one honest participant suffices. But the sociology says something different. With six participants, you can name them, investigate them, pressure them, bribe them, subpoena them. With over a hundred thousand anonymous contributors from around the world, many submitting their randomness through a simple web browser, collusion becomes operationally infeasible. The security is sociological as much as cryptographic: you are not trusting a specific person. You are trusting that it is impossible to corrupt *everyone*.

The evolution of ceremonies tells the story of this insight pushed to its limit:

- **2016 – Zcash Sprout.** Six pre-selected participants. Air-gapped machines physically destroyed. Two days. The first trusted setup ceremony in production.
- **2018 – Zcash Sapling.** Approximately ninety participants across two phases (87 in phase 1, 91 in phase 2) via the BGM17 "MMORPG" framework [Bowe, Gabizon, Miers, 2017]. 2.5 hours per contribution. The first scalable ceremony protocol.

- **2019-2022 – Proliferation.** Dozens of projects (Tornado Cash, Hermez, Aztec, Loopring) ran their own ceremonies, mostly for Groth16 circuits on the BN254 curve.
- **2023 – Ethereum KZG Summoning.** 141,416 contributors. Permissionless. Web-based. Anyone could participate. The ceremony ran for months. The "more-the-merrier" model at maximum scale.
- **2025+ – On-chain ceremonies.** Smart-contract-mediated setups where contributions are verified on-chain and no coordinator is needed [Nikolaenko, Ragsdale, Bonneau, Boneh, 2022].

The SRS that emerged from the Ethereum KZG ceremony is *universal*: it works for any circuit up to a certain size. This distinction matters. The older Groth16 system [Groth, 2016] required a *circuit-specific* setup – every new program needed a new ceremony. PLONK [Gabizon, Williamson, Ciobotaru, 2019] and its descendants changed this: one ceremony, one universal SRS, and then deterministic, public key derivation for each specific circuit. The ceremony is the capital expenditure. Everything after it is operating expense.

2.3.2 What It Felt Like: The Human Interior of a Ceremony

The timeline above tells the institutional story. It says nothing about what it was like to be there – to be one of those 141,416 people, sitting at a laptop, waiting for a progress bar to crawl across a browser tab, knowing that you were participating in something whose purpose you might only half understand.

Consider the six who gathered for the Zcash Sprout ceremony in 2016. They came from different countries. They did not all know each other. Some were cryptographers by training; others were engineers or entrepreneurs who happened to believe that financial privacy was a human right worth defending with mathematics. They met in locations that were, by design, not disclosed in advance. Each brought a computer – purchased new, never connected to the internet, destined to be destroyed within hours of its first and only use.

Peter Todd, one of the six, later described his experience. He purchased a laptop with cash from a store he chose at random, drove to a location he had selected from a map, and generated his contribution in a room with no wireless signals. When he was done, he filmed himself incinerating the machine. The video exists. You can watch a man feed a laptop into a furnace and understand that what you are witnessing is not performance art. It is a protocol step. The computer had touched a secret – a single random number, a fragment of toxic waste – and the protocol required that the computer cease to exist. Todd performed the ceremony alone, with no audience except the camera, in a deliberate act of informational suicide: the knowing destruction of something known, so that the knowing would end.

There is something ancient in this. Cultures have always understood that certain acts of creation require corresponding acts of destruction. The Navajo sand painting is swept away upon completion. The Tibetan mandala is dissolved into a river. The meaning of the creation is inseparable from the certainty of its erasure. What the Zcash participants were doing – generating a secret, using it exactly once, and annihilating every trace of it – belongs to this lineage. The difference is that here, the stakes were not symbolic. A single surviving photon of information, a single bit of toxic waste persisting in a swap file or a memory register, would compromise the soundness of every proof the system would ever produce.

Andrew Miller, another Sprout participant, took a different approach. He used a CD-ROM-based operating system that ran entirely in volatile memory – nothing was ever written to permanent storage. When he powered down the machine, the secret evaporated with the electrical charge in the RAM chips. The computer itself survived. The information did not. This was, in its way, more elegant than incineration: rather than destroying the vessel, you ensured the vessel never held the secret in any durable form. The secret existed for perhaps ninety seconds as a pattern of electrical charges in silicon, performed its mathematical function, contributed its randomness to the growing SRS, and then returned to thermal noise. If you had frozen the RAM chips in liquid nitrogen within those ninety seconds, you might have recovered something. Nobody did.

These are not colorful anecdotes. They are the actual security analysis. When a cryptographer evaluates the Sprout ceremony, they must reason about furnaces, cash purchases, volatile memory, and the thermal decay time of DRAM cells. The mathematics of the 1-of-N trust model is impeccable. The implementation of that model passes through the physical world, where "destruction" is an engineering judgment, not a mathematical theorem.

The Sapling ceremony in 2018 changed the texture of participation entirely. Sean Bowe, Ariel Gabizon, and Ian Miers designed the BGM17 protocol specifically to lower the barrier. You no longer needed to buy a disposable laptop or own a furnace. You needed a computer, an internet connection, and approximately 2.5 hours of patience. The protocol was sequential – each participant received the current state of the SRS, multiplied it by their own secret randomness, and passed the result to the next participant – but the infrastructure was centralized enough to be manageable and decentralized enough to be credible.

Ninety people participated. They came from the cryptography community, from the Zcash ecosystem, from academia, from the broader blockchain world. Some ran the computation on high-end workstations. Others used modest laptops. One participant, reportedly, ran their contribution on a Raspberry Pi – the $35 single-board computer popular with hobbyists – just to prove it could be done. The contribution took longer on the Pi. The mathematics did not care.

But ninety people is still a small enough group to name, to investigate, to subpoena. The operational security improvement over six participants was real but bounded. A nation-state adversary with suf-

ficient motivation could, in principle, identify and compromise all ninety. The names were not all public, but the ceremony's sequential nature meant that the coordinating infrastructure knew who was participating and when. The 1-of-N model was theoretically sound. The N was not yet sociologically overwhelming.

The KZG Summoning Ceremony that Ethereum conducted in 2023 was something qualitatively different. It was not a ceremony in the way the Zcash events were ceremonies – a small group of known individuals performing careful, bespoke acts of cryptographic hygiene. It was a mass participatory event. A ritual scaled to the size of a small city.

The interface was a web page. You navigated to ceremony.ethereum.org, connected a wallet or signed in with an Ethereum account, and waited in a queue. When your turn came, your browser generated a random number, performed the necessary elliptic curve multiplication, submitted the result, and discarded the secret. The entire contribution took between twenty seconds and two minutes, depending on your hardware. You did not need to understand polynomial commitments or KZG or elliptic curves. You needed to click a button and wait.

One hundred and forty-one thousand, four hundred and sixteen people did this.

They did it from bedrooms and offices and coffee shops. They did it on phones and tablets and gaming rigs and ancient ThinkPads. They did it in Tokyo and Lagos and Sao Paulo and Reykjavik and places that no one will ever catalog because the ceremony did not require identification beyond an Ethereum address. Some understood exactly what they were contributing and why. Others participated because someone they followed on social media said it mattered. Others did it out of curiosity, or boredom, or a vague sense of civic duty toward an infrastructure they used but did not fully comprehend.

And here is what matters: the ceremony did not need any of them to understand. The mathematics does not require informed consent. It requires honest randomness. A participant who clicks the button because a friend told them to, generating a random number from hardware entropy they could not explain, contributes exactly as much security as a participant who has read every paper on polynomial commitments since 2010. The secret is not in the understanding. The secret is in the entropy.

This is the sense in which the KZG ceremony was a ritual in the anthropological meaning of the word. A ritual is a formalized action whose efficacy does not depend on the participant's comprehension of the underlying mechanism. A child who is baptized does not understand the theology. A voter who marks a ballot does not understand constitutional law. A ceremony participant who clicks a button does not understand the discrete logarithm problem. In each case, the act is valid because the protocol says it is valid, not because the actor grasps the full chain of reasoning that connects the act to its consequences.

Anthropologists who study ritual observe that large-scale ceremonies serve functions beyond their stated purpose. They create shared experience. They generate social cohesion. They mark boundaries

between in-groups and out-groups. They transform abstract commitments into embodied actions. The Ethereum KZG ceremony did all of these things. Participants shared screenshots of their contribution receipts on social media. They discussed queue times and hardware performance. They created a shared memory of participation in something that mattered, even if they could not articulate exactly what it was or why.

The ceremony also served a function that its designers may not have fully anticipated: it created a constituency. One hundred and forty-one thousand people who personally contributed to the SRS have a stake in its legitimacy that goes beyond economic interest. They performed an act – a small act, a few seconds of computation – and that act binds them to the outcome in a way that merely reading about a ceremony does not. The SRS is not just a mathematical object to them. It is something they helped build. This psychological investment is not captured in any security model, but it is real, and it matters for the long-term governance and social resilience of the system.

2.3.3 Ritual as Protocol, Protocol as Ritual

The progression from Sprout to the KZG Summoning traces an arc that any historian of technology would recognize: the democratization of a priestly function.

In 2016, generating ceremony contributions required expertise, specialized equipment, and operational security practices borrowed from intelligence agencies. The participants were, in effect, priests – a small caste entrusted with a dangerous sacrament, selected for their competence and their willingness to undergo the destruction ritual. The community watched from outside. They could verify the transcript after the fact, but they could not participate. Trust flowed downward, from the priests to the congregation.

By 2023, the priestly function had been automated and distributed. The browser handled the cryptography. The hardware random number generator provided the entropy. The protocol enforced the sequencing. The participant's role was reduced to a single act of will (the decision to show up) and a single act of trust (the belief that their browser was not compromised). The ceremony had become, in the language of distributed systems, *permissionless*. Anyone could join. No one could be excluded. The trust model had inverted: instead of the congregation trusting the priests, each participant trusted only themselves.

This inversion changes how we should think about the foundations of cryptographic systems. The traditional model of security assumes a small number of trusted parties who perform critical operations on behalf of a larger community. Certificate authorities sign TLS certificates. Key escrow agents hold recovery keys. Hardware security module operators maintain root secrets. The ceremony model

– especially at the scale of the KZG Summoning – replaces this with something closer to a vote. Not a vote on a proposition, but a vote of presence. By participating, each contributor cast a ballot that said: "I exist, I generated honest randomness, and I destroyed my secret." The SRS is the tally.

The algebra is worth seeing. The SRS is a sequence of elliptic curve points: $[s]G, [s^2]G, [s^3]G, \ldots, [s^d]G$, where s is the toxic waste and G is the generator point. During the ceremony, each participant i takes the current SRS, multiplies every point by their own secret τ_i, and passes the result forward. After all N participants have contributed, the effective secret is $s = \tau_1 \cdot \tau_2 \cdot \tau_3 \cdots \tau_N$. To recover s, an adversary must know *every* τ_i – every participant's secret. If even one participant's τ_i is truly random and truly destroyed, the product s is irrecoverable. The secrets do not add. They multiply. And a product of unknowns is unknown.

This multiplicative structure is why the 1-of-N model works, and it is also why the human story matters. Each τ_i is not just a number. It is a moment in a person's life: the instant their hardware random number generator sampled thermal noise, the few seconds of computation, the silent deletion that followed. The SRS encodes 141,416 such moments, multiplied together, fused into a mathematical object that no participant can reverse and no adversary can deconstruct. The ceremony is a protocol that transforms individual acts of trust into a collective mathematical artifact. It is, in this precise sense, an act of collective authorship – a document written by 141,416 hands, each contributing a single character in a language none of them can read, producing a text that none of them can forge.

Whether this is beautiful or terrifying depends on your disposition. From the perspective of pure mathematics, the construction is elegant: a simple algebraic property (the hardness of factoring a product when the factors are destroyed) scaled to sociological proportions. From the perspective of operational security, it is a source of permanent anxiety: the entire edifice rests on the assumption that physical destruction is final, that no forensic technique will ever recover a τ_i from a formatted hard drive or a power-cycled RAM chip or the electromagnetic emanations of a laptop that existed for three hours in 2016 and then met a furnace.

Both perspectives are correct simultaneously. The ceremony is the most robust trust-minimization technique we know how to build. It is also a construction whose security guarantee is, at bottom, a claim about the physical world – about the irreversibility of certain physical processes – and the physical world is full of surprises. Nobody has ever recovered toxic waste from a destroyed ceremony machine. But nobody has ever proved, in any rigorous sense, that it cannot be done.

2.3.4 The Transparent Stage: No Secrets, No Waste, No Ceremony

The alternative is to build the stage from glass.

STARKs – Scalable Transparent ARguments of Knowledge – were introduced by Ben-Sasson, Bentov, Horesh, and Riabzev in 2018 [Ben-Sasson et al., 2018]. The word "transparent" is the operative one. A STARK requires no trusted setup at all. The only "setup" is agreeing on a hash function – a publicly known algorithm like SHA-256 or BLAKE3.

A note on the arithmetic that underlies everything in this book: all the numbers in a zero-knowledge system live in a "finite field" – a set of numbers (say, 0 through some large prime minus 1) where arithmetic wraps around, like a clock. On a 12-hour clock, 7 + 8 gives 3, not 15. Field arithmetic works the same way, but the "clock" has billions or trillions of hours. Every time you see "field element," "field arithmetic," or "field size" in the intervening chapters, this is what we mean: clock arithmetic with a very large clock. There is no secret. There is no toxic waste. There is no ceremony. There is nothing to destroy because nothing dangerous was ever created.

The security of a STARK rests on a different mathematical assumption: collision resistance of the hash function. It must be computationally infeasible to find two inputs that produce the same output. This is a weaker trust assumption than the discrete logarithm problem underlying KZG, and it has a property worth remembering: collision-resistant hash functions are believed to resist quantum computers. We will return to this point shortly, because it shapes every architectural decision in the ZK ecosystem.

A third family deserves mention: Bulletproofs [Bunz et al., 2017]. Bulletproofs are transparent (no ceremony needed) and produce logarithmic-size proofs, but they require linear verification time. Monero adopted Bulletproofs for its confidential transactions. The Inner Product Argument (IPA) at the core of Bulletproofs inspired the Halo approach to recursion without pairings. But Bulletproofs' linear verification cost makes them unsuitable for on-chain verification of large circuits, and – importantly – they rely on the discrete logarithm assumption. They are *not* quantum-resistant.

The setup spectrum, ordered from most to least trust required, looks like this:

Setup Type	Trust Assumption	Example	Proof Size	PQ Secure?
Circuit-specific trusted	1-of-N honest in ceremony	Groth16	192 bytes	No
Universal trusted	1-of-N honest in ceremony	PLONK, Marlin	~880 bytes	No

Setup Type	Trust Assumption	Example	Proof Size	PQ Secure?
Transparent (DL-based)	Discrete log hard	Bulletproofs	~700 bytes	No
Transparent (hash-based)	Collision resistance	STARKs	~100 KB	Yes

Notice the pattern: as you move down the table, trust decreases but proof size increases. No known construction breaks this tradeoff. The SoK paper on trusted setups [Wang, Cohney, Bonneau, 2025] identifies the "ideal polynomial commitment scheme" – transparent setup, constant-size proofs, constant-time verification – as the central open problem in the field. Nobody has built it. Nobody has proved it impossible. It is the holy grail.

2.4 The Capex/Opex Framework

The most useful lens for understanding setup economics is the capital expenditure / operating expenditure distinction.

Ceremony costs are capex. The Ethereum KZG ceremony required approximately $2-5 million in coordination, engineering, and security auditing. This is a one-time cost. Because the resulting SRS is universal, every system that uses it amortizes that cost. With fifty or more rollups sharing the same SRS, the per-rollup cost converges to approximately $60,000 – trivial relative to the annual operating budget of any serious blockchain project.

Per-proof costs are opex. Once the stage is built, the cost that matters is the cost of each proof. Here the gap between trusted and transparent setups opens wide:

- **Groth16 on-chain verification**: approximately 200,000-300,000 gas, which at typical Ethereum gas prices translates to about $0.50-$1.00. This is the cheapest on-chain verification available. Groth16 proofs are exactly 192 bytes – three elliptic curve group elements. Smaller than a tweet.
- **Raw STARK on-chain verification**: approximately 2-5 million gas, translating to $5-$25. STARK proofs run roughly 100 KB – about 500 times larger than Groth16 proofs.

These numbers explain why trusted setups persist despite their trust assumptions. The argument is not philosophical. It is economic. And it explains why the dominant production pattern in 2026 is neither pure trusted nor pure transparent, but *hybrid*: use a transparent STARK as the inner proof

(no ceremony required, post-quantum security for the computation), then wrap it in a Groth16 or KZG-based outer proof for cheap on-chain verification. You get the transparency of STARKs and the economics of SNARKs. The cost is complexity – you now maintain two proof systems – and the outer wrapper still requires a trusted setup.

2.5 The 141,416-Person Question

Let us dwell on a question that sounds naive but turns out to be fundamental.

If the 1-of-N trust model says you only need one honest participant, why did 141,416 people participate in the Ethereum KZG ceremony? What is the marginal value of participant number 141,417?

The mathematical answer: zero. One honest participant is sufficient. The 141,415 others add nothing to the mathematical security guarantee.

The operational answer: everything. Security is not a binary state – it is a confidence level. With six participants, a well-resourced adversary could conceivably investigate, coerce, or compromise all six. With 141,416 anonymous participants from around the world, many contributing through ephemeral browser sessions, the attack surface becomes unmanageable. You would need to compromise not just the people but their hardware, their network connections, their randomness sources, their memories.

There is also a signaling dimension. A ceremony with 141,416 participants is a public demonstration that the community takes the trust assumption seriously. It is *social consensus made visible*. The ceremony functions as both a cryptographic protocol and a coordination event.

But a deeper question hides beneath this one.

"If the toxic waste is destroyed, how does anyone *know* it was actually destroyed?"

The epigraph that opened this chapter asked it. Here is the honest answer: nobody knows. "Destruction" is a physical claim about a digital artifact. You cannot prove you deleted something from your own computer. You can only promise you did. The 1-of-N model rests on: "I trust that at least one person is honest, *and* that their computer was not compromised, *and* that no backup exists anywhere, *and* that no side-channel leaked the secret during generation."

This is weaker than trusting a single entity. But it is not zero trust. The word "trustless" does not apply here. The accurate word is "trust-minimized." This distinction matters, and it will recur at every layer.

2.5.1 The Sociology of Trust at Scale

The 141,416-person question admits a deeper analysis than the one just given, one that touches the foundations of how human societies construct shared belief.

Consider what it means to "trust" a cryptographic ceremony. You are not trusting that a specific person told the truth. You are not trusting that a specific institution acted in good faith. You are trusting a *statistical claim about a population*: that among 141,416 independent actors, at least one behaved honestly. This is a fundamentally different kind of trust than anything that existed before networked cryptography. It is closer to the trust you place in thermodynamic laws – not trust in any particular molecule, but trust in the aggregate behavior of an astronomically large ensemble.

The sociologist Niklas Luhmann distinguished between *confidence* and *trust*. Confidence is the passive expectation that things will continue to work as they have – you are confident the sun will rise, confident your bank will not vanish overnight. Trust is the active decision to make yourself vulnerable to another agent's choices – you trust a surgeon, trust a business partner, trust a babysitter. The 1-of-N ceremony model sits at the boundary between these two concepts. You are not trusting any specific ceremony participant. You are placing confidence in a system designed so that trust in any specific individual is unnecessary. The confidence is in the design, not in the people.

This distinction has practical consequences. When trust fails – when a specific person or institution betrays you – the natural response is blame, accountability, punishment. When confidence fails – when the sun does not rise, or when a financial system collapses – the natural response is existential shock, a rewriting of your model of the world. If the Ethereum KZG ceremony were ever found to be compromised, the response would be closer to the second kind. There would be no individual to blame, no villain to prosecute. There would only be the discovery that a system designed to be trustworthy was not – that the statistical claim about the population was wrong.

This is why the ceremony's social dimensions matter as much as its cryptographic ones. The ceremony must not only *be* secure. It must be *perceived* as secure by a community large and diverse enough to sustain confidence over decades. Perception is not a frivolous concern. It is an engineering requirement. A ceremony that is cryptographically perfect but that the community does not believe in provides no security in practice, because projects will not build on an SRS the community doubts, and users will not deposit funds into systems built on questioned foundations.

The Ethereum KZG ceremony achieved its social credibility through three mechanisms that future ceremony designers should study.

First, *radical openness*. The ceremony's source code was published. The coordination protocol was specified in public documents. The queue was visible in real time. Every contribution was logged in

a publicly verifiable transcript. This transparency did not make the ceremony secure – security came from the 1-of-N assumption – but it made the ceremony *auditable*, which is a precondition for sustained confidence.

Second, *permissionless participation*. The decision to let anyone with an Ethereum address contribute was not merely a design choice. It was a statement about who the SRS belongs to. A ceremony restricted to credentialed cryptographers would have been more efficient and arguably more secure in a narrow technical sense – each contribution would have been generated with better operational hygiene. But it would have been a ceremony *for* the community, not *by* the community. The permissionless design traded some individual contribution quality for a qualitative shift in collective ownership.

Third, *diversity of entropy sources*. Participants contributed from different hardware, different operating systems, different network configurations, different physical locations, different jurisdictions. An adversary attempting to compromise the ceremony would need to mount simultaneous attacks across an impossibly heterogeneous set of targets. This is the cryptographic equivalent of biodiversity: monocultures are fragile, but ecosystems with many species are resilient. The heterogeneity of 141,416 independent computing environments is itself a security property, one that no formal model fully captures.

There is a fourth mechanism, less often discussed: the *unforgeable cost of participation*. Each contributor spent real time – waiting in the queue, running the computation, verifying the receipt. This time cannot be faked or automated away entirely (anti-Sybil measures ensured that each Ethereum address could contribute only once). The aggregate time investment of 141,416 people, even at a few minutes each, represents thousands of person-hours of attention. This attention is a scarce resource, and its expenditure signals genuine commitment in a way that no amount of on-chain capital can replicate. A ceremony with 141,416 participants who each spent three minutes is harder to dismiss than a ceremony with six participants who each spent three days, even though the per-person investment is radically different, because the *collective* investment is distributed across a population too large to coordinate against.

The game-theoretic analysis reinforces this intuition. In a ceremony with six participants, the cost of corrupting the ceremony is the maximum of the costs of corrupting each participant – you need all six. With 141,416 participants, the cost is still the maximum of individual corruption costs (since you need all of them), but the *minimum* of those individual costs is now determined by the most resilient participant in a population of 141,416. Among that many people, there is almost certainly someone whose operational security is excellent, whose hardware is air-gapped, whose randomness source is physical, and whose motivation is ideological rather than financial. The ceremony's security is, in a precise sense, determined by its strongest link rather than its weakest.

This is the inverse of the chain metaphor that dominates security thinking. A chain is only as strong as its weakest link. A ceremony is only as *insecure* as its *strongest* link. This inversion – this structural op-

timism built into the mathematics of the 1-of-N model – is what makes mass-participation ceremonies viable. You do not need to ensure that every participant is careful. You need to ensure that the population is large enough and diverse enough that at least one participant, somewhere, is careful enough.

The question of how large is "large enough" has no clean mathematical answer, because it depends on the adversary model. Against a lone hacker, six participants may suffice. Against a corporate adversary, ninety. Against a nation-state with global surveillance capabilities, perhaps tens of thousands. Against a coalition of nation-states – the most paranoid threat model, but not an absurd one for infrastructure securing tens of billions of dollars – perhaps hundreds of thousands. The Ethereum ceremony's 141,416 participants do not guarantee security against any specific adversary. They guarantee that the *sociological* bar for a successful attack is extraordinarily high, higher than any previous cryptographic ceremony has set, and plausibly higher than any adversary can clear.

Plausibly. Not provably. And that gap – between plausible and provable – is where the honest assessment of ceremony security must live.

2.6 The Bug That Was Not a Ceremony Failure

In 2019, a vulnerability designated CVE-2019-7167 surfaced in the BCTV14 construction used by Zcash [BCTV14]. The bug was devastating: it would have allowed unlimited counterfeiting of Zcash tokens. And it had nothing to do with the trusted setup ceremony.

The flaw lived in the cryptographic construction itself. The BCTV14 protocol included an extra element in the SRS that was unnecessary for the proof system but that an attacker could exploit to forge proofs. The Zcash ceremony could have been conducted perfectly – every participant honest, every piece of toxic waste destroyed – and the system would still have been vulnerable.

This case study carries a lesson that extends far beyond Layer 1: *ceremony integrity is necessary but not sufficient*. You can build a perfect stage and still get the show wrong. The construction must be correct independently of the ceremony. The program must be correct independently of the proof system. Security is not a chain where one strong link compensates for a weak one. It is a conjunction: every link must hold simultaneously.

Sean Bowe and Ariel Gabizon discovered the BCTV14 bug before anyone exploited it. The team deployed a fix transparently. But the episode is a permanent reminder that the most dangerous vulnerabilities are not the ones in the ceremony. They are the ones in the mathematics that the ceremony exists to protect.

2.7 Universal versus Circuit-Specific Setups

The transition from circuit-specific to universal setups changed how zero-knowledge systems are built. Understanding it requires understanding what "circuit-specific" means in practice.

Groth16 [Groth, 2016] produces the most compact proofs ever constructed: exactly 192 bytes, three group elements, verified with three pairing operations. No other system comes close. But Groth16's setup is circuit-specific. The SRS encodes the structure of a particular circuit: the specific polynomial constraints, the wiring, the gates. Change the circuit – add a feature, fix a bug, upgrade the protocol – and you need a new ceremony. New coordination. New participants. New toxic waste to destroy. At $2-5 million per ceremony, this is unsustainable for any system that evolves.

PLONK [Gabizon, Williamson, Ciobotaru, 2019] and Marlin [Chiesa et al., 2019] solved this by making the SRS *universal*. The SRS encodes a generic mathematical structure (powers of a secret value on an elliptic curve, the same structure as KZG commitments). Any circuit up to a maximum size can use the same SRS. The "derived" per-circuit setup – selecting the evaluation domain, computing selector polynomials, generating proving and verification keys – is entirely *deterministic and public*. No new secrets. No new ceremony. No new toxic waste.

Midnight, the privacy-focused blockchain, adopted this model. Midnight uses a PLONK-family proof system (specifically, a variant of Halo2) with a universal SRS on the BLS12-381 elliptic curve [Midnight Developer Guide, 2026]. The Compact compiler takes each smart contract, compiles it to a circuit intermediate representation (ZKIR), and derives per-circuit proving and verification keys from the universal SRS. One ceremony serves all contracts. The per-contract compilation runs in seconds, produces deterministic output (same source code yields same keys), and requires no trust beyond the original ceremony.

The architecture diagram from Midnight's developer documentation shows this cleanly:

- `midnight-trusted-setup` – the ceremony, producing the universal SRS
- `midnight-zk` – the proof system (PLONK / Halo2 / BLS12-381), consuming the SRS
- `compactc compile` – the compiler, deriving per-circuit keys deterministically

Each compiled contract produces three artifact types: TypeScript bindings for the dApp frontend, ZKIR circuit files for the proof server, and proving/verification key pairs derived from the universal SRS. The proving key for a simple counter contract is 13.7 KB. The verification key is 1.3 KB. These keys inherit their security from the original ceremony – no additional trust assumptions enter during compilation.

The universal setup model has a consequence for the capex/opex analysis that is easy to miss: the capital expenditure of the ceremony is paid once and amortized indefinitely. Every new contract, every new

circuit, every upgrade deploys under the same SRS. The marginal cost of adding a new application to the ecosystem converges to the cost of compilation – effectively zero.

2.8 The Quantum Shelf Life

Every pairing-based proof system – Groth16, PLONK, Marlin, Sonic, KZG – rests on the hardness of the discrete logarithm problem on elliptic curves. Shor's algorithm, running on a sufficiently powerful quantum computer, solves the discrete logarithm problem in polynomial time.

This is not "might break." This is "does break, given sufficient hardware." The question is when, not whether.

The implications for Layer 1 are stark. A quantum computer would not need to compromise any ceremony participant. It would not need to break into anyone's hardware or bribe anyone. It would simply take the *public SRS* – the list of elliptic curve points that everyone can see – and extract the original trapdoor from it. Mathematically, irreversibly, without detection.

Once the trapdoor is extracted, every proof ever generated under that SRS becomes suspect. The attacker can forge proofs of false statements indistinguishable from legitimate ones. The six-figure ceremony becomes irrelevant. The toxic waste was not in anyone's memory or on anyone's hard drive – it was encoded in the public parameters, waiting for a quantum computer powerful enough to read it.

Conservative estimates place cryptographically relevant quantum computers – machines capable of breaking 256-bit elliptic curve cryptography – in the 2032-2035 timeframe. The National Institute of Standards and Technology (NIST) finalized its first post-quantum cryptography standards (FIPS 203, 204, 205) in August 2024 and published NIST IR 8547, a deprecation roadmap that targets 2035 for the retirement of pre-quantum cryptographic algorithms.

This creates a concept we will call the *quantum shelf life* of a trusted setup. A KZG ceremony conducted in 2023 produces an SRS that is secure against classical computers indefinitely but has a finite lifespan against quantum adversaries. If that SRS secures a blockchain expected to operate for 20 years, the setup's shelf life may expire before the system's intended lifetime ends.

STARKs, by contrast, rely on collision-resistant hash functions, which are believed to resist quantum computers. (The caveat "believed to be" is doing real work here: Grover's algorithm provides a quadratic speedup for hash preimage search, which means security parameters need to be doubled – 128-bit classical security becomes 64-bit quantum security – but this is a quantitative adjustment, not a qualitative break.) A transparent STARK-based setup has no quantum shelf life problem because there is no trapdoor to extract.

This brings us to the concept of *option value*. A transparent setup preserves the option to migrate to post-quantum cryptography without re-ceremony. A trusted setup on a pairing-friendly curve burns this option: if the post-quantum deadline arrives and your SRS lives on BLS12-381, you need a new setup from scratch. Given the NIST 2035 timeline, systems designed today with 10+ year lifespans should seriously consider whether the performance advantages of pairing-based setups justify the loss of post-quantum migration flexibility.

Midnight's choice of BLS12-381 illustrates this tension concretely. BLS12-381 provides approximately 128-bit classical security but offers zero post-quantum security. Every component of Midnight's proof system – the SRS, the polynomial commitments, the proof verification – rests on assumptions that quantum computers break. Midnight originally experimented with Pluto-Eris curves (which enabled recursive proof composition), but switched back to BLS12-381 for pragmatic reasons: faster proof generation, wider ecosystem compatibility, and better tooling support. The switch was a deliberate choice to optimize for present-day deployment at the cost of future quantum vulnerability.

The alternative approach – exemplified by systems like Neo [Nguyen, Setty, 2025] – uses lattice-based cryptography (Module-SIS and Module-LWE assumptions) that is conjectured to resist quantum computers. The tradeoff: larger proofs, more expensive verification, and a transparent setup that requires no ceremony at all. The trust assumption shrinks from "1-of-N ceremony participants were honest" to "the lattice problem is hard" – a purely mathematical assumption with no sociological component.

Neither choice is unambiguously correct. They represent different bets on the future: Midnight bets that pairing-based cryptography will remain secure long enough for the system to establish itself and migrate later. Lattice-based systems bet that the quantum threat justifies paying the performance cost now. Nobody knows which bet is right, because nobody knows when quantum computers will arrive at cryptographic scale.

2.9 BN254's Eroding Security Margin

There is a more immediate security concern than quantum computers, and it affects the most widely deployed curve in the ZK ecosystem.

BN254 (also called alt_bn128 or BN128) is the curve hardcoded into Ethereum's elliptic curve precompile opcodes. Every Groth16 proof verified on Ethereum's base layer uses BN254. But the estimated security of BN254 has been eroding. The Tower Number Field Sieve (Tower NFS) – a family of discrete log algorithms that exploits the tower structure of extension fields – has reduced BN254's estimated security from 128 bits to approximately 100 bits [Kim, Barbulescu, 2016; Menezes, Sarkar, Singh, 2016].

One hundred bits of security is not broken. But it sits below the 128-bit threshold that NIST mandates for new cryptographic deployments. The ZK ecosystem has been slowly migrating from BN254 to BLS12-381, which provides a comfortable 128-bit security margin even under Tower NFS analysis. Migration is slow, though: existing smart contracts reference the BN254 precompile directly, and changing the curve means changing the verification logic, which means upgrading every contract that verifies proofs.

The setup layer casts a long shadow. Choices made at Layer 1 – which curve, which parameters, which ceremony – propagate forward through years or decades of deployment. The curve choice that was state-of-the-art in 2018 shows its age by 2026. The ceremony that was sufficient in 2023 may be insufficient by 2035. Building the stage is not a one-time act. It is a commitment with a time horizon.

2.10 The ADOPT Framework

The most comprehensive survey of trusted setup ceremonies, the SoK paper by Wang, Cohney, and Bonneau [Wang, Cohney, Bonneau, 2025], catalogs over forty real-world ceremonies and evaluates them against five properties (the ADOPT framework):

- **A**vailable: Can anyone access and verify the SRS?
- **D**ecentralized: Is there a single coordinator who could manipulate the ceremony?
- **O**pen: Can anyone participate without permission?
- **P**ersistent: Will the ceremony data survive long-term for auditing?
- **T**ransparent: Is every step of the ceremony publicly observable?

Their finding: *no existing ceremony satisfies all five properties*. The Ethereum KZG ceremony scores well on openness and availability but still relied on a coordinating entity (the Ethereum Foundation). Many older ceremonies fail on persistence – the intermediate transcript data for projects like Hermez has already become unrecoverable just a few years later.

This is not a failure of any specific project. It is a structural limitation of the ceremony model. Ceremonies are social events, and social events are messy. The gap between the abstract "1-of-N honest participant" security model and the operational reality of running a ceremony with hundreds of thousands of participants – each using different hardware, different software, different randomness sources, across different jurisdictions – is wide. The protocol can be mathematically perfect. The ceremony is always imperfect.

The transparent alternative – STARKs, hash-based commitments, no ceremony at all – avoids this entire problem class. There is no ceremony to evaluate, no transcript to preserve, no coordinator to trust. The cost is measured in proof size and verification time, not in coordination complexity and social trust.

2.11 Midnight's BLS12-381 Stage

Let us trace the full Layer 1 story through a real system.

Midnight is a privacy-focused blockchain built on the Cardano ecosystem. It uses BLS12-381 with a PLONK-family proof system. Here is how its Layer 1 choices cascade through the stack:

The ceremony. Midnight operates a `midnight-trusted-setup` repository for conducting its Powers-of-Tau ceremony. The ceremony produces a universal SRS on BLS12-381 – a set of elliptic curve points encoding powers of a secret trapdoor. The SRS bounds the maximum circuit size: you cannot prove statements about circuits larger than the SRS supports.

Per-circuit keys. The Compact compiler (`compactc compile`) takes each contract's circuit description (ZKIR) and the universal SRS to produce per-circuit proving and verification keys. This is deterministic: same source plus same compiler yields same keys. No new trust assumption enters. A simple counter contract produces a 13.7 KB proving key and a 1.3 KB verification key.

Deployment flow. The proving key goes to the proof server (running locally at localhost:6300 – witnesses never cross the network). The verification key reaches the blockchain via `submitInsertVerifierKeyTx`. Every node that validates transactions uses this key to check proofs.

The Pluto-Eris detour. Midnight originally adopted Pluto-Eris, a cycle of curves that enabled recursive proof composition (KZG on Pluto, IPA on Eris). In April 2025, the team announced a switch back to BLS12-381. The reason: faster proof generation, better ecosystem tooling, and support for higher transaction volumes. Theoretical optimality yielded to engineering pragmatism. This is a pattern we will see repeatedly: the choices that win in production are not always the choices that win in papers.

The quantum exposure. BLS12-381 provides approximately 128-bit classical security but zero post-quantum security. Every ZK proof in Midnight – state transitions, shielded transfers, Zswap privacy operations – becomes forgeable if a quantum computer extracts the trapdoor from the SRS. Midnight's entire proof system would need replacement in a post-quantum world. The migration path is unclear.

The Midnight case study illustrates a broader truth about Layer 1: the setup choice is a bet on the future. Midnight bet on BLS12-381 because it is the most mature, best-tooled, most widely deployed pairing-friendly curve in the ecosystem. That bet optimizes for today's performance and ecosystem compatibility. Whether it survives the quantum transition is an open question – one that the Midnight team will face, along with every other pairing-based system, before the decade is out.

2.12 Option-Value Analysis

We close the economic analysis with a concept from financial options theory that the ZK ecosystem has not yet internalized: *option value*.

A trusted setup on BLS12-381 buys performance today but forecloses certain futures. When – not if – post-quantum migration becomes necessary, every system on BLS12-381 will need a new setup. New ceremony. New coordination. New toxic waste. If the system has accumulated ten years of state and ten million users, the migration cost is not just the $2-5 million of the ceremony. It is the coordination cost of upgrading every node, every contract, every verifier key.

A transparent setup on hash-based primitives costs more per proof today but preserves the option to migrate without re-ceremony. The hash function can be swapped. The security parameters can be adjusted. No toxic waste needs to be re-destroyed because none was ever created.

What is this option worth? The answer depends on your estimate of the quantum timeline. If you believe cryptographically relevant quantum computers are 20+ years away, the option has low present value, and the performance advantage of pairing-based setups dominates. If you believe the timeline is 10-15 years – consistent with NIST's 2035 deprecation target – the option is substantial. And if the "Harvest Now, Decrypt Later" threat model applies (where adversaries record encrypted data today to decrypt it when quantum computers arrive), the option may be critical even for systems launched tomorrow.

The Federal Reserve's FEDS 2025-093 working paper on quantum threats to financial infrastructure uses exactly this option-value framework. Their conclusion: systems with 10+ year operational lifespans should use post-quantum or post-quantum-ready primitives. Transparent setups satisfy this criterion by default. Trusted setups on pairing-friendly curves do not.

2.13 The Setup Tradeoff

The stage is built. Let us take stock.

Layer 1 is where the trust story begins. The choice between trusted and transparent setups is not a philosophical preference – it is an economic, operational, and temporal decision with consequences that persist for the lifetime of the system. Trusted setups produce smaller proofs and cheaper verification but require a ceremony, create toxic waste, and carry a quantum shelf life. Transparent setups avoid all ceremony-related risks but pay a cost in proof size and verification expense. The hybrid approach – transparent inner proof, trusted outer wrapper – dominates production because it captures most of the benefits of both, at the price of added complexity.

The 1-of-N trust model is a genuine advance over trusting a single entity, but it is trust-minimized, not trustless. The BCTV14 bug proves that ceremony integrity alone is insufficient – the construction must be independently correct. The quantum shelf life means that no pairing-based setup is permanent. And the ADOPT framework reveals that no ceremony conducted to date achieves the ideal of full availability, decentralization, openness, persistence, and transparency.

We have introduced our two characters – the magician (prover) and the audience (verifier) – and we have seen that even the first layer of their interaction involves non-trivial trust decisions. The magician cannot perform without a stage. The audience cannot judge without a ruler. And someone, somewhere, had to build both.

Now, the stage is ready. The mathematical parameters are in place. The SRS is published. The toxic waste is (we hope) destroyed. The verifier keys are derived and deployed.

The magician needs a script. She needs to express her computation – the thing she wants to prove – in a language the proof system can understand. That language choice turns out to be the single most consequential decision for the security of the entire system. Not because of the cryptography. Because of the bugs. Sixty-seven percent of real-world SNARK vulnerabilities trace back to the script, not the stage – to the program layer, not the ceremony or the proof system. The next chapter explains why, and what the field is doing about it.

The stage is built. Now, who writes the script?

Part II: The Craft

The audience has seen the stage. They know the rules. Now the curtain rises, and the real work begins – backstage, in the mathematics, where every step must be recorded, encoded, sealed, and verified. The next six chapters trace the magician's craft from the first line of code to the final proof on the blockchain.

Chapter 3: Choreographing the Act

Layer 2 – Language

3.1 RISC-V Won. Why Taxonomy Still Matters.

If RISC-V has won – if eight of ten major zero-knowledge virtual machines now target the same general-purpose instruction set – why bother presenting a taxonomy of competing philosophies at all? Why not just say "write Rust, compile to RISC-V, the proof system handles the rest"?

Because the taxonomy is not about which instruction set the machine runs. It is about what *the developer sees*. And what the developer sees determines what bugs the developer makes. Those bugs – not cryptographic breaks, not quantum computers, not governance attacks – are the single largest source of real-world failures in zero-knowledge systems. Sixty-seven percent of all known SNARK vulnerabilities are under-constrained circuits: programs where the developer said less than they meant, and the proof system happily proved false statements as a result.

The language layer is where the magician writes the choreography for the act. The choice of notation determines what mistakes are possible. Some notations let the performer accidentally step into the

audience's view. Others physically prevent it. That distinction is worth understanding, even in a world where RISC-V has won the instruction set war.

The Running Example: A Sudoku Proof

To ground the abstractions that follow, we will trace a single computation through every layer of the stack. The computation: proving you know the solution to a 4x4 Sudoku puzzle without revealing it.

The puzzle has these givens:

```
+---+---+---+---+
| 1 |   |   | 4 |
+---+---+---+---+
|   | 4 | 1 |   |
+---+---+---+---+
|   | 1 |   |   |
+---+---+---+---+
| 4 |   |   | 1 |
+---+---+---+---+
```

The program checks: every row contains {1,2,3,4} with no repeats, every column contains {1,2,3,4}, every 2x2 box contains {1,2,3,4}, and every filled cell matches the given clue. The prover knows the solution. The verifier knows only the puzzle. At Layer 2, this is a program. We follow it through every layer to come.

3.2 From Circuits to Virtual Machines: A Brief Evolution

To understand where we are, we need to understand where we came from.

The first generation of zero-knowledge programming looked nothing like programming. In 2018, if you wanted to prove a computation in zero knowledge, you wrote *circuits* – not programs, but direct descriptions of mathematical relationships. The dominant tool was Circom, a domain-specific language created by Jordi Baylina and the iden3 team. In Circom, you did not write `if balance >= amount then approve`. You wrote constraint templates: mathematical equations that the proof system would verify. The developer was simultaneously writing two programs in one file – one that computed the witness (the private data), and one that generated the constraints (the mathematical rules). These two programs had to agree perfectly on every input. When they did not, the result was an under-constrained circuit: a proof system that would certify false statements as true.

Imagine asking a playwright to write both the script and the stage directions in a single document, using two different notations that had to be perfectly synchronized, with no compiler to check whether they matched. That is what early ZK development felt like.

Circom was powerful. It gave developers complete control over the constraint system. But that control came at a cost. The Chaliasos SoK confirmed the scale of the problem: 95 of 141 catalogued vulnerabilities were under-constrained circuits – the epidemic described at the opening of this chapter. The Tornado Cash bug was a single character: = where <== was needed. One character. Complete soundness break. A malicious prover could generate proofs for false statements, and the verifier would accept them.

The second generation asked a different question: what if the developer never saw the constraints at all? What if they wrote a program in a language they already knew, and a compiler handled the translation to mathematics?

Cairo, created by StarkWare in 2021, pioneered this approach. Cairo defined a new instruction set architecture – a virtual CPU designed from the ground up so that every instruction's execution could be efficiently encoded as polynomial constraints. The developer wrote programs. The compiler generated constraints. The proof system verified the constraints. The developer never touched a mathematical equation.

But Cairo required learning a new language, a new toolchain, a new way of thinking about computation. The programs you had already written – in Rust, in C++, in Python – could not run on Cairo. You had to rewrite everything.

The third generation asked the obvious follow-up: what if we proved a processor that developers already targeted? What if the instruction set was not some exotic ZK-native design, but plain RISC-V – the open standard that Rust, C, and C++ compilers already produce code for?

This is the generation we live in now. SP1 (Succinct), RISC Zero, Airbender (ZKsync), ZisK (the team formerly known as Polygon Hermez), and Pico Prism all prove RISC-V execution. The developer writes standard Rust. The compiler targets standard RISC-V. The proof system proves the execution trace. The developer may never know they are working with zero-knowledge proofs at all.

But this triumphant narrative – circuits to custom VMs to RISC-V – leaves out a fourth thread. There is another approach, one that does not fit the evolutionary story. It does not prove a processor at all. It proves *state transitions*. And its compiler does something no instruction-set-based approach can do: it prevents privacy leaks at compile time.

3.3 The Four Philosophies

Every approach to Layer 2 answers the same question differently: how should a developer express a computation that will be proven in zero knowledge?

Four distinct philosophies have emerged. Each makes a different bet about what matters most.

Philosophy A: EVM-Compatible. Prove the Ethereum Virtual Machine itself. The developer writes Solidity, the same language used for ordinary Ethereum smart contracts. The proof system executes the EVM bytecode and proves that execution was correct. The advantage is obvious: every existing Ethereum application "just works." The Solidity ecosystem – its tooling, its auditors, its libraries, its millions of lines of battle-tested code – transfers wholesale.

The leading examples are Scroll, with $748 million in total value locked, and Linea (ConsenSys), with over $2 billion. Both prove EVM execution using different strategies: Scroll uses a custom zkEVM circuit, while Linea has been converging toward a RISC-V backend with EVM compatibility layered on top.

The disadvantage is equally obvious: the EVM was not designed for proving. It has 256-bit arithmetic (proof systems prefer 31-bit or 64-bit fields). It has dynamic gas metering. It has a complex memory model. Proving every quirk of the EVM is computationally expensive – like translating a novel into a language that has no word for half the concepts.

Philosophy B: ZK-Native ISA. Design a new instruction set from scratch, optimized for proving. The developer learns a new language, but every instruction translates efficiently into polynomial constraints. Cairo is the canonical example. Its instruction set was co-designed with StarkWare's STARK proof system. Layer 2 was literally shaped by Layer 4 – the language exists *because* of the arithmetization.

Cairo's execution trace model – columns for program counter, flags, operands, with polynomial constraints between adjacent rows – became the template for every subsequent zkVM. The Goldilocks prime field ($2^{64} - 2^{32} + 1$) that Cairo adopted has since been used by RISC Zero, SP1, and others. Cairo bet on algebraic efficiency over developer familiarity, and for the Starknet ecosystem, that bet paid off: it is the most battle-tested ZK system after Circom, handling real assets on mainnet since 2020.

The cost is ecosystem isolation. Cairo programs do not run anywhere except Starknet. The tooling, libraries, and developer community are Cairo-specific. Every line of code is a bet on one ecosystem. But that bet has paid returns: Starknet processes real assets, real DeFi, real NFTs. Cairo is not a research project. It is production infrastructure.

The evolution from Cairo 0 to Cairo 1.0 is itself instructive. The original Cairo was barely recognizable as a programming language – it looked more like assembly with syntactic sugar. Cairo 1.0 aligned with mainstream language features: Rust-like syntax, a borrow checker, generic types. The lesson for the

field: even a language designed for proof efficiency eventually converges toward familiar developer ergonomics, because adoption requires accessibility.

Philosophy C: General-Purpose ISA. Prove a standard processor. RISC-V has won this category so decisively that it is no longer a competition. SP1 (Succinct), RISC Zero (with its R0VM 2.0 reducing Ethereum block proving from 35 minutes to 44 seconds), Airbender (ZKsync, achieving 21.8 million cycles per second on a single H100 GPU), ZisK (the ex-Polygon team), and Pico Prism all target RISC-V.

The appeal is obvious: the developer writes Rust, C, or C++. The compiler produces RISC-V machine code. The zkVM executes that code and generates a proof. Standard toolchains, standard debuggers, standard testing frameworks. The ZK-specific complexity hides behind the compilation boundary.

Even projects that started with EVM compatibility are converging on RISC-V. ZKsync's Airbender proves RISC-V execution and layers EVM compatibility on top. The trend is clear: RISC-V is the assembly language of the zero-knowledge world.

Philosophy D: Application-Specific DSL. This is the philosophy that does not fit the evolutionary narrative. Instead of proving a processor, these languages prove *state transitions*. Instead of hiding ZK complexity behind a compilation boundary, they make privacy a first-class language concept.

Three languages define this category.

Compact (Midnight/IOG) is a TypeScript-like language for privacy-preserving smart contracts. Its compiler produces three artifacts from a single source file: a ZKIR circuit (the constraint system), TypeScript bindings (the application interface), and proving keys (the cryptographic setup material). No other ZK language generates all three from one source. More importantly, Compact's compiler includes a *disclosure analysis* pass that statically rejects any program where a private value might leak to a public surface without explicit developer consent. We will return to this in detail.

Noir (Aztec Labs) is a Rust-inspired, backend-agnostic ZK language. It compiles to an Abstract Circuit Intermediate Representation (ACIR). Noir does not target a specific proof system – it targets multiple backends. This makes it the closest thing the ZK world has to a "write once, prove anywhere" language.

Noir 1.0 was pre-released in late 2025. It became an officially recognized language on GitHub. NoirCon conferences have been held (NoirCon0 in November 2024). The ecosystem includes over 600 projects and 900 GitHub stars. Key adopters include zkEmail, zkPassport, and zkLogin. Aztec's Ignition Chain went live in November 2025 as the first decentralized L2 on Ethereum, with 185+ operators across 5 continents and 3,400+ sequencers – and its core cryptography was rewritten in Noir.

Noir breaks the three-philosophy taxonomy because it is neither ISA-based nor chain-specific: it is a universal circuit language. Its privacy model is straightforward – all inputs are private unless explicitly declared `pub` – but it lacks the compile-time disclosure analysis that Compact provides. The developer

bears responsibility for correctly managing the public/private boundary. In exchange, Noir programs can target any proving backend that accepts ACIR, making them portable across proof systems in a way that no other ZK language achieves.

Leo (Aleo) is a privacy-first language for the Aleo blockchain, with syntax borrowing from Rust and TypeScript. Leo targets Aleo's record-based (UTXO-like – a UTXO, or Unspent Transaction Output, is a model where each digital coin is a discrete object created by one transaction and consumed by another, like physical bills in a wallet) privacy model and includes hooks for formal verification. With over 400,000 CLI downloads, Leo represents the privacy-specialized variant of the application DSL approach.

To understand why these three languages matter – why they are not merely syntactic alternatives to writing Rust and compiling to RISC-V – you need to see what development actually looks like in each one. The differences are not cosmetic. They are structural. Each language imposes a different mental model on the developer, and that mental model determines what classes of bug are possible, what classes of privacy leak are preventable, and what the compiler can guarantee before a single proof is generated.

3.3.1 Compact: Privacy by Compilation

Consider a simple scenario: a private token transfer. The sender wants to prove they have sufficient balance without revealing what that balance is. In Compact, the developer writes something that looks, at first glance, like an ordinary TypeScript function:

```
export circuit transfer(
  recipient: Bytes<32>,
  amount: Unsigned Integer
): [] {
  const my_balance = disclose(get_balance());
  assert(my_balance >= amount, "insufficient funds");

  const new_sender_balance = my_balance - amount;
  const new_recipient_balance = get_recipient_balance(recipient) + amount;

  ledger.sender_balances[sender()] = new_sender_balance;
  ledger.recipient_balances[recipient] = new_recipient_balance;
}
```

The keyword `circuit` is the first departure from ordinary programming. This function will not execute on a server. It will be compiled into a zero-knowledge circuit – a set of polynomial constraints that a prover can satisfy and a verifier can check. Every variable inside this function will become a wire in that circuit. Every operation will become a gate.

The keyword `disclose` is the second departure, and the more consequential one. The function `get_balance()` is a witness function – it retrieves the sender's private balance from off-chain storage. That value is, by default, invisible to anyone but the prover. The `disclose()` call is the developer's explicit declaration: "I acknowledge that this private value will influence public state." Without it, the compiler rejects the program. Not at runtime. Not during testing. At compile time, before any proof is generated, before any key material is created, before any circuit is emitted.

The practical consequence is that a developer cannot accidentally write a transfer function that leaks the sender's balance. They can intentionally leak it – `disclose()` is an explicit consent mechanism, not a prohibition. But the accidental case, which accounts for the majority of real-world privacy bugs, is eliminated by the compiler's disclosure analysis pass.

The compilation itself is worth understanding. Compact's 26-pass nanopass pipeline transforms the source through a sequence of increasingly specialized intermediate languages. The program begins as `Lsrc` – essentially the developer's TypeScript-like code. It passes through type inference (`Ltypes`), where the compiler determines the bit-width of every value. It passes through disclosure analysis (`Lnodisclose`), where the compiler traces every data-flow path from witness inputs to public outputs. It passes through loop unrolling (`Lunrolled`), where bounded loops are expanded into straight-line code – necessary because ZK circuits have no concept of iteration. It passes through circuit flattening (`Lflattened`), where nested expressions are decomposed into individual gates. And it emerges as ZKIR: a JSON-formatted circuit description ready for the proof system.

At no point does the developer interact with constraints, gates, or polynomials. The entire mathematical substrate is hidden behind the compilation boundary. The developer writes TypeScript-like code. The compiler produces a zero-knowledge circuit. The gap between intent and implementation – the gap where under-constrained bugs live – is bridged by the compiler, not by the developer.

3.3.2 Noir: Write Once, Prove Anywhere

Noir takes a different approach to the same problem. Where Compact is chain-specific and privacy-first, Noir is backend-agnostic and correctness-first. A Noir program compiles not to a specific proof system's constraint format, but to ACIR – Abstract Circuit Intermediate Representation – which can then be lowered to any compatible backend.

Consider the same token transfer scenario in Noir:

```
fn main(
    sender_balance: Field,
    amount: pub Field,
    recipient_balance: Field,
) -> pub Field {
    assert(sender_balance >= amount);
    let new_recipient_balance = recipient_balance + amount;
    new_recipient_balance
}
```

The privacy model is visible in the function signature. Parameters without the `pub` keyword are private – they are part of the witness, known only to the prover. Parameters marked `pub` are public inputs, visible to the verifier. The return value, also marked `pub`, is the public output.

This is simpler than Compact's disclosure analysis. There is no `disclose()` mechanism, no 26-pass pipeline tracing data-flow paths. The developer declares privacy at the function boundary: this input is private, that input is public, and the compiler enforces the declaration. It is the developer's responsibility to get the declaration right. Noir will not warn you if you accidentally mark a sensitive value as `pub`. But it will guarantee that every private input remains invisible to the verifier – that the proof reveals nothing about private inputs beyond what the public outputs logically imply.

Where Noir stands apart is composability. A Noir developer can write a library of circuit components – hash functions, signature verifiers, Merkle tree checkers – and reuse them across projects targeting different proof systems. The same Noir code that runs on Aztec's Barretenberg backend today could, in principle, run on a PLONK backend, a Groth16 backend, or a future proof system that does not yet exist. This is not theoretical: the Noir ecosystem already includes standard libraries for common cryptographic primitives, and projects like zkEmail, zkPassport, and zkLogin use these libraries to build real applications.

A more realistic Noir program demonstrates the language's expressiveness. Here is a simplified credential verification – proving you are over 18 without revealing your birthdate:

```
use std::hash::poseidon;

fn main(
    birth_year: Field,
    birth_month: Field,
    birth_day: Field,
    credential_hash: pub Field,
    current_year: pub Field,
    threshold_age: pub Field,
) {
    // Verify the credential hash matches the private birthdate
    let computed_hash = poseidon::bn254::hash_3([birth_year, birth_month, birth_day]);
    assert(computed_hash == credential_hash);

    // Verify age threshold without revealing exact birthdate
    let age = current_year - birth_year;
    assert(age >= threshold_age);
}
```

The developer writes Rust-like code. The standard library provides cryptographic primitives. The compiler handles the translation to constraints. The program is readable, auditable, and portable across proof backends. What the program cannot do – what no Noir program can do – is enforce at compile time that the developer has not accidentally marked `birth_year` as `pub`. That responsibility rests with the developer, not the compiler.

3.3.3 Leo: Privacy as a Record System

Leo takes a third approach, one rooted in Aleo's record-based execution model. Where Compact models privacy as a property of data flow and Noir models it as a property of function signatures, Leo models privacy as a property of *records* – discrete objects that are created, consumed, and transferred, much like physical banknotes.

A Leo token transfer looks different from both Compact and Noir:

```
program token.aleo {
    record Token {
        owner: address,
        amount: u64,
    }

    transition transfer(
        input: Token,
        recipient: address,
        amount: u64,
    ) -> (Token, Token) {
        let remaining: u64 = input.amount - amount;

        let sender_token: Token = Token {
            owner: self.caller,
            amount: remaining,
        };

        let recipient_token: Token = Token {
            owner: recipient,
            amount: amount,
        };

        return (sender_token, recipient_token);
    }
}
```

The keyword `record` defines a private data structure. Records in Leo are encrypted on-chain – only the owner can decrypt them. When a `transition` consumes a record and produces new records, the old record is nullified (marked as spent) and the new records are encrypted for their respective owners. The entire UTXO lifecycle – creation, transfer, consumption – is expressed in the language itself.

The keyword `transition` is Leo's equivalent of Compact's `circuit`. It defines a function that will be proven in zero knowledge. But where Compact's circuits operate on abstract state and Noir's functions

operate on field elements, Leo's transitions operate on records – typed, owned, encrypted objects with a lifecycle managed by the Aleo runtime.

Leo's privacy model is structural rather than analytical. Privacy does not emerge from a disclosure analysis pass or from `pub` annotations on function parameters. It emerges from the record model itself: records are encrypted, transitions consume and produce records, and the only public artifact is a nullifier (proving a record was spent) and a commitment (proving a new record was created). The developer does not choose what is private. Everything inside a record is private by default. Publicity is the exception, declared through explicit `public` annotations on transition inputs.

The tradeoff is the same one that appears throughout Philosophy D: Leo programs run only on Aleo. The record model, the transition semantics, the encryption scheme – all are Aleo-specific. But for developers building on Aleo, Leo provides something that general-purpose languages cannot: a programming model where privacy is not a layer of annotation on top of ordinary computation, but the fundamental unit of state.

3.3.4 The Philosophy D Synthesis

What unites these three languages is a conviction that privacy cannot be an afterthought. In Philosophies A, B, and C, the proof system guarantees computational integrity – it proves the computation was done correctly. But it says nothing about what information the computation reveals. Privacy depends on the developer correctly managing which values are public and which are private. A mistake does not produce a compiler error. It produces a privacy leak.

Philosophy D languages treat privacy as a compiler concern. The language itself knows the difference between public and private. And in Compact's case, the compiler physically prevents the developer from accidentally crossing that boundary.

What unites Philosophies A, B, and C is a shared assumption: the developer does not need to think about privacy. The proof system guarantees computational integrity – it proves the computation was done correctly. But it says nothing about what information the computation reveals. Whether to encrypt inputs, hide outputs, or shield metadata is left to the application layer, if it is considered at all.

Philosophy D breaks this assumption. Privacy is not a layer above the language – it is embedded in the language itself.

The following table summarizes the four philosophies:

Philosophy	Representative	What It Proves	Privacy Model	Developer Experience	Tradeoff
A: EVM-Compatible	Scroll, Linea	EVM execution	None (transparent)	Familiar (Solidity)	Proving the EVM is expensive
B: ZK-Native ISA	Cairo (Starknet)	Custom CPU trace	None (transparent)	New language required	Locked to one ecosystem
C: General-Purpose ISA	SP1, RISC Zero, Airbender	RISC-V execution	None (transparent)	Familiar (Rust, C++)	Arithmetization overhead
D: Application DSL	Compact, Noir, Leo	State transitions or circuits	Compiler-enforced	Domain-specific syntax	Locked to one chain (Compact/Leo) or one IR (Noir)

The taxonomy is not a ranking. Each philosophy serves a different constituency. The following decision guide distills each philosophy's sweet spot:

Philosophy	Choose This When	Avoid When
A: EVM-Compatible	Existing Solidity codebase; need L1 security inheritance; team knows EVM tooling	Building from scratch; performance-critical new system; proving cost is binding constraint
B: ZK-Native ISA	Maximum proving efficiency; willing to learn Cairo; building within Starknet ecosystem	Need ecosystem portability; team cannot invest in new language; multi-chain deployment required

Philosophy	Choose This When	Avoid When
C: General-Purpose ISA	Standard Rust/C++ codebase; want ecosystem portability; broadest developer pool; zkVM as proving backend	Need compile-time privacy enforcement; need lowest possible proof latency for simple circuits
D: Application DSL	Privacy-preserving smart contracts; want compiler-enforced disclosure rules; domain-specific state model	Need general-purpose computation; team wants familiar language; multi-backend portability (except Noir)

To make these four philosophies concrete: **Philosophy A** is like translating a novel into a language that has no word for half the concepts – faithful but expensive. **Philosophy B** is like building a custom theater for a specific play – the acoustics are perfect, but the theater can only stage that one production. **Philosophy C** is like staging the play in any theater in the world by writing it for a universal stage: RISC-V is the universal stage, and the play (your Rust program) works anywhere. **Philosophy D** is like writing a play where the script physically prevents the actors from breaking character – in Compact, the compiler will not let a private value reach a public surface without explicit consent. Philosophy A serves Ethereum developers who want to reuse existing code. Philosophy B serves ecosystems willing to invest in a custom stack for maximum proof efficiency. Philosophy C serves the broadest developer community with the least friction. Philosophy D serves applications where privacy is not optional – where the language must prevent the developer from accidentally revealing what should stay hidden.

3.4 The Polygon zkEVM Cautionary Tale

Before we proceed, a cautionary tale.

The original version of this book listed Polygon zkEVM as the flagship example of Philosophy A – the EVM-compatible approach. Five independent reviewers flagged the same problem: Polygon zkEVM was shut down.

Polygon acquired Hermez, the team behind one of the most ambitious zkEVM implementations, for approximately $250 million. The project aimed to prove every Ethereum opcode, faithfully reproducing

the EVM's behavior inside a zero-knowledge circuit. It was technically impressive. It was also, in the end, commercially unviable.

In 2025, Polygon announced the sunsetting of zkEVM Mainnet Beta. The Hermez team, led by co-founder Jordi Baylina, spun off to form ZisK – and pivoted to RISC-V. The very team that had spent years building the most faithful EVM-compatible proof system concluded that proving the EVM directly was not the path forward. Better to prove a clean, efficient instruction set (RISC-V) and layer EVM compatibility on top.

The lesson is not that EVM compatibility is wrong. Scroll and Linea are thriving. The lesson is that *how* you achieve compatibility matters enormously. Polygon bet on faithfully reproducing every EVM quirk at the circuit level. The cost – in engineering complexity, in proving time, in the sheer number of constraints required to model 256-bit arithmetic – proved prohibitive. Scroll and Linea survived because they found more efficient paths to the same goal.

For a system architect evaluating ZK infrastructure, the takeaway is concrete: a $250 million investment with a world-class team is not sufficient if the architectural approach creates an exponential constraint-generation problem. Philosophy A is viable, but only if you resist the temptation to prove the EVM literally. Abstract where you can. Approximate where you must. And keep one eye on the RISC-V projects that may make your compatibility layer unnecessary.

The cautionary tale has a coda. Jordi Baylina, who led Polygon Hermez and co-created Circom, took the team to ZisK and chose RISC-V. The person who knew more about EVM-compatible ZK proving than almost anyone else alive concluded that the direct approach was not the path forward. When the world's leading practitioner abandons a technique, the rest of the field should pay attention.

3.5 The Developer's Actual Experience

The taxonomy of philosophies describes how *architects* think about Layer 2. But what does a *developer* actually do?

The original paper described choosing a language but never described using one. A smart reader would ask: "Fine, I picked a language. Now what?" The answer involves a lifecycle that most ZK documentation glosses over.

Step 1: Write. The developer writes source code. In SP1, this is standard Rust. In Circom, this is a template-based constraint description language. In Compact, this is TypeScript-like code with `witness` declarations and `circuit` exports. The writing experience varies enormously. An SP1 developer uses familiar tools – VS Code, cargo, clippy. A Circom developer works in a specialized IDE with no debugger,

no step-through execution, and error messages that refer to constraint indices rather than variable names.

Step 2: Compile. The compiler translates source code into a form the proof system can work with. For SP1, this means Rust to RISC-V machine code via the standard LLVM backend. For Circom, this means templates to R1CS constraint systems. For Compact, this means a 26-pass nanopass compilation pipeline that transforms the source through 26 intermediate languages – from `Lsrc` through type checking (`Ltypes`), disclosure analysis (`Lnodisclose`), loop unrolling (`Lunrolled`), circuit flattening (`Lflattened`), and finally ZKIR output.

A critical finding from recent research: standard LLVM optimization passes (-O3) yield over 40% improvement when targeting zkVMs, compared to much larger gains on traditional CPUs. This is because LLVM's optimization heuristics are tuned for hardware features – cache locality, branch prediction, instruction-level parallelism – that do not exist in a zkVM. By refining a small set of LLVM passes to use a ZK-aware cost model, researchers achieved up to 45% on individual benchmarks (with average gains of 1-4%). The compiler is an underexplored optimization surface.

Step 3: Test. The developer tests their program. In the RISC-V world, this means running the program natively (without proof generation) and checking outputs. SP1 supports this directly: you can execute your Rust program on a standard CPU and verify correctness before paying the cost of proof generation. In Compact, the SDK provides a local execution oracle that simulates the blockchain environment – block time, token balances, contract state – without a running chain.

Step 4: Prove. The developer generates a proof. This is where the cost hits. Proof generation for a Compact circuit takes seconds to tens of seconds on local development hardware (detailed timings appear in Chapter 6). For an SP1 program, proving time depends on the number of RISC-V cycles executed – a simple computation might take seconds; an Ethereum block might take minutes even on GPU clusters.

What does "prove" actually feel like? The experience is unlike anything else in software development. There is no analogy in web development, in systems programming, in machine learning training. It is its own thing, and it deserves honest description.

PLACEHOLDER_PROVE_REST

Step 5: Deploy. The proven program is deployed. For rollup-based systems, this means posting the proof and public inputs to Ethereum. For Compact, this means deploying the ZKIR circuit, TypeScript bindings, and proving keys to Midnight's network. Contract deployment on Midnight's devnet is dominated by proof generation for the constructor circuit (see Chapter 6 for measured latencies).

Step 6: Monitor. In production, the developer monitors for correctness, performance, and security. This step is almost entirely undocumented in the ZK ecosystem. There are no standard monitoring tools for ZK deployments. No dashboards for constraint utilization. No alerting for proof generation failures. The gap between "deploy" and "done" is where real-world systems fail.

One emerging bright spot: LLM-assisted ZK development. The ZK-Coder system improved Circom circuit generation success rates from 20% (baseline large language model) to 88%. This suggests that the developer experience barrier – the steep learning curve, the unfamiliar constraint semantics, the cryptic error messages – may be partially addressable through AI tooling. But 88% is not 100%, and the 12% failure cases may be precisely the subtle under-constrainedness bugs that are hardest to detect. An LLM that generates a circuit with a missing constraint is more dangerous than an LLM that fails to generate a circuit at all.

The developer workflow reveals something about Layer 2: the *language* is the part the developer sees, but the *compiler* is the part that matters. A language with beautiful syntax and a buggy compiler is worse than an ugly language with a correct compiler. The field is beginning to understand this. CirC, a unifying compiler infrastructure from Stanford, demonstrated that the compilation problem for ZK circuits shares fundamental structure with SMT solving and software verification. The same optimizations – constant folding, dead code elimination, common subexpression elimination – apply across all targets. This suggests that investment in ZK compiler infrastructure could pay off disproportionately, improving every language simultaneously rather than optimizing each one independently.

3.6 Under-Constrained Circuits: The Dominant Failure Mode

Here is the fact that should keep every ZK developer awake at night: the most common way a zero-knowledge system fails in practice is not a cryptographic break. It is not a quantum computer. It is not a governance attack. It is a bug in the program.

The under-constrained vulnerability epidemic documented in Section 3.1 – 95 of 141 real-world bugs – is not an abstraction. To understand why it happens at this scale, you need to understand the dual-track problem.

In Circom – the most widely deployed ZK language by project count – every line of code simultaneously describes two things: how to *compute* a value (witness generation), and how to *constrain* that value (the mathematical rule the proof system enforces). These two descriptions use different operators. The arrow `<--` computes a value. The triple-equals `===` constrains it. The combined operator `<==` does both.

The Tornado Cash bug was this: a developer used = (JavaScript assignment) where <== (constrained assignment) was needed. The witness generator computed the correct value. The constraint system did not enforce it. A malicious prover could substitute any value, and the proof would still verify. One character. Complete soundness break.

This is not a rare edge case. The ZKAP static analysis framework, which introduced a Circuit Dependence Graph abstraction to detect vulnerabilities in Circom circuits, found 34 previously unknown vulnerabilities across 258 circuits in 15 open-source projects. Its analysis identified three root causes:

First, *nondeterministic signals* – circuit outputs that can take multiple values for a given input because constraints are missing. Twenty-four percent of ZKAP's findings fell in this category.

Second, *unsafe component usage* – sub-circuits invoked without properly constraining their inputs or outputs. This created gaps where values passed between components were computed but not verified.

Third, *constraint-computation discrepancies* – places where the witness generator and the constraint system diverged. Division is the classic example: in witness generation, division computes a quotient. In constraints, division is expressed as multiplication (if $a/b = c$, the constraint is $b \cdot c = a$). When the divisor can be zero, these two formulations behave differently. Division-by-zero was the single most common vulnerability class in ZKAP's findings, accounting for 41% of all bugs.

The defensive tooling is evolving. Picus (also called QED^2) uses SMT-based techniques to automatically detect under-constrained circuits in R1CS, reducing the under-constrainedness problem to queries on systems of polynomial equations over finite fields. ZKAP introduced the Circuit Dependence Graph abstraction – combining data flow edges (witness computation) with constraint edges (R1CS) – and achieved an F1 score of 0.82, compared to 0.64 for the earlier Circomspect tool. zkFuzz, a fuzz-testing framework for ZK circuits, found 66 bugs including 38 zero-days. MTZK discovered 21 bugs across 4 different ZK compilers.

But these tools share a limitation: they work primarily on Circom circuits. The Rust-based systems that dominate production – halo2 (used by Scroll and ZK Bridge projects), Plonky3 (used by SP1 and Stwo), and custom constraint systems in RISC Zero and Jolt – are not covered. The most common ZK bug class has automated detection for the oldest ZK language but not for the systems where new code is being written.

NAVe, a formal verification tool for Noir programs announced in 2025, begins to close this gap. It formalizes Noir's ACIR intermediate representation and uses the cvc5 SMT solver to verify program properties. But formal verification at scale – for circuits with millions of constraints – remains beyond current tools. The combination of compile-time prevention (refinement types, disclosure analysis) and post-hoc verification (static analysis, formal methods) could provide comprehensive coverage, but no system achieves both today.

The evidence is clear: the most common failure mode in zero-knowledge systems is not a failure of cryptography. It is a failure of software engineering. The magician's choreography has a typo, and the proof system performs the typo faithfully.

3.7 Compact's Disclosure Analysis

Compact, Midnight's smart contract language, takes a fundamentally different approach to the bug problem. Rather than giving developers direct access to constraints and hoping they get it right, Compact's compiler enforces a *disclosure rule*: witness values are private by default, and any attempt to use a private value in a public context without explicit consent is a compile-time error.

This is not a style guide. It is not a best practice recommendation. It is a hard compiler rejection.

Here is how it works. In Compact, private data enters the circuit through *witness* functions – declared in Compact, implemented in TypeScript:

```
witness get_secret(): Bytes<32>;
```

The implementation runs off-chain, in the user's browser or node:

```
const witnesses = {
  get_secret: (ctx) => [ctx.privateState, hexToBytes("aabb...")]
};
```

The witness value is private. It exists only on the user's device. To use it inside a circuit – where it will be constrained and proven – the developer must explicitly call `disclose()`:

```
export circuit verify(): [] {
  const sk = disclose(get_secret());
  const my_hash = persistentHash([pad(32, "auth:"), sk]);
  assert(my_hash == stored_hash, "not authorized");
}
```

Without `disclose()`, the compiler rejects the program. The error message traces the complete path from the witness value to the public surface:

```
potential witness-value disclosure must be declared but is not:
```

```
  witness value potentially disclosed:
    the return value of witness get_amount at line 8 char 1
  nature of the disclosure:
    ledger operation might disclose the witness value
  via this path through the program:
    the argument to increment at line 11 char 8
```

The compiler catches three categories of accidental leakage: witness values used in ledger operations (writing to on-chain state), witness values returned from circuits (visible in the proof's public outputs), and witness values passed to kernel operations (token transfers, balance queries).

The disclosure analysis pass runs as part of the 26-stage nanopass compilation pipeline, at the `Lnodisclose` intermediate language stage. The privacy boundary is verified before any code generation occurs. The compiler has already type-checked the program, analyzed data flow paths, and confirmed that every potential disclosure is explicitly marked – or the compilation fails.

Consider a concrete case. The Midnight developer guide documents that the first attempt at implementing a private voting contract – using naive if/else branching on witness values – was rejected by the compiler with 11 disclosure errors. Each error traced the path from a witness value to a public surface. The compiler forced a fundamental redesign: Merkle trees replaced per-slot branching, nullifiers replaced voted-flags, and arithmetic tallying replaced conditional increments. The resulting design was not just compiler-compliant – it was architecturally superior. The naive approach would have leaked which candidate each voter chose through the pattern of ledger writes. The compiler-forced redesign made this impossible.

No other ZK language provides this guarantee. In Circom, Noir, and Cairo, privacy depends on the developer correctly managing which values are public and which are private. A mistake does not produce a compiler error. It produces a privacy leak that may not be discovered until an attacker exploits it. Compact makes privacy a compiler guarantee rather than a developer responsibility.

The tradeoff is clear: Compact contracts are locked to Midnight's proof system (PLONK on BLS12-381), token model (Zswap), and ledger architecture. They cannot be deployed on another chain. In exchange, the developer gets something no general-purpose approach can offer: the compiler will not let you accidentally show the audience what is behind the curtain.

3.8 Midnight: Compiler, IR, Circuit

Compact's three-part compilation architecture illustrates a principle that applies across all of Layer 2: the compilation target shapes the developer's world.

From a single `.compact` source file, the compiler produces three distinct artifacts:

Artifact 1: ZKIR circuits. ZKIR (Zero-Knowledge Intermediate Representation) is a JSON-formatted circuit description with 24 typed instructions organized into eight categories: arithmetic (`add`, `mul`, `neg`), constraints (`assert`, `constrain_bits`, `constrain_eq`), control flow (`cond_select`, `copy`), type encoding (`decode`, `encode`), cryptographic operations (`ec_mul`, `hash_to_curve`, `persistent_hash`), and I/O (`impact`, `output`, `private_input`, `public_input`).

Every ZKIR circuit has two transcript channels. The `publicTranscript` records ledger operations – reads, writes, comparisons – visible to the on-chain verifier. The `privateTranscriptOutputs` contains witness-derived values visible only to the prover. The ZKIR checker verifies that the serialized public transcript matches exactly what the circuit computed. Tampering with either transcript causes rejection with specific, diagnostic error messages.

Artifact 2: TypeScript bindings. The compiler generates type-safe JavaScript/TypeScript API code that handles contract interaction from the dApp frontend. This includes witness provider interfaces (the functions that supply private inputs), serialization between TypeScript types and field elements, and the transaction construction and submission pipeline. The witness functions run off-chain with access to private state, external APIs, and databases – computation that could never run inside a ZK circuit.

Artifact 3: Proving keys. Contract-specific cryptographic material for the PLONK-based proof system on BLS12-381. Different circuits produce different keys. The proof server requires these keys to generate proofs; validators require them to verify proofs.

This three-part split is not an implementation detail. It reflects the fundamental architecture of privacy-preserving computation: what can be proven (ZKIR), what runs privately (TypeScript), and what makes proofs possible (keys). No other ZK language produces all three from a single source file with first-class blockchain integration. Circom produces R1CS plus a witness generator but no blockchain API. Noir produces ACIR but no TypeScript bindings and no blockchain integration. Cairo produces execution traces but no privacy separation. Compact unifies the entire dApp development pipeline.

For a system architect, the lesson generalizes beyond Midnight: the choice of Layer 2 language is not just a choice of syntax. It is a choice of compilation target, developer tooling, privacy model, and deployment pipeline. The language shapes everything downstream.

The choreography is written. The developer has expressed their computation in a language – whether Rust targeting RISC-V, Cairo targeting Starknet, or Compact targeting Midnight's ZKIR. The compiler has translated the program into a form the proof system can work with.

But the program is just the *plan*. It describes what the computation should do. It does not contain the private data. It does not contain the execution trace. It does not contain the secret.

Now the magician goes backstage. The curtain closes. The audience waits. Behind the curtain, the magician will run the computation with real data – real bank balances, real identity credentials, real votes – and record every step. This recording is the witness: the complete execution trace that later layers will prove properties about without ever revealing.

The recording is where the real cost lives. And it is where the real vulnerabilities hide.

Chapter 4: The Secret Performance

Layer 3 – Witness Generation

4.1 The Hidden Bottleneck

Here is a number that should have been a scandal: 50-70%.

That is the fraction of total proving time consumed by witness generation in modern GPU-accelerated systems. Not 10-25%, as the field commonly claimed until recently. The outright majority. If witness generation accounts for more than half the cost of producing a zero-knowledge proof, why has the field treated it as a minor backstage interlude? Why have billions of dollars in optimization effort focused on the cryptographic proving step while the dominant bottleneck hid in plain sight?

The answer reveals something important about how technology communities deceive themselves. When GPU acceleration made the cryptographic proving step 10 times faster, witness generation did not get faster. It stayed the same speed. But its *share* of total time climbed from a modest 20% to a dominant 67%. The better the proof system got, the worse the witness gap became. The field celebrated its proving breakthroughs while the actual bottleneck quietly grew.

This chapter is about what happens backstage. The curtain has closed. The audience (the verifier) cannot see what the magician does next. She takes her private data – your bank balance, your identity, your vote – and runs the computation, recording every step. This recording is the witness: the complete execution trace. Later layers will prove properties about it without revealing its contents.

But three problems lurk behind that curtain. The recording is expensive to make. The recording room has thin walls. And if the recording is wrong, the entire system breaks.

4.2 Execution Traces

Start with the simplest possible example. You want to prove you know a number whose square is 25. The witness is that number: 5. The proof convinces the verifier that you know such a number, without revealing that the number is 5. The verifier learns "this person knows a square root of 25." The verifier does not learn "the number is 5." (In fact, the verifier does not even learn whether you chose 5 or -5 – both are valid witnesses for the same statement.)

Now scale up. You want to prove that an Ethereum state transition is valid – that a block of transactions, when applied to the current state, produces the claimed new state. The witness is not a single number. It is the *entire execution trace*: every memory access, every register value, every instruction execution, every intermediate hash computation, every storage read and write. For a complex Ethereum block, this means millions of steps, each with dozens of columns of data. The execution trace for a block proved by SP1 or RISC Zero might contain billions of field elements.

The witness is, in the most literal sense, a complete recording of everything that happened during the computation. Think of it as a security camera that films the magician backstage: it captures not just the final result but every intermediate movement, every prop placement, every sleight of hand. The recording exists so that the proof system can later verify that every step was consistent with the rules – without the verifier ever watching the footage.

Here is what a witness looks like for a trivial computation: proving that $x^2 + x = 12$ where $x = 3$.

Step	Operation	Result
1	Load x	3
2	$t_1 = x \times x$	9
3	$t_2 = t_1 + x$	12

Step	Operation	Result
4	Assert $t_2 = 12$	?

Four field elements. Four rows in the trace table. Each intermediate value is a cell the prover must fill in and the constraint system must verify. For a real zkVM executing a RISC-V program, the trace has columns for every register (32 registers), the program counter, the current instruction, memory addresses accessed, and intermediate ALU results – thousands of columns, millions of rows, but the same fundamental structure: a table where each row is one clock cycle of execution.

This distinction between the *computation* (what the magician does) and the *recording* (the witness) matters more than it looks. The computation might take milliseconds. The recording is vastly larger because it includes every intermediate value. And generating the recording – running the computation while capturing every detail – is the expensive part.

The Running Example: The Sudoku Witness

For our 4x4 Sudoku, the witness is the completed grid – sixteen field elements:

```
+---+---+---+---+
| 1 | 2 | 3 | 4 |
+---+---+---+---+
| 3 | 4 | 1 | 2 |
+---+---+---+---+
| 2 | 1 | 4 | 3 |
+---+---+---+---+
| 4 | 3 | 2 | 1 |
+---+---+---+---+
```

The execution trace records every check: "cell (0,0) = 1, matches given? yes. Row 0 sum = 10, all distinct? yes. Column 0 = {1,3,2,4}, all distinct? yes. Box (0,0) = {1,2,3,4}, all distinct? yes." Sixteen values, plus every intermediate comparison and boolean result – roughly 80 field elements in total. The verifier never sees this grid. The verifier sees only the original puzzle (the public input) and, eventually, the proof.

In a zkVM like SP1 or RISC Zero, witness generation means *emulating the entire RISC-V processor*. Every instruction is fetched, decoded, and executed. Every register update is recorded. Every memory access is logged. This is full virtual machine emulation, step by step, sequentially. It cannot be easily parallelized

because each instruction depends on the state left by the previous instruction. The program counter moves forward one step at a time, and the witness generator must follow.

Here is why witness generation is CPU-bound. Polynomial arithmetic – the core of the proving step – is naturally parallel. Number-theoretic transforms, multi-scalar multiplications, and polynomial evaluations split naturally across thousands of GPU cores. But VM emulation is inherently sequential. The next instruction depends on the current instruction's result. You cannot execute instruction 1,000 before you know the outcome of instruction 999.

4.3 Witness Generation Costs

The paper that this book revises claimed witness generation accounted for 10-25% of total proving time. That figure was approximately correct in 2023, when the proving step was slow enough to dwarf everything else. It is no longer correct.

Modern GPU-accelerated provers have transformed the cost structure. When the cryptographic proving step runs on an NVIDIA H100 GPU – or a cluster of them – it becomes 10 to 100 times faster than on a CPU. Multi-scalar multiplications, the traditional bottleneck, have been optimized to the point where number-theoretic transforms (NTTs) now account for up to 90% of GPU proving time. And NTTs themselves have been accelerated through pipelining, with systems like BatchZK achieving over 3,000 times speedup over CPU baselines for Merkle tree commitment operations.

But witness generation did not ride this wave. It remains CPU-bound. Sequential. Memory-intensive. Accelerate the proving step by 10x and leave witness generation unchanged, and watch the proportions shift:

Before GPU acceleration: witness generation takes 2 seconds, proving takes 8 seconds. Witness share: 20%.

After GPU acceleration: witness generation still takes 2 seconds, proving takes 0.8 seconds. Witness share: 71%.

Welcome to the Witness Gap – and it is growing, not shrinking. Every improvement to the proving step makes the witness gap worse by proportion. The field has been optimizing the fast part and ignoring the slow part.

The numbers from recent profiling studies confirm this. ZKPOG, the first end-to-end GPU acceleration system that treats witness generation as a first-class optimization target, demonstrated that moving witness generation to the GPU can yield 3-10x speedups. But this requires different parallelization strategies than proving. Proving parallelizes naturally because polynomial arithmetic is regular and

data-independent. Witness generation requires analyzing the circuit's dependency graph, topologically sorting gates to identify independent clusters, and mapping irregular computation patterns onto GPU hardware. The parallelism is there, but extracting it is harder.

BatchZK took a different approach: pipelining. Instead of generating the entire witness first and then proving, BatchZK overlaps witness generation with proof computation. The witness is generated in chunks and fed into the prover as a stream. This significantly improves GPU utilization throughout the pipeline compared to sequential approaches where the GPU sits idle during witness generation.

The most radical proposal comes from Nair, Thaler, and Zhu, who showed that the Jolt zkVM can be implemented with *streaming witness generation* – the prover never materializes the full witness in memory. Instead, it generates witness chunks on the fly, consumes them immediately in the sum-check protocol, and discards them. Checkpoints at regular intervals enable parallel regeneration. The space requirement drops from linear in the trace length to the square root of the trace length, with less than 2x time overhead. For a computation with 2^{35} cycles, this means roughly 100 GB of working memory instead of terabytes.

A third approach attacks the problem from the constraint side rather than the computation side. Ozdemir, Laufer, and Boneh developed new algebraic interactive proofs for RAM consistency checking – the process of verifying that memory reads and writes in the execution trace are consistent. Memory checking dominates zkVM witness generation cost because the standard approach (Merkle tree commitments) requires approximately $600 \cdot A \cdot \log(N)$ constraints for A accesses to N-sized memory. Each Poseidon hash in the Merkle tree costs roughly 300 field multiplications. Their approach reduces this to $3N + 2A + O(1)$ constraints – up to 51.3x fewer for persistent memory and even more for sparse memory. By shrinking the constraint count for memory operations, this work proportionally shrinks the witness that must be generated, since memory-related witness entries often dominate the total witness size.

These are not theoretical proposals. They represent the frontier of a field that has finally recognized where the real bottleneck lives.

4.3.1 The Bottleneck That Flipped

The before-and-after illustrates a pattern that recurs throughout engineering: the phenomenon where solving one problem promotes the next problem to dominance.

In 2023, a typical zero-knowledge proof for a moderately complex computation – say, verifying a batch of Ethereum transactions – took approximately 10 seconds end to end. Of those 10 seconds, approximately 2 seconds were spent on witness generation: emulating the virtual machine, recording every register value, logging every memory access, building the complete execution trace. The remaining 8 seconds

were spent on the cryptographic proving step: computing multi-scalar exponentiations, performing number-theoretic transforms, building polynomial commitments. The witness generation share was 20%. It was a minor cost. Nobody optimized it because the proving step was the obvious target.

The field poured billions of dollars into making the proving step faster. GPU implementations replaced CPU implementations. Custom NTT kernels exploited the butterfly structure of the transform. Batched MSM algorithms amortized curve operations across thousands of scalar multiplications. Pipeline architectures overlapped memory transfers with computation. It worked. By 2025, GPU acceleration had reduced the proving step from 8 seconds to 0.8 seconds – a 10x improvement, and some systems achieved 100x.

Witness generation was still 2 seconds.

The arithmetic is pitiless. Before: $2 / (2 + 8) = 20\%$. After: $2 / (2 + 0.8) = 71\%$. The proving step had become a solved problem. The witness step had become the problem. And the worse the proving step got (in the sense of "faster"), the more dominant the witness step became. At 100x GPU acceleration, the proving step drops to 0.08 seconds, and witness generation accounts for 96% of total time. At that point, further GPU optimization yields approximately zero improvement to the end-to-end latency. You could make the proving step infinitely fast and the proof would still take 2 seconds.

This is Amdahl's Law applied to zero-knowledge proofs. Amdahl's Law states that the speedup of a program is limited by the fraction of the program that cannot be parallelized. If witness generation accounts for 71% of total time and cannot be parallelized (because VM emulation is inherently sequential), then even infinite parallelization of the remaining 29% yields at most a 3.4x overall speedup. The bottleneck is not a bottleneck you can throw hardware at. It is a bottleneck in the structure of the computation itself.

4.3.2 Why Witness Generation Resists Parallelization

The reason witness generation is sequential is not an accident of implementation. It is a consequence of what witness generation *is*.

Consider a RISC-V program executing inside a zkVM. Each instruction reads from registers, performs an operation, and writes the result to a register. The next instruction reads from registers that may have been written by the previous instruction. The program counter advances by one. Branches depend on comparison results that were just computed. Memory loads depend on addresses that were just calculated.

This is a dependency chain. Instruction 1000 cannot execute before instruction 999 because instruction 1000 might read a register that instruction 999 wrote. Instruction 999 cannot execute before instruction

998 for the same reason. The entire execution trace is a single sequential thread of dependencies, from the first instruction to the last.

Compare this to the proving step. A number-theoretic transform operates on a vector of field elements. Each butterfly operation in the transform reads two elements, computes two outputs, and writes them back. The butterfly operations within a single stage of the transform are *independent* – they read and write disjoint memory locations. This means they can execute in parallel across thousands of GPU cores. The parallelism is not extracted by clever engineering; it is inherent in the mathematical structure of the transform.

Witness generation has no such structure. The "butterfly" of VM emulation is a single instruction, and each instruction depends on the previous one. There is no way to execute instruction 1000 before instruction 999 completes, because you do not know what instruction 1000 *is* until instruction 999 has updated the program counter. A branch instruction at position 999 could send execution to position 1000 or to position 5000 or anywhere else. You cannot know until the branch condition is evaluated.

This is why three different research groups attacked the problem from three different angles – each targeting a different dimension of the sequential bottleneck.

Pipelining (BatchZK). Instead of generating the entire witness first and then proving, BatchZK overlaps the two phases. The witness is generated in chunks: the first chunk of the execution trace is produced, then immediately fed to the GPU prover while the CPU generates the next chunk. The GPU is never idle. The CPU is never idle. The total wall-clock time drops because the two phases execute concurrently. But the witness generation itself is not faster – only the overlap is new. Pipelining does not solve the sequential bottleneck; it hides it behind the proving step.

Streaming (Nair, Thaler, Zhu). The streaming approach attacks the memory dimension. Instead of materializing the full witness – which for large computations can require hundreds of gigabytes of RAM – the streaming prover generates witness chunks on demand, feeds them into the sum-check protocol, and discards them. Checkpoints at regular intervals (every $\sqrt{T}$ steps) allow the prover to restart from any checkpoint, enabling a form of parallelism: different proving threads can work on different segments of the trace, each regenerating its segment from the nearest checkpoint. The space requirement drops from $O(T)$ to $O(\sqrt{T})$, and the time overhead is less than 2x. For a computation with 2^{35} cycles, this means roughly 100 GB of working memory instead of terabytes.

Algebraic RAM reduction (Ozdemir, Laufer, Boneh). The most radical approach does not try to make witness generation faster or smaller. It tries to make the witness *simpler*. Memory checking – verifying that memory reads and writes in the execution trace are consistent – is the dominant cost in zkVM witness generation because the standard approach uses Merkle trees. Each memory access requires a Poseidon hash, and each Poseidon hash costs approximately 300 field multiplications. For a program

with A memory accesses to N-sized memory, the standard approach requires approximately $600 \cdot A \cdot \log(N)$ constraints. Ozdemir et al. replace Merkle tree checking with algebraic interactive proofs that require only $3N + 2A + O(1)$ constraints – up to 51.3x fewer. Fewer constraints means a smaller witness for the memory-checking portion, which often dominates the total witness.

Each approach has trade-offs. Pipelining requires careful synchronization between the CPU witness generator and the GPU prover. Streaming requires checkpoint management and sacrifices some proving speed for memory savings. Algebraic RAM reduction requires new interactive proof protocols that are not yet implemented in production systems. But together, they represent the first serious engineering effort to close the witness gap – the gap that the field spent years ignoring because the proving step was the shinier problem.

The key performance numbers tell the story:

Metric	Value	Source
Witness generation share (with GPU proving)	50-70% of total time	ZKPOG
NTT share of GPU proving time	up to 90%	ZKProphet
GPU pipeline speedup over CPU (sum-check protocol)	3,040x	BatchZK
GPU pipeline speedup over CPU (Merkle tree)	793x	BatchZK
GPU pipeline memory per proof	0.08-0.44 GB	BatchZK
Streaming prover space reduction	$O(\sqrt{KT})$	Nair et al.
RAM constraint reduction	up to 51.3x	Ozdemir et al.
ZKPOG end-to-end GPU speedup	22.8x average	ZKPOG

4.4 Memory: The Binding Constraint

The Witness Gap is also a memory problem.

GPU proving requires a minimum of 24 GB of VRAM – which excludes every consumer GPU below the NVIDIA RTX 4090 (approximately $2,000). Large computations demand far more. Jolt and ZKM can require 128 GB of system RAM. Groth16 for circuits with 2^{25} constraints needs approximately 200 GB of RAM. An Ethereum block execution trace, with its millions of state accesses and storage operations, can require specialized hardware with 512 GB or more.

These are not abstract numbers. They determine who can generate proofs and who cannot.

If you are a rollup operator running a proving cluster in a data center with 16 NVIDIA H100 GPUs and dual-socket servers with 512 GB of RAM, the memory requirements are a budgeted operating expense. You buy the hardware, you run the provers, you amortize the cost across millions of transactions.

But if you are an individual user generating proofs on your own device – the scenario that provides maximum privacy, because your private data never leaves your machine – the memory requirements become a barrier to entry. A laptop with 16 GB of RAM cannot generate proofs for non-trivial computations. A phone cannot even attempt it.

This leads to an uncomfortable conclusion, and it deserves to land with full force: *privacy is, in part, a luxury good*. The privacy-maximizing architecture (client-side proving, where your secrets never leave your device) requires hardware that most people do not own. The privacy-minimizing architecture (delegated proving, where you send your private data to a proving service) works on any device but requires trusting the service with your secrets.

Read that again. The architecture that protects your data the most demands hardware that costs the most. The architecture that exposes your data to third parties is the one available to everyone. This is not a theoretical concern. It is the economic structure of privacy in 2026, and it should disturb anyone who believes privacy is a right rather than a commodity.

4.4.1 The Hardware Ladder

The abstract claim becomes concrete when you map it to specific hardware tiers. Here is what zero-knowledge proving looks like at each rung of the hardware ladder, from the bottom up.

Tier 1: A laptop with 16 GB of RAM (~$800-1,500). You can prove trivial computations – a few thousand constraints, the kind found in simple identity attestations or basic Merkle membership proofs. Anything resembling a real application (a token transfer with privacy, a complex smart contract execution, a state transition proof) exceeds your memory budget. The witness alone may require more RAM than you have. At this tier, you must delegate proving to a remote service. Your private data – the witness, which contains every secret the proof is supposed to protect – leaves your machine. You are trusting someone else's hardware with your secrets.

Tier 2: An NVIDIA RTX 4090 with 24 GB VRAM (~$2,000 for the GPU alone, ~$4,000 for a workstation that can host it). You can prove moderately complex circuits locally. This is the minimum hardware for client-side privacy-preserving proofs of meaningful complexity. ZKPOG targets this tier explicitly, arguing that it represents the frontier of "democratized" proving. Midnight's proof server targets similar hardware. At this tier, your secrets stay on your machine. You have genuine privacy. But

you have also spent $4,000 on a workstation, which places you in the top 5% of computing hardware ownership globally.

Tier 3: A data center server with an NVIDIA H100 GPU, 80 GB of HBM3 memory (~$30,000 for the GPU, ~$50,000-80,000 for a server). You can prove Ethereum blocks in real time. You can run a rollup's proving infrastructure. You can handle the most complex circuits that production systems generate today. This is the rollup operator tier – the hardware that Succinct, RISC Zero, and other proving services deploy. The H100's HBM3 provides 3.35 TB/s of memory bandwidth, which is the binding constraint for NTT performance at large polynomial sizes. No consumer GPU comes close.

Tier 4: A cluster of 16 NVIDIA H100 GPUs (~$500,000 for the GPUs alone, ~$1 million or more for the full cluster with networking, cooling, and redundancy). You can prove the most complex computations that exist in the ZK ecosystem: large zkVM programs with billions of cycles, full Ethereum block proofs with all precompiles, recursive proof compositions with deep nesting. This is the proving-as-a-service tier. Companies like Succinct and Gevulot operate at this level. A single proof that would take an RTX 4090 several minutes completes in seconds when the computation is sharded across 16 H100s with high-bandwidth interconnect.

Tier 0: A smartphone with 4-8 GB of RAM (~$200-800). This is what most of the world actually owns. At this tier, zero-knowledge proving of any meaningful complexity is not slow – it is impossible. The memory is insufficient. The compute is insufficient. The thermal envelope is insufficient. A phone cannot generate a ZK proof for a shielded transaction, a private vote, or a verifiable computation of any significant size. Not "cannot generate it quickly" – cannot generate it at all.

The uncomfortable math: approximately 5.5 billion people on Earth own a smartphone. Approximately 200 million own a desktop or laptop with a discrete GPU capable of ZK proving. Of those, perhaps 10-20 million own hardware at Tier 2 or above. That means roughly 95% of the world's population – including nearly all smartphone-only users in developing economies – cannot perform client-side ZK proving. They must delegate. They must trust. The cryptographic guarantee of privacy is available to them only through the intermediation of someone else's hardware.

This is not a bug in the technology. It is a structural feature of the cost curve. Moore's Law may eventually bring proving hardware to lower price points. Algorithmic improvements (streaming provers, algebraic RAM reduction) may lower the hardware floor. But in 2026, the privacy hierarchy is clear: the richer your hardware, the more private your computation. The poorer your hardware, the more you must trust others with your secrets.

There is a historical parallel. In the 1990s, strong encryption was classified as a munition by the United States government. Export-grade encryption was deliberately weakened to 40-bit keys, ensuring that only domestic users (and the NSA) had access to real cryptographic security. The rest of the world got a

pantomime of privacy. The “Crypto Wars” ended when the government relented and strong encryption became universally available. The current ZK hardware barrier is not a government restriction – it is an economic one. But the effect is similar: strong privacy for the few, weak privacy (or no privacy) for the many. Whether this barrier will fall, as the export restrictions did, depends on whether the field can make proving cheap enough to run on the hardware that people actually own.

The field is aware of this tension. ZKPOG specifically targets the NVIDIA RTX 4090 (24 GB VRAM, approximately $2,000) as its hardware platform, arguing that democratizing ZK proving requires targeting accessible consumer hardware rather than data center GPUs. Streaming witness generation reduces memory requirements at the cost of additional computation. And proof delegation with trusted execution environments (TEEs) offers a middle path – your data is processed inside a secure enclave that even the hardware operator cannot inspect. But TEEs have their own vulnerability history (Foreshadow, AEPIC Leak, Downfall), and Intel deprecated SGX on consumer processors in 2021.

The memory constraint also interacts with NTT performance in a way that matters for system design. NTT – the number-theoretic transform used in polynomial commitment schemes – is memory-bandwidth-limited at large sizes. The butterfly structure of the NTT requires global memory accesses with poor locality in later stages, meaning that the speed of proving is ultimately limited not by compute (FLOPS) but by memory bandwidth (GB/s). HBM (High Bandwidth Memory) on data center GPUs provides the bandwidth; consumer GPUs do not.

For a system architect making infrastructure decisions, the takeaway is this: when evaluating ZK proving solutions, ask about memory, not just speed. A system that generates proofs in 3 seconds but requires 256 GB of RAM is not the same as a system that generates proofs in 10 seconds but runs in 32 GB. The first requires a data center. The second runs on a workstation. A proving service that advertises “sub-second proofs” but requires an H100 cluster is making a different claim than one that advertises “ten-second proofs” on consumer hardware.

The memory question also intersects with the privacy question. If client-side proving requires 24 GB of VRAM, and only data center GPUs and the NVIDIA RTX 4090 have that much, then privacy-preserving client-side proving is available to a narrow slice of users. Everyone else must delegate proving to a service, which means sending their private witness to someone else’s hardware. The Midnight architecture partially addresses this by running the proof server locally alongside the dApp – but “locally” still means “on hardware with sufficient resources.” The 18-24 second proof generation time observed on Midnight’s development environment reflects desktop-class hardware. On a mobile device, the same computation might be infeasible.

4.5 Side-Channel Attacks: When the Walls Leak

The mathematical definition of zero-knowledge is precise: the proof reveals nothing about the witness beyond the truth of the statement being proved. But the mathematical definition describes the *proof*. It does not describe the *process of generating the proof*.

The process can leak.

The most vivid demonstration came in a 2020 USENIX Security paper by Tramer, Boneh, and Paterson. They showed that Zcash's Groth16 prover leaked information about transaction amounts through proof generation time. The attack was simple in concept: the prover's multi-scalar exponentiation (MSM) implementation optimized away terms where the witness coefficient was zero. More zeros in the binary representation of the transaction amount meant fewer curve multiplications, which meant faster proof generation. By measuring how long proof generation took – remotely, across the network – an attacker could estimate the Hamming weight of the transaction amount.

The correlation coefficient was $R = 0.57$. Not perfect, but far from zero. A timing measurement that should have been meaningless – how long did the proof take? – revealed information about the secret that the proof was supposed to protect.

Monero's Bulletproofs implementation was safe. The correlation was $R = 0.04$ – essentially noise. The difference was architectural: Bulletproofs operates on both the binary decomposition of the amount *and its complement*, making the number of curve operations constant regardless of the value. The proof generation time was the same whether the amount was 1 or 1,000,000. Constant-time implementation is not an optimization. It is a security requirement.

4.5.1 The Zcash Timing Attack: A Detective Story

The Zcash attack deserves to be told as the detective story it was, because the investigative method reveals how side-channel analysis works in practice – and why it is so difficult to defend against.

The researchers began with a hypothesis: if the Groth16 prover's multi-scalar exponentiation skips zero-valued scalar multiplications, then the proof generation time should correlate with the number of nonzero bits in the witness. They did not need to break any cryptography. They did not need to find a flaw in the mathematics. They needed a stopwatch.

They set up a Zcash node and generated shielded transactions with known amounts. For each transaction, they measured the wall-clock time of the `r1cs_gg_ppzksnark_prover` function – the core Groth16 proving routine in libsnark. They varied the transaction amount systematically, from round

numbers like 1.000 ZEC (which has a simple binary representation with many trailing zeros) to numbers with many nonzero digits like 1.337 ZEC (which has a denser binary representation).

The pattern emerged immediately. Round numbers produced faster proofs. Dense numbers produced slower proofs. The relationship was not subtle. A transaction of exactly 1.000 ZEC generated a proof measurably faster than a transaction of 1.337 ZEC, because the scalar representation of 1.000 ZEC in the finite field contained more zero coefficients, and each zero coefficient allowed the MSM algorithm to skip a point addition.

The correlation coefficient was $R = 0.57$ – not strong enough to determine the exact amount, but strong enough to distinguish broad categories. An attacker could not tell whether you sent 1.337 ZEC or 1.338 ZEC. But the attacker could distinguish "approximately 1 ZEC" from "approximately 100 ZEC" with meaningful confidence. The "zero-knowledge" proof leaked the order of magnitude of the transaction amount through the duration of its generation.

What makes this attack memorable is not its severity – $R = 0.57$ is a partial leak, not a catastrophic one – but what it reveals about the gap between mathematical proof and physical implementation. The Groth16 proof system is provably zero-knowledge. The mathematical proof of this property is correct. The implementation of the prover was not constant-time, and so the *process* of generating the proof leaked information that the *proof itself* was mathematically guaranteed to conceal. The proof was zero-knowledge. The prover was not.

The fix was straightforward: replace the variable-time MSM with a constant-time implementation that processes all scalar coefficients identically, whether they are zero or nonzero. The constant-time version is slower – it performs multiplications that the variable-time version skips – but it eliminates the timing channel. Zcash implemented this fix. The performance cost was real. The privacy gain was essential.

4.5.2 The Poseidon Cache-Timing Attack

The second attack class is subtler, and in some ways more disturbing, because it targets a component that was specifically designed for zero-knowledge systems.

Poseidon is an "algebraic" hash function. Unlike SHA-256, which was designed for general-purpose hashing and happens to be usable (expensively) inside ZK circuits, Poseidon was built from the ground up to minimize the number of constraints required to express its computation as a polynomial relation. Where SHA-256 requires approximately 25,000 constraints per hash in a Groth16 circuit, Poseidon requires approximately 300. This 80x reduction in constraint count translates directly into faster proving and smaller proofs. Poseidon is, by design, the ideal hash function for zero-knowledge systems. Every major ZK project uses it or a close variant.

But Poseidon's S-box – the nonlinear component that provides cryptographic security – involves computing x^5 (or x^7, depending on the variant) over a large prime field. In software, this is typically implemented using lookup tables that map input values to output values. The S-box computation accesses these tables at indices determined by the internal state of the hash, which depends on the secret input being hashed.

This is where cache timing enters. Modern CPUs use a hierarchy of caches (L1, L2, L3) to speed up memory access. When a program accesses a memory location, the CPU loads the surrounding cache line (typically 64 bytes) into the L1 cache. Subsequent accesses to the same cache line are fast (a few cycles). Accesses to different cache lines that map to the same cache set can evict earlier entries, making them slow again (hundreds of cycles).

An attacker sharing the same physical CPU – a realistic scenario in cloud computing environments where virtual machines share hardware – can observe which cache lines the victim's Poseidon computation accesses. The attacker primes the cache (fills it with known data), waits for the victim's hash computation to execute, then probes the cache to see which of the attacker's entries were evicted. The eviction pattern reveals which table entries the victim accessed, which reveals information about the internal state of the hash, which reveals information about the secret input.

The hash function was designed to be ZK-friendly in algebra. It was not designed to be constant-time in hardware. The algebraic design and the implementation security were treated as separate concerns, and the gap between them is exploitable.

This is not hypothetical. Cache-timing attacks are a well-studied attack class with decades of published results against AES, RSA, and other cryptographic primitives. The novelty of Mukherjee et al.'s work is showing that the same attack class applies to ZK-specific constructions – and that the ZK community's emphasis on algebraic efficiency has, in some cases, actively increased vulnerability by encouraging table-based designs.

4.5.3 The Electromagnetic Channel

The third attack class operates at a physical level that most software engineers never consider.

Every electronic circuit, when it operates, produces electromagnetic emanations. A transistor switching from 0 to 1 consumes a different amount of current than a transistor remaining at 0. This current difference creates a magnetic field that propagates outward from the chip. The field is weak – microwatts – but it is measurable with equipment that costs a few hundred dollars: a near-field electromagnetic probe, a low-noise amplifier, and a digital oscilloscope.

Field operations in elliptic curve arithmetic are particularly vulnerable. When a prover computes a point addition on an elliptic curve, the specific operations performed (and their power consumption) depend on the coordinates of the points being added, which depend on the witness values. A modular multiplication where both operands are large consumes more power than one where an operand is small. A conditional branch (reduce or do not reduce after multiplication) produces a different electromagnetic signature depending on which path is taken.

Measuring these emanations from a few centimeters away – close enough to touch the device but not close enough to require physical modification – can reconstruct the scalar values used in multi-scalar exponentiation. This means reconstructing the witness coefficients. This means reconstructing the private inputs to the proof.

This is not science fiction. Electromagnetic side-channel attacks are a published, demonstrated attack class. They have been used to extract AES keys from smartcards, RSA private keys from laptops, and ECDSA signing keys from hardware security modules. The equipment required is modest: a near-field probe ($50-200), an amplifier ($100-500), and an oscilloscope ($500-5,000). A university research lab can mount this attack. A well-funded adversary can mount it from across a room using more sensitive antennas.

For zero-knowledge provers running on commodity hardware – laptops, desktops, even data center servers without electromagnetic shielding – the EM channel is an open question. No major ZK implementation has published an electromagnetic side-channel analysis. The attack surface is real, the equipment is cheap, and the countermeasures (electromagnetic shielding, randomized execution ordering, amplitude-flattening power regulation) are not part of any ZK prover's design requirements.

The three attack channels – timing, cache, electromagnetic – form a hierarchy of increasing physical intimacy. Timing attacks can be mounted remotely, across a network. Cache attacks require co-location on the same physical machine. Electromagnetic attacks require physical proximity to the hardware. But all three extract information from the same fundamental source: the fact that computation is a physical process, and physical processes leave physical traces. The mathematical abstraction of a zero-knowledge

proof exists in a world of pure information. The implementation exists in a world of transistors, cache lines, and electromagnetic fields. The gap between those worlds is where privacy leaks.

The attack extended beyond timing. Mukherjee, Rechberger, and Schofnegger published the first systematic study of cache timing leakages in zero-knowledge protocols in 2024. They examined ZK-friendly hash functions (Poseidon, Reinforced Concrete, Tip5, Monolith) and popular proof systems (Groth16, Plonky2, Plonky3, halo2, Circle STARKs). Here is what they found.

ZK-friendly hash functions were designed for *algebraic* efficiency – they minimize the number of constraints required to express a hash computation inside a circuit. But nobody designed them for *implementation* security. Reinforced Concrete uses large lookup tables (256 KB for its Bars function) indexed by secret-dependent data. These table lookups create cache access patterns that vary with the secret. In a shared cloud environment, where the attacker runs on the same physical machine as the prover, these cache patterns are observable.

The irony cuts deep: the move toward lookup-based designs in ZK hash functions – motivated by the algebraic efficiency gains that Lasso and Jolt demonstrated – actively increases the side-channel attack surface. The very optimization that makes proving faster makes the proving process less private.

Field arithmetic itself leaks. The Goldilocks field ($2^{64} - 2^{32} + 1$) uses conditional reductions after arithmetic operations. If the result exceeds the modulus, a reduction step is needed; if not, it is skipped. This conditional branch creates a timing signal. Assembly implementations with branch-free code mitigate this, but many deployed field arithmetic libraries use branch-dependent paths for performance.

The accurate assessment: "zero-knowledge" is a mathematical property of the *proof*. Implementation zero-knowledge depends on the hardware, the operating system, the runtime, and the network. The gap between "the proof reveals nothing" and "the timing reveals everything" is the gap between theory and practice. The backstage walls are thinner than the magician thinks.

For defensive implementation, the standard is clear:

- No zero-skipping optimizations in multi-scalar exponentiation.
- No secret-dependent table lookups in hash functions.
- Branch-free field arithmetic, especially for conditional reduction steps.
- Cache-aligned memory access patterns.
- Constant-time prover implementations throughout the pipeline.

These requirements conflict with performance optimization at almost every turn. Making a prover constant-time means foregoing shortcuts that can halve computation time. The tension between proving speed and implementation privacy is real and ongoing.

The interaction between side channels and GPU proving adds another dimension. GPU architectures use SIMT (Single Instruction, Multiple Threads) execution, where groups of 32 threads (warps on NVIDIA hardware) execute the same instruction simultaneously. Constant-time code requires all threads in a warp to follow the same execution path. When some threads need a conditional reduction and others do not, the warp must execute both paths, with inactive threads masking their results. This "thread divergence" reduces GPU utilization – the very parallelism that makes GPUs fast for proving works against the constant-time requirement.

The open question is whether witness generation can be made fully constant-time on GPUs without unacceptable performance loss. The answer is not yet clear. What is clear is that any system claiming both GPU-accelerated proving and zero-knowledge must address this tension explicitly. Most do not.

For the reader who wants a single mental model: the backstage walls are made of different materials at different heights. The cryptographic walls (the proof itself) are mathematically perfect – no information passes through. The implementation walls (the proving process) are made of timing signals, cache patterns, and memory access traces. They leak. Not catastrophically, not in every deployment, but measurably and exploitably in the wrong environment. The system architect's job is not to eliminate all leakage – that may be impossible – but to understand where the walls are thin and what information an attacker on the other side could extract.

4.6 Witness-Constraint Divergence

The witness is not just the most expensive artifact in the ZK pipeline. It is also the most dangerous place for bugs.

Remember the dual-track problem from Chapter 3: in many ZK systems, the witness generator and the constraint system are two separate programs that must compute identical functions on all inputs. When they disagree, the result is either a soundness bug (the proof system accepts false statements) or a completeness bug (the proof system rejects true statements). Both are bad. Soundness bugs are worse.

Two real-world examples illustrate the stakes.

RISC Zero disclosed CVE-2025-52484: a missing constraint in the RISC-V circuit that allowed confusion between the rs1 and rs2 register operands. The witness generator computed the correct values for both registers. The constraint system did not enforce that the registers were distinct. A malicious prover could substitute one register for another, and the proof would still verify. The computation would appear valid – the proof would pass – but the result would be wrong.

The zkSync Era MemoryWriteQuery bug was worse. The struct that handled memory write operations failed to call `lc.enforce_zero(cs)` on the highest 128 bits of a 256-bit value, leaving those bits unconstrained. A malicious prover could modify the highest 128 bits of any memory write – including withdrawal amounts. The proof system would accept the modification. An attacker could change a withdrawal of 0.00002 ETH to a withdrawal of 100,000 ETH, and the proof would verify.

These bugs share a common structure: the witness was correct, but the constraints were insufficient. The magician performed the trick honestly backstage, but the constraint system's description of what "honest" meant was incomplete. The proof certified that the computation matched the constraints. The constraints did not match the intended computation.

Call it the correctness gap: the distance between what the developer meant and what the constraints actually enforce. It is measured not in bits of security but in lines of code that were not written.

Multiple valid witnesses can exist for the same statement, and the proof system does not care which one the prover uses. If you prove that you know a square root of 25, the proof system accepts whether you use 5 or -5. This is by design – the proof system guarantees soundness (you cannot prove a false statement), not uniqueness (there is only one valid witness).

This property – that multiple valid witnesses exist – is fundamental, not a bug. The proof system guarantees that the prover knows *some* valid witness. It does not guarantee which one. For most applications, this is exactly right: if you prove you know a valid password, it does not matter whether you know the first password in the hash table or the last.

Non-deterministic hints exploit this deliberately. Instead of computing a square root step by step – which is expensive inside a circuit – the prover *guesses* the answer (as a witness value) and the circuit verifies that the square of the guess equals 25. The computation goes from expensive (sequential square root algorithm) to cheap (one multiplication and one comparison). This "guess and check" pattern is a standard programming technique in ZK systems, not a bug. But it requires that the checking constraints are complete. If the check only verifies $x \cdot x = 25$ without constraining that x is in the correct range, a malicious prover might find a field element that satisfies the equation but is not the intended square root.

4.7 The `disclose()` Boundary: Midnight's Witness Architecture

Midnight's approach to witness generation illustrates both the power and the subtlety of the witness/circuit separation.

In Compact, the two worlds are sharply delineated:

Property	Witnesses	Circuits
Where they run	Off-chain (user's browser or node)	Compiled to ZKIR, proven locally, verified on-chain
What they can do	Arbitrary computation (JavaScript)	Only ZK-provable computation (field arithmetic, hashing, comparisons)
What they access	Private state, external APIs, databases	Only circuit inputs (public and disclosed private values)
Privacy	Completely private – never leaves the device	Proven but not revealed

Witness functions are *declared* in Compact but *implemented* in TypeScript:

```
witness get_secret(): Bytes<32>;  // declared in Compact
```

```
// implemented in TypeScript
const witnesses = {
  get_secret: (ctx) => [ctx.privateState, secretKey]
};
```

Each witness function receives the current context – including a `privateState` object that persists across invocations – and returns a tuple of the updated private state and the witness value. This means witnesses can maintain state, query external services, read databases, and perform arbitrary computation. They are full JavaScript programs, not circuit-compatible snippets.

The `disclose()` operator is the *sole gateway* from the witness world to the circuit world. Without it, witness values are invisible to the circuit, the proof, and the chain. With it, the value enters the circuit as a `private_input` in the ZKIR – the verifier never sees the value, but the constraints verify properties about it.

The compiled ZKIR circuit has two transcript channels that encode this separation physically:

The `publicTranscript` records every ledger operation – reads, writes, comparisons – as a sequence of VM operations. The on-chain verifier sees this transcript and checks that it is consistent with the proof. If the developer's circuit reads a counter value from the ledger, that read appears in the public

transcript. If the circuit increments the counter, that increment appears. The public transcript is the complete audit trail of on-chain state changes.

The `privateTranscriptOutputs` contains the witness values that entered the circuit through `disclose()`. Only the prover sees these. They are consumed during proof generation by `private_input` instructions in the ZKIR and then discarded. The ZKIR checker verifies that both transcripts are fully consumed – no extra values, no missing values, no tampering.

This two-transcript model achieves something precise: the verifier knows *what changed* on the ledger (from the public transcript) and is convinced that the changes are valid (from the ZK proof), but does not know *why* the changes were made (the private inputs that drove the computation). The "what" is public. The "why" is private.

In practice, the pipeline for a Midnight transaction involves four sequential steps:

1. `contracts.callTx()` reads current ledger state via the indexer, executes the circuit locally with current state and private witnesses, and produces an unproven transaction.
2. `proofProvider.proveTx()` generates the ZK proof – the dominant latency step, as detailed in Chapter 6 – producing a proven transaction.
3. `walletProvider.balanceTx()` binds the transaction, runs token balancing (unshielded, shielded, and dust), signs UTXO inputs with BIP-340 Schnorr signatures, and merges the balancing transaction with the original.
4. `midnightProvider.submitTx()` submits to the blockchain, where the node verifies the ZK proof, checks that the public transcript matches the ledger state, and applies the state transition.

At no point do witnesses cross the network. The developer guide states this explicitly: "Witnesses stay local. Never sent to chain."

The side-channel implications matter here. Proof generation takes a fixed amount of time regardless of witness values – dominated by the cryptographic proving step, not the witness computation. This provides natural but unintentional timing uniformity: the fixed cost of proof generation drowns out any timing variation in witness computation. But the documentation does not address cache timing, network timing (when a user queries the indexer immediately before submitting a transaction, the timing correlation reveals which contract state they are acting on), or transaction structure analysis (the number of segments in a transaction could reveal which circuit was called).

Privacy on Midnight is genuine at the cryptographic level. It is unexamined at the implementation level. This is not a criticism unique to Midnight – it applies to every privacy-preserving system in production. But it is worth noting that the same project that provides the most rigorous compile-time privacy guarantees (disclosure analysis) has the least documented runtime privacy analysis.

One accidental privacy benefit: the fixed cost of proof generation – dominated by the PLONK proving step – provides natural timing padding. Whether the witness contains a trivial secret or a complex multi-step computation, the proof generation time is approximately the same. An observer measuring transaction timing cannot easily distinguish between different witness computations. The padding is natural but not designed. A dedicated attacker measuring sub-second timing variations in the witness computation phase (which occurs before proof generation) might still extract information. But the 18-second proving step provides a large, fixed-duration buffer that dominates the total transaction time.

The developer guide's demonstration of the private voting dApp provides a concrete end-to-end example. The off-chain witness construction computes Poseidon hashes using the same `persistentHash` function available in both the Compact circuit and the JavaScript SDK. The voter provides their secret key, vote choice, Merkle sibling hash, and direction flag as witnesses. The circuit reconstructs the Merkle root from these witnesses and asserts it matches the on-chain root. The circuit also computes a nullifier – `persistentHash([pad(32, "nullf:"), secret_key])` – using domain separation so that the nullifier hash and the voter's attestation hash are cryptographically unlinkable. The nullifier is disclosed (it must appear on-chain to prevent double-voting), but it cannot be traced back to the voter's identity.

What the on-chain verifier sees per vote: a nullifier hash and updated vote tallies. What the verifier does not see: which voter, their exact identity, or which Merkle leaf they occupy. The privacy boundary is sharp, and it is enforced at every level: by the compiler (disclosure analysis), by the ZKIR checker (transcript integrity), and by the cryptographic construction (domain-separated hashing).

4.8 The Witness as a Multi-Dimensional Problem

The research literature reveals that the "Witness Gap" is not a single bottleneck but a convergence of four distinct challenges, each requiring different solutions.

The Performance Gap. Witness generation has been neglected relative to MSM and NTT optimization because it does not parallelize in the same way. ZKPOG shows that GPU-accelerated witness generation is possible (3-10x speedups) but requires analyzing the circuit's dependency graph and topologically sorting gates to identify independent clusters. The parallelism exists, but extracting it is harder than parallelizing polynomial arithmetic.

The Memory Gap. The full witness for a large computation can require hundreds of gigabytes of RAM. Streaming witness generation – never materializing the full witness, instead generating chunks on the fly and consuming them immediately – is the path forward. Nair, Thaler, and Zhu showed this can be

achieved with $O(\sqrt{T})$ space and less than 2x time overhead, using checkpoints at regular intervals for parallel regeneration.

The Security Gap. The witness is the most sensitive artifact in the system. It contains the private inputs. Side-channel attacks can leak witness information through timing (R=0.57 in Zcash), cache patterns (Mukherjee et al. 2024), and network metadata. Constant-time implementation is a security requirement, not a performance optimization.

The Correctness Gap. The witness generator and the constraint system must compute identical functions. When they disagree, the result is a soundness bug. Static analysis tools like ZKAP (F1 score 0.82, 34 previously unknown vulnerabilities discovered) can detect divergence, but they currently work only on Circom. Extending them to Rust-based systems (halo2, Plonky3) remains an open problem.

These four gaps interact. Solving the performance gap (GPU acceleration) can worsen the security gap (GPU thread divergence from constant-time code reduces SIMT utilization). Solving the memory gap (streaming) changes the architecture in ways that affect the correctness gap (streaming provers must handle state differently than batch provers). There is no single fix. The witness problem is systemic.

The analogy holds: the magician's backstage is not just dark – it is expensive, fragile, and surveilled. The recording equipment costs a fortune. The walls have cracks. And if the recording is wrong, the audience will believe a lie. Layer 3 is where the practical reality of zero-knowledge systems diverges most sharply from the elegant theory. The mathematics is beautiful. The engineering is brutal.

For the system architect, Layer 3 generates the most concrete questions in any ZK evaluation:

- What is the witness generation time for your target workload, and how does it compare to the proving time? If it exceeds 50%, your proving GPU is idle most of the time.
- What are the memory requirements? Can the witness fit in the VRAM of your target GPU, or must it be streamed from system RAM?
- Is the prover constant-time? If not, what information does the timing profile reveal about private inputs?
- Is client-side proving feasible on your users' hardware? If not, what is the trust model for delegated proving?
- How does the witness generator handle the correctness gap? Is the constraint system formally verified, statically analyzed, or tested against the witness generator?

These are not theoretical questions. They determine whether a ZK system provides the properties it claims. A system with fast proving but slow witness generation, insufficient memory, timing leaks, and unverified constraints is a system that looks good on benchmarks and fails in production.

The recording is made. It is expensive to produce. It is vulnerable to side channels. It is the most common site of implementation bugs.

But the witness is just a recording. It proves nothing by itself. Anyone could fabricate a recording. The question that Layer 4 must answer is: how do we turn this recording into a mathematical puzzle – a system of polynomial equations – such that checking the puzzle is vastly cheaper than re-doing the computation? How do we encode a million steps of execution into a form where a few random spot-checks are enough to guarantee that every step was correct?

That transformation is the subject of Layer 4: the most technically demanding layer in the stack, and the one where the magic trick metaphor will finally strain to its breaking point.

Chapter 5: Encoding the Performance

5.1 Layer 4 – Arithmetization

In the previous chapter, we watched the magician go backstage and produce a detailed recording of every step in the computation – the witness. That recording is private. Now comes the hardest transformation in the entire stack: converting that recording into a form that can be checked mathematically, without re-doing the computation, and without revealing the private data.

Arithmetization: the art of turning a computer program into a system of polynomial equations.

If the previous chapter was about what the magician does backstage, this chapter is about the notation system used to write down what happened. The notation must be precise enough to catch any error, compact enough to be checked quickly, and structured enough to reveal nothing about the performance except its correctness. Finding such a notation – and making it efficient enough for practical use – has been the central technical challenge of the zero-knowledge field for the past decade.

The Sudoku analogy implies a unique solution. Does a ZK proof have one solution or many?

The answer is: many. There are typically many valid witnesses for a given public statement. If you are proving you know a number whose square is 25, both 5 and -5 work. If you are proving you have a valid passport, any valid passport will do. The Sudoku comparison, popular in introductory ZK writing, misleads precisely because it implies uniqueness – one grid, one solution, one truth. A zero-knowledge proof is closer to proving you hold *a* winning lottery ticket without showing which one. The distinction matters because the entire machinery of this chapter exists to handle a richer, messier reality than any single-solution puzzle can capture.

Let us be honest at the outset: this is where the magic trick analogy strains hardest. A sealed scorecard, a crossword puzzle, a spreadsheet with rules – every metaphor we reach for captures one aspect and distorts another. So we will do what Feynman recommended when analogies fail: state what is actually happening in plain language, and trust the reader to follow.

This chapter is longer and more technical than the others. That is because arithmetization is where the conceptual rubber meets the mathematical road. The ideas here – constraint systems, polynomial identities, lookup arguments, the sumcheck protocol – are the load-bearing structures of every ZK system in existence. A reader who understands this chapter understands why zero-knowledge proofs work. A reader who skips it must take the rest of the book on faith.

What is actually happening is this. The computation – every addition, every comparison, every memory access – gets encoded as relationships between numbers in a finite field. These relationships take the form of polynomial equations. If the computation was performed correctly, all the equations are satisfied

simultaneously. If the prover cheated at any step, at least one equation is violated. And here is the key insight that makes the entire field of zero-knowledge proofs possible: checking whether all these polynomial equations hold can be done by evaluating them at a few random points, which is vastly faster than re-executing the original computation.

This chapter tells the story of how the encoding schemes evolved, from the rigid first attempts to the unified framework that powers every modern proof system. It is also, unavoidably, a story about the overhead this encoding imposes – and whether that overhead is an immutable tax or a temporary engineering constraint.

The story has five acts. First, we establish the spreadsheet metaphor that makes constraint systems intuitive. Second, we trace the evolution from R1CS to AIR to PLONKish, with concrete worked examples showing how each system encodes computation differently. Third, we encounter CCS – the unifying grammar that reveals all three systems as dialects of the same language – and the sumcheck protocol that powers verification. Fourth, we follow the lookup revolution from Plookup through Jolt, watching as table lookups replace polynomial constraints as the primary computation paradigm. Fifth, we confront the overhead tax honestly, with concrete numbers showing what the encoding costs in practice and where those costs are falling.

5.2 The Spreadsheet Metaphor (And Where It Works)

Before we encounter the formal constraint systems, we need a mental model. The best available one is the spreadsheet.

The spreadsheet metaphor is not perfect – we will say where it breaks down – but it is the most productive starting point. Every major constraint system (R1CS, AIR, PLONKish, CCS) can be understood as a particular way of organizing a spreadsheet and writing rules for its cells. The differences between the systems are differences in how the rules are structured, not in the underlying idea.

Imagine a computation with a thousand steps. You create a giant table. Each row represents one moment in time – one step of the computation. Each column represents a variable: a processor register, a memory value, a boolean flag. Every cell contains a number drawn from a finite field (think: integers modulo a large prime).

Consider a tiny computation: $x = 3$, $y = 4$, $z = x + y = 7$, then $w = z \times 2 = 14$.

Row	A (input 1)	B (input 2)	C (result)	Rule
1	3	4	7	C = A + B
2	7	2	14	C = A * B

If every rule holds across every row, the spreadsheet faithfully records the computation. Change any cell, and at least one rule breaks. Suppose a cheating prover changes the result in row 1 from 7 to 9. Row 1 now violates its own rule ($3 + 4$ is not 9), and row 2 also breaks (because row 2 expects to read 7 from row 1's output, not 9). Errors propagate. This is a two-row example, but the principle scales to millions of rows: one wrong cell poisons the entire spreadsheet.

Now you write rules. "The value in column B at row 5 must equal the value in column A at row 4 plus the value in column C at row 4." "If the opcode column at row 7 says 'multiply,' then column D at row 7 must equal column B at row 7 times column C at row 7." These rules are polynomial equations that relate cells to one another.

Notice that the rules are not arbitrary. They are polynomial equations – expressions built from addition, subtraction, and multiplication of cell values. This restriction is fundamental. A polynomial rule like "$A \cdot B = C$" is checkable by the proof system. A non-polynomial rule like "if $A > B$ then $C = 1$ else $C = 0$" cannot be directly encoded as a polynomial equation because comparison is not a polynomial operation. (It can be encoded *indirectly*, by decomposing A and B into bits and constraining the bit-level comparison, but this adds many auxiliary constraints.) The polynomial restriction is the price of admittance to the proof system. Only relationships expressible as polynomial equations over finite fields can be directly verified. Everything else must be translated into polynomial form first.

If every rule holds across every row, the spreadsheet is *consistent* – it faithfully records a valid computation. If any rule is violated, the computation was not performed correctly. The prover's job is to fill in the spreadsheet (this is the witness from the previous chapter) and then convince the verifier that all the rules hold. The verifier's job is to check – but not by examining every cell. Instead, the verifier picks random evaluation points and checks whether the polynomial equations are satisfied there. By the Schwartz-Zippel lemma, a polynomial that is not identically zero will be nonzero at a random point with high probability. The Schwartz-Zippel lemma is the mathematical fact that makes this work: a nonzero polynomial of degree d, evaluated at a random point from a field of size q, is zero with probability at most d/q. For the fields used in ZK (where q is astronomically large), this probability is negligible. One random check is almost as good as checking everywhere. So if the equations check out at the random points, the spreadsheet is almost certainly correct everywhere.

That is the core idea of arithmetization. Every constraint system in this chapter is a different way of organizing the spreadsheet, choosing the rules, and encoding the computation.

The Running Example: The Sudoku Constraints

Our Sudoku witness becomes a 16-row constraint system. Each cell must satisfy:

- **Range constraint**: $(\text{cell}-1)(\text{cell}-2)(\text{cell}-3)(\text{cell}-4) = 0$. This polynomial evaluates to zero only when the cell contains a valid value. Four values, one degree-4 polynomial per cell.
- **Given-cell constraint**: For each clue, $\text{cell}_i = \text{given}_i$. Eight equalities for our 8-given puzzle.
- **Uniqueness constraint**: For each row, column, and 2x2 box, the product $(a-b)$ for all pairs must be nonzero. Equivalently: the polynomial product over all pairs of $(a-b)$ must be nonzero for each group. Eight groups, $\binom{4}{2} = 6$ pairs each, yielding 48 pair checks.

Total: 16 range constraints + 8 given-cell constraints + 48 uniqueness checks = 72 constraints over 16 witness variables. In R1CS form, each degree-4 range constraint decomposes into intermediate multiplications, expanding to roughly 120 R1CS constraints. In CCS form, the higher-degree constraints can be expressed directly. The witness (the completed grid) satisfies all 72 constraints. A wrong value in any cell makes at least one polynomial nonzero, and the Schwartz-Zippel lemma catches it with overwhelming probability at a random evaluation point.

Why polynomials? Because of a fact about polynomials: a polynomial of degree d is completely determined by its values at any $d+1$ points. If you know a line (degree 1), two points fix it exactly. If you know a cubic (degree 3), four points fix it exactly. This means that if a polynomial "misbehaves" at even a single point, it must be the wrong polynomial – and checking it at a random point catches this misbehavior with near certainty. A polynomial commitment scheme exploits this: the prover seals a polynomial into a short commitment, and the verifier can spot-check it at random points to confirm it is correct – without ever seeing the full polynomial.

The metaphor is imperfect in one important respect: a real spreadsheet has rows and columns with human-readable labels. A constraint system is a set of abstract polynomial equations over vectors of field elements. The "rows" and "columns" are a convenient way to think about structure, but the mathematics does not require a rectangular layout. Keep this in mind as we move through the specific systems.

With the spreadsheet image in hand, we can state the central question of this chapter: What is the best way to organize the rules? Should each row have its own custom rule (like R1CS)? Should all rows share

the same rule (like AIR)? Should rows have switchable rules controlled by flags (like PLONKish)? Or should all these approaches be unified under a single framework (like CCS)? The history of arithmetization is the history of answering this question, and the answer keeps changing as proof systems evolve and new mathematical tools become available. The constraint systems in the next sections are not mere notation. They are architectural decisions that determine the performance, flexibility, and security of every zero-knowledge proof system built on top of them.

5.3 The Constraint System Evolution: R1CS, AIR, PLONKish

The three major constraint systems – R1CS, AIR, and PLONKish – emerged in a span of just seven years (2012-2019). Each was designed to solve a specific limitation of its predecessor, but each also introduced new trade-offs. Understanding this evolution is not optional for understanding modern ZK: every proof system, every zkVM, and every privacy protocol in production today is built on one of these three foundations (or, increasingly, on CCS, which unifies all three).

The history of arithmetization is a history of increasing expressiveness. Each new constraint system solved a specific limitation of its predecessor. Understanding this genealogy is essential because the constraint system you choose determines which proof systems you can use, which fields are efficient, and how much overhead the encoding imposes.

The genealogy also reveals how rapidly the field moves. R1CS was introduced in 2012. By 2023, it was already the "legacy" format – still deployed in production (Groth16 is not going away), but superseded by more expressive systems for new development. The eleven-year span from R1CS to CCS saw more architectural innovation in constraint system design than the previous four decades of theoretical computer science produced. This acceleration is driven by practical pressure: the economic value of efficient zero-knowledge proofs creates strong incentives for better arithmetization.

5.3.1 R1CS: The Assembly Language (2012)

The first practical arithmetization emerged from the work of Gennaro, Gentry, Parno, and Raykova (GGPR) in 2012, who introduced the QAP (Quadratic Arithmetic Program) framework that R1CS later formalized. The name "Rank-1 Constraint System" describes the mathematical structure precisely: each constraint has rank 1 (it is the product of two linear functions), and the system is a collection of such constraints. The "rank-1" designation means each constraint captures exactly one multiplication – a bilinear relationship between variables. Addition is free (it does not require a constraint, because linear combinations can be folded into the matrix entries). Only multiplication generates constraints. This is

why the number of R1CS constraints for a circuit equals the number of multiplication gates, not the total number of gates.

Before the mathematical notation, let us ground it in the spreadsheet from the previous section. Imagine your spreadsheet has three special columns – call them A, B, and C. For each row, column A and column B each contain a combination of the variables, and column C contains the result. The rule for every row is: (what is in column A) multiplied by (what is in column B) must equal (what is in column C). That is all R1CS is: a spreadsheet where every row enforces one multiplication rule. The mathematical notation below says exactly this, just more precisely.

R1CS encodes a computation as a list of constraints, each of the form:

(linear combination of variables) times (linear combination of variables) equals (linear combination of variables)

Or, in mathematical notation: $(\mathbf{A} \cdot \mathbf{z}) \circ (\mathbf{B} \cdot \mathbf{z}) = \mathbf{C} \cdot \mathbf{z}$, where $\mathbf{A}$, $\mathbf{B}$, and $\mathbf{C}$ are sparse matrices and $\mathbf{z}$ is the vector of all variables (public inputs, private witness, and intermediate values). Each row of the matrices defines one constraint. Each constraint captures one multiplication gate.

For the tiny spreadsheet example ($3 + 4 = 7$, then $7 * 2 = 14$), the witness vector is $\mathbf{z} = (1, 3, 4, 7, 2, 14)$ – the constant 1 followed by the variables x, y, z, w, and the final result. The first row of A selects "x" (entry 1 in position corresponding to x), the first row of B selects "1" (indicating the addition is encoded as $(x + y) * 1 = z$ after reformulation), and the first row of C selects "z". The second row of A selects "z" (value 7), the second row of B selects "w" (value 2), and the second row of C selects "result" (value 14). The matrices are mostly zeros – only a handful of entries are nonzero. This sparsity is typical: a circuit with millions of constraints has matrices with millions of rows but only a few nonzero entries per row.

R1CS is the assembly language of constraint systems – simple, well-understood, and directly amenable to proof systems like Groth16 and Spartan. Groth16, deployed across most of the Ethereum ecosystem, works natively with R1CS and produces the smallest possible proofs – three elliptic curve group elements, constant-time verification.

But R1CS has a fundamental limitation: each constraint is bilinear – degree 2. You can encode a multiplication ($a \cdot b = c$) directly. But what about a computation that requires checking a hash function, where a single invocation might need thousands of multiplications? You encode each multiplication as a separate constraint, one after another. There is no way to express a higher-degree relationship in a single constraint, and there is no notion of "this constraint applies uniformly across all time steps." Every gate gets its own row.

For small circuits, R1CS works beautifully. For large, repetitive computations – like executing millions of processor instructions in a zkVM – the lack of structure becomes a liability.

To see this concretely, consider encoding a computation with 10 million multiplication gates in R1CS. You need 10 million rows in the matrices A, B, and C. Each matrix is sparse (most entries are zero), but the total number of nonzero entries – and hence the prover's work – scales linearly with the gate count. There is no compression possible: R1CS treats each gate independently, with no awareness that gate 5,000,001 might be performing exactly the same operation as gate 1. Compare this to a function that repeats the same 1,000-gate circuit 10,000 times: R1CS still requires 10,000,000 separate constraints, while a structured constraint system could potentially describe the repeating pattern once and instantiate it 10,000 times. This structural blindness is what motivated the search for richer constraint formats.

Yet R1CS persists. Groth16 proofs (which require R1CS) remain the gold standard for on-chain verification because of their unmatched proof size: 3 group elements, roughly 128 bytes, verifiable in constant time. No other proof system achieves this compactness. When Ethereum smart contracts verify ZK proofs, the gas cost of verification is proportional to proof size – and Groth16's tiny proofs mean minimal gas costs. Many production systems (Zcash, Tornado Cash, Worldcoin) use Groth16 for precisely this reason, accepting the constraint system's limitations in exchange for the proof system's efficiency. R1CS is the assembly language: nobody wants to write it directly, but the machine code it produces is unbeatable.

5.3.2 AIR: The State Machine (2018)

The Algebraic Intermediate Representation arrived alongside STARKs, introduced by Ben-Sasson, Bentov, Horesh, and Riabzev in 2018. The name is precise: "Algebraic" because the constraints are polynomial equations over fields (not boolean circuits or SAT formulas). "Intermediate" because AIR sits between the high-level computation and the low-level proof system – it is the "compiled" form of the computation, analogous to LLVM IR in a compiler toolchain. "Representation" because it is a way of representing the computation, not the computation itself. AIR solves the structure problem by embracing the spreadsheet metaphor directly.

An AIR consists of two things: an execution trace (a 2D matrix where rows are time steps and columns are algebraic registers) and transition constraints (polynomial equations that must hold between consecutive rows). The constraints are *uniform* – the same polynomial equations apply at every row.

This uniformity is AIR's great strength and its great limitation. It is a perfect fit for sequential computations where the same operation repeats: hash chains, state machine execution, virtual machine instruction cycles. The constraint "if the opcode is ADD, then the output register equals the sum of the two input registers" applies identically at every step. You write it once; it is enforced everywhere.

The word "uniform" deserves emphasis. In R1CS, each constraint row can encode a completely different relationship between variables. Row 1 might enforce $a+b=c$; row 2 might enforce $d \cdot e = f$; row 3 might enforce $g = h$. Each row is independent. In AIR, every row obeys the same set of transition polynomials. If the transition constraint says "column_3[next] = column_1[current]+column_2[current]," then this relationship holds at every pair of consecutive rows. The prover cannot make exceptions. This rigidity is what enables compression: a single polynomial equation describes the entire computation, regardless of how many steps it contains.

The limitation is that AIR cannot natively handle non-uniform computation. If your program has different instruction types – additions, multiplications, hash invocations, memory accesses – you need tricks to encode the selection logic ("which instruction is executing at this row?") within the uniform framework. This is possible but adds complexity and overhead.

In practice, real STARK-based systems handle non-uniformity by using multiple AIR traces – one per "sub-machine." Cairo's architecture, for example, decomposes the VM into separate traces for the CPU, memory, range checks, and each built-in operation (Pedersen hash, ECDSA, bitwise operations). Each trace is a separate AIR with its own transition constraints. Cross-trace consistency is enforced through permutation arguments and lookup arguments that connect the traces: when the CPU trace records "hash instruction at step 1000," the hash trace must contain a corresponding row with matching inputs and outputs. This multi-trace design preserves AIR's uniformity within each trace while handling the

non-uniformity of a full instruction set across traces. The cost is the cross-trace connection overhead, but for computations dominated by a single operation type (as hash-heavy applications often are), the overhead is manageable.

AIR became the foundation of the STARK ecosystem: StarkWare's Stone and Stwo provers, Polygon Miden, and others. Its tight coupling with FRI (the hash-based polynomial commitment scheme) means AIR-based systems are transparent (no trusted setup) and plausibly post-quantum secure.

The AIR-FRI coupling deserves a moment of attention because it illustrates how Layers 4 and 5 (arithmetization and proof system) become inseparable. FRI works by repeatedly folding a polynomial in half – reducing its degree by a factor of 2 at each step – and checking consistency at random points. This folding requires the polynomial to be evaluated on a domain with a specific multiplicative structure (a "coset" of a subgroup of the field). AIR traces are naturally expressed as polynomials on such domains because the trace rows correspond to consecutive powers of a group generator. The uniformity of AIR's transition constraints means the constraint polynomial has the same degree structure as the trace polynomial, which is exactly what FRI needs. Try to use FRI with a non-uniform constraint system (like PLONKish), and you need additional machinery (permutation polynomials, selector commitments) that adds overhead. AIR and FRI were born for each other.

A Tiny AIR: The Counter

To make AIR concrete, consider the simplest possible state machine: a counter that starts at 0 and increments by 1 each step. The execution trace has two columns – `counter` (the current value) and `flag` (whether to increment) – and three rows:

Row	counter	flag
0	0	1
1	1	1
2	2	1

The transition constraint is a single polynomial equation that must hold between every pair of consecutive rows:

$\text{counter}[i+1] = \text{counter}[i] + \text{flag}[i]$

Check it. Between rows 0 and 1: does $1 = 0 + 1$? Yes. Between rows 1 and 2: does $2 = 1 + 1$? Yes. The trace is valid.

Now suppose a cheating prover submits a trace where row 2 claims `counter = 5`. The transition constraint between rows 1 and 2 becomes: does $5 = 1 + 1$? No. The constraint is violated, and the proof fails.

There is a subtlety that the transition constraint alone does not capture: the starting value. The transition constraint says "each row follows correctly from the previous row," but it says nothing about where the counter begins. A trace starting at counter = 1000 with flag = 1 at every row would satisfy the transition constraint perfectly – 1000, 1001, 1002 – even though the counter was supposed to start at 0. This is where **boundary constraints** enter. A boundary constraint pins a specific cell to a specific value: "counter at row 0 must equal 0." In an AIR, you typically have transition constraints (which apply uniformly between consecutive rows) and boundary constraints (which apply at specific rows, usually the first and last). The transition constraints ensure the computation proceeds correctly; the boundary constraints ensure it starts and ends in the right place.

The full AIR for this counter is therefore:

- Transition constraint: $\text{counter}[i+1] = \text{counter}[i] + \text{flag}[i]$ (for all consecutive row pairs)
- Boundary constraint: $\text{counter}[0] = 0$ (the counter starts at zero)
- Boundary constraint: $\text{flag}[i] \cdot (1 - \text{flag}[i]) = 0$ (each flag is boolean – either 0 or 1)

The boolean constraint on the flag deserves attention. Without it, a cheating prover could set flag = 7 at some row, making the counter jump by 7 instead of 0 or 1. The constraint $\text{flag} \cdot (1 - \text{flag}) = 0$ is satisfied only when flag is 0 or 1 (plug in either value and one factor is zero). This is a polynomial constraint of degree 2 – exactly the kind of equation that AIR handles naturally.

The critical observation is what the prover *wrote down* to define this constraint system. Not three separate rules – one for each row – but a small set of polynomial equations applied uniformly across the entire trace. One transition rule and a few boundary conditions, enforced everywhere. If the counter had a million rows instead of three, the constraint description would be identical: the same equations. Only the trace grows; the constraints stay fixed.

This is the structural difference from R1CS. In R1CS, you would write a separate constraint for each row: "row 0's output equals row 0's input plus row 0's flag," then "row 1's output equals row 1's input plus row 1's flag," and so on – one constraint per step. For a million-step computation, you need a million constraints. In AIR, you write the rule once. The prover fills in the trace; the polynomial machinery checks the rule everywhere simultaneously.

For repetitive computations – hash chains where the same compression function executes thousands of times, virtual machines where the same instruction cycle repeats for every step – AIR's uniformity is not just convenient. It is a compression of the constraint description itself, from linear in the number

of steps to constant in the number of distinct transition rules. That compression is what made STARKs practical for proving large computations.

One further detail illuminates how the polynomial machinery works behind the scenes. The prover does not submit the raw trace table to the verifier. Instead, the prover interpolates each column of the trace as a polynomial. For the counter column with values (0, 1, 2), the prover finds a polynomial $P(x)$ such that $P(0) = 0$, $P(1) = 1$, $P(2) = 2$ – in this case, simply $P(x) = x$. For the flag column with values (1, 1, 1), the polynomial is $F(x) = 1$. The transition constraint "$P(x + 1) = P(x) + F(x)$" becomes a polynomial identity that must hold at $x = 0$ and $x = 1$ (every pair of consecutive rows). The proof system checks this identity at a random evaluation point – not at $x = 0$ or $x = 1$, but at some random r chosen by the verifier – and the Schwartz-Zippel lemma guarantees that a false identity will fail this random check with overwhelming probability.

The trace, the constraints, and the verification all reduce to polynomials. The trace columns are polynomials. The transition constraint is a polynomial identity. The verification is a polynomial evaluation. This is what "arithmetization" means in practice: every aspect of the computation becomes a polynomial, and every check becomes a polynomial evaluation.

The polynomial encoding also reveals the source of the overhead. The counter trace has 3 rows and 2 columns – 6 values. But the polynomials that interpolate these columns have degree 2 (you need a degree-2 polynomial to pass through 3 points in general). The transition constraint, when expressed as a polynomial, produces a "constraint polynomial" whose degree is the sum of the degrees of the trace polynomials it involves. If the trace polynomial has degree d and the transition constraint has algebraic degree k, the constraint polynomial has degree roughly $d \cdot k$. For a trace with n rows, d is roughly n, so the constraint polynomial has degree roughly $n \cdot k$. This polynomial must be shown to vanish on all consecutive-row pairs, which means it is divisible by a "vanishing polynomial" $Z(x)$ that has roots at the evaluation domain. The quotient $T(x) = \text{constraint}(x)/Z(x)$ is the polynomial the prover commits to; if the division is exact (no remainder), the constraints are satisfied. If the prover cheated, the division leaves a remainder, and the random evaluation check catches it.

This is where the NTTs come in. Converting between coefficient and evaluation representations of these high-degree polynomials requires the Number Theoretic Transform – the finite-field version of the FFT – which dominates the prover's computation time.

The limitation is equally visible. Suppose you want some rows to add and other rows to multiply. With a single uniform constraint, you cannot express "do addition at row 5 and multiplication at row 6" without encoding the selection logic into the polynomial itself – adding flag columns, conditional terms, and degree overhead. The counter example is clean because every row does the same thing. Real programs do not.

5.3.3 PLONKish: The Custom Workshop (2019)

PLONK, introduced by Gabizon, Williamson, and Ciobotaru in 2019, took a different approach. Instead of uniform constraints, PLONKish arithmetization uses *selector columns* to enable non-uniform gates.

The key innovation separates the constraint system into two components:

1. **Gate constraints**: polynomial equations controlled by selector polynomials. Different rows can have different gate types. If the selector for "addition" is active at row 5, the addition constraint is enforced there. If the selector for "multiplication" is active at row 6, the multiplication constraint is enforced there. You can define custom gates for any operation you need.
2. **Copy constraints**: a permutation argument that enforces wiring – ensuring that the output of one gate is correctly fed as input to another gate. This replaces R1CS's matrix-based variable assignment with a more flexible connection mechanism.

PLONKish sits between R1CS and AIR in expressiveness. Like AIR, it uses a structured trace with rows and columns. Unlike AIR, different rows can follow different rules. Like R1CS, it can handle arbitrary circuits. Unlike R1CS, it supports custom gates that capture complex operations in fewer constraints.

PLONKish became the dominant arithmetization in deployed systems. Halo2 (used by Zcash and Scroll), Polygon zkEVM (before its shutdown), and numerous other production systems chose PLONKish because its flexibility handles the diverse instruction sets of real-world computations. The Halo2 library, originally developed by the Electric Coin Company for the Zcash Orchard protocol, became the de facto standard for PLONKish circuit development. Its "region-based" API lets developers define gates, assign cells, and specify copy constraints in a structured way that catches many common errors at compile time. Scroll's zkEVM – one of the most ambitious ZK projects ever attempted – encoded the entire Ethereum Virtual Machine instruction set as Halo2 PLONKish circuits, using custom gates for EVM opcodes, lookup arguments for bytecode verification, and copy constraints to wire the data path. The resulting circuit has millions of constraints per block and requires GPU clusters to prove, but it works – which says something about PLONKish's flexibility.

A Tiny PLONKish Circuit: Compute 3 + 4, Then Multiply, Then Add Again

Here is a three-row PLONKish trace that computes $(3 + 4) * 2 + 1 = 15$. The trace has three "witness" columns (a, b, c) and two "selector" columns (q_add, q_mul):

Row	a	b	c	q_add	q_mul
0	3	4	7	1	0

Row	a	b	c	q_add	q_mul
1	7	2	14	0	1
2	14	1	15	1	0

The gate constraint is a single equation evaluated at every row:

$q_{\text{add}} \cdot (a + b - c) + q_{\text{mul}} \cdot (a \cdot b - c) = 0$

At row 0: $q_{\text{add}} = 1$, $q_{\text{mul}} = 0$, so the equation becomes $1 \cdot (3 + 4 - 7) + 0 \cdot (\ldots) = 0$. Check: $0 = 0$. The addition gate is active.

At row 1: $q_{\text{add}} = 0$, $q_{\text{mul}} = 1$, so the equation becomes $0 \cdot (\ldots) + 1 \cdot (7 \cdot 2 - 14) = 0$. Check: $0 = 0$. The multiplication gate is active.

At row 2: $q_{\text{add}} = 1$, $q_{\text{mul}} = 0$, so the equation becomes $1 \cdot (14 + 1 - 15) + 0 \cdot (\ldots) = 0$. Check: $0 = 0$. The addition gate is active again.

The selectors act as switches. When $q_{\text{add}} = 1$ and $q_{\text{mul}} = 0$, only the addition constraint is "on." When the selectors flip, only the multiplication constraint is "on." One polynomial equation, evaluated identically at every row, enforces different gate types depending on which selector is active.

But the gate constraint alone does not guarantee correctness. Look at the trace: row 0 produces c = 7, and row 1 consumes a = 7. How does the proof system know these two 7s are the *same* value – that the output of row 0 actually flows into the input of row 1?

This is the job of the **copy constraint**. A permutation argument – a separate cryptographic mechanism outside the gate equation – enforces that the cell (row 0, column c) contains the same value as the cell (row 1, column a). Similarly, (row 1, column c) must equal (row 2, column a). The permutation argument works by proving that a certain set of values is a rearrangement of another set, which can only be true if the "wired" cells agree. Without copy constraints, a cheating prover could fill row 1 with a = 999 and the gate equation would still pass (as long as 999 * 2 = c at row 1). The copy constraint is what stitches the circuit together.

To see the copy constraint at work, consider what happens without it. A cheating prover submits this trace:

Row	a	b	c	q_add	q_mul
0	3	4	7	1	0

Row	a	b	c	q_add	q_mul
1	99	2	198	0	1
2	198	1	199	1	0

Every gate constraint passes: $3 + 4 = 7$, $99 * 2 = 198$, $198 + 1 = 199$. But the computation is wrong – row 1 should have used $a = 7$ (the output of row 0), not $a = 99$. Without the copy constraint binding cell (row 0, c) to cell (row 1, a), the prover is free to insert any value it likes. The copy constraint catches this: it requires that position (row 0, column c) and position (row 1, column a) hold the same value. Since 7 is not 99, the permutation check fails, and the proof is rejected.

This reveals why PLONKish is more flexible than AIR. In the AIR counter example, every row obeyed the same transition rule. Here, row 0 adds, row 1 multiplies, and row 2 adds again – three different operations in three rows, controlled by selector values. You can define custom gates for any operation: a "range check" gate, a "Poseidon hash round" gate, an "elliptic curve addition" gate. Each gets its own selector column, and the prover activates whichever gate the computation requires at each row. The trace is a heterogeneous computation log, not a uniform state machine.

The power of custom gates becomes clearer with a slightly more complex example. Suppose you want to enforce that a value lies in the range $[0, 255]$ – an 8-bit range check. In R1CS, you would decompose the value into 8 bits, constrain each bit to be boolean (8 constraints), and constrain the sum to equal the original value (1 constraint) – 9 constraints total. In PLONKish, you can define a single custom "range gate" that encodes the entire check in one row, using a lookup argument or a specialized polynomial identity. One row, one gate, one constraint. The circuit designer creates the gate once; the prover activates it wherever a range check is needed.

The cost of this flexibility is the copy constraint machinery. The permutation argument adds overhead – both in proof size and in prover computation – that AIR avoids because AIR's uniform structure implicitly handles data flow between consecutive rows. But for computations that mix many different operations (as real programs do), the overhead is worth paying.

The Same Computation, Three Encodings

To crystallize the differences, consider encoding the same simple computation – "compute $x \cdot (x + 1)$ where $x = 3$, so the result is 12" – in all three systems.

In R1CS: You need two constraints. First, an addition: an intermediate variable $t = x + 1 = 4$. Then a multiplication: result $= x \cdot t = 3 \cdot 4 = 12$. Each constraint takes one row in the R1CS matrix. The matrices A, B, C encode the variable wiring. Two rows, two constraints, done.

In AIR: You set up a two-row trace. Row 0 holds $x = 3$ and computes $t = x + 1 = 4$. Row 1 holds the multiplication result $= x \cdot t = 12$. The transition constraint relates consecutive rows. But here is the awkwardness: the addition and the multiplication are *different* operations, and AIR wants uniform constraints across all rows. You either need to encode both operations into a single transition polynomial (using conditional logic with flag columns, which increases the constraint degree), or you define a two-step cycle where even rows add and odd rows multiply (which works but means half the trace structure is "wasted" on selection logic). For this tiny example, AIR is overkill.

In PLONKish: Row 0 uses the addition gate: $a = 3$, $b = 1$, $c = 4$ ($q_{\text{add}} = 1$). Row 1 uses the multiplication gate: $a = 3$, $b = 4$, $c = 12$ ($q_{\text{mul}} = 1$). A copy constraint links (row 0, column c) to (row 1, column b), and another links the input $x = 3$ to (row 1, column a). Two rows, two different gate types, clean and direct.

The comparison reveals each system's natural habitat. R1CS handles this computation most directly – two bilinear constraints, no overhead. PLONKish handles it almost as directly, with slight overhead from the copy constraints. AIR handles it least naturally, because the computation is not repetitive – there is no pattern that repeats across many rows. For a computation that *is* repetitive (running the same hash compression 1000 times), the ranking reverses: AIR wins by writing the constraint once, while R1CS and PLONKish must either repeat the constraint description 1000 times or use recursion to simulate repetition.

The constraint count for the same computation across different systems is instructive:

System	Constraints for x*(x+1)	Constraints for 1000 hash rounds	Why
R1CS	2	~30,000,000	1 constraint per gate, ~30,000 per hash round
AIR	~4 (with padding)	~30,000	One transition polynomial, reused 1000 times
PLONKish	2 (+ copy constraints)	~15,000,000	Custom hash gates cut per-round cost in half

The numbers are approximate, but the ratios are revealing. For the tiny computation, all three systems are roughly comparable. For the hash chain, AIR's constraint count is independent of the number of repetitions – it depends only on the number of distinct transition types. This is why STARKs dominate in hash-heavy workloads (blockchain state verification, recursive proof composition) while PLONKish dominates in mixed workloads (smart contract execution, general-purpose circuits).

A clarification for the precise reader: AIR's "~30,000" for 1000 hash rounds refers to the *constraint description* size – the number of distinct polynomial equations that must hold. The actual *trace* still has 1000 * (rows per hash round) rows, each of which must satisfy the constraints. The prover's work is proportional to the trace size, not the constraint description size. But the constraint description size matters for the verifier (who must check the polynomial identity, not every row) and for the proof size (which depends on the degree of the constraint polynomial, not the number of trace rows). The asymmetry between "small constraint description, large trace" is precisely what makes AIR efficient for repetitive computations: the verifier's work grows with the constraint complexity, not with the number of repetitions.

5.3.4 Three Dialects, One Problem

By 2022, the ZK ecosystem had three constraint system families, each with its own proof systems, tooling, and community:

System	Year	Constraint Structure	Best For	Key Proof Systems
R1CS	2012	Bilinear (degree 2)	Small circuits, Groth16	Groth16, Spartan, Nova
AIR	2018	Uniform polynomial	VM traces, STARKs	STARKs (Stone, Stwo)
PLONKish	2019	Selector-gated, custom	Flexible circuits	PLONK, Halo2

A folding scheme designed for R1CS could not accept AIR input. A proof system built for AIR could not handle PLONKish circuits. A developer choosing an arithmetization was simultaneously choosing a proof system ecosystem – and switching later meant rewriting everything.

The fragmentation had real costs. When the Polygon team decided to migrate from Hermez (PLONK-based) to a STARK-based architecture, the circuit rewrite took years. When Scroll built their zkEVM on Halo2 (PLONKish), they could not easily adopt the newer sumcheck-based proof systems that emerged in 2023-2024 without rewriting their entire constraint system. Research teams working on folding schemes had to choose: target R1CS (like Nova did) and exclude the STARK ecosystem, or target a

custom format and exclude everyone else. The constraint system choice was a one-way door – enter through it, and you are locked into the corresponding proof system family for the life of the project.

The field needed a unifier. Not a compromise format that sacrificed efficiency for generality, but a mathematical framework that could express R1CS, AIR, and PLONKish as special cases – preserving the efficiency of each while providing a single target for proof systems to implement.

5.4 CCS: The Rosetta Stone

In 2023, Srinath Setty, Justin Thaler, and Riad Wahby published a paper that changed constraint system design. Customizable Constraint Systems (CCS) unified R1CS, AIR, and PLONKish into a single mathematical framework, without overhead.

The word "without overhead" is the point. Previous attempts at unification existed – for example, you can always convert AIR to R1CS by expanding every transition into individual constraints, or convert R1CS to AIR by padding with identity transitions. But these conversions incur blowup: the converted instance is larger, sometimes much larger, than the original. CCS achieves something different: it captures each constraint format in its native form, preserving the sparsity and structure that makes each format efficient. An R1CS instance becomes a CCS instance of exactly the same size. An AIR instance becomes a CCS instance of exactly the same size. Nothing is wasted in translation.

5.4.1 The Idea

A CCS instance is defined by a set of sparse matrices $M_1, \ldots, M_t$ over a finite field, a collection of multisets $S_1, \ldots, S_q$ (each specifying which matrices to multiply together element-wise), and constants $c_1, \ldots, c_q$. The satisfying condition is:

$\sum_{i=1}^{q} c_i \cdot \bigcirc_{j \in S_i} (M_j \cdot \mathbf{z}) = \mathbf{0}$

(The Hadamard product is simply element-wise multiplication: $[a, b, c] \circ [d, e, f] = [ad, be, cf]$. When the formula says "Hadamard product of $M_j \cdot \mathbf{z}$," it means: compute each matrix-vector product separately, then multiply the resulting vectors element by element.)

This looks abstract. Here is what it means concretely.

R1CS is CCS with two terms. Set $q = 2$, $S_1 = \{1, 2\}$, $S_2 = \{3\}$, $c_1 = 1$, $c_2 = -1$. The satisfying condition becomes $(M_1 \cdot \mathbf{z}) \circ (M_2 \cdot \mathbf{z}) - (M_3 \cdot \mathbf{z}) = 0$, which is exactly the R1CS equation $\mathbf{Az} \circ \mathbf{Bz} = \mathbf{Cz}$ when you identify $M_1 = A, M_2 = B, M_3 = C$.

AIR is CCS with matrices encoding shift relations. The transition constraints between consecutive rows become matrix-vector products with appropriate shift structure.

PLONKish is CCS with matrices encoding selector-weighted gate equations. The selector polynomials become entries in the matrices; the copy constraints map to specific matrix structures.

The key insight is not that CCS enables new computations – any NP statement can already be expressed in R1CS. The insight is that CCS provides a *uniform interface*. A proof system that targets CCS automatically handles R1CS, AIR, and PLONKish inputs without conversion overhead. Write one folding scheme for CCS, and it works with every constraint format the industry has produced.

5.4.2 Why CCS Matters Now

CCS is the native constraint system for every major modern folding scheme:

- **HyperNova** (Kothapalli and Setty, 2023): multi-folding for CCS, using the sumcheck protocol to fold multiple CCS instances simultaneously.
- **ProtoStar** and **ProtoGalaxy** (2023): folding schemes that generalize Nova to higher-degree constraint systems – which CCS naturally supports.
- **Neo** (Nguyen and Setty, 2025): the first lattice-based folding scheme for CCS, achieving post-quantum security with native small-field efficiency.
- **LatticeFold+** (Boneh and Chen, 2025): extends LatticeFold with faster, simpler lattice-based folding and shorter proofs.

Without CCS, none of these systems could claim generality. Each would be locked to R1CS (like Nova) or would need separate implementations for each constraint format. CCS is the abstraction layer that made the folding revolution possible.

The gap between research and deployment is real, however. Production systems in early 2026 still largely use PLONKish (Halo2, Scroll) or AIR (StarkWare, Stwo). The CCS-native stack is approximately two to three years behind the research frontier. But the trajectory is clear: as folding-based proof systems move from research prototypes to production deployments, CCS will become the standard target.

The parallel to programming language history is instructive. In the 1960s, each computer had its own instruction set, its own assembler, and its own operating conventions. Writing a program for an IBM 7090 required completely different code than writing for a UNIVAC 1108. Then came C and UNIX, which provided a common language and a common operating system interface. Programs written in C could run on any machine with a C compiler. CCS plays the same role for constraint systems: it provides a common mathematical interface that any proof system can target. Write your constraints in CCS, and any CCS-compatible proof system – HyperNova, ProtoStar, Neo – can prove them. The

"operating system" for zero-knowledge proof systems is being standardized, even if the "applications" (production deployments) have not yet caught up.

5.4.3 The Degree Parameter

One subtle but important feature of CCS is the degree parameter d, which captures the maximum degree of the constraint polynomials. R1CS has $d = 2$ (bilinear constraints). PLONKish can have $d = 2$ or higher, depending on the custom gate design. CCS handles arbitrary degree without modification.

This matters because higher-degree constraints can capture more complex operations in fewer constraints. A single degree-4 constraint can express relationships that would require multiple degree-2 R1CS constraints. The tradeoff is that higher-degree constraints require more sophisticated proof techniques – but the sumcheck protocol, which we turn to next, handles arbitrary degrees naturally.

To see the degree parameter in action, return to the "x * (x + 1)" computation. In R1CS ($d = 2$), this requires two constraints: $t = x + 1$ (degree 1, but padded to the bilinear form as $(x + 1) \cdot 1 = t$) and result $= x \cdot t$ (degree 2). In a CCS instance with $d = 3$, you could express the entire computation in a single constraint: $x \cdot (x + 1) - \text{result} = 0$, which is a degree-2 polynomial in x. With $d = 4$, you could encode $x \cdot (x + 1) \cdot (x + 2) - \text{result} = 0$ in a single constraint – a relationship that would require three R1CS constraints (one for each pairwise multiplication). Higher degree means more computation packed into fewer constraints, at the cost of more complex proof machinery.

5.4.4 Three Dialects, One Grammar

Return to the three micro-examples we built in the previous sections and look at them through the CCS lens. What CCS reveals is that R1CS, AIR, and PLONKish are not three different formalisms. They are three dialects of the same language.

R1CS is CCS with $q = 2$. You need exactly two multisets: $S_1 = \{1, 2\}$ (which multiplies the results of matrices M_1 and M_2 element-wise) and $S_2 = \{3\}$ (which provides M_3's result). The CCS equation becomes $c_1 \cdot (M_1 \cdot \mathbf{z} \circ M_2 \cdot \mathbf{z}) + c_2 \cdot (M_3 \cdot \mathbf{z}) = 0$, with $c_1 = 1$ and $c_2 = -1$. This is exactly $(\mathbf{A} \cdot \mathbf{z}) \circ (\mathbf{B} \cdot \mathbf{z}) - \mathbf{C} \cdot \mathbf{z} = 0$ – the R1CS equation, expressed in CCS notation. Two matrix-vector products, one Hadamard product, one subtraction. That is the entire constraint system. Every R1CS instance that has ever been deployed – every Groth16 proof, every Spartan verification – is a CCS instance with $q = 2$.

For the "3 * 4 = 12" multiplication from the spreadsheet example, the CCS encoding has $t = 3$ matrices ($M_1 = A$, $M_2 = B$, $M_3 = C$), $q = 2$ multisets ($S_1 = \{1, 2\}$ and $S_2 = \{3\}$), and the witness vector $\mathbf{z} = (1, 3, 4, 12)$ where the first entry is the constant 1. The matrix A selects the left operand (3), B selects the right operand (4), and C selects the output (12). The Hadamard product $(\mathbf{A} \cdot \mathbf{z}) \circ (\mathbf{B} \cdot \mathbf{z})$ computes

$3 \cdot 4 = 12$ element-wise, and subtracting $\mathbf{C} \cdot \mathbf{z} = 12$ yields zero. One constraint, three matrices, one Hadamard product.

AIR is CCS with shift matrices. The counter example from earlier had the transition constraint counter$[i+1]$ = counter$[i]$ + flag$[i]$. In CCS, this becomes a set of matrices where one matrix M_{shift} extracts the "next row" values and another M_{current} extracts the "current row" values. For a 3-row trace, M_{current} might have ones on the diagonal (selecting counter[0], counter[1], counter[2]) while M_{shift} has ones on the superdiagonal (selecting counter[1], counter[2], counter[0] with wraparound). The shift operation – looking at row i + 1 instead of row i – is encoded in the matrix structure itself. The polynomial constraint between consecutive rows becomes a matrix-vector product where the matrix has ones on a shifted diagonal. What looked like a fundamentally different formalism (rules between consecutive rows, rather than rules within a single row) turns out to be a specific matrix pattern within CCS.

PLONKish is CCS with selector-weighted matrices. The PLONKish trace with its q_{add} and q_{mul} columns maps to CCS matrices where the selector values are baked into the matrix entries. The gate equation $q_{\text{add}} \cdot (a+b-c) + q_{\text{mul}} \cdot (a \cdot b - c) = 0$ becomes a CCS instance where one multiset captures the addition term (weighted by q_{add}) and another captures the multiplication term (weighted by q_{mul}). The copy constraints – the permutation argument that wires outputs to inputs – map to additional matrix structure that enforces equality between specific positions in the witness vector.

The visual is this: imagine three spreadsheets, each with different column headers and different rules. The R1CS spreadsheet has columns A, B, C with the rule "A times B equals C." The AIR spreadsheet has columns for registers with the rule "next row relates to current row by this transition polynomial." The PLONKish spreadsheet has witness columns and selector columns with the rule "the active gate constraint must be satisfied." Three different layouts. Three different conventions. But CCS says: they are all just matrices times a witness vector, combined with Hadamard products and summed to zero. One grammar, three dialects.

This is not a metaphor. It is a theorem. Any R1CS instance, any AIR instance, any PLONKish instance can be mechanically translated into a CCS instance with no increase in constraint count or witness size. The translation preserves everything – the structure, the sparsity, the degree. When a proof system like HyperNova targets CCS, it is not accepting a lowest-common-denominator format. It is accepting the universal format that contains every existing constraint dialect as a special case.

The practical consequence is immediate. Before CCS, a developer choosing R1CS was simultaneously choosing Groth16 or Spartan. A developer choosing AIR was choosing STARKs. A developer choosing PLONKish was choosing Halo2 or PLONK. Switching constraint systems meant rewriting the circuit and the proof system integration. CCS breaks this coupling. Write your constraints in whichever dialect

is natural for your computation – R1CS for simple circuits, AIR for VM traces, PLONKish for mixed-gate workloads – and any CCS-compatible proof system will accept them without translation overhead. The grammar is universal; the dialects are a matter of convenience.

There is a deeper mathematical point here, one that Penrose would appreciate. The existence of a universal constraint grammar is not obvious. One might have expected that the structural differences between R1CS (bilinear, flat), AIR (uniform, sequential), and PLONKish (selector-gated, permutation-wired) would require genuinely different proof techniques – that no single algebraic framework could capture all three without paying some conversion tax. CCS demonstrates that the differences are shallow. At the level of sparse matrix-vector products and Hadamard products, all three constraint systems are doing the same thing. The "three families" narrative that dominated ZK from 2018 to 2022 was a historical artifact, not a mathematical necessity.

CCS provides the universal grammar. Sumcheck provides the universal verification engine. The two are partners: CCS tells us *what* the constraints look like – a sum of Hadamard products of matrix-vector pairs – and sumcheck tells us *how to check* that sum without evaluating every term. The verifier does not inspect every cell of the constraint spreadsheet. Instead, sumcheck reduces the problem: "does this multilinear polynomial sum to zero over the boolean hypercube?" becomes, after n rounds of interaction, "does this polynomial evaluate correctly at one random point?" The reduction is exponential – from 2^n checks to n rounds – and it is the reason modern proof systems can verify in time logarithmic in the computation size.

5.5 The Sumcheck Protocol: The Hidden Foundation

If there is one protocol that deserves to be called the backbone of modern zero-knowledge proof systems, it is the sumcheck protocol. Lund, Fortnow, Karloff, and Nisan introduced it in 1992 – decades before practical ZK systems existed. Sumcheck has since become the common thread running through every major proof system of the current era.

5.5.1 What Sumcheck Does

Before stating the problem, one definition. A multilinear polynomial is one where no variable appears with degree higher than one – for example, $g(x_1, x_2) = 3x_1x_2+2x_1+x_2+1$. Each variable is either present or absent in any given term, but never squared or cubed. The sumcheck protocol works naturally with multilinear polynomials because their structure matches the binary hypercube over which the sum is taken: each variable takes the value 0 or 1, so higher powers would collapse anyway (since $0^k = 0$ and

$1^k = 1$). Multilinear polynomials are the native representation for most sumcheck-based proof systems, including Spartan, HyperNova, and Jolt.

The problem sumcheck solves is deceptively simple. You have a multivariate polynomial $g(x_1, \ldots, x_n)$ over a finite field, and you want to verify that its sum over all binary inputs equals a claimed value:

$\sum_{(x_1,\ldots,x_n)\in\{0,1\}^n} g(x_1, \ldots, x_n) = T$

Naively, checking this requires evaluating g at all 2^n binary inputs. For a polynomial over 30 variables, that is a billion evaluations. Sumcheck reduces this to n rounds of interaction (or, via the Fiat-Shamir transform, n rounds of hash-based challenge generation), each involving a single univariate polynomial of low degree.

The protocol is interactive: the prover and verifier exchange messages in rounds. In practice, the Fiat-Shamir transform replaces the verifier's random challenges with hash outputs, making the protocol non-interactive. But the conceptual structure remains round-based.

Here is the intuition. In round 1, the prover sends a univariate polynomial $p_1(x_1)$ that claims to be the sum of g over all remaining variables: $p_1(x_1) = \sum_{x_2,\ldots,x_n} g(x_1, x_2, \ldots, x_n)$. The verifier checks that $p_1(0) + p_1(1) = T$ (this ensures consistency with the claimed total), then sends a random challenge r_1. In round 2, the prover sends $p_2(x_2) = \sum_{x_3,\ldots,x_n} g(r_1, x_2, x_3, \ldots, x_n)$. The verifier checks $p_2(0) + p_2(1) = p_1(r_1)$, then sends another random challenge. This continues until all variables are bound to random values, at which point the verifier checks one evaluation of g at the random point.

The result: verifying a sum over 2^n inputs reduces to checking n low-degree univariate polynomials and one evaluation of g. For $n = 30$, that is 30 polynomial checks instead of a billion evaluations.

To make this concrete, suppose we want to verify that a polynomial $g(x_1, x_2)$ sums to T over all binary inputs. There are four inputs: $g(0,0) + g(0,1) + g(1,0) + g(1,1) = T$. Instead of checking all four, the prover sends a univariate polynomial $p_1(x_1)$ that claims to be the partial sum over x_2. The verifier checks: does $p_1(0) + p_1(1) = T$? If yes, the verifier picks a random r_1 and asks for the next round. Now the prover sends $p_2(x_2)$ claiming to sum $g(r_1, x_2)$. The verifier checks $p_2(0) + p_2(1) = p_1(r_1)$. After two rounds, the verifier holds a single point $g(r_1, r_2)$ that can be checked directly. Two rounds replaced four evaluations. For n variables, n rounds replace 2^n evaluations.

This exponential compression is why sumcheck appears everywhere in modern ZK. The reduction from 2^n evaluations to n rounds is not merely a constant-factor improvement. It is an exponential improvement – the kind that turns impossible problems into trivial ones. For a polynomial over 100 variables, naively verifying the sum would require 2^{100} evaluations (more than the number of atoms in the observable universe). Sumcheck reduces this to 100 rounds. The gap between "impossible" and "trivial" is the gap that sumcheck bridges.

To see how sumcheck serves CCS specifically, consider a CCS instance with a single constraint: $M_1 \cdot \mathbf{z} \circ M_2 \cdot \mathbf{z} = M_3 \cdot \mathbf{z}$ (where $\circ$ is the Hadamard product). The verifier needs to check that this equation holds at every row. Equivalently, the verifier needs to check that the polynomial $h(x) = (M_1 \cdot \mathbf{z})(x) \cdot (M_2 \cdot \mathbf{z})(x) - (M_3 \cdot \mathbf{z})(x)$ sums to zero over all row indices. This is exactly a sumcheck instance. The prover sends the univariate restriction of h in the first variable, the verifier checks its degree and evaluates at a random point, and the process recurses on the remaining variables. After $\log_2(n)$ rounds, the verifier holds a single evaluation claim that can be checked against the committed polynomials. The CCS constraint – which could be R1CS, AIR, or PLONKish in disguise – has been verified without the verifier ever touching the witness.

Spartan uses it for R1CS verification. HyperNova uses it for CCS folding. Jolt and Lasso use it for lookup arguments. SP1 Hypercube builds its entire polynomial stack on sumcheck. When the Ethereum Foundation's zkEVM effort evaluated proof system designs, sumcheck-based architectures won – not because they are simplest to implement, but because the exponential reduction in verifier work is too large to ignore.

5.5.2 Why Sumcheck Is Everywhere

The sumcheck protocol is the verification mechanism that makes polynomial-based arithmetization practical. Every time a modern proof system needs to verify that a polynomial identity holds over a large domain, sumcheck is how it does so.

- **Spartan** (Setty, 2019) uses sumcheck to verify R1CS satisfaction directly, without FFTs. The prover expresses the R1CS check as a multilinear polynomial sum and runs sumcheck to prove it holds.
- **HyperNova** uses sumcheck as the core of its multi-folding protocol. Folding multiple CCS instances reduces to a sumcheck instance.
- **Jolt and Lasso** reduce lookup verification to sumcheck instances. Every table lookup becomes a polynomial sum that sumcheck can verify.
- **LogUp-GKR** combines the sumcheck protocol with the GKR interactive proof to verify lookup arguments with logarithmic overhead.
- **SP1 Hypercube** (Succinct, 2025) uses sumcheck over the Boolean hypercube as its primary verification strategy.
- **Binius** (Irreducible, 2025) applies sumcheck over binary tower fields.

The sumcheck protocol is to modern ZK proof systems what the internal combustion engine was to early automobiles: the mechanism that makes the entire apparatus work, even though the user never sees it directly. Understanding that sumcheck exists, and that it reduces exponential verification to

linear communication, is essential for understanding why the overhead of arithmetization is not as catastrophic as it might first appear.

A note on presentation order: this chapter covers arithmetization (Layer 4) before the proof system (Layer 5) and cryptographic primitives (Layer 6). But in practice, the causal arrow often runs the other way. The sumcheck protocol is a proof technique that shaped which arithmetization formats became practical. The field choice at Layer 6 determines which polynomial representations are efficient at Layer 4. These three layers – field, commitment scheme, polynomial representation – form an inseparable "proof core." We present them in the standard order, but the reader should understand that the dependency is circular, not linear.

The sumcheck protocol also illustrates a recurring theme in this book: the most important technical ideas are often invisible to the end user. A developer writing a Compact smart contract on Midnight, or a Solidity developer deploying a Groth16 verifier on Ethereum, will never interact with the sumcheck protocol directly. They will never see a multilinear polynomial or check a partial sum. But sumcheck is running underneath, silently reducing the verification cost from exponential to linear, making the entire stack practical. The seven layers of the magic trick include mechanisms that the audience never sees – and sumcheck is the most consequential of them all.

5.6 Lookup Arguments

In the classical approach to arithmetization, every operation in a computation is encoded as polynomial constraints. Addition and multiplication are natural: they are already arithmetic operations over the field. But what about operations that are *not* naturally arithmetic?

Consider the problem from the perspective of a circuit designer building a zkVM. The RISC-V instruction set contains 47 base instructions. Of these, roughly half are "arithmetic-friendly" – ADD, SUB, MUL, and their variants map naturally to field operations. But the other half are "arithmetic-hostile": AND, OR, XOR, SLL (shift left logical), SRL (shift right logical), SLT (set less than), BEQ (branch if equal), and the memory load/store operations. Each of these requires bitwise decomposition or comparison logic that does not map neatly to field arithmetic.

Bitwise AND, comparison, range checks, hash functions – these require decomposing the values into individual bits, constraining each bit to be 0 or 1, and then reconstructing the result. A single SHA-256 hash invocation can require tens of thousands of constraints.

The cost is staggering when you trace through a concrete example. Consider XOR – the bitwise exclusive-or of two 8-bit numbers. On a CPU, this is one instruction, one clock cycle, done. In a polynomial

constraint system, you must first decompose each 8-bit input into 8 individual bits (8 range-check constraints per input, 16 total), then constrain each output bit to be the XOR of the corresponding input bits (each bit-level XOR requires the polynomial $a + b - 2ab$, which is a degree-2 constraint, so 8 more constraints), and finally reconstruct the output from its bits (8 more constraints). That is roughly 32 constraints for an operation that takes a single machine cycle. SHA-256 calls XOR, AND, NOT, and rotation thousands of times. Multiply 32 constraints per operation by thousands of operations and you arrive at the tens of thousands of constraints that a single hash invocation demands.

This is not a fixable inefficiency in the constraint system design. It is a fundamental mismatch between the polynomial language (which speaks addition and multiplication over large fields) and the bitwise language (which speaks AND, OR, XOR over individual bits). No amount of cleverness in the constraint layout will make polynomials natively express bit manipulation. The operations live in different algebraic worlds.

The mismatch created a two-tier cost structure in early ZK systems. "Arithmetic-friendly" operations (Poseidon hash, MiMC, elliptic curve arithmetic) – operations designed from the ground up to minimize constraint count – were cheap. "Arithmetic-hostile" operations (SHA-256, Keccak, AES, bitwise logic) – operations from the traditional computing world – were expensive by comparison. This is why the ZK community designed entirely new hash functions (Poseidon, Rescue, Neptune) that use only field additions and multiplications, avoiding bitwise operations entirely. A Poseidon hash costs roughly 300 constraints in R1CS. A SHA-256 hash costs roughly 25,000 constraints. Same security level. Hundred-fold difference in proving cost. The constraint system penalizes any computation that crosses the boundary between field arithmetic and bit arithmetic.

Lookup arguments offer a fundamentally different approach: instead of encoding the operation as constraints, you look up the answer in a pre-computed table. Instead of proving *how* you computed XOR, you prove *that* your answer appears in a table of all correct XOR results. The philosophical shift is from verification-by-recomputation to verification-by-membership.

5.6.1 Plookup: The First Practical Lookup (2020)

Gabizon and Williamson introduced Plookup in 2020. The idea: if you have a table of pre-approved input-output pairs (for example, all possible 8-bit XOR results), you can prove that a set of values appears in the table without recomputing the operation.

Consider that table of 8-bit XOR results. It has 256 * 256 = 65,536 entries, each of the form (a, b, a XOR b). If the prover claims that 0x3F XOR 0xA7 = 0x98, the verifier does not check the XOR. Instead, the verifier checks that the triple (0x3F, 0xA7, 0x98) appears somewhere in the table. If it does, the answer

is correct – because the table was constructed correctly, and membership in a correct table implies correctness of the result.

Plookup works by sorting the lookup values and the table entries into a single sorted sequence, then verifying the sorting through a grand product argument. If every lookup value appears in the table, the sorted sequence has a specific structure that the grand product captures. The prover merges the table and the lookup values into one sorted list, then proves that the merged list is a valid interleaving of the original table with the queried entries. A polynomial identity – checked via a grand product over the entire sorted sequence – catches any entry that does not belong.

The catch: sorting costs $O(n \log n)$, and the grand product requires committing to the sorted sequence. For a circuit with n lookups into a table of size T, the prover must commit to a sorted list of length $n + T$. This makes Plookup a meaningful optimization for expensive operations (hashes, range checks) but not a universal solution. The sorting overhead is the bottleneck, and it resists parallelization – you cannot sort half a list on one machine and half on another without a merge step.

Despite its limitations, Plookup was immediately adopted. Lookup arguments for range checks (proving a value is between 0 and 2^N) replaced the naive approach of decomposing into N bits and constraining each one. For a 16-bit range check, the naive approach requires 16 boolean constraints plus a reconstruction constraint (17 total). A Plookup-based range check requires one lookup into a table of 65,536 entries. The lookup is more expensive in absolute prover time (sorting overhead), but far cheaper in constraint count – and for systems where constraint count is the bottleneck, this tradeoff is worthwhile. By 2021, every major PLONKish system used lookup arguments for range checks.

5.6.2 LogUp: The Sorting-Free Revolution (2022)

Ulrich Haboeck's LogUp paper in 2022 replaced Plookup's sorting with an observation that is, in retrospect, elegant to the point of inevitability. The name "LogUp" comes from "logarithmic derivative" – the key mathematical technique. If you have a polynomial P(X) = Product of (X - r_i) whose roots are exactly the lookup values, then the logarithmic derivative of P is P'(X)/P(X) = Sum of 1/(X - r_i). This transforms a product (which is hard to check incrementally) into a sum (which is easy to accumulate and verify). The idea comes from complex analysis, where logarithmic derivatives convert multiplicative structures into additive ones. Haboeck's insight was to apply this classical technique to the lookup problem.

Instead of sorting, LogUp observes that if every lookup value f_i appears in the table t, then a specific identity over rational functions must hold:

$\sum_i \frac{1}{X-f_i} = \sum_j \frac{m_j}{X-t_j}$

where m_j counts how many times table entry t_j is looked up. The left side sums one term per lookup. The right side sums one term per table entry, weighted by the number of times it was accessed. If every lookup value is in the table, these two sums are equal as formal rational functions – and therefore they are equal at a random evaluation point with overwhelming probability.

A tiny example makes this concrete. Suppose the table contains {1, 2, 3} and the prover claims to look up the values {2, 3, 2}. The left side (one term per lookup) is: $1/(X-2) + 1/(X-3) + 1/(X-2) = 2/(X-2) + 1/(X-3)$. The right side (one term per table entry, weighted by frequency) is: $0/(X-1) + 2/(X-2) + 1/(X-3)$ – because entry 1 is looked up 0 times, entry 2 is looked up twice, and entry 3 is looked up once. Both sides equal $2/(X-2) + 1/(X-3)$. The identity holds. Now suppose the prover cheats and claims to look up {2, 3, 5}, where 5 is not in the table. The left side becomes $1/(X-2)+1/(X-3)+1/(X-5)$. No assignment of multiplicities to the table entries {1, 2, 3} can produce a term $1/(X-5)$ on the right side. The identity fails at a random evaluation point with overwhelming probability.

This transforms the lookup argument from a product check to a *sum check* – and sums, unlike products, compose beautifully. Why does this matter? Because a product of n terms can be thrown off by a single corrupted factor (the product becomes wrong, but localizing the error requires inspecting every factor). A sum of n terms, by contrast, is naturally decomposable: you can split the sum into batches, compute partial sums independently, and aggregate them. The algebraic structure of addition is friendlier than the algebraic structure of multiplication.

The advantages are substantial:

- **No sorting required.** Eliminates the $O(n \log n)$ overhead entirely. The prover's cost drops to $O(n+T)$, linear in the number of lookups plus the table size.

- **Natural batching.** Multiple lookup tables can be handled simultaneously via random linear combinations. If your circuit uses a XOR table, an AND table, and a range-check table, LogUp handles all three in one pass.
- **Parallelizable.** The summation structure means partial lookups can be computed independently on separate machines and aggregated with a single addition. This is exactly the property that GPU-based provers exploit.
- **Sumcheck-compatible.** The rational function identity can be verified using the sumcheck protocol, connecting lookups directly to the same verification backbone that handles constraint checking.

LogUp became the production standard. It replaced Plookup in deployed systems and enabled the next generation of lookup-based architectures. The transition was rapid: by 2024, virtually every new proof system design used LogUp or a LogUp variant for its lookup needs.

5.6.3 LogUp-GKR: The Verifier Gets Faster (2023)

LogUp made the prover efficient. But the verifier still had to check the rational function identity, which naively requires work proportional to the number of lookups. Papini and Haboeck combined LogUp with the GKR interactive proof protocol to solve this.

The GKR protocol provides an efficient way to verify layered arithmetic circuits – circuits where the computation flows through layers, each layer depending only on the one before it. The fractional sum computation in LogUp (adding up all the 1/(X - f_i) terms) has exactly this layered structure: it is a sum reduction tree. LogUp-GKR applies the GKR protocol to this tree, reducing the verifier's work from linear to logarithmic in the number of lookups.

The result: logarithmic proof size and verification time for lookup arguments. The verifier does $O(\log n)$ work regardless of how many lookups the prover performed.

LogUp-GKR is now used in Polygon's Plonky3 framework and in SP1 Hypercube. It makes lookups nearly free for the verifier, which is critical for recursive proof composition – where the verifier's circuit size directly affects the cost of the next recursion step. If verifying a lookup takes $O(n)$ work, then a recursive verifier circuit must be $O(n)$ in size, which is expensive to prove in the next recursion layer. With LogUp-GKR, the recursive verifier circuit is $O(\log n)$, making deep recursion practical.

The combination of LogUp (efficient prover) and GKR (efficient verifier) made lookups a first-class operation in the proof system – no longer an optimization to be applied selectively, but a general-purpose tool to be used wherever a pre-computed table exists. This set the stage for the two results that completed

the lookup revolution: Lasso, which made table size irrelevant, and Jolt, which made lookups the only computation paradigm needed.

5.6.4 Lasso: The Table Size Disappears (2023)

The fundamental limitation of both Plookup and LogUp is that the prover must somehow touch the entire table. For a table of 2^{16} entries (65,536 rows), this is manageable. For a table of all possible 64-bit operations – 2^{128} entries – it is physically impossible to even store the table, let alone commit to it. The table of all 64-bit additions alone has 2^{128} rows. At one byte per row, that is 10^{38} bytes – more than the number of atoms in the observable universe. No amount of hardware solves this.

Lasso, by Setty, Thaler, and Wahby (2023), solves this through *decomposition*. The insight is that most useful tables have internal structure that can be exploited. Specifically, if the table's multilinear extension (MLE) can be evaluated efficiently – meaning you can compute the table's value at any point without materializing the entire table – then each lookup can be decomposed into lookups into much smaller subtables.

The intuition is best seen through an analogy. Suppose you have a multiplication table for two-digit numbers. Instead of storing all 90 * 90 = 8,100 entries (for digits 10-99), you could decompose each two-digit number into its tens and units digits, store separate multiplication tables for single digits (only 10 * 10 = 100 entries each), and reconstruct the full product from partial products. The full table has 8,100 entries; the subtables have a combined 200 entries. You traded one large lookup for several small lookups plus some arithmetic glue. Lasso does exactly this, but for arbitrary structured tables over finite fields, using the multilinear extension as the decomposition mechanism.

For a table of size 2^{2W}, Lasso decomposes each lookup index into c chunks. Instead of one lookup into a table of size 2^{2W}, the prover performs c lookups into subtables of size $2^{2W/c}$. For 64-bit RISC-V operations with $c = 6$, each subtable has roughly 2^{22} entries – about 4 million rows. That fits comfortably in memory.

The prover's cost becomes $O(n \cdot c \cdot \log(N)/c)$, which is proportional to the number of lookups (n) and the number of chunks (c), but *independent of the table size* (N). You can look up values in a table of 2^{128} entries without ever materializing the table. The table exists as a mathematical function – its MLE – not as a stored data structure. The prover only pays for the subtable entries it actually accesses.

This is the kind of result that reshapes what is considered possible. Before Lasso, "table size" was a hard constraint on lookup arguments. After Lasso, table size is irrelevant – only table structure matters. A structured table of 2^{128} entries is no harder to use than a structured table of 2^{16} entries. The prover's

work scales with the number of lookups it performs, not with the number of entries it could theoretically look up.

5.6.5 Jolt: The Lookup Singularity Realized (2023)

Arun, Setty, and Thaler's Jolt paper took Lasso's theoretical framework and applied it to its logical conclusion: what if *every* instruction in a processor were a lookup?

The concept, originally proposed by Barry Whitehat as the "lookup singularity," posits that circuits should be expressed entirely as lookups into pre-computed tables. Jolt demonstrates this is achievable for a complete RISC-V instruction set:

1. Every RISC-V instruction has an evaluation table mapping inputs to outputs.
2. All instruction tables are MLE-structured (their multilinear extensions can be evaluated efficiently).
3. Lasso's decomposition makes these lookups efficient regardless of the theoretical table size.
4. Memory consistency is verified through offline memory checking (fingerprint-based techniques), not Merkle trees.

For each instruction, the prover decomposes the operands into chunks, performs lookups into small subtables (typically around 4 million entries each), and commits to roughly 18 field elements per instruction (3 per chunk, with $c = 6$ chunks).

The memory-checking component (point 4) is worth highlighting separately. In a real processor, memory is read-write: the program loads and stores values freely. Proving that every load returns the value of the most recent store to the same address is the memory consistency problem discussed in the overhead section. Jolt handles this through "offline memory checking" – a technique where the prover computes a cryptographic fingerprint of the sequence of all reads and writes, and the verifier checks that the fingerprint is consistent with a valid read-write memory. This avoids the per-access cost of Merkle tree proofs and makes memory checking nearly as cheap as instruction checking. The technique is not specific to Jolt; it was developed by Blum et al. in the 1990s and adapted for ZK by Setty (Spartan) and others. But Jolt's integration of offline memory checking with Lasso-based instruction lookups produces a complete zkVM architecture where every component – instruction verification, memory consistency, program counter management – is handled by either a lookup or a fingerprint check.

The result is a zkVM where the constraint system is almost entirely lookups, with minimal arithmetic "glue." This is not an optimization applied to an existing constraint system – it is a different paradigm for encoding computation.

Pause and consider what Jolt achieved. A complete RISC-V instruction set – ADD, SUB, AND, OR, XOR, SLL, SRL, SRA, SLT, SLTU, BEQ, BNE, BLT, BGE, LW, SW, and dozens more – expressed without writing a single arithmetic constraint by hand. Every instruction is a table lookup. The "constraint system" is a collection of tables plus the Lasso machinery to prove that every instruction's result appears in the correct table. No custom gates. No selector polynomials. No hand-optimized constraint layouts. Just tables.

To see how this works for a specific instruction, trace through a 32-bit ADD. The prover needs to prove that register_a + register_b = register_c. The "addition table" for 32-bit inputs has 2^{64} entries – impossibly large to store. But Lasso decomposes each 32-bit operand into $c = 4$ chunks of 8 bits each. Each chunk lookup goes into a subtable of size $2^{16} = 65{,}536$ entries (all possible 8-bit additions, accounting for carry). The prover performs 4 small lookups instead of one impossible lookup, commits to the chunk values and the carry bits, and uses the Lasso sumcheck machinery to prove that the chunks reconstruct the full addition correctly.

Compare this to how a traditional zkVM would prove the same 32-bit ADD. In RISC Zero's earlier architecture, the prover would encode the addition as a polynomial constraint over the full 32-bit values, with range checks to ensure the operands fit in 32 bits (costing roughly 32 constraints for bit decomposition per operand), a constraint for the addition itself, and further constraints for carry propagation and overflow detection. Roughly 70 to 100 constraints per ADD instruction. In Jolt, the same instruction costs approximately 18 field element commitments (3 per chunk, 6 chunks for 64-bit RISC-V) and a handful of sumcheck rounds. The constraint count per instruction drops by roughly 4x.

This is genuinely surprising. For years, the ZK community assumed that building a practical zkVM required painstaking constraint engineering – hand-crafting gate designs for each instruction type, optimizing selector layouts, minimizing constraint counts through algebraic tricks. Jolt demonstrates that all of that complexity can be replaced by a single, uniform mechanism: look up the answer. The engineering effort shifts from "design clever constraints" to "design decomposable tables," and the latter turns out to be systematically easier.

5.6.6 The Genealogy in Full

The progression from auxiliary technique to primary computation paradigm took just three years:

System	Year	Technique	Sorting?	Table Size Limit	Key Innovation
Plookup	2020	Grand product	Yes ($O(n \log n)$)	Fixed, materializable	First practical lookup
LogUp	2022	Logarithmic derivatives	No	Fixed, materializable	Sum-based, parallelizable
LogUp-GKR	2023	LogUp + GKR	No	Fixed, materializable	Logarithmic verifier
Lasso	2023	Decomposition	No	Unlimited (structured)	Table-size independent
Jolt	2023	Lasso for full ISA	No	Unlimited (structured)	Lookup singularity realized

The three-year progression is worth pausing on. In 2020, lookups were an auxiliary optimization – useful for range checks and hash functions, but secondary to the main constraint system. By 2023, lookups had become a complete computation paradigm – capable of replacing the constraint system entirely for general-purpose ISA execution. Nothing else in ZK moved this fast.

The industry has not yet fully absorbed this shift. Jolt is in alpha (open-sourced by a16z), with key missing features including full recursion support and GPU-accelerated proving. Production systems in 2026 still largely use LogUp or LogUp-GKR for specific operations (range checks, hash functions) while relying on polynomial constraints for the core computation. But the trajectory is clear: lookups are moving from a useful optimization to the primary computation paradigm.

One qualification is important. The lookup singularity works well for ISA-level computation – adding two registers, comparing values, shifting bits. For application-specific circuits with rich algebraic structure (elliptic curve operations, pairing computations), direct polynomial constraints remain more efficient. The lookup approach excels for *general-purpose* computation; specialized circuits may still prefer custom constraints.

The lesson of the lookup revolution is about the relationship between computation and verification. The classical approach to ZK asks: "How do I re-express this computation as polynomial constraints?" The lookup approach asks a different question: "How do I prove that the answer is correct, without re-expressing the computation at all?" The table is a certificate of correctness. If the answer is in the table, the answer is correct. The proof reduces to a membership test. This conceptual shift – from "prove the

computation" to "prove membership in a table of correct answers" – may be the most consequential shift in arithmetization since CCS unified the constraint systems. It is also, notably, the idea that connects most directly to the sumcheck protocol: LogUp's rational function identity is verified by sumcheck, Lasso's decomposition is verified by sumcheck, and Jolt's per-instruction correctness reduces to sumcheck instances. Lookups and sumcheck are two sides of the same coin.

Three advances – CCS, sumcheck, and lookups – form a complete verification stack. CCS provides the universal constraint format: any polynomial relation, any degree, any structure, expressed as sums of Hadamard products. Sumcheck provides the universal verification engine: any polynomial sum over an exponential domain, reduced to a single-point check in logarithmic rounds. Lookups replace the most expensive constraint-by-constraint encoding with table references: instead of proving that a value satisfies a complex relation through dozens of polynomial constraints, the prover demonstrates that the value appears in a precomputed table. Together, they answer a question that was open as recently as 2022: can we build a proof system that handles any constraint format, verifies in near-linear time, and avoids the worst overhead of hand-crafted constraint engineering? By 2024, the answer was yes – and the combination of CCS + sumcheck + lookups is the engine inside every frontier proof system built since.

5.7 The Overhead Tax: 10,000x to 50,000x

We have spent this chapter describing how computation is encoded as mathematics. Now we must confront the cost.

A computation that runs natively in 1 millisecond – executing instructions directly on a processor at billions of operations per second – takes 10 to 50 seconds to prove in a zkVM. That is an overhead of 10,000x to 50,000x. Where does this multiplier come from?

The answer is not a single bottleneck but three interlocking sources of overhead that multiply together. Each source has its own physics, its own improvement trajectory, and its own fundamental limits. Understanding the decomposition is essential because it determines which engineering improvements will matter most for which applications.

5.7.1 Source 1: Field Arithmetic Encoding

Native computation uses 32-bit or 64-bit integers with hardware-accelerated arithmetic. A single addition or multiplication takes one CPU cycle – roughly 0.3 nanoseconds on a modern processor.

ZK computation uses finite field arithmetic. If the proof system requires a 254-bit prime field (as BLS12-381 and BN254 do), every "addition" becomes multi-precision arithmetic over four 64-bit machine words. Each field multiplication requires a Barrett or Montgomery reduction. The per-operation cost is roughly 10 to 100 times higher than native integer arithmetic, depending on the field size.

The small-field revolution – BabyBear (31-bit), Mersenne-31 (31-bit), Goldilocks (64-bit) – addresses this directly. A 31-bit field element fits in a single 32-bit register; a 64-bit Goldilocks element fits in a single machine word. Arithmetic in these fields runs 10x to 100x faster than in 254-bit fields. But even in the smallest fields, the overhead of field arithmetic versus native integer operations remains significant.

The Mersenne-31 field is particularly elegant. Its modulus is $2^{31} - 1$, which is a Mersenne prime. Reduction modulo $2^{31} - 1$ is exceptionally fast: after computing the product of two 31-bit numbers (yielding a 62-bit result), you split the result into a high part and a low part at bit position 31, and add them together. If the sum exceeds $2^{31} - 1$, subtract once. Two shifts and two additions – faster than any other prime modular reduction. This is why SP1 and Stwo chose Mersenne-31 as their base field: the per-operation cost is nearly as fast as native 32-bit integer arithmetic, closing the gap between "native" and "field" computation to a factor of roughly 3x to 5x.

5.7.2 Source 2: Constraint Expansion

A single native instruction (say, a 64-bit addition) becomes multiple constraints in the arithmetized form. The addition itself is one constraint, but proving that the operands are within the correct range (range checks), that the memory was read correctly (memory consistency), and that the instruction was selected properly (opcode decoding) can require dozens to hundreds of additional constraints. This multiplicative blowup is the most counter-intuitive aspect of arithmetization: the "interesting" computation (the actual addition) accounts for a tiny fraction of the total constraint count. The vast majority of constraints are devoted to proving that the computational environment is correctly maintained – that registers hold the right values, that memory is consistent, that the instruction pointer advanced correctly.

To make this tangible, here is a rough breakdown of the constraints required to prove a single ADD instruction in a typical zkVM (based on published analyses of RISC Zero and SP1 architectures):

- **Instruction decode**: the prover must prove that the opcode field of the instruction word equals the ADD opcode. This requires extracting bit fields and constraining them – roughly 5 to 10 constraints.
- **Register read**: the prover must prove it correctly read the values of the source registers (rs1 and rs2) from the register file. Memory consistency for each read requires 3 to 5 constraints (depending on the memory-checking technique).
- **Arithmetic operation**: the actual addition is 1 constraint.
- **Overflow handling**: the result must be reduced modulo 2^{32} (for 32-bit RISC-V). This requires proving that the result fits in 32 bits – a range check costing 32 constraints (one per bit) in naive approaches, or fewer with lookup-based range checks.
- **Register write**: the prover must prove it correctly wrote the result to the destination register (rd). Another 3 to 5 memory consistency constraints.
- **Program counter update**: the prover must prove that the PC advanced by 4 (the size of one instruction). 2 to 3 constraints.

Total: roughly 50 to 80 constraints for a single ADD instruction that, on a real CPU, takes one clock cycle. The arithmetic itself (the actual addition) accounts for exactly 1 of those constraints. The other 49 to 79 are bookkeeping: proving that the right values were read from the right places, that the result was stored correctly, and that the instruction was decoded properly. This bookkeeping overhead is the dominant cost in constraint expansion.

Memory consistency is particularly expensive. In a native processor, reading from memory is a single operation – the cache or main memory returns the value, and the hardware guarantees it is the value that

was most recently written to that address. The CPU does not need to *prove* this; the hardware enforces it physically. In a constraint system, there is no hardware. The prover claims "I read value 42 from address 0x1000," and the verifier has no way to check this claim without a mathematical argument.

Classical approaches use Merkle tree hashing – roughly 300 multiplication constraints per Poseidon hash invocation – to authenticate every memory access. The idea: maintain a Merkle tree over the entire memory state. Before each read, prove that the value at the target address is consistent with the Merkle root. After each write, update the Merkle tree and prove the new root is correct. Each access requires a Merkle proof (log-depth hash chain), and each hash costs hundreds of constraints. For a program with millions of memory accesses, this becomes the dominant cost.

Ozdemir and others have demonstrated algebraic approaches that reduce memory checking costs by 50x to 150x by replacing Merkle proofs with "offline memory checking" – a technique where the prover accumulates a fingerprint of all reads and writes, and the verifier checks that the fingerprint is consistent at the end. No per-access Merkle proofs, just a global consistency check. This is the approach used by Jolt and by SP1's latest architecture. But even the improved methods add substantial overhead compared to native memory access, which costs exactly zero proof constraints.

5.7.3 Source 3: Polynomial Commitment

After the constraints are constructed, the prover must commit to the polynomial representations and prove their evaluations. The commitment step is where the "zero-knowledge" part happens: the prover seals the polynomials into cryptographic commitments that reveal nothing about the underlying values, then proves properties of the committed polynomials without opening them. This is also, by far, the most computationally expensive step.

The commitment process typically involves Number Theoretic Transforms (NTTs) – the finite-field analog of the Fast Fourier Transform – which can consume up to 90% of GPU proving time. Multi-scalar multiplications (MSMs) for group-based commitments add further cost. For FRI-based systems (used with STARKs), the commitment involves building Merkle trees over polynomial evaluations and performing multiple rounds of degree-halving with random challenges. For KZG-based systems (used with PLONK and Groth16), the commitment involves computing elliptic curve group operations – MSMs of size proportional to the polynomial degree.

The three sources multiply. If field encoding costs 10x, constraint expansion costs 50x, and polynomial commitment costs 10x, the total overhead is not 70× but 5,000×. This multiplicative composition is why the overhead is so large and why improvements in any single source yield modest overall gains. Reducing field encoding overhead by 10x (moving from 254-bit to 31-bit fields) and reducing constraint

expansion by 2x (using lookup-based approaches) yields a combined 20x improvement – meaningful, but still leaving a 250x overhead. All three sources must be attacked simultaneously.

5.7.4 Is the Overhead Fundamental?

No. Every source of overhead is under active attack by engineering and mathematical innovation:

- **Field size**: The shift from 254-bit to 31-bit fields reduced per-operation cost by roughly 100x. A 31-bit field multiplication is a single machine word multiply followed by a modular reduction – roughly 2 to 3 clock cycles versus the 50 to 100 cycles required for a 254-bit Montgomery multiplication.
- **Memory checking**: Algebraic memory checking (Ozdemir et al.) reduces overhead by 50-150x versus Merkle-based approaches. Instead of hashing at every memory access, algebraic techniques use offline fingerprinting – accumulating a running product that can be checked at the end of the execution.
- **Bit-level encoding**: Binius (Irreducible, 2025) reduces embedding overhead by 100x for bit-heavy workloads by working directly over binary tower fields, where a single bit *is* a field element (no embedding required).
- **Hardware acceleration**: GPU-based provers (BatchZK, ZKProphet) achieve throughput improvements of 10x to 100x through massive parallelism in NTT and MSM computations.
- **Lookup-based architectures**: Jolt eliminates many constraint expansion costs by replacing polynomial constraints with table lookups. The 50-80 constraints per instruction in a traditional zkVM drop to roughly 18 field element commitments per instruction.
- **Folding schemes**: Nova, HyperNova, and Neo amortize the cost of proving many similar statements by "folding" them into a single accumulated instance. Instead of proving each step independently, the prover maintains a running accumulation that grows by a constant amount per step.

The cumulative effect is multiplicative. If small fields give 100x, algebraic memory checking gives 50x, and GPU acceleration gives 10x, the combined improvement is not 160x but potentially 50,000x – enough to close much of the gap between native and proven computation. The catch is that these improvements compound only if they apply to the same bottleneck; in practice, eliminating one bottleneck exposes the next. But the engineering trajectory points down.

5.7.5 What the Overhead Feels Like in Practice

Abstract multipliers are hard to internalize. Here are three concrete examples that reveal the texture of the overhead tax.

The single addition. A function that adds two 64-bit integers takes approximately 1 nanosecond on a modern CPU – one clock cycle, one instruction, done. The same addition, proven in zero knowledge, requires: encoding the addition as a field operation (the 64-bit integer must be represented as one or more field elements, with range checks to prove it fits in 64 bits), committing to the input and output values (a polynomial commitment involving a multi-scalar multiplication or hash-based Merkle path), and generating a proof that the commitment is consistent with the constraint (running the full SNARK or STARK prover pipeline). Total time: 10 to 100 microseconds, depending on the proof system. Overhead: 10,000x to 100,000x. A single addition – the simplest possible computation – pays the full fixed cost of the proof machinery.

The Ethereum block. An Ethereum block execution takes roughly 100 milliseconds of native computation: verifying signatures, executing smart contract bytecode, updating the state trie. Proving the same block in a zkEVM takes 6 to 35 seconds on GPU clusters (as reported by SP1, RISC Zero, and Succinct in 2025 benchmarks). Overhead: 60x to 350x. This is far less than the theoretical 10,000x to 50,000x because GPU parallelism and algorithmic optimizations have eaten most of the overhead for large computations. The NTTs and MSMs that dominate prover time are embarrassingly parallel – they decompose into millions of independent operations that map naturally onto GPU architectures with thousands of cores.

The Midnight transaction. A Midnight shielded transfer involves a Compact smart contract that reads and updates token balances behind zero-knowledge proofs. The native computation – checking a balance, subtracting from one account, adding to another – would take a few microseconds in any programming language. Proving the same transaction in Midnight's ZK pipeline takes approximately 20 seconds: the Compact compiler produces ZKIR instructions, the backend lowers them to PLONKish constraints over BLS12-381, and the Halo2-style prover generates the proof. Overhead: roughly 1,000,000x if measured against the pure arithmetic of balance updates. But the comparison is misleading, because the proof is doing far more than the arithmetic – it is proving that the balance update is consistent with the entire ledger state, that the sender has sufficient funds, that the nullifier has not been previously spent, and that the cryptographic commitments are correctly formed. The "computation" being proved is not the balance update itself but the entire integrity argument surrounding it.

The gap between the three. A single addition suffers 10,000x to 100,000x overhead. An Ethereum block suffers 60x to 350x. A Midnight transaction suffers a nominal 1,000,000x on the pure arithmetic but a

more reasonable 1,000x when measured against the full security computation it replaces. The difference is not a measurement error. It reflects a fundamental asymmetry in the cost structure: zero-knowledge proving has large fixed costs (setting up the polynomial commitment, running the Fiat-Shamir transcript, computing the proof) and relatively small marginal costs per additional constraint. A single addition amortizes those fixed costs over one operation. An Ethereum block – with millions of constraints – amortizes them over millions. The per-constraint overhead might be identical, but the ratio of total proving time to native execution time drops as the computation grows.

A table makes the pattern visible:

Computation	Native time	Proof time	Overhead	Why
Single 64-bit addition	~1 ns	10-100 us	10,000-100,000x	Fixed costs dominate
SHA-256 hash (one block)	~300 ns	1-10 ms	3,000-30,000x	Constraint expansion for bitwise ops
Ethereum block execution	~100 ms	6-35 s	60-350x	GPU parallelism amortizes fixed costs

Computation	Native time	Proof time	Overhead	Why
Midnight shielded transfer	~5 us (arithmetic)	~20 s	~4,000,000x (arithmetic)	Large cryptographic circuit, BLS12-381 field

The key insight: overhead is not uniform. Small computations suffer disproportionately. Large computations amortize the fixed costs. Computations over large fields (BLS12-381 at 254 bits) pay more per operation than those over small fields (BabyBear at 31 bits). And computations that are inherently bitwise (hashes, comparisons) pay more than those that are inherently arithmetic (field operations, polynomial evaluations).

This is why zkVMs are viable for block-level proving (where the overhead is 100x to 500x, manageable with GPU clusters) but impractical for individual function calls (where the overhead is 10,000x to 100,000x, making a 1-microsecond function take 100 milliseconds to prove). The economics of zero-knowledge computation favor batching – proving large computations in bulk rather than small computations one at a time.

Where does the time actually go for a block-level proof? In a typical GPU-based zkEVM prover (SP1, RISC Zero, or similar systems as measured in 2025 benchmarks), the breakdown looks roughly like this:

- **NTT (Number Theoretic Transform):** 40-60% of total proving time. The finite-field analog of the FFT, used to convert polynomials between coefficient and evaluation representations. These are the workhorses of polynomial commitment. An NTT of size 2^{24} involves roughly $24 \cdot 2^{24} = 400$ million field multiplications – each of which, even in a fast 31-bit field, takes a few nanoseconds.
- **Polynomial commitment (MSM or hash-based):** 15-30% of total proving time. For KZG-based systems, multi-scalar multiplications over elliptic curves. For FRI-based systems, Merkle tree construction over hash evaluations. FRI commitments require hashing the polynomial evaluations into a Merkle tree, then performing multiple rounds of folding (each requiring NTTs of decreasing size) and opening consistency proofs.

- **Witness generation and constraint evaluation:** 10-20% of total proving time. Filling in the execution trace and checking that all constraints are satisfied. This is the "spreadsheet" work: computing the values for every cell and verifying that every rule holds.
- **Memory and communication overhead:** 5-15%. Moving data between CPU and GPU, allocating buffers, serializing proof elements. For large proofs, the witness can be several gigabytes, and transferring it across the PCIe bus takes non-trivial time.

The dominance of NTT explains why GPU parallelism helps so much: NTTs decompose into independent butterfly operations that map directly onto GPU warp-level parallelism. A single NVIDIA A100 GPU can perform NTTs over 2^{24} field elements in under 100 milliseconds – a task that would take several seconds on a CPU. The algorithmic improvements from 2024 to 2026 (circle STARKs, WHIR, lattice-based schemes that avoid NTTs entirely) are systematically attacking this bottleneck.

One emerging approach avoids NTTs entirely. Sumcheck-based proof systems (like Spartan, HyperNova, and Jolt) work with multilinear polynomials over the Boolean hypercube rather than univariate polynomials over multiplicative subgroups. Multilinear polynomials do not require NTTs for evaluation or commitment. Instead, the prover performs structured summations – which decompose into independent, parallel operations without the butterfly dependency pattern of NTTs. This is why the sumcheck-based architectures are gaining ground: they eliminate the NTT bottleneck entirely, replacing it with a computation pattern that is even more GPU-friendly.

This asymmetry also explains why recursive proof composition matters so much. If you can batch thousands of small proofs into one large proof, and then prove the large proof recursively, you move the computation into the regime where amortization works in your favor. The fixed costs are paid once; the marginal costs scale linearly. Recursion is an overhead-amortization strategy, not merely a proof-size optimization.

The implications for system design are immediate. If you are building a zkVM for general-purpose computation, you should optimize for large batch sizes: prove an entire block at once, not individual transactions. If you are building a privacy-preserving application (like Midnight), where each transaction requires its own proof, you should invest in reducing the fixed costs: smaller fields, faster commitment schemes, and more efficient constraint systems. The overhead tax is not one number. It is a function of computation size, field choice, constraint system, and proof system – and the design space offers different tradeoffs for different applications.

The 10,000-50,000x overhead of 2024 is not a permanent feature of provable computation. It is the current state of a rapidly improving engineering frontier. A reasonable projection is that overhead will decrease to 1,000-5,000x within two to three years for general-purpose zkVMs, with application-specific

circuits already achieving lower ratios. Whether it can ever approach 100x or below for general computation remains an open research question.

The trajectory is visible in the benchmarks. In 2022, proving a single Ethereum block took minutes on specialized hardware. By 2024, it took 30 to 60 seconds on GPU clusters. By early 2026, the fastest systems (SP1 Hypercube, RISC Zero 1.0) demonstrate 6 to 15 seconds for the same workload, with further improvements expected as circle STARKs, WHIR, and lattice-based commitments reach production maturity. Each generation of improvements comes from a different source: the move from 254-bit to 31-bit fields (2022-2023), the adoption of LogUp-GKR for lookups (2023-2024), the shift to sumcheck-based architectures (2024-2025), and GPU kernel optimization for NTTs and MSMs (ongoing). The overhead is falling not because of one breakthrough but because of compounding engineering progress across every layer of the stack.

For architects comparing systems, the following table normalizes the overhead by system and field, using Ethereum block proving as the benchmark workload:

System	Base Field	Eth Block Time	Approx. Overhead	Year	Key Innovation
RISC Zero (v0.x)	BN254 (254-bit)	~60 s (GPU)	~50,000x	2023	First general-purpose zkVM
SP1 (v1)	BabyBear (31-bit)	~15 s (16 GPU)	~10,000x	2024	Small-field + multilinear STARK
SP1 Hypercube	BabyBear (31-bit)	6.9 s (16 GPU)	~5,000x	2025	Sumcheck + precompiles
Stwo	Mersenne-31 (31-bit)	~10 s (cluster)	~3,000-5,000x	2025	Circle STARK + 940x vs. Stone
Airbender	BabyBear (31-bit)	~35 s (1 H100)	~8,000x	2025	Single-GPU design

5.8 Midnight's ZKIR: A Concrete Layer 4

Abstract discussions of constraint systems benefit from a concrete example. Midnight's ZKIR (Zero-Knowledge Intermediate Representation) provides one – and it reveals that real-world arithmetization carries more structure than the mathematical formalism might suggest.

5.8.1 The 24-Opcode DAG

ZKIR is not itself a constraint system. It is a typed instruction-level intermediate representation that sits *above* the constraint system in the compilation stack:

```
Compact (source language)
    |
    v
Compact IR (typed AST)
    |
    v
ZKIR (instruction DAG)  <-- This is what we are examining
    |
    v
PLONKish constraints (Halo2-style)
    |
    v
ZK proof (over BLS12-381)
```

A ZKIR circuit is a directed acyclic graph of 24 base instructions organized into eight categories:

- **Arithmetic** (3 opcodes): `add`, `mul`, `neg` – basic field arithmetic modulo the BLS12-381 scalar field (approximately 2^{253}). There is no subtraction opcode; the compiler implements a - b as add(a, neg(b)).
- **Constraints** (4 opcodes): `assert`, `constrain_eq`, `constrain_bits`, `constrain_to_boolean` – the enforcement mechanism.
- **Comparison** (2 opcodes): `test_eq`, `less_than` – produce boolean results without enforcing them.
- **Control flow** (2 opcodes): `cond_select`, `copy` – conditional multiplexing and variable aliasing.
- **Type encoding** (3 opcodes): `reconstitute_field`, `encode`, `decode` – type-level serialization.
- **Division** (1 opcode): `div_mod_power_of_two` – integer-style division for byte extraction.

- **Cryptographic** (5 opcodes): `transient_hash`, `persistent_hash`, `ec_mul_generator`, `ec_mul`, `hash_to_curve` – elliptic curve and hash operations over the Jubjub curve embedded in BLS12-381.
- **I/O** (4 opcodes): `private_input`, `public_input`, `output`, `impact` – the boundary between circuit and ledger state.

Each instruction consumes inputs (field elements or references to earlier instruction outputs) and produces outputs. The DAG structure emerges from data dependencies: instruction i depends on instruction j if it references j's output. Variables are numbered sequentially (0, 1, 2, ...), and instructions can only reference outputs of earlier instructions.

5.8.2 constrain_eq and constrain_bits: The Enforcement Backbone

Two opcodes embody the fundamental challenge of arithmetization: ensuring that abstract mathematical objects faithfully represent concrete computational values.

constrain_eq enforces that two field elements are identical. It produces no output. If the values differ, the circuit rejects. This is the fundamental correctness enforcement mechanism – it appears after computations to verify results, in transcript verification to bind circuit values to on-chain state, and as the implicit check inside assertions.

There is a critical distinction between `constrain_eq` (which *enforces* equality and fails the circuit if violated) and `test_eq` (which *produces* a boolean result without enforcement). The Compact compiler uses `test_eq` for equality comparisons in program logic and `constrain_eq` for internal correctness checks. Confusing the two is precisely the kind of constraint error that causes the under-constrained vulnerabilities discussed in Chapter 3.

constrain_bits enforces that a field element lies within a range $[0, 2^N - 1]$. This is essential because ZKIR values are elements of the BLS12-381 scalar field – numbers up to approximately 2^{253}. But Compact types often have bounded ranges: `Uint<8>` must be in $[0, 255]$, `Uint<32>` in $[0, 2^{32} - 1]$, `Boolean` must be exactly 0 or 1.

Without `constrain_bits`, a malicious prover could substitute any 253-bit field element where an 8-bit value was expected. If a circuit adds two `Uint<8>` values, the honest result is at most 510 – but without range checking, a prover could claim the result is an arbitrary 253-bit number, potentially extracting value or corrupting state. Every `Uint<N>` value in compiled Compact code includes a corresponding `constrain_bits` to enforce the range constraint.

A general principle is at work: in constraint systems, everything that is *not* explicitly constrained is implicitly *allowed*. The prover will satisfy exactly the constraints you write, and nothing more. If you

forget a constraint, the prover is free to exploit the gap. This is why under-constrained circuits are the dominant failure mode in ZK systems.

5.8.3 Where ZKIR Sits in the Taxonomy

ZKIR's relationship to the standard constraint system taxonomy is instructive:

Property	R1CS	AIR	PLONKish	CCS	ZKIR
Basic unit	Rank-1 constraint	Transition polynomial	Custom gate + wiring	Matrix-vector product	Typed instruction
Structure	Flat constraint list	Uniform trace	Gate array + permutation	Matrix equation	DAG of instructions
Abstraction level	Low	Low	Medium	Medium	**High**
Proof system binding	Groth16, Spartan	STARKs	Halo2, PLONK	Any IOP	PLONKish (via backend)

ZKIR is not a competitor to R1CS, AIR, PLONKish, or CCS. It operates at a higher abstraction level. Each ZKIR opcode *generates* one or more underlying PLONKish constraints: `add` generates an addition gate, `mul` generates a multiplication gate, `constrain_bits` generates range-check constraints (potentially many gates for N-bit range), `ec_mul` generates a full scalar multiplication circuit (many internal gates), `persistent_hash` generates a hash circuit (many internal gates).

The ZKIR-to-PLONKish lowering is handled by the proof system backend. ZKIR documents the *semantic* layer – what the circuit means. The actual arithmetization into PLONK gates happens below, invisible to the Compact developer.

Midnight's design philosophy becomes clear at this boundary. In Circom, the developer writes constraints directly. In SP1, the zkVM generates constraints automatically from the RISC-V execution trace. In Midnight, the Compact compiler produces ZKIR instructions that carry type information and semantic meaning (including blockchain-specific operations like ledger reads and writes), and the backend translates these into PLONKish constraints. The developer never touches the constraint system.

A ZKIR circuit could, in principle, be lowered to CCS instead of PLONKish. The typed instruction set would need to be decomposed into the matrix-vector product form that CCS requires. Whether Midnight's proof system will eventually migrate from PLONKish to CCS depends on the maturity of CCS-based proof systems and the availability of lattice-based folding schemes for production use – a question that connects Layer 4 directly to the post-quantum considerations at Layer 6.

5.8.4 The BLS12-381 Field Consequence

ZKIR operates over the BLS12-381 scalar field: a prime of approximately 2^{253}, requiring 255 bits to represent. This is roughly 4x wider than the Goldilocks field (64-bit) used by Neo and Plonky2, and approximately 8x wider than the BabyBear (31-bit) or Mersenne-31 (31-bit) fields used by SP1, RISC Zero, and Stwo.

The large field is necessary for Midnight's architectural choices. BLS12-381 is a pairing-friendly curve, enabling KZG polynomial commitments and Groth16 verification. The Jubjub twisted Edwards curve embeds natively in BLS12-381's scalar field, enabling in-circuit elliptic curve operations for Pedersen commitments and key derivation. The mature ecosystem (Zcash, Ethereum 2.0) provides audited tooling.

But the large field is also the primary performance cost. Each field operation operates on 255-bit numbers using multi-precision arithmetic, while BabyBear or M31 operations use single-register native arithmetic. This is why Midnight's proof generation takes the order of 20 seconds per circuit (see Chapter 6 for exact measurements) – acceptable for privacy-preserving blockchain transactions, but orders of magnitude slower than what small-field STARK systems achieve.

The field choice at Layer 6 determines the arithmetic cost at Layer 4. There is no escaping this dependency.

5.9 Where the Layers Collapse

This chapter has presented arithmetization as a distinct layer. But the evidence from real systems shows that the boundary between Layer 4 and its neighbors is porous.

5.9.1 Layers 3 and 4: Jolt's Merger

In Jolt, witness generation *is* the arithmetization. Every instruction in the execution trace is decomposed into lookups on small subtables. The decomposition happens simultaneously with trace generation – there is no meaningful step where "first you generate the witness, then you arithmetize it." The two processes are fused.

This has concrete implications. When the Feynman analysis asked "if Layers 3 and 4 collapse into one in Jolt, does the seven-layer model actually work?", the answer is that the model is descriptive, not prescriptive. It identifies conceptual concerns (witness generation, constraint encoding) that are always present, even when the implementation fuses them.

5.9.2 Layers 2 and 4: Cairo's Co-Design

Cairo, StarkWare's ZK-native language, was designed specifically so that its instruction set would map efficiently to AIR constraints. The ISA *is* the constraint system. Language design (Layer 2) was dictated by arithmetization efficiency (Layer 4).

The dependency runs opposite to the top-down model the book follows. In Cairo's case, the constraint system came first, and the language was designed to match it. Cairo's memory model is "write-once" – once a value is written to an address, it cannot be overwritten – because write-once memory is much cheaper to prove in AIR constraints than read-write memory. A conventional language designer would never choose a write-once memory model. But the constraint system designer knows that proving memory consistency for write-once memory requires a simple sorted-access check (much cheaper than Merkle trees or fingerprinting), so the language was shaped to match. The pedagogical order (language before arithmetization) hides a real engineering dependency. Cairo was not "compiled to" AIR; it was "born from" AIR.

5.9.3 The Proof Core: Layers 4, 5, and 6

The most significant cross-layer dependency is the "proof core" – the inseparable triad of {finite field, polynomial commitment scheme, polynomial representation} that straddles Layers 4, 5, and 6.

Choose a 31-bit field (Layer 6) and you get fast arithmetic but need FRI-based commitments (Layer 5) and AIR or multilinear representations (Layer 4). Choose a 254-bit pairing-friendly field (Layer 6) and you can use KZG commitments (Layer 5) with univariate polynomials in Lagrange basis (Layer 4). Choose a lattice-based commitment over a 64-bit Goldilocks field (Layer 6) and you get post-quantum security with CCS-native constraints (Layer 4) and sumcheck-based folding (Layer 5).

These are not three independent choices. They are one choice with three manifestations. The seven-layer model usefully separates the *concerns* (what is being encoded? how is it committed? what field operations are available?) even when the *implementations* cannot be separated.

This is the collapse we warned about in Chapter 1. The seven-layer model is a pedagogical map. The engineering territory has three layers at the proof core, not seven, and the edges between them are bidirectional. Hold both models as you read: the pedagogical stack (useful for learning each concern in isolation) and the engineering DAG (useful for building real systems). Chapter 10 will draw the honest map – seven nodes, fourteen directed edges, no pretense of independence.

A concrete example of this coupling: RISC Zero originally used a 254-bit field with KZG commitments and R1CS constraints. In 2023, they migrated to BabyBear (31-bit field) with FRI commitments and AIR constraints. The migration was not "swap out the field and keep everything else." It required simultaneously changing the field (Layer 6), the commitment scheme (Layer 5), and the constraint format (Layer 4) – because none of the three could be changed independently. BabyBear does not support KZG (which needs a pairing-friendly curve), and FRI does not work naturally with R1CS (which lacks the evaluation-domain structure that FRI requires). The three layers moved as a unit, confirming that the "proof core" is a single design decision dressed up as three.

5.10 Where the Analogies Break

We promised at the beginning of this chapter to say where the analogies break down.

Arithmetization is the hardest layer to explain because it is the layer where computer science, algebra, and information theory collide in ways that resist simplification. The "spreadsheet with polynomial rules" captures the structure but not the mechanism. The "Sudoku puzzle" captures the constraint-satisfaction flavor but misleads about uniqueness. The "encoding" metaphor captures the transformation but hides the overhead.

What actually happens at Layer 4 is a lossy translation. A computation in the real world involves pointers, variable-length data, exceptions, floating-point approximation, and timing. The arithmetized version strips all of that away and replaces it with fixed-size field elements, fixed-structure polynomial constraints, and deterministic evaluation. The gap between the two – the "abstraction tax" of 10,000x to 50,000x – is the price of making computation mathematically verifiable.

Consider what is lost in the translation. A native C program uses 64-bit integers with overflow semantics (values wrap around at 2^{64}). The arithmetized version uses field elements modulo a prime – where overflow does not exist, because field arithmetic is always exact. To faithfully represent 64-bit overflow

behavior, the constraint system must decompose the values into 64 individual bits, check that each is boolean, compute the sum, and check that only the low 64 bits are retained. The "overflow" that hardware handles in zero cycles costs dozens of constraints. Similarly, a floating-point multiplication that the CPU executes in one cycle using a dedicated FPU requires hundreds of constraints to simulate in field arithmetic – because there is no floating-point hardware in a finite field, only integers. Every gap between native computation and field arithmetic generates constraints. The overhead is not laziness or bad engineering. It is the cost of bridging two incompatible computational models.

That price is falling. It fell when AIR replaced R1CS for VM-style computations. It fell when PLONKish introduced custom gates. It fell when CCS unified the constraint systems. It fell when LogUp eliminated sorting from lookups. It fell when Lasso made table sizes irrelevant. It fell when small fields replaced 254-bit primes. It falls every time a researcher finds a way to encode more computation in fewer constraints.

But it has not fallen to zero, and it may never fall to zero. Provable computation is inherently more expensive than unprovable computation. The magician who performs backstage with no audience can cut corners. The magician who must produce a sealed certificate – one that any stranger can verify – must record every step with mathematical precision. The overhead of arithmetization is the cost of making the performance verifiable.

There is a theoretical lower bound that clarifies the situation. Any computation that produces n bits of output requires at least n bits of communication to verify (you need to at least read the output). The overhead above this information-theoretic minimum comes from the cryptographic machinery: polynomial commitments, random challenge generation, and the proof that the polynomial identities hold. Whether this cryptographic overhead can be reduced to $O(1)$ multiplicative factor remains an open question. The current answer is: not yet, but the constant factor is shrinking every year.

The honest summary of Layer 4 in 2026: arithmetization is hard, expensive, and getting better fast. The constraint systems are converging toward CCS. The lookup revolution is replacing hand-crafted constraints with table lookups. The overhead is falling from 10,000x toward 1,000x and below. And the sumcheck protocol – invented in 1992, long before anyone imagined practical zero-knowledge proofs – has become the universal verification engine that makes it all work.

From this point forward, the magician-and-audience framing will recede. Layers 5, 6, and 7 operate at a level of abstraction where the metaphor obscures more than it reveals. The sealed certificate, the deep craft, the verdict – these section titles keep the theatrical frame alive as a mnemonic, but the explanations will be increasingly technical. We will still speak of provers and verifiers, because those are the real actors, but the stage curtains come down here. When the metaphor returns in full force, it will be in Chapter 12, where Midnight provides a concrete theater that makes the abstraction physical again.

The computation is encoded as mathematics. Every instruction, every memory access, every comparison has been transformed into polynomial equations over a finite field. The equations are organized – as R1CS, as AIR, as PLONKish, or as CCS – and the lookup arguments have replaced the most expensive operations with table references. The sumcheck protocol stands ready to verify that the equations hold, without checking every cell in the spreadsheet.

The constraint system is the scorecard. But a scorecard means nothing until someone seals it into a certificate that cannot be tampered with. The next chapter enters the proof system – the cryptographic mechanism that seals the certificate.

5.10.1 Reference Data

- **R1CS** (2012): bilinear constraints (degree 2). One constraint per multiplication gate. Native format for Groth16 (128-byte proofs, constant-time verification) and Spartan.
- **AIR** (2018): uniform transition constraints over execution traces. Native format for STARKs (transparent, post-quantum). Constraint description size independent of trace length.
- **PLONKish** (2019): selector-gated custom gates with copy constraints via permutation arguments. Native format for Halo2, PLONK. Dominant in deployed systems (2020-2025).
- **CCS** (Setty, 2023): unifies R1CS, AIR, and PLONKish without overhead. Native target for HyperNova, Neo, ProtoStar, ProtoGalaxy.
- **Sumcheck protocol** (Lund et al., 1992): reduces verification of polynomial sums over 2^n inputs to n rounds of interaction. Backbone of Spartan, HyperNova, Jolt, and SP1 Hypercube.
- **Plookup** (2020): first practical lookup argument. Sorting-based, $O(n \log n)$.
- **LogUp** (2022): sorting-free lookup via logarithmic derivatives. $O(n)$ prover cost.
- **LogUp-GKR** (2023): logarithmic verifier cost for lookups. Used in SP1 Hypercube and Stwo.
- **Lasso** (2023): lookups into tables of size 2^{128}, prover cost independent of table size.
- **Jolt** (2023): approximately 6x faster than RISC Zero in theoretical commitment cost analysis. Full RISC-V ISA via lookups.
- **Overhead tax**: 10,000-50,000x versus native execution (2024-2025 systems). Falling to 1,000-5,000x by 2027-2028.
- **Overhead breakdown**: field encoding (10-100x), constraint expansion (50-100x), polynomial commitment (10-50x). Sources multiply.
- **Ozdemir et al.**: 50-150x reduction in memory checking constraints via algebraic approaches.
- **Binius** (2025): 100x reduction in bit-level embedding overhead via binary tower fields.
- **Mersenne-31**: field modulus $2^{31} - 1$. Fastest known modular reduction. Used by SP1 and Stwo.

- **ZKIR**: 24 typed instructions, compiling Compact to PLONKish constraints over BLS12-381 ($\sim 2^{253}$).

5.10.2 Sources

- [R-L4-1] Gennaro, Gentry, Parno, Raykova. "Quadratic Span Programs and Succinct NIZKs without PCPs." EUROCRYPT 2013. ePrint 2012/215.
- [R-L4-2] Ben-Sasson, Bentov, Horesh, Riabzev. "Scalable, Transparent, and Post-Quantum Secure Computational Integrity." ePrint 2018/046.
- [R-L4-3] Gabizon, Williamson, Ciobotaru. "PLONK." ePrint 2019/953.
- [R-L4-4] Setty, Thaler, Wahby. "Customizable Constraint Systems for Succinct Arguments." ePrint 2023/552.
- [R-L4-5] Setty. "Spartan: Efficient and General-Purpose zkSNARKs without Trusted Setup." ePrint 2019/550.
- [R-L4-6] Gabizon, Williamson. "Plookup." ePrint 2020/315.
- [R-L4-7] Haboeck. "LogUp." ePrint 2022/1530.
- [R-L4-8] Papini, Shahar and Ulrich Haboeck. "LogUp-GKR." ePrint 2023/1284.
- [R-L4-9] Setty, Thaler, Wahby. "Lasso." ePrint 2023/1216.
- [R-L4-10] Arun, Setty, Thaler. "Jolt." ePrint 2023/1217.
- Midnight ZKIR Reference (v2/v3), 119 oracle traces. Compact compiler v0.29.0.
- Lund, Fortnow, Karloff, Nisan. "Algebraic Methods for Interactive Proof Systems." JCSS 1992.

A note on the next three chapters. Chapters 5, 6, and 7 cover arithmetization, proof systems, and cryptographic primitives – what this book calls the "proof core." In practice, these three layers are inseparable: the choice of field (Layer 6) determines which arithmetization works (Layer 4), which determines which proof system is viable (Layer 5). We present them sequentially because a book must be linear, but they are best understood as a single coupled design unit. If a choice in Chapter 7 seems to contradict a claim in Chapter 5, it is because the dependency runs in both directions. Read all three, then revisit.

Chapter 6: Layer 5 – The Sealed Certificate

6.1 Sealing the Certificate

The magician has performed the trick backstage. Every step of the computation has been recorded in a mathematical trace and encoded as a system of polynomial equations. But a recording that never leaves the backstage is worthless. The audience needs something it can hold in its hands, inspect, and trust – without ever seeing the performance itself.

That something is a sealed certificate.

The certificate attests that the trick was performed correctly – "this transaction is valid," "this person is over 18," "this block was executed correctly." It is small enough to carry in your pocket. It is unforgeable. Anyone can verify it. Nobody needs to see the original performance. And a forged certificate is not merely unlikely – it is a mathematical impossibility, its probability shrinking exponentially as the security parameter grows.

Think of it this way. In the previous two chapters, we watched the computation get written down (the language), performed backstage (the witness), and translated into a mathematical puzzle (the arithmetization). Now we reach the moment the puzzle gets sealed into a certificate that will travel

out into the world, to be checked by strangers who have no reason to trust us and no access to our private data. This is Layer 5: the proof system. It is the mechanism that presses the wax seal.

The Running Example: The Sudoku Proof

Our 72 constraints over 16 witness variables are now sealed into a certificate. The prover commits to the constraint polynomials using the chosen commitment scheme (KZG, FRI, or Ajtai – depending on the architectural path from Chapter 10). The verifier sends random challenge points (or they are derived via Fiat-Shamir from the transcript hash). The prover evaluates the committed polynomials at those points and returns the evaluations with opening proofs.

The verifier checks: do the evaluations satisfy the constraint relationships at the challenged points? If yes, Schwartz-Zippel guarantees that the polynomials themselves satisfy the constraints everywhere – except with probability at most $d/|\mathbb{F}|$, where d is the polynomial degree and $|\mathbb{F}|$ is the field size. For our 4x4 Sudoku over a 256-bit field, d is roughly 4 and $|\mathbb{F}|$ is roughly 2^{256}, so the soundness error is vanishingly small.

The result: a Groth16 proof of 192 bytes, or a STARK proof of around 50 KB, or a folded lattice commitment – depending on the path. The verifier learns that the prover knows a valid solution. The verifier learns nothing about which numbers go where. The sixteen secret values never leave the prover's machine. The 4x4 grid that was the witness has been compressed into a handful of group elements and field evaluations – the sealed certificate.

Here is how the seal works. The proof system commits to a set of polynomial evaluations. If the prover cheated anywhere in the computation, those evaluations will be inconsistent – the polynomial will disagree with itself at a random point the verifier picks. The probability that the prover can guess which point the verifier will pick, and cheat in exactly the right way to pass that specific check, is negligible – meaning it shrinks exponentially as the security parameter grows.

A forged certificate does not look like a convincing imitation that might fool someone. It looks like an impossible object. The proof system is designed so that forged certificates simply cannot be produced in polynomial time, assuming the underlying mathematical hardness assumptions hold. This chapter explains how the sealing mechanism works, how it evolved from a single method into a family of techniques with radically different properties, and why the engineering choices within Layer 5 are changing the entire economics of blockchain computation.

6.2 The Three Families

Every zero-knowledge proof system in production today belongs to one of three families. They differ in what they require before you start, how large a proof they produce, and what mathematical assumptions keep them secure. Choosing among them is not a philosophical exercise. It is an engineering decision with direct consequences for cost, speed, security, and quantum resistance.

6.2.1 Groth16: The Gold Standard for Size

In 2016, Jens Groth published a proof system that produces the smallest possible proofs: three elliptic curve group elements, totaling 192 bytes (two G1 points and one G2 point on BLS12-381). Verification requires three pairing computations (the Ethereum EVM implementation batches this as four pairings via EIP-1108) – about 250,000 gas on Ethereum. Nearly a decade later, nothing comes close to this combination of proof size and verification cost. Groth16 remains the final wrapping target for almost every production ZK system that posts proofs on-chain.

The catch is severe. Groth16 requires a per-circuit trusted setup ceremony. You cannot change the circuit without re-running the ceremony. The ceremony produces a structured reference string (SRS) that contains "toxic waste" – secret randomness that, if any single participant retains, would allow them to forge proofs for that circuit forever. The 1-of-N trust model (discussed in Chapter 2) mitigates this, but the ceremony is expensive, inflexible, and not quantum-resistant.

Put differently: Groth16 seals the smallest possible certificate – just 192 bytes, three elliptic curve points. But you need a custom seal for every circuit you want to prove, and manufacturing that seal requires a ceremony involving thousands of people, any one of whom could secretly keep a master key to forge future certificates. If a quantum computer ever appears, every seal ever manufactured becomes useless.

Despite these limitations, Groth16's on-chain economics are so favorable that it persists as the outer shell of nearly every hybrid proving pipeline. The inner proof system might be a STARK, a folding scheme, or something else entirely. But the last step – the proof that actually touches the blockchain – is almost always Groth16 over the BN254 curve, because Ethereum's EVM has precompiled contracts that make BN254 pairings cheap.

6.2.2 PLONK: The Universal Workhorse

In 2019, Gabizon, Williamson, and Ciobotaru published PLONK, which solved Groth16's biggest practical problem: the per-circuit setup. PLONK uses a universal structured reference string. Run one ceremony, and you can use the resulting SRS for any circuit up to a fixed size. Change your program? No new ceremony needed. Just compile a new circuit against the same SRS.

PLONK introduced "PLONKish" arithmetization – a system of gate constraints and copy constraints glued together by a permutation argument. This turned out to be very flexible. Custom gates allow specialized operations (elliptic curve arithmetic, hash functions, range checks) to be encoded efficiently. Lookup tables (via Plookup and its successors) let the prover reference pre-computed values instead of re-deriving them from scratch. The result was a proof system that could be tuned for specific workloads while retaining universality.

Halo 2, developed by the Electric Coin Company for Zcash and later adopted by other projects, extended PLONK with custom gates, lookup tables, and a flexible backend that supports multiple commitment schemes. Note: the original Halo paper (2019) used IPA commitments with no trusted setup. Halo 2, as deployed by Zcash for Orchard and adopted by Midnight, uses KZG commitments requiring a ceremony – a different trust model despite the shared name. When instantiated with IPA instead of KZG, Halo 2 eliminates the trusted setup entirely, and the proof size grows from constant to logarithmic, but verification becomes somewhat more expensive.

PLONK and its variants (UltraPlonk, TurboPlonk, Halo 2) are the workhorses of production ZK today. They sit at the center of a design space between Groth16's extreme compactness and STARKs' extreme transparency. Most ZK applications that need flexibility – circuits that change frequently, applications where a per-circuit ceremony is impractical – choose a PLONK-family system.

6.2.3 STARKs: The Transparent Path

STARKs (Scalable Transparent ARguments of Knowledge), introduced by Ben-Sasson, Bentov, Horesh, and Riabzev in 2018, took a radically different approach. No trusted setup at all. No pairings. No elliptic curves. The only cryptographic assumption is the existence of collision-resistant hash functions – a primitive that is believed to be quantum-resistant.

The core idea is clean. STARKs prove statements about Algebraic Intermediate Representations (AIR): polynomial constraints on an execution trace, where each row represents one time step and the constraints enforce that consecutive rows are consistent. The key mechanism is the FRI protocol (Fast Reed-Solomon Interactive Oracle Proof), which verifies that a committed function is close to a low-degree polynomial. If the computation was performed correctly, the trace satisfies all constraints, and

the polynomial is low-degree. If the prover cheated, the polynomial's degree blows up, and FRI catches the inconsistency with overwhelming probability.

The cost of this transparency is proof size. A STARK proof is hundreds of kilobytes – roughly 1,000 times larger than a Groth16 proof. On Ethereum, where every byte costs gas, this difference translates directly into money. But STARKs offer something the other families cannot: a path to post-quantum security, and a proving architecture that scales almost linearly with computation size.

So the three families seal certificates with different properties. Groth16 seals the smallest possible certificate – just 192 bytes – but requires a custom seal for every circuit. PLONK seals a slightly larger certificate but can use the same seal for any trick, eliminating the per-circuit ceremony. STARKs seal certificates without any secret ingredient at all – no ceremony, no toxic waste, no trust assumptions beyond collision-resistant hashing – but the certificates are much bulkier: hundreds of kilobytes instead of hundreds of bytes.

6.2.4 Three Envelopes: An Intuition

The technical differences between the three families are real, but their *character* is better grasped through analogy. Each family seals a certificate. Think of each certificate as a letter placed inside an envelope. The families differ not in what the letter says but in how the envelope is made, what it costs, and what it reveals about the process of sealing.

Groth16 is the smallest possible envelope. Three numbers – two points on a curve and one more – encode an entire computation. Consider the compression ratio. A circuit proving that an Ethereum block was executed correctly might involve billions of constraints, millions of intermediate values, a witness consuming gigabytes of memory. The Groth16 proof of that computation is 192 bytes. Three elliptic curve elements. It is as though someone handed you a novel – a thick, sprawling saga with hundreds of characters and interlocking subplots – and you compressed it into a haiku. Seventeen syllables. And yet a reader who knows the rules of haiku composition can verify that those seventeen syllables faithfully capture the plot. Not a summary. Not an approximation. A mathematically exact encoding from which any deviation would be detectable. The haiku either satisfies the verification equation or it does not. There is no room for a "pretty good" forgery.

The cost of this extreme compression is inflexibility. The haiku form must be custom-designed for each novel. You cannot take a haiku mold built for *War and Peace* and use it to compress *Moby Dick*. In Groth16 terms, this means a new trusted setup ceremony for every circuit. The ceremony produces the specific algebraic structure – the structured reference string – that makes the compression possible for that particular computation. Change the computation, and you need a new ceremony. This is why

Groth16 is almost never used as the primary proof system. It is used as the *final wrapper* – the outermost envelope – because you only need one ceremony for the wrapping circuit, and that circuit does not change.

PLONK is the universal envelope. One envelope fits all letters. The envelope factory runs a single setup ceremony and produces a structured reference string that works for any circuit up to a certain size. Write a new smart contract? Same envelope. Update your program logic? Same envelope. The setup cost is paid once. After that, any computation that fits within the size bound can be sealed without a new ceremony.

The analogy is a standardized shipping container. Before containerization, every type of cargo required its own packaging, its own crane, its own dockworker expertise. A container does not care whether it holds televisions, bananas, or machine parts. It is a universal interface between the thing being shipped and the infrastructure that moves it. PLONK's universal SRS is the container. The circuit – whatever it computes – is the cargo. The infrastructure – the verifier, the blockchain, the precompiled contract – handles every circuit the same way, because they all arrive in the same container.

The proofs are slightly larger than Groth16's (a few hundred bytes to a few kilobytes, depending on the variant), and verification is slightly more expensive. These are the costs of universality. You pay a modest premium for the ability to change your message without manufacturing a new envelope. For most applications, this tradeoff is overwhelmingly favorable, which is why PLONK-family systems are the workhorses of production ZK.

STARKs are glass envelopes. Nothing is hidden in the construction. There is no trusted setup, no toxic waste, no secret randomness that could compromise the system if leaked. The envelope-making process is entirely public. Anyone can inspect it, audit it, reproduce it. The cryptographic assumption is minimal: collision-resistant hash functions exist. That is all.

The trade: glass envelopes are bulkier than paper ones. A STARK proof is hundreds of kilobytes – roughly a thousand times larger than a Groth16 proof. On a blockchain where every byte costs gas, this bulk translates directly into dollars. Glass is heavier than paper. You pay more to ship it. But glass has a property that paper lacks: you can see through it. The transparency is not a metaphor. It is a literal statement about the trust model. There is no ceremony to trust, no participant to worry about, no toxic waste to dispose of. The security rests on the hardest-to-break foundation in cryptography: the belief that hash functions do not have secret backdoors.

And glass has another property that matters more each year: it does not shatter under quantum impact. Hash-based cryptography is believed to resist quantum attacks. Paper envelopes – those built on elliptic curve assumptions – will dissolve the moment a sufficiently powerful quantum computer runs Shor's

algorithm. Glass envelopes will still be standing. The bulk that seems like a disadvantage today may prove to be the price of survival.

But notice: the three envelope types are not competing products on a shelf. They are components in a supply chain. The glass envelope (STARK) is manufactured first, because it is transparent and quantum-resistant. Then the contents are transferred into a paper envelope (Groth16) for shipping, because paper is lighter and the postal system (the blockchain) charges by weight. The glass envelope never reaches the destination. It does its job backstage – proving the computation with full transparency – and then the result is repackaged into the smallest, cheapest container for the final mile. Understanding this supply chain is what the next section is about.

6.3 The Hybrid Pipeline

Here is the secret that the field's own marketing has obscured: in production, the three families are not competitors. They are components of a single pipeline.

The dominant architecture in 2025 looks like this:

1. **Prove the computation with a STARK.** The inner proof system generates a large, transparent proof using fast, small-field arithmetic. No trusted setup is required. The prover runs on GPUs.
2. **Recursively compress the STARK.** Apply one or more rounds of recursive STARK verification to shrink the proof from hundreds of kilobytes to tens of kilobytes.
3. **Wrap the compressed STARK in a Groth16 SNARK.** A small circuit verifies the STARK proof and produces a Groth16 proof: 192 bytes, 250K gas to verify on-chain. The wrapping circuit adapts the STARK's field arithmetic (say, Mersenne-31) to the BN254 field that Ethereum's precompiles support.
4. **Post the SNARK on-chain.** The Ethereum verifier contract checks the Groth16 proof. It has no idea that the inner computation used a STARK. From the chain's perspective, everything looks the same.

Every major production system follows this architecture: SP1 (Succinct Labs), Stwo (StarkWare), Polygon, ZKM, and most others. The STARK generates the raw material; the SNARK seals it into a certificate the blockchain will accept. The STARK provides transparency and prover efficiency. The SNARK provides on-chain cost efficiency. Each family contributes what it does best.

The implication is worth spelling out. The "SNARK vs. STARK" debate that dominated conference panels from 2019 to 2023 was a false dichotomy. The field converged on "STARK inside, SNARK outside" because the engineering tradeoffs demanded it. Transparency for the prover. Compactness for the chain.

The only remaining question is whether the outer SNARK wrapper can itself become post-quantum – a problem that lattice-based proof systems (discussed later in this chapter) are beginning to address.

6.3.1 A Concrete Pipeline: From 1,000 Transactions to a 192-Byte Proof

The hybrid architecture becomes vivid when you trace a specific workload through it, step by step. Consider a ZK rollup operator – running on Ethereum mainnet – who receives a batch of 1,000 transactions. Each transaction is a token transfer, a swap, or a contract call. The operator must prove that executing all 1,000 transactions against the current state produces the claimed new state root. Here is how the pipeline processes that batch in 2025.

Step 1: Execute and generate the witness. The operator's sequencer replays all 1,000 transactions against a local copy of the rollup's state. Every storage read, every storage write, every arithmetic operation is recorded in an execution trace – the giant spreadsheet from Chapter 4. The witness includes all private data: account balances, nonces, intermediate computation values. This step is ordinary software execution, no cryptography involved. On a modern server, it takes 1 to 2 seconds. The output is a trace with millions of rows and dozens of columns.

Step 2: Generate a STARK proof over a small field. The prover takes the execution trace and produces a STARK proof using BabyBear (31-bit) or Mersenne-31 arithmetic. This is where the heavy computation happens. The trace is interpolated into polynomials, the polynomials are committed via FRI (Merkle trees of evaluations), and the FRI protocol verifies low-degree proximity. On a cluster of GPUs – say, four NVIDIA A100s – this step takes 3 to 5 seconds. The output is a STARK proof: transparent, hash-based, quantum-resistant, and roughly 200 to 400 kilobytes in size.

Step 3: Recursively compress the STARK. The raw STARK proof is too large to post on-chain economically. So the operator generates a second STARK proof that verifies the first one. This is recursion: a proof about a proof. The verifier circuit for a STARK is much smaller than the original computation circuit, so the recursive proof is faster to generate and produces a smaller output. One or two rounds of recursive compression shrink the proof from hundreds of kilobytes to tens of kilobytes. This step takes 1 to 2 seconds.

Step 4: Wrap in Groth16 over BN254. The compressed STARK proof is now small enough to verify inside a Groth16 circuit. A specialized wrapping circuit takes the STARK verifier computation – check the Merkle paths, verify the FRI folding, confirm the constraint evaluations – and expresses it as an R1CS instance over the BN254 field. The Groth16 prover then seals this into a 192-byte proof: two G1 points and one G2 point. This is the field-crossing step, where 31-bit STARK arithmetic is translated into 254-bit BN254 arithmetic. It is computationally expensive per field operation, but the circuit is small (it

is only verifying a STARK, not re-executing the original computation). On a single GPU, this step takes 5 to 10 seconds.

Step 5: Post the proof to Ethereum. The operator submits a transaction to the rollup's on-chain verifier contract. The transaction contains the 192-byte Groth16 proof, the new state root, and a commitment to the batch of transactions. The Ethereum verifier contract calls the BN254 pairing precompile (EIP-1108), checks the three pairings, and accepts or rejects. Verification costs approximately 250,000 gas – roughly $0.50 to $1.00 at typical gas prices. The state root is updated. The 1,000 transactions are finalized.

The audience sees only Step 5. A 192-byte proof appears on-chain. A smart contract checks it in a few milliseconds. The state updates. Nobody knows – or needs to know – that behind those 192 bytes lie four NVIDIA GPUs, two rounds of recursive compression, a field-crossing circuit, and the execution traces of 1,000 individual transactions. The entire pipeline, from receiving the batch to posting the proof, completes in under 15 seconds and costs under $1.00.

That is the hybrid pipeline in concrete terms. The STARK did the heavy lifting: proving the computation with transparency and quantum resistance. The Groth16 wrapper did the packaging: compressing everything into the smallest possible on-chain footprint. Each proof system contributed what it does best. The audience – Ethereum's verifier contract – received the finished product and asked no questions about the manufacturing process.

Two years earlier, the same pipeline took minutes and cost tens of dollars. Two years before that, it was a research prototype that could not process a full Ethereum block at all. The economics shifted not because anyone invented a fundamentally new proof system, but because each component in the pipeline got faster – small-field arithmetic, GPU parallelism, recursive compression, optimized wrapping circuits – and the improvements compounded multiplicatively across stages. A 3x improvement in STARK proving, combined with a 2x improvement in recursive compression, combined with a 2x improvement in Groth16 wrapping, produces a 12x improvement end-to-end. This is why the cost curve has been steeper than Moore's Law.

6.4 Recursion vs. Folding: Russian Dolls and Snowballs

Within any proof system architecture, there is a second design choice that matters just as much: how do you handle computation that happens in steps?

A blockchain processes blocks sequentially. A zkVM executes instructions one at a time. A rollup batches transactions and proves them in order. In every case, the prover needs to demonstrate not just "this single step is correct" but "every step from the beginning until now was correct." This is

Incrementally Verifiable Computation (IVC), first formalized by Valiant in 2008, and it sits at the heart of how proof systems scale.

6.4.1 Recursion: The Russian Doll

The first approach to IVC was recursive composition. The idea is deceptively simple: to prove that steps 1 through N are all correct, you prove step N is correct *and* that you have a valid proof that steps 1 through N-1 are correct. The verifier for steps 1 through N-1 is itself expressed as a circuit, and the proof for step N includes a verification of the previous proof.

Think of Russian nesting dolls. Each doll contains a smaller doll inside it, and that smaller doll contains an even smaller one. Opening the outermost doll and confirming it contains a properly formed inner doll gives you confidence in the entire chain. You never need to open all the dolls at once.

The problem is cost. The SNARK verifier is a complex circuit – for pairing-based systems like Groth16, it involves millions of gates. Embedding a Groth16 verifier inside a Groth16 circuit means every step of computation must pay the cost of a full SNARK verification on top of the actual computation. Zexe (Bowe, Chiesa, Green, Miers, Mishra, Wu, 2018) demonstrated this approach using depth-2 recursion over a 2-chain of pairing-friendly curves, achieving constant-size 968-byte transactions regardless of offline computation. But the proving cost per step was enormous.

6.4.2 Folding: The Snowball

In 2022, Abhiram Kothapalli, Srinath Setty, and Ioanna Tzialla published Nova, and the field changed direction overnight.

Nova introduced a fundamentally different approach: instead of proving each step and recursively verifying previous proofs, you *fold* two claims into one. Here is the intuition. In recursion, each step generates a complete proof, and the next step verifies it. In folding, each step generates a *claim* – an incomplete, unverified assertion – and combines it with the running claim from all previous steps. The combination is a random linear combination: the prover takes the two claims, the verifier picks a random challenge, and the prover produces a single new claim that is valid if and only if both original claims were valid. No full proof is ever generated during the accumulation phase. The certificate is not sealed along the way. Claims accumulate, and the expensive seal is deferred to the very end. Only when all steps have been folded together does the prover generate a single SNARK proof for the final accumulated claim.

The snowball analogy captures it precisely. Each new step is a handful of snow. Instead of building a separate snowman for each step (recursion), you pack the new snow onto the growing snowball (fold-

ing). The snowball gets slightly larger with each step, but you never have to build an entire snowman until the very end.

Nova's key technical insight was *relaxed R1CS*. This is where the mathematics earns its keep, and it is worth understanding why.

Standard R1CS says $Az \circ Bz = Cz$ (where A, B, C are matrices and z is the witness vector). Now suppose you try to combine two standard R1CS instances by taking a random linear combination: $(A(z_1 + r \cdot z_2)) \circ (B(z_1 + r \cdot z_2))$. When you expand this product, cross-terms appear – terms involving both z_1 and z_2 multiplied together – that do not fit the $Az \circ Bz = Cz$ format. The product of two linear combinations is quadratic, but R1CS demands bilinear structure. The cross-terms break the format.

This is the kind of obstacle that looks fatal until someone sees through it. Nova's insight was to relax the constraint. Instead of requiring $Az \circ Bz = Cz$ exactly, allow $Az \circ Bz = u \cdot Cz + E$, where u is a scalar and E is an error vector. When $u = 1$ and $E = 0$, you recover standard R1CS. But the relaxed form is closed under random linear combination: when you combine two relaxed instances, the cross-terms that would have destroyed the standard format are absorbed into the error vector E. The format survives. The scalar u tracks the linear combination's coefficient, and E accumulates the algebraic debris that would otherwise break the structure.

The folding verifier is tiny – one scalar multiplication plus hashing, roughly 10,000 multiplication gates – compared to millions for a full SNARK verifier. This is the breakthrough: fold cheaply at every step, prove expensively only once at the end. The per-step overhead of IVC dropped by orders of magnitude.

6.4.3 The Snowball Does Not Fall Apart

The natural worry about the snowball analogy is: snowballs fall apart. What is the failure mode of folding?

The answer involves a subtlety that trips up even experienced cryptographers. Folding does not provide soundness on its own. It provides a *reduction*: if the folded claim is valid, then both original claims were valid (with overwhelming probability over the verifier's random challenge). But "valid" here means "satisfies the relaxed constraint system." The final proof – the "decider" SNARK at the end of the chain – is what provides soundness. If the underlying commitment scheme is binding and the random challenges are honestly generated, then a cheating prover cannot produce a valid folded claim for an invalid computation. The security proof works backward: a valid final claim implies all intermediate claims were valid, which implies all computation steps were correct.

The practical penalty comes from the size of the accumulated instance. In classical Nova, the error vector E grows with each folding step. The 10,000-gate figure for Nova's folding verifier is specifically the cost of

the non-native scalar multiplication required when working over a 2-cycle of curves. CycleFold (Kothapalli and Setty, 2023) delegated this scalar multiplication to a co-processor circuit on the secondary curve, where it can be computed natively in approximately 1,500 gates. The folding verifier itself – the hash and the random linear combination – is much cheaper. But the fundamental architecture remains: fold cheaply, prove expensively but only once, at the end.

6.5 The Folding Genealogy

Folding is not a single technique. It is a research program that has produced a lineage of increasingly general schemes over the past four years. Understanding this lineage matters for seeing where proof systems are headed – including the post-quantum frontier.

Before tracing each step, here is the map. Read it the way a naturalist reads a field guide: not to memorize every species, but to understand the territory they occupy. The folding genealogy advances along four independent axes:

- **Constraint generality:** R1CS (Nova) to CCS (HyperNova), covering all major arithmetizations.
- **Instance generality:** Single-instance (Nova) to multi-instance (ProtoGalaxy), enabling parallel proving.
- **Instruction generality:** Uniform IVC (Nova) to non-uniform IVC (SuperNova, SuperNeo), enabling VM execution.
- **Cryptographic generality:** Elliptic curves (Nova through CycleFold) to lattices (LatticeFold, Neo, Symphony), enabling post-quantum security.

Every scheme in the genealogy that follows advances along at least one of these axes. The reader can track the progress by asking a single question of each paper: which axis did it push forward?

6.5.1 Nova (2022): The Origin

Kothapalli, Setty, and Tzialla. Folding for R1CS. The verifier performs one scalar multiplication plus hashing – roughly 10,000 R1CS gates of recursive overhead. Prover cost is linear in the circuit size. This is the paper that made folding practical. Every subsequent folding scheme either extends, generalizes, or adapts Nova's core ideas.

6.5.2 SuperNova (2022): Non-Uniform IVC

Kothapalli and Setty. Extended Nova to support multiple circuit types. Instead of one step function that repeats, SuperNova allows different step functions at each step – exactly what you need for a virtual machine, where each instruction has a different circuit. The scheme maintains one running instance per instruction type and pays only for the circuit of the instruction actually executed. This is the "pay-per-instruction" property that makes folding-based zkVMs viable.

6.5.3 HyperNova (2023): The Generalization Point

Kothapalli and Setty again. This paper is the pivot of the entire genealogy.

HyperNova generalized folding from R1CS to CCS – Customizable Constraint Systems – which subsume R1CS, PLONKish, and AIR in a single framework. The key enabler was the sumcheck protocol, originally due to Lund, Fortnow, Karloff, and Nisan in 1992, one of the most elegant tools in theoretical computer science. It reduces the task of checking a sum over a large domain (say, 2^{20} evaluations of a multilinear polynomial) to checking a single evaluation at a random point. The verifier sends random challenges in each round; the prover responds with univariate polynomials. After $\log(n)$ rounds, the verifier has a single-point claim that can be checked directly.

HyperNova uses sumcheck to fold CCS instances without degree blowup. A CCS constraint has the form: sum of products of matrix-vector multiplications equals zero. This is inherently higher-degree than R1CS. Without sumcheck, folding such constraints would require the verifier to handle cross-terms whose degree grows with the constraint degree. Sumcheck reduces the multilinear CCS equation to a single-point evaluation, allowing the folded instance to retain its structure. The verifier cost remains $O(\log m)$ field operations plus one scalar multiplication – essentially the same as Nova, despite handling a strictly more general constraint system.

HyperNova is the generalization point because everything after it works in the CCS framework. If Nova gave folding a body, HyperNova gave it a universal language. CCS is to constraint systems what SQL is to databases: a common tongue that lets you express any query regardless of the underlying storage engine.

6.5.4 ProtoStar and ProtoGalaxy (2023): Alternative Paths

These two schemes took different approaches to generalizing folding. ProtoStar (Bunz and Chen, ePrint 2023/620) built a generic accumulation framework for any interactive argument satisfying "special soundness" – a more abstract starting point than HyperNova's CCS-specific approach. ProtoGalaxy (Eagen and Gabizon) introduced multi-instance folding: folding k instances simultaneously in a single round, rather than folding pairs sequentially. This enables high-arity proof-carrying data trees, which are needed for parallel proof generation.

Both contributed important ideas to the field, but the main trunk of the genealogy runs through HyperNova because CCS became the dominant constraint language.

6.5.5 CycleFold (2023): The Practical Fix

Kothapalli and Setty. This is the engineering paper that made all the theoretical folding schemes practical over elliptic curves. The problem: Nova's folding verifier includes a scalar multiplication, which requires non-native field arithmetic when working over a 2-cycle of curves. CycleFold delegates this scalar multiplication to a co-processor circuit on the second curve, where it can be computed natively. This reduced the second-curve circuit from roughly 10,000 gates to 1,500. Not glamorous, but essential. CycleFold is the standard technique used in every practical implementation of Nova and HyperNova.

6.5.6 LatticeFold (2024): Crossing Into Post-Quantum Territory

Boneh and Chen, published at ASIACRYPT 2025. LatticeFold was the first folding scheme based on lattice assumptions rather than elliptic curve assumptions. This matters enormously, because lattice problems (Module-SIS, Module-LWE) are believed to resist quantum attacks, while elliptic curve discrete logarithm falls to Shor's algorithm.

LatticeFold replaced elliptic curve commitments with Ajtai-style lattice commitments: $\text{Commit}(A, m, r) = A \cdot [m; r] \bmod q$, where A is a public matrix, m is the message, and r is randomness. These commitments are linearly homomorphic – exactly the property that folding needs to combine instances via random linear combination. But LatticeFold worked over large fields ($q \approx 2^{128}$), which made arithmetic expensive.

6.5.7 Neo (2025): Small Fields and Pay-Per-Bit

Wilson Nguyen and Srinath Setty. Neo adapted HyperNova's CCS folding to lattice-based commitments over small fields – specifically the Goldilocks field ($q = 2^{64} - 2^{32} + 1$) with the cyclotomic ring $\mathbb{F}_q[X]/(\Phi_{81})$, where $\Phi_{81}(X) = X^{54} + X^{27} + 1$. Working over a 64-bit field instead of a 128-bit field makes arithmetic dramatically faster, particularly on GPUs where 64-bit integer operations are natively supported.

Neo introduced a property called "pay-per-bit commitments." In an Ajtai commitment, the cost of committing depends on the binary representation of the witness. Committing to a single bit costs 32 times less than committing to a 32-bit value. This means the prover can commit to a binary witness at a fraction of the cost of committing to field elements – a property unique to the lattice setting that has no analog in elliptic curve commitments.

SuperNeo, presented within the same paper, extended Neo to non-uniform IVC (the lattice analog of SuperNova), supporting multiple instruction types for VM execution.

6.5.8 Symphony (2026): Production-Grade Lattice Folding

Independently extending lattice-based folding to high-arity settings, Symphony refined the protocol for practical deployment. Key additions included optimized Number Theoretic Transforms over the cyclotomic ring for fast polynomial multiplication, a GPU-friendly architecture (lattice operations – matrix-vector products, NTTs – parallelize naturally), and formal bridge theorems connecting the folding scheme's knowledge soundness to the underlying Module-SIS and Module-LWE assumptions.

Symphony demonstrated that lattice-based folding can be practical, not just theoretical. With GPU acceleration, it achieves practical proving times, closing the performance gap with elliptic-curve-based systems while maintaining plausible post-quantum security.

Each step in the genealogy broadened the reach of folding without sacrificing its fundamental advantage: logarithmic verifier cost and linear prover cost per step. The entire genealogy, from Nova to Symphony, spans just three years. This rate of progress is unusual even by the standards of a fast-moving field.

6.6 Nightstream: What a Folding Engine Looks Like From the Inside

The genealogy reads like a family tree. But a family tree does not show you the plumbing. It tells you who begat whom, not how the pipes connect, where the pressure builds, or which joints leak under load.

Nightstream is a real implementation of the lattice-folding lineage described above. Fifteen Rust crates, a Lean formal model, and a proving pipeline that turns execution traces into folded obligations over the Goldilocks field. It is a research prototype, not a production system – its own README says so. But it is the most complete public implementation of CCS-native lattice-based folding available for study. And studying its engineering reveals something that theory papers consistently leave out: the hardest problem in a folding system is not any single algorithm. It is the alignment between all of them.

6.6.1 A Pipeline, Not a Library

The spine of Nightstream is a sequence of crates, each doing one thing, each depending on the ones before it:

`neo-params` and `neo-math` fix the algebraic world – Goldilocks field, cyclotomic ring, norm bounds. `neo-ccs` defines the constraint and evaluation relations. `neo-ajtai` provides the linear commitment backend. `neo-transcript` enforces Fiat-Shamir sequencing through Poseidon2 hashing. `neo-reductions` implements the algebraic proof kernel. `neo-memory` turns execution traces into per-step witness bundles. `neo-fold` coordinates the shard-and-session folding runtime and emits obligations.

This is Chapter 4's proof core triad – constraint system, commitment scheme, information-theoretic protocol – made concrete in Rust. Each crate does one thing. The system works only because every crate agrees with every other crate on field choice, ring structure, witness layout, commitment semantics, and transcript ordering. Break one agreement and the pipeline does not produce wrong proofs. It produces no proofs at all.

That is the nature of a pipeline. A library offers tools. A pipeline imposes discipline. You can use a library selectively. You must use a pipeline as designed, or not at all.

6.6.2 The Shared Bus

How does an execution trace become a foldable witness?

The answer lives in `neo-memory`, and the design is a shared CPU bus. The execution trace – every instruction, every memory access, every register state – is laid out as a single columnar spine. Each step in the computation produces a row. The columns follow a fixed schema defined in the bus layout. Twist and Shout memory arguments, which verify that the prover's claimed memory operations are consistent, consume bus-extracted columns from the same spine rather than maintaining separate committed witnesses.

This is what the backstage recording machinery from Chapter 4 looks like when you unroll it into actual data structures. The magician's assistant is not just scribbling notes – she is filling in a spreadsheet with a fixed schema, one row per step, and every downstream verification process reads from that same spreadsheet.

The design works because it is unified. One trace spine means one source of truth. No reconciliation between separate witness formats. No risk of the memory argument seeing a different witness than the folding runtime.

The bus architecture connects six components in a fixed data flow:

Component	Role	Reads From	Writes To
Execution trace	Records each computation step as a row	Program input + state	Columnar spine
Bus layout	Defines column schema (registers, flags, memory)	(architectural constant)	All downstream consumers
neo-memory	Builds per-step witness bundles	Columnar spine	Folding runtime

Component	Role	Reads From	Writes To
Twist memory args	Enforces memory consistency via permutation	Bus columns	Obligation streams (val lane)
neo-fold	Accumulates shard/session folding obligations	Per-step bundles	main + val obligation streams
Finalizer	Consumes both lanes and produces outer proof	main + val obligations	Final proof artifact

The design is fragile because unified. Bus layout, witness builder, CPU constraints, and oracle expectations must all agree on the exact column semantics. If `neo-memory` produces a witness with columns in one order and `neo-fold` expects them in another, the mismatch is not caught by type checking. It is caught by a constraint that fails to satisfy, deep in the pipeline, with an error message that does not point back to the layout disagreement. This is the kind of bug that theory papers never discuss, because in theory, witness layout is not a concept. In engineering, it is the concept.

6.6.3 Three Reductions and Two Lanes

The algebraic heart of Nightstream lives in `neo-reductions`, which implements three reductions:

Pi_CCS reduces CCS constraint satisfaction into evaluation claims. This is the step that transforms "does this witness satisfy these constraints?" into "does this polynomial evaluate correctly at this random point?" – the same transition from constraint checking to evaluation checking that appears throughout modern proof systems.

Pi_RLC performs random linear combination, batching multiple evaluation claims into one. This is where the folding happens: two claims become one, weighted by a verifier challenge.

Pi_DEC handles decomposition, ensuring that the committed witness values stay within the norm bounds required by the lattice commitment scheme. Without this, the prover could fold claims using witness values that violate the short-vector assumptions that make Ajtai commitments binding.

The reductions crate preserves a three-path architecture that deserves attention. `Optimized` is the fast runtime path. `PaperExact` mirrors the published protocol specification step for step, sacrificing performance for auditability. `OptimizedWithCrosscheck` runs both paths and compares their outputs. This is not paranoia. It is engineering discipline. The optimized path inevitably diverges from the paper's notation, and a reference implementation that can be run in parallel provides a semantic anchor.

After the reductions, the folding runtime in `neo-fold` introduces the two-lane obligation model. The `main` lane carries the primary folded claims. The `val` lane carries a separate obligation stream for the Twist-related verification path, derived from a distinct random challenge. These are not interchangeable. Verification reconstructs obligations in both lanes, but verification alone does not finish the proof. Completion depends on a finalizer that consumes both lanes and produces the outer proof. The system emits a structured proof state, not a finished certificate.

What does a two-lane divergence look like in practice? Suppose the prover, at folding step 3, manipulates a register value in the witness – setting r_7 to 42 instead of the correct value 37. The `main` lane, using random challenge α, accumulates an obligation that combines this register with others: $\alpha \cdot r_1 + \alpha^2 \cdot r_2 + \cdots + \alpha^7 \cdot r_7 + \cdots$ The wrong value shifts the accumulated sum, but with a single challenge, the prover might get lucky – if α happens to land on a root of the error polynomial, the corruption is invisible.

The `val` lane exists to prevent this. It uses an independent challenge β, drawn from a different Fiat-Shamir transcript fork. The probability that the same error is invisible under *both* α and β is at most $d/|K|^2$, where d is the constraint degree and $|K|$ is the extension field size. For Neo's parameters ($K = \mathbb{F}_{q^2}$ with $q \sim 2^{64}$), this probability is less than 2^{-127}. The two lanes provide soundness amplification: an error that survives one random challenge is vanishingly unlikely to survive both. The finalizer checks that

both lanes produce consistent results – and if the manipulated r_7 corrupted the `main` lane's obligation, the `val` lane catches the inconsistency and the proof fails.

6.6.4 The Lean Boundary

Nightstream includes a formal subproject in Lean 4 that closes theorem surfaces for the core protocol. The Lean model covers the algebraic reductions, the commitment binding properties under Module-SIS, and the soundness chain from folded claims back to original computation steps.

This is unusual for a research prototype. Most implementations at this stage rely entirely on paper proofs and test suites. Nightstream's formal model provides a higher standard of assurance for the mathematical core.

But there is a boundary that formal methods cannot cross on their own. The Lean proofs verify that the *mathematics* is correct – that the reductions preserve soundness, that the commitment scheme is binding under stated assumptions. The Rust runtime must then consume those Lean-closed theorem surfaces exactly as intended. The proofs live in one world; the code lives in another. The residual risk is not that the theorems are wrong. It is that the code, through a layout mismatch, an off-by-one index, or a misread parameter, might inhabit a slightly different mathematical world than the one the theorems describe. Mathematics proven correct in the abstract; the question is whether the implementation faithfully instantiates it. This is a higher-quality problem than most systems have. It is still the real remaining assurance boundary.

To be concrete about what Lean proves and what it does not: the Lean formalization verifies that the mathematical reductions are sound – that if the folded claim passes the verifier's checks, then the original CCS instance was satisfiable. It proves the *if-then* chain from folded proof to original computation. What Lean does *not* prove is that the Rust code in `neo-fold` correctly instantiates the reduction. An off-by-one index in a matrix multiplication, a transposed loop bound in the commitment computation, a misread parameter from `neo-params` – any of these could cause the Rust implementation to inhabit a slightly different mathematical world than the one Lean verified. The residual risk is implementation fidelity, not mathematical unsoundness. SP1's approach – formal verification of opcode constraints against the RISC-V Sail specification – attacks the same gap from the opposite direction: proving the code matches the spec rather than proving the spec is sound. Neither project has closed the full loop from spec to code to hardware. That loop is the frontier.

6.6.5 Unfinished Scaffolding

The core proving runtime – reductions, memory, folding – is maintained and tested. The rest of the system is at varying stages of completion.

The finalizer that consumes both obligation lanes and produces an outer proof is work in progress. `neo-spartan-bridge`, which would compress the folded obligations into a Spartan-style SNARK, is explicitly experimental. `neo-midnight-bridge`, which would connect Nightstream's output to Midnight's PLONK/KZG verification layer, exists as a roadmap interoperability path rather than a maintained product surface.

The project's README describes it as a research prototype. This is honest, and the honesty matters. A system that accurately describes its own incompleteness is more trustworthy than one that claims a completeness it has not achieved. The core is real. The edges are still under construction. That distinction should be preserved in any evaluation of the system.

6.6.6 What the Plumbing Reveals

The main lesson from Nightstream is not about any single cryptographic primitive. It is about alignment.

The hardest engineering in a modern folding system is not inventing a new commitment scheme or designing a new reduction. It is making the witness layout match the fold expectations. Making the reductions agree with the commitment semantics. Making the transcript bind every public input before challenges are sampled. Making the bus columns line up between the builder that writes them and the constraints that read them.

After 2023, the interesting work in proof systems moved from individual algorithmic breakthroughs to pipeline coordination. Nova was a breakthrough. HyperNova was a breakthrough. But getting fifteen crates to agree on column ordering, norm bounds, transcript sequencing, and obligation semantics – that is not a breakthrough. It is the slow, patient, unglamorous work of making a real system function. And it is where most of the time goes.

That lesson does not appear in any genealogy. It can only come from a codebase.

6.7 Circle STARKs and Stwo: A Generational Leap

While the folding lineage was evolving, a parallel revolution was happening in the STARK world. In 2024, Haboeck (Polygon Labs), Levit, and Papini (StarkWare) published Circle STARKs, and their production implementation – Stwo – became the fastest proof system ever deployed.

The key insight is a change in the algebraic structure underlying the FFT, which is the computational backbone of every STARK prover. Traditional STARKs use multiplicative subgroups of finite fields for their FFT domains. These subgroups exist in large fields (like BN254's 254-bit field), but finding smooth-order subgroups in small fields is difficult. Circle STARKs replace the multiplicative subgroup with the *circle group* of a Mersenne prime field.

Why the circle group? The Mersenne-31 field ($M31 = 2^{31} - 1$) is a prime with a special property: the circle group $C(\mathbb{F}_p) = \{(x, y) : x^2 + y^2 = 1 \text{ over } \mathbb{F}_p\}$ has order $p + 1 = 2^{31}$, which is a perfect power of 2. This enables a radix-2 FFT (the Circle FFT, or CFFT) analogous to the standard Cooley-Tukey FFT but operating over circle points. Each FRI step halves the domain using the circle group's "squaring" map, and the entire protocol adapts naturally to the circle geometry.

Why does the field size matter so much? Because 31-bit numbers fit in a single 32-bit machine word. On modern CPUs with SIMD instructions, you can process 8 or 16 M31 elements in parallel per instruction. On GPUs, the advantage is even more dramatic. Arithmetic over M31 is roughly 100 times faster than arithmetic over BN254's 254-bit field. When your proof system spends most of its time doing field arithmetic, a 100x speedup in the field operations translates almost directly into a 100x speedup in proving.

The numbers confirm this. Stwo, StarkWare's production implementation of Circle STARKs, went live on Starknet mainnet in 2025. Every Starknet block is now proven by Stwo. The benchmarks:

- **940x throughput improvement** over Stone (StarkWare's previous prover)
- **50x improvement** over ethSTARK (the academic reference implementation)
- GPU acceleration via ICICLE-Stwo adds another 3.25x to 7x on top of the CPU SIMD backend
- Sub-second recursive proving is within reach: roughly 20 milliseconds for 10,000 Poseidon hash evaluations

These are not projections. This is production performance, running on mainnet, proving real blocks with real transactions. The gap between "academic proof system" and "deployed infrastructure" has closed.

6.7.1 Why the Circle Group Matters

The phrase "circle group" sounds like an abstraction. It is not. It is a geometric fact about numbers that makes everything else possible, and it deserves a careful explanation.

Start with what came before. Traditional STARKs need a domain of points where they can evaluate polynomials – a set of evenly-spaced "sampling points" that enable the FFT. In large fields like BN254, you find these points in the multiplicative group: pick a generator g, and the powers $g, g^2, g^3, \ldots, g^{2^k}$ form a subgroup of order 2^k. These powers wrap around like hours on a clock face. The FFT works because the subgroup has smooth order (a power of 2), so the Cooley-Tukey butterfly decomposition applies perfectly.

But in small fields, this strategy collapses. The Mersenne-31 field has only $2^{31} - 2$ nonzero elements. Its multiplicative group has order $2^{31} - 2 = 2 \times 3 \times 357{,}913{,}941$. The largest power-of-2 subgroup has order just 2 – it contains only {1, -1}. You cannot run an FFT of size 2^{24} over a group of size 2. The multiplicative group of M31 is, for FFT purposes, useless.

This is where the circle enters. Consider the set of all pairs (x, y) in M31 that satisfy $x^2 + y^2 = 1$. This is the unit circle over the finite field – not a continuous curve but a discrete set of points. The key fact: this set has exactly $p + 1 = 2^{31}$ points. Not $2^{31} - 2$. Not some awkward composite. Exactly 2^{31}. A perfect power of 2.

The coincidence is not a coincidence. It is a theorem. For any Mersenne prime $p = 2^n - 1$, the circle group $C(\mathbb{F}_p)$ has order $p + 1 = 2^n$. This is because the circle group over $\mathbb{F}_p$ is isomorphic to the multiplicative group of $\mathbb{F}_{p^2}$ modulo $\mathbb{F}_p^*$ – a quotient that inherits the 2-adic structure of $p + 1$. When p is a Mersenne prime, that structure is maximally smooth: a pure power of 2.

This perfect power-of-2 structure gives you the cleanest possible FFT. The Circle FFT (CFFT) decomposes a polynomial evaluation over 2^{31} points into layers of half-size evaluations, just as the standard Cooley-Tukey FFT does over multiplicative subgroups. Each layer halves the domain. After 31 layers, you are done. No padding, no awkward leftovers, no compromises.

6.7.2 The Squaring Map: Folding a Circle in Half

The FRI protocol – the heart of every STARK's proof of low degree – works by repeatedly halving the evaluation domain. At each step, the prover combines pairs of evaluations into single values, reducing the domain by a factor of 2. After enough steps, the polynomial is so small that the verifier can check it directly.

In a traditional STARK over a multiplicative group, the halving map is squaring: the map $x \mapsto x^2$ sends a subgroup of order 2^k to a subgroup of order 2^{k-1}. Each element and its "twin" (its negation in the group) map to the same value, so the domain folds in half.

On the circle, the halving map is also a squaring – but a *geometric* squaring. The circle has a group law: you can "add" two points by a formula analogous to angle addition. The squaring map sends a point to its double under this group law. Concretely, for a point (x, y) on the circle $x^2 + y^2 = 1$, the doubling map sends (x, y) to $(2x^2 - 1, 2xy)$. This map is 2-to-1: each image point has exactly two preimages. So the circle of 2^k points folds onto a circle of 2^{k-1} points.

The visual intuition is literal. Imagine folding a circle in half. The top half maps onto the bottom half. Each point on the bottom half corresponds to two points on the original circle – one on top, one on bottom. The FRI protocol does exactly this, algebraically. At each step, the prover commits to evaluations on a circle, the verifier sends a random challenge, and the prover uses the challenge to combine each pair of evaluations (the point and its "fold partner") into a single value on the half-size circle. After $\log(n)$ steps, only a constant-size polynomial remains.

This geometric folding is why Circle STARKs achieve the same asymptotic efficiency as traditional STARKs – $O(n \log n)$ prover time, $O(\log^2 n)$ verifier time, $O(\log^2 n)$ proof size – while operating over a field where the elements are only 31 bits wide.

6.7.3 Why 31-Bit Arithmetic Is So Fast

The final piece of the Circle STARK advantage is not algebraic but architectural. It concerns the physical reality of how modern processors handle numbers.

A 64-bit CPU register holds 64 bits. A 254-bit BN254 field element requires four 64-bit "limbs" and careful carry propagation between them. A single field multiplication over BN254 involves roughly 16 limb multiplications and a cascade of additions and carries – effectively 20 to 30 machine instructions per field multiplication. This is multi-precision arithmetic, and it is inherently serial: each carry depends on the result of the previous limb multiplication.

A 31-bit M31 field element fits in a single 32-bit word. A single field multiplication is one 32-bit multiply followed by one modular reduction – and because $M31 = 2^{31} - 1$ is a Mersenne prime, the reduction

is a single addition and a conditional subtraction. Two machine instructions. The ratio is severe: 2 instructions for M31 versus 20-30 for BN254. A factor of 10 to 15, per operation.

But the advantage compounds under parallelism. A 64-bit register can hold two M31 elements side by side. An AVX-256 SIMD register holds eight. An AVX-512 register holds sixteen. A single SIMD instruction can multiply sixteen M31 elements simultaneously, producing sixteen field products in the time it takes to perform one BN254 multiplication. The theoretical throughput ratio is not 10x. It is 100x or more.

On a GPU, the effect is even more dramatic. A GPU's streaming multiprocessors are optimized for 32-bit integer and floating-point operations – the native word size of graphics workloads. M31 arithmetic maps directly onto the hardware's sweet spot. BN254 arithmetic requires emulation using multiple 32-bit operations per limb, with register pressure and carry chains that reduce occupancy (the fraction of the GPU's compute units that are actively working). In practice, Stwo's GPU backend – implemented via ICICLE – achieves 3.25x to 7x additional speedup on top of the already-fast CPU SIMD implementation. The cumulative advantage of small-field arithmetic, from instruction-level to chip-level, is why Circle STARKs proved to be not a modest improvement over traditional STARKs but a qualitative jump.

The Mersenne prime M31 is not the only small field in production. BabyBear ($p = 2^{31} - 2^{27} + 1 = 15 \times 2^{27} + 1$) offers similar 31-bit arithmetic with a multiplicative group of smooth order (2^{27} divides $p - 1$), enabling traditional multiplicative-subgroup FFTs rather than circle FFTs. SP1 uses BabyBear for its inner proof system. The choice between M31 and BabyBear is a choice between circle-group FFTs and multiplicative-group FFTs – two paths to the same destination of fast, small-field proving. Both paths converge on the same insight: the biggest optimization in proof system engineering is not a cleverer algorithm. It is a smaller number.

6.8 Real-Time Ethereum Proving

The performance revolution in proof systems has had a direct economic consequence worth pausing to appreciate, because it changes the fundamental calculus of what this technology is good for.

In December 2023, proving a single Ethereum block cost approximately $80.21. By December 2025, the cost had fallen to roughly $0.04 – a 2,000x reduction in 24 months. Airbender (ZKsync's prover) achieved $0.0001 per transfer. These numbers come from CastleLabs and Ethproofs benchmarks, and they represent a cost curve steeper than Moore's Law.

But cost is only half the story. Speed matters equally, because Ethereum produces a new block every 12 seconds. If proving takes longer than 12 seconds, the prover cannot keep up with the chain. For years, this was a distant goal. In late 2025, four teams crossed the threshold:

- **SP1 Hypercube** (Succinct Labs): Proved 99.7% of Ethereum Layer 1 blocks in under 12 seconds, using 16 NVIDIA RTX 5090 GPUs (hardware cost approximately $32,000). SP1 Hypercube uses a multilinear polynomial stack built entirely on the sumcheck protocol, with a "jagged" polynomial commitment scheme that enables pay-per-use proving.
- **ZKsync Airbender**: Achieved 21.8 million cycles per second on a single H100 GPU, proving Ethereum blocks in approximately 35 seconds. Open-source, moving toward formal verification.
- **Two additional teams** from the Ethereum Foundation's proving ecosystem demonstrated sub-12-second proving under various hardware configurations.

The Ethereum Foundation responded by declaring the speed race "operationally viable" in December 2025 and shifting its targets. The new requirements: less than 10 seconds per block, less than $100,000 in hardware, less than 10 kilowatts of power, 128-bit provable security, and proof sizes under 300 kilobytes. The pivot from speed to security signals that the raw performance problem, which dominated proof system research for five years, has been substantially solved. The next frontier is formal security guarantees – a topic we will revisit in Chapter 7.

This cost drop matters for reasons beyond Ethereum. When proving costs $80, ZK proofs are a luxury – viable only for high-value transactions or well-funded rollups. When proving costs four cents, ZK proofs become infrastructure – cheap enough to apply to every transaction, every block, every state transition. The technology moves from "expensive security upgrade" to "default operating mode." That shift was enabled almost entirely by improvements at Layer 5.

6.9 The Proof Core: Why Layers 4, 5, and 6 Are Inseparable

At this point in our tour of the seven-layer stack, a structural observation is unavoidable. The boundaries between Layer 4 (arithmetization), Layer 5 (proof system), and Layer 6 (cryptographic primitives) are not clean lines. They are gradients.

Consider the design decisions involved in building a proof system:

- The **finite field** (Layer 6) determines which arithmetic is fast. Mersenne-31 enables SIMD-friendly 32-bit operations. Goldilocks enables efficient 64-bit operations on GPUs. BN254 requires expensive 254-bit multi-precision arithmetic. The field choice propagates upward through every layer.
- The **commitment scheme** (Layer 6) determines the trust model and proof size. KZG commitments (from pairings, Layer 6) give constant-size proofs but require a trusted setup. FRI commitments (from hash functions, Layer 6) give logarithmic proofs with transparency. Ajtai commitments (from lattices, Layer 6) give post-quantum security with larger proofs. The commitment scheme shapes the proof system architecture.
- The **arithmetization** (Layer 4) determines which constraints the proof system must handle. R1CS, CCS, AIR, PLONKish – each format has different properties, and the proof system must be designed to handle the chosen format efficiently. CCS folding (HyperNova) requires the sum-check protocol. AIR proofs (STARKs) require FRI. The arithmetization and the proof system co-evolve.

These three choices – field, commitment, arithmetization – form a tightly coupled triad. Change one, and the other two must adapt. This is why we call them the "proof core": they function as a single design unit, even though our seven-layer model places them in separate layers.

The layered model is still useful for understanding. It separates concerns that are conceptually distinct: what the mathematics *is* (Layer 6), how computation is *encoded* (Layer 4), and how the encoding is *verified* (Layer 5). But the reader should understand that in practice, these layers are designed together, optimized together, and constrained by each other's choices. A proof system is not assembled from independent components like bricks in a wall. It is forged as a single alloy, where the properties of each ingredient determine the properties of the whole.

6.10 Fiat-Shamir Vulnerabilities

Every proof system we have discussed relies on a technique called the Fiat-Shamir transform to convert an interactive protocol into a non-interactive one. In the interactive version, the verifier sends random challenges to the prover. In the non-interactive version, the prover generates the challenges by hashing the protocol's transcript – all previous messages – into pseudorandom values. This eliminates the need for the verifier to be online during proving.

The Fiat-Shamir transform is simple in principle but treacherous in practice. The transcript that gets hashed must include *everything* that the verifier would have seen in the interactive version. If the prover omits anything – a public input, a commitment, a previous challenge – then the resulting hash is not a faithful simulation of the interactive protocol, and soundness can break.

6.10.1 Frozen Heart and Fiat-Shamir Vulnerabilities

Why does this matter at Layer 5 specifically? Because the Fiat-Shamir transform is the *binding* mechanism between prover and verifier. In the interactive version of any proof protocol, the verifier generates fresh random challenges that force the prover to commit before seeing what will be checked. The Fiat-Shamir transform replaces these live challenges with hash-derived challenges – but only if the hash input includes *every* public value the verifier would have seen. The transcript is the contract between the two parties. Omit a single term, and the prover can retroactively choose commitments that satisfy whatever challenges arise. The mathematical proof of soundness assumes the transcript is complete. The implementation must deliver on that assumption, term by term, or the proof of soundness is vacated.

This is not a theoretical concern. The "Frozen Heart" vulnerability class, disclosed by Trail of Bits in 2022, affected six independent implementations across three proof systems. The "Last Challenge Attack" of 2024 compromised gnark's PLONK verifier, used by multiple Ethereum rollups. In 2025, Solana's ZK ElGamal repeated the pattern. Chapter 8 catalogs these incidents in forensic detail – what was omitted, how the forgery was constructed, and what the governance implications are for on-chain verification. Here at Layer 5, the lesson is narrower but no less urgent: the gap between a proof system's mathematical specification and its implementation is where real-world attacks live.

Fiat-Shamir vulnerabilities are the "SQL injection" of zero-knowledge cryptography: a well-understood class of bug that keeps recurring because it is easy to get wrong and hard to detect by inspection. They remind us that the security of a proof system is not just a property of its mathematical design. It is a property of its implementation, its specification, and the gap between the two. This is why formal veri-

fication of proof system implementations – not just their mathematical specifications – is increasingly recognized as essential. SP1 Hypercube's formal verification of all 62 RISC-V opcode constraints against the official RISC-V Sail specification represents the state of the art in this direction.

6.11 Case Study: Midnight's Sealed Certificate

To see how these abstract proof system choices play out in a real system, consider Midnight – the privacy-focused blockchain developed by Input Output Global (IOG) for the Cardano ecosystem. Midnight's sealed certificate tells us what Layer 5 looks like when theory meets production.

6.11.1 The Proof System: Halo 2 / UltraPlonk over BLS12-381

Midnight chose a PLONK-family proof system: specifically Halo 2, an extension of UltraPlonk with custom gates and lookup tables. The curve is BLS12-381, a pairing-friendly curve with better security properties than BN254 (roughly 128-bit security vs. BN254's ~100 bits after Tower NFS advances). Although the original Halo paper (2019) used inner product arguments (IPA) that required no trusted setup, Midnight's deployment uses KZG polynomial commitments over BLS12-381, which require a Powers-of-Tau ceremony – a universal trusted setup with a 1-of-N trust assumption.

This places Midnight firmly in the "classical SNARK" camp: pairing-based, recursion-capable, not post-quantum. It is a mature, well-understood choice. The PLONK arithmetization is flexible enough to support Midnight's privacy-preserving smart contract model, where transactions carry zero-knowledge proofs of valid state transitions.

6.11.2 The Four-Phase Transaction Pipeline

Midnight's proof generation follows a four-phase pipeline that illustrates how the sealed certificate gets manufactured in practice:

Phase 1: Circuit Execution (callTx). The DApp calls a contract function. The Compact compiler has already compiled this function into a ZKIR (Zero-Knowledge Intermediate Representation) circuit. The circuit executes locally – on the user's machine – producing an "unproven transaction" that contains the execution trace but no proof.

Phase 2: Proof Generation (proveTx). The unproven transaction is sent to a local proof server running on localhost:6300. This is a separate process, launched via Docker, that generates the ZK proof. The

witness (private inputs) never leaves the client machine. The proof server runs Halo 2's prover and returns a proven transaction. This step dominates latency. This is where the certificate gets sealed.

Phase 3: Fee Balancing (balanceTx). The proven transaction is bound, balanced (fee inputs are added from the user's DUST wallet), and signed. This step is sub-second.

Phase 4: Submission (submitTx). The finalized transaction – containing the ZK proof and the public transcript of state reads and writes – is submitted to the blockchain. The node verifies the proof against the on-chain verifier key, checks that the transcript is consistent with the current ledger state, and applies the state transition. The sealed certificate has reached the audience.

6.11.3 The Performance Reality

Measured performance on Midnight's devnet reveals the cost of sealing in concrete terms:

Operation	Time	Bottleneck
Deploy (constructor circuit)	17-27 seconds	Proof generation
Circuit call (simple increment)	17-18 seconds	Proof generation
Circuit call (sealed bid)	22-24 seconds	Proof generation
Circuit call (execute escrow)	23.8 seconds	Proof generation
Balance + submit	< 1 second	Network
Failed assertion (local)	0.1-0.5 seconds	No proof needed

The pattern is clear: proof generation accounts for 95% or more of every transaction's latency. A simple counter increment (the "hello world" of smart contracts) takes 17 seconds because the Halo 2 prover must generate a full zero-knowledge proof. A more complex circuit (escrow execution) takes 24 seconds. Everything else – fee balancing, signing, network submission, on-chain verification – is negligible by comparison.

From the user's perspective, Layer 5 is a 17-to-28-second pause during which the proof server is pressing the seal into wax. The cryptography is working. The privacy is being maintained. But the user is waiting.

6.11.4 Midnight vs. the Frontier

Midnight's architecture makes different choices than the proof systems at the performance frontier. It does not use STARKs. It does not use folding. It does not use GPU acceleration (the proof server appears to be CPU-only). It does not use small-field arithmetic. These are deliberate engineering choices that prioritize maturity and correctness over raw speed.

A comparison with Ethereum is instructive:

Property	Midnight (Halo 2)	Neo/Symphony (Lattice folding)	SP1 Hypercube
Proof system family	PLONK	Folding (CCS)	STARK + sumcheck
Hardness assumption	Discrete log	Module-SIS/LWE	Collision-resistant hashing
Post-quantum	No	Yes (plausible)	Yes (inner proof)
Field	BLS12-381 (255-bit)	Goldilocks (64-bit)	M31/BabyBear (31-bit)
Composition strategy	Recursion	Folding	STARK recursion + SNARK wrap
Proof time (simple circuit)	17-18 seconds	GPU-accelerated NTT	Sub-second
Proof size	~hundreds of bytes	Larger (lattice-based)	~192 bytes (after Groth16 wrap)
Trust model	Trusted (universal SRS)	Transparent	Transparent inner, trusted outer

Midnight's 17-second proof time for a simple increment reflects BLS12-381's expensive 255-bit arithmetic, the absence of GPU acceleration, and the overhead of a full Halo 2 proof per transaction. SP1 Hypercube's sub-second proving reflects M31's cheap 31-bit arithmetic, GPU parallelism, and an architecture optimized for throughput. These are not different implementations of the same idea. They are

different points in a design space where field size, hardware utilization, and proof architecture interact multiplicatively.

None of this is meant to criticize Midnight. Its choices are appropriate for a privacy-focused system where correctness and auditability matter more than raw speed, and where the developer toolchain (Compact language, ZKIR, local proof server) provides a coherent end-to-end experience. But it illustrates how the choices made when sealing the certificate – amplified by the field and commitment choices at Layer 6 – determine the user experience at the application layer.

6.12 SNARK Recursion vs. Folding: The Full Picture

Now that we have seen both approaches in action – Midnight's recursive Halo 2 architecture and the folding genealogy leading to Neo and Symphony – we can draw a more complete picture of how they compare.

6.12.1 When Recursion Wins

Recursive proof composition is the right choice when:

- The chain of computation is short (tens of steps, not millions)
- The proof system already has a cheap verifier circuit (as Halo 2 does for its IPA-based verification)
- The application needs per-step proofs (every transaction gets its own proof, as in Midnight)
- The infrastructure is mature and the priority is correctness over performance

Recursion is conceptually simpler: each step produces a proof, and the next step verifies it. There are no accumulated instances to manage, no decider to run at the end, and no error vectors that grow with the chain length. For short chains and mature tooling, recursion is the pragmatic choice.

6.12.2 When Folding Wins

Folding is the right choice when:

- The chain of computation is long (millions of VM steps)
- Per-step cost must be minimized (the overhead of a full SNARK verification per step is unacceptable)
- The application is a zkVM or rollup that processes large batches of transactions
- Post-quantum security is a requirement (lattice-based folding via Neo/Symphony)
- Parallel proving is needed (multi-instance folding via ProtoGalaxy enables tree-structured PCD)

The asymptotic advantage of folding is clear: $O(|F|)$ prover cost per step (where $|F|$ is the step circuit size) plus roughly 1,500 gates of folding overhead (with CycleFold), compared to $O(|F| + |V|)$ for recursion, where $|V|$ is the size of the verifier circuit (potentially millions of gates). For a zkVM executing millions of RISC-V instructions, this difference is the gap between practical and impractical.

6.12.3 The Convergence

In practice, the distinction between recursion and folding is blurring. Most production systems use a hybrid: folding for the inner loop (accumulate computation steps cheaply), then a monolithic SNARK (Spartan, Groth16) as the "decider" that produces the final succinct proof. Nova uses Spartan as its decider. Mangrove (Nguyen, Datta, Chen, Tyagi, Boneh, 2024) builds a k-arity PCD tree where leaf nodes fold computation chunks and internal nodes merge folded instances, with a final SNARK for the NP statement. The boundary between "folding" and "recursion" dissolves into a pipeline where both techniques serve different stages.

The key decision variable is step count. For computations under approximately 1,000 steps, recursion with a fast inner proof system (Groth16 or PLONK) is competitive in both prover time and implementation complexity – the per-step overhead of a full recursive proof is high but amortizes over few steps. For computations exceeding 10,000 steps – the regime of zkVMs proving full program executions – folding's $O(|F|)$ per-step cost with ~1,500 gates of folding overhead dominates recursion's $O(|F| + |V|)$ per-step cost, where $|V|$ is the verifier circuit size. The crossover region between 1,000 and 10,000 steps depends on the specific constraint system, field size, and hardware: folding wins earlier on small fields (where $|V|$ is relatively large compared to $|F|$) and later on pairing-friendly fields (where recursive verification is cheap). In practice, the distinction is blurring – the dominant architecture uses folding for the inner accumulation loop and a single recursive SNARK compression as the final decider step.

6.13 The Post-Quantum Horizon

Everything we have discussed in this chapter faces an existential question: what happens when quantum computers arrive?

Shor's algorithm breaks the discrete logarithm problem in polynomial time. This means every proof system built on elliptic curve cryptography – Groth16, PLONK, Halo 2, KZG commitments, all of Nova's original elliptic-curve-based instantiations – becomes insecure. Not "might become insecure." Becomes insecure. The mathematical fact is established; only the engineering timeline is uncertain.

The NIST IR 8547 deprecation schedule targets 2035 for phasing out pre-quantum cryptography. Conservative estimates for "Q-Day" – the date when a cryptographically relevant quantum computer exists – range from 2032 to 2035. For blockchain systems with 10+ year lifespans, this means systems deployed today must either plan for migration or be built on quantum-resistant foundations from the start.

The STARK family is already partially quantum-resistant: its security rests on collision-resistant hash functions, which resist quantum attacks (though Grover's algorithm reduces their effective security, SHA-256's collision resistance drops from 128 bits to roughly 85 bits under the BHT algorithm, though this attack requires quantum random-access memory, which is widely considered physically impracticable with current technology). But the STARK-to-SNARK wrapping pipeline reintroduces quantum vulnerability at the final step, because the Groth16 wrapper uses BN254 pairings.

The lattice folding branch of the genealogy – LatticeFold, Neo, Symphony – represents the most direct path to post-quantum proof systems. Neo achieves 127-bit security under plausible lattice hardness assumptions (Module-SIS, Module-LWE), operates over the GPU-friendly Goldilocks field, and supports the full CCS constraint framework via sumcheck-based folding. Symphony extends this to production-grade performance with GPU-optimized NTTs.

The remaining gap is on-chain verification. No post-quantum on-chain verifier exists in production. Lattice-based proofs are larger than elliptic-curve-based proofs (tens of kilobytes vs. hundreds of bytes), and no blockchain has precompiled contracts for lattice operations. Closing this gap – either through lattice-friendly L1 verification or through novel compression techniques – is one of the open problems at the frontier of the field.

6.14 From Speed Race to Security Race

The story of Layer 5 over the last three years is a story of two races.

The first race was about speed. From 2022 to 2025, the question was: can you prove computation fast enough for it to matter? Can you prove an Ethereum block before the next block arrives? Can you bring the cost below a dollar, below a dime, below a penny? This race has been substantially won. Real-time proving is operational. Costs are in the single-digit cents. The hardware stack – GPUs, SIMD, and potentially FPGAs and ASICs – has been mobilized.

The second race is about security. The Ethereum Foundation's December 2025 announcement marked the pivot: the target shifted from "prove fast" to "prove with 128-bit provable security." This means not just believing the proof system is secure, but having a formal proof that a computationally bounded adversary cannot forge proofs with probability better than 2^{-128}. It means formally verifying the im-

plementation against the specification. It means accounting for the gap between the random oracle model (where Fiat-Shamir uses ideal hash functions) and reality (where Fiat-Shamir uses SHA-256 or Poseidon).

SP1 Hypercube's formal verification of all 62 RISC-V opcode constraints against the RISC-V Sail specification is a milestone in this second race. It demonstrates that production proof systems can achieve the level of formal rigor that was previously associated only with academic papers. But verifying opcodes is only the beginning. The full stack – from the Fiat-Shamir transform through the polynomial commitment scheme through the field arithmetic – must be verified end-to-end. This is a harder problem, and it is the one that will define Layer 5's trajectory over the next several years.

6.15 The Sealed Certificate

Layer 5 is where the mathematics becomes a machine. The abstract polynomial constraints from Layer 4 are sealed into a tamper-evident certificate that can travel across networks, be verified by strangers, and survive adversarial scrutiny. The certificate's properties – its size, its verification cost, its security assumptions, its quantum resistance – are determined by the proof system that seals it.

Three families of proof systems (Groth16, PLONK, STARKs) have converged into a single hybrid pipeline where each contributes its strengths. Folding schemes evolved from Nova's R1CS-specific innovation into a family of increasingly general techniques spanning CCS, multi-instance proving, non-uniform IVC, and lattice-based post-quantum security. Circle STARKs and Stwo delivered a 940x throughput improvement through the simple insight that 31-bit field arithmetic is 100 times faster than 254-bit arithmetic. Proving costs plummeted by three orders of magnitude in two years, and proving speed crossed the real-time threshold for Ethereum blocks.

But the vulnerabilities are equally real. Fiat-Shamir bugs (Frozen Heart, the Last Challenge Attack) demonstrate that the gap between a proof system's mathematical security and its implementation security is where real-world attacks live. The proof core – the inseparable triad of field, commitment scheme, and arithmetization – means that Layer 5 cannot be understood in isolation from the layers above and below it.

And we have seen a real system, Midnight, deploy Halo 2 in production with a four-phase transaction pipeline whose proof generation latency we measured. This is the state of the art for privacy-preserving blockchain computation: mathematically rigorous, practically functional, and waiting for the performance frontier to catch up.

The certificate is sealed. But we have been treating the sealing mechanism as a black box – we know it produces trustworthy certificates, but we have not examined what makes the seal unforgeable. What mathematical hardness assumptions prevent a forger from creating a convincing fake? Why do we believe these assumptions hold? And what happens if a quantum computer shatters them?

These are the questions of Layer 6. The seal works because certain mathematical problems are hard. The next chapter examines those problems – the foundations that make the entire magic trick possible.

Chapter 7: Layer 6 – The Deep Craft

7.1 The Laws That Break

Richard Feynman liked to say that the laws of physics do not change. You can test them in New York or on the moon, today or in a billion years, and you get the same answers. The constants are constant. The symmetries hold. Nature does not update its firmware.

Zero-knowledge proof systems have their own "laws of physics" – mathematical assumptions about which problems are hard to solve. These assumptions sit beneath every layer we have examined so far. The setup ceremonies of Layer 1, the constraint systems of Layer 4, the proof engines of Layer 5 – all of them rest on a handful of beliefs about the difficulty of certain computations. If those beliefs are correct, the entire tower stands. If they are wrong, it collapses – not gracefully, not partially, but completely.

Here is the uncomfortable question that Feynman would have asked, leaning forward with that half-grin that meant he had spotted something everyone else was politely ignoring: *What happens when quantum computers change these "laws"?*

Physics does not change. Mathematics does not change either. But our *assumptions* about which mathematical problems are hard – those change whenever someone invents a better attack. And a quantum

computer running Shor's algorithm is not a better attack. It is not a faster way to pick the same lock. It is a different kind of physics applied to the same mathematics, and it renders certain problems trivially easy that we have spent fifty years assuming were impossibly hard.

This chapter descends to the deepest layer of the zero-knowledge stack: the cryptographic primitives that everything else is built upon. It is about hardness assumptions, commitment schemes, finite fields, and the coming quantum reckoning. It is also about a revolution in progress – a shift from one family of mathematical foundations to another that may dissolve what looked like permanent tradeoffs.

Here the metaphor reaches its limit. We cannot avoid the mathematics. But we can make it concrete. Every abstraction in this chapter corresponds to a specific engineering choice made by real teams building real systems. When we say "the Goldilocks field," we mean a specific 64-bit prime number. When we say "Module-SIS," we mean a specific problem involving short vectors in high-dimensional lattices. The goal is not to teach the mathematics but to explain why these choices matter and what they cost.

7.2 Three Hardness Assumptions, Three Worlds

Every cryptographic system rests on a *hardness assumption*: a belief that a specific mathematical problem cannot be solved efficiently. The entire security guarantee is conditional. "This proof system is sound" really means "this proof system is sound *assuming* that problem X is hard." If someone finds a fast algorithm for problem X, the security guarantee vanishes. Not slowly. Immediately.

There is a useful way to think about this. A hardness assumption is like a combination lock. Classical computers try every combination one at a time – for a lock with a trillion trillion combinations, they will never finish. Quantum computers do not try faster. They exploit the lock's internal structure to narrow the possibilities. Shor's algorithm does exactly this to the discrete logarithm problem: it uses quantum interference to find the answer directly. The lock opens. For hash-based problems, quantum computers get a modest advantage but the lock still holds if you make it big enough. For lattice problems (Module-SIS), no one has found a quantum trick that exploits the lock's structure at all. The tumblers do not vibrate. The lock holds.

Three hardness assumptions dominate zero-knowledge cryptography. Each creates a different world of possibilities and constraints.

7.2.1 The Discrete Logarithm Problem

The oldest and most widely deployed assumption. Given a number g and a value $h = g^x$, find x. On ordinary computers, the best known algorithms require roughly 2^{128} operations for carefully chosen groups – effectively impossible. This assumption powers all elliptic curve cryptography, which in turn powers KZG commitments, Groth16 proofs, PLONK, and every pairing-based SNARK.

The DLP (Discrete Logarithm Problem – the mathematical puzzle of figuring out how many times a number was multiplied by itself to produce a given result, which is easy to state but very hard to solve) world offers deep algebraic richness. Elliptic curve groups have a bilinear pairing operation – a special function that takes two curve points and produces an element in a "target group." This pairing is the engine behind KZG polynomial commitments, which produce constant-size proofs (a single curve point, about 48 bytes) and enable constant-time verification (one pairing check). Nothing else in cryptography achieves this combination of succinctness and speed.

The cost is existential. A quantum computer running Shor's algorithm solves the discrete logarithm problem in polynomial time. Not "might solve" – *does solve*, given enough qubits. The DLP world has an expiration date. We do not know the day. But the clock is ticking, and the hands do not run backward.

7.2.2 Collision-Resistant Hash Functions

A completely different kind of assumption. A hash function takes arbitrary input and produces a fixed-size output. "Collision resistance" means it is hard to find two different inputs that produce the same output. SHA-256, BLAKE3, and Poseidon are all collision-resistant hash functions (we believe).

The CRHF world is simpler and more conservative. Hash functions require no algebraic structure – no groups, no pairings, no special number theory. This simplicity is both a strength (fewer assumptions to break) and a weakness (fewer mathematical tools to work with). FRI-based commitment schemes and STARKs live in this world. They are transparent (no trusted setup) and plausibly post-quantum, since hash functions are not broken by Shor's algorithm.

But "plausibly post-quantum" deserves scrutiny, and scrutiny reveals cracks. Grover's algorithm gives a quantum computer a quadratic speedup for brute-force search, halving the effective security level of hash preimage resistance: a 256-bit hash drops to 128-bit quantum security. More subtly, the BHT algorithm (Brassard-Hoyer-Tapp) can reduce collision resistance by a factor of three: SHA-256's 128-bit classical collision resistance becomes roughly 85-bit quantum collision resistance, though this attack requires impractical amounts of quantum random-access memory. And the FRI protocol's post-quantum security depends on the soundness of the Fiat-Shamir transform in the quantum random oracle model – a reduction that is known but carries non-tight security bounds.

The honest statement is that hash-based systems *probably* survive quantum computers with appropriate parameter adjustments, but the unqualified claim that they are "post-quantum secure" gives false confidence. Intellectual honesty demands we say: this is not yet fully understood.

7.2.3 Module-SIS (Module Short Integer Solution)

The newest and most mathematically demanding assumption. Given a matrix M over a polynomial ring, find a short nonzero vector z such that $M \cdot z = 0$. "Short" means the coefficients of z are small. The best known algorithms (both classical and quantum) require exponential time for properly chosen parameters.

Module-SIS is the foundation of lattice-based cryptography – the family that the post-quantum community has rallied around. NIST's post-quantum standards (FIPS 203, 204, and 205, published August 2024) are built on lattice problems. The assumption has been studied for over two decades, and no quantum algorithm significantly outperforms classical ones against it.

The lattice world offers a distinctive property: *module homomorphism*. An Ajtai commitment (the lattice analogue of a Pedersen commitment – a Pedersen commitment is a cryptographic method for "sealing" a number using elliptic curve arithmetic, so the committed value can be verified later but cannot be changed after commitment) satisfies the equation $\rho \cdot \mathrm{Com}(Z) = \mathrm{Com}(\rho \cdot Z)$, where ρ is a ring element. This is the algebraic structure that makes lattice-based folding schemes possible. It is weaker than what pairings provide (no bilinear map to a target group) but stronger than what hash functions provide (which have no algebraic structure at all).

The upshot is that lattice-based schemes occupy a useful middle ground: they have enough algebraic structure for folding and efficient composition, plus post-quantum security, plus transparent setup. Whether they can match the succinctness of pairing-based schemes is the open research question – and the answer is converging toward "close enough."

7.3 Four Families of Commitment Schemes

The hardness assumptions crystallize into concrete cryptographic tools called *polynomial commitment schemes*. A commitment scheme lets a prover "seal" a polynomial into a short commitment, then later prove that the polynomial evaluates to a specific value at a specific point. This is the mechanism that makes zero-knowledge proofs work – it is how the prover demonstrates knowledge without revealing the underlying data.

Four families dominate. Each inherits the properties and limitations of its underlying assumption, the way a building inherits the geology of the ground it stands on.

7.3.1 KZG (Kate-Zaverucha-Goldberg)

Built on the DLP and the bilinear pairing. The prover commits to a polynomial $p(x)$ as a single elliptic curve point $C = g^{p(s)}$, where s is a secret from a trusted setup ceremony (the "powers of tau"). To prove that $p(z) = y$, the prover produces a single group element – an evaluation proof – that the verifier checks with one pairing operation.

KZG is the gold standard for succinctness. Proof size: constant, roughly 48 bytes on BLS12-381 regardless of polynomial degree. Verification time: constant, one pairing check. These are numbers that no other scheme matches. They are, in a precise mathematical sense, optimal.

The costs are equally clear. KZG requires a trusted setup – a structured reference string generated by a multi-party computation ceremony. At least one participant must honestly destroy their secret contribution, or the entire system can be broken. The ceremony for Ethereum's EIP-4844 drew the six-figure participant count described in Chapter 2. And KZG is not post-quantum: Shor's algorithm breaks the pairing assumption along with the DLP.

KZG powers Groth16, PLONK, Marlin, and virtually every pairing-based SNARK. It is used by Midnight, Zcash, most Ethereum rollups (at the final verification layer), and the EIP-4844 blob commitment scheme.

To understand what a polynomial commitment *feels like*, consider what it accomplishes. A polynomial of degree n encodes $n+1$ independent values – it is, in a precise sense, a compressed representation of an entire dataset. A polynomial of degree one million contains a million pieces of information. KZG seals all of that information into a single elliptic curve point: 48 bytes. One point on a curve, and behind it, a million values, invisible but committed. Later, anyone can ask "what does the polynomial evaluate to at point z?" and the prover produces a single additional curve point as proof. Not a proof proportional to the polynomial's size. Not a proof that grows with the complexity of the claim. A single point. Constant size. Whether the polynomial has degree ten or degree ten million, the proof is 48 bytes.

This is, in a precise mathematical sense, miraculous. It is worth pausing to feel the weight of that claim, because no amount of familiarity should make it seem ordinary.

The miracle rests on the bilinear pairing – a function $e(P, Q)$ that takes two elliptic curve points and produces an element in a target group, satisfying $e(aP, bQ) = e(P, Q)^{ab}$. This bilinearity allows the verifier to check polynomial relationships without ever seeing the polynomial. The proof that $p(z) = y$ consists of a commitment to the quotient polynomial $q(x) = (p(x) - y)/(x - z)$. The verifier checks one

equation: $e(C - yG, H) = e(\pi, H_s - zH)$, where C is the commitment, π is the proof, and H_s is a point from the trusted setup encoding the secret s. If the equation holds, the polynomial evaluates to y at z. If it does not, the prover is lying. One equation. Two pairing evaluations. Done.

The trusted setup deserves its own intuition. Think of it as a ruler with markings at positions $s, s^2, s^3, \ldots, s^n$, where s is a secret that no one knows. The ruler is published as a sequence of elliptic curve points: $g, g^s, g^{s^2}, \ldots, g^{s^n}$. Anyone can use these markings to "measure" polynomial evaluations – to compute $g^{p(s)}$ for any polynomial p by combining the markings with the polynomial's coefficients. But no one can recover s itself, because extracting s from g^s requires solving the discrete logarithm.

The analogy is not casual. It captures the essential structure: the SRS is a set of calibrated instruments whose internal workings are opaque but whose external behavior is perfectly reliable. You do not need to know the secret to use the ruler. You need to trust that someone built the ruler honestly – that the markings correspond to genuine powers of a single unknown s, and that s was destroyed after construction. This is the trust assumption that the multi-party ceremony enforces. If even one participant destroys their contribution, the ruler is sound. If all participants collude (or are compromised), they can forge proofs. The ceremony scales trust across thousands of independent parties, diluting the assumption to its practical vanishing point.

The mathematical miracle, then, is this: pairings allow you to verify claims about a polynomial you have never seen, committed in a form that reveals nothing about its coefficients, using a ruler whose markings you cannot read but whose geometry you can trust. Constant-size commitments. Constant-size proofs. Constant-time verification. These three constants are the reason KZG has dominated practical zero-knowledge for half a decade, and they are the benchmark against which every alternative scheme is measured.

7.3.2 FRI (Fast Reed-Solomon Interactive Oracle Proof of Proximity)

Built on collision-resistant hashing alone. FRI tests whether a function (represented as a table of evaluations) is close to the evaluations of a low-degree polynomial. It works by recursive "folding" – halving the domain at each step and checking consistency via random linear combinations, with Merkle trees providing the commitment structure.

FRI proofs are transparent (no trusted setup) and plausibly post-quantum (security from hashing, not from algebraic structure). Proof sizes are polylogarithmic – $O(\log^2 n)$ in theory, typically 50 to 200 kilobytes in practice. This is orders of magnitude larger than KZG's 48 bytes, but the absence of a trusted setup and the quantum resistance are compelling compensations.

FRI is the commitment scheme inside every STARK: StarkWare's Stwo, Polygon's Plonky2 and Plonky3, SP1 Hypercube, and RISC Zero. It works best over fields with large multiplicative subgroups of order 2^k, which is why STARK-friendly fields like Goldilocks and BabyBear exist.

The limitation: FRI has no algebraic homomorphism. You cannot add two FRI commitments and get a commitment to the sum of the polynomials. This means FRI cannot support folding directly, which is why STARK systems use recursion (proving that a proof is valid) rather than folding (combining two claims into one).

The intuition behind FRI is proximity testing, and it deserves a concrete picture. Suppose you have a table of values – say, 2^{20} entries – and you claim these values are the evaluations of a polynomial of degree at most 2^{10}. In other words, you claim there exists a smooth, low-degree curve that passes through all million-odd points. FRI's job is to test that claim without reading the entire table.

Here is how it works. Imagine the claimed polynomial plotted on a graph – a smooth curve undulating through a million points on the horizontal axis. FRI asks: "Is this really a smooth low-degree curve, or is it a jagged high-degree impostor that happens to agree with a low-degree polynomial at most points?" The test proceeds by *folding*. The verifier sends a random challenge α. The prover uses α to combine pairs of evaluations: for each point x in the domain, the prover computes a new value from $f(x)$ and $f(-x)$, weighted by α. This produces a new table of half the size, over a domain of half the width. If the original function was degree d, the folded function has degree $d/2$.

Now repeat. Another random challenge. Another folding. The domain halves again. The degree halves again. Each round is a "zoom in" – the verifier is looking at the function at finer and finer resolution, checking whether the smoothness persists. If the original function was truly a low-degree polynomial, every zoomed-in version remains smooth. The degree drops: 2^{10}, then 2^9, then 2^8, all the way down to a constant. At the end, the prover reveals the final polynomial directly – it is small enough to check by inspection.

But if the original function was *not* close to a low-degree polynomial – if it was a high-degree impostor with hidden bumps – then folding amplifies the bumps. Each round of folding, guided by the random challenge, mixes pairs of evaluations in a way that smooths genuine structure but destabilizes imposture. The bumps do not cancel; they compound. By the time you have zoomed in far enough, the remaining function is visibly not low-degree. The impostor is caught.

The commitment mechanism is beautifully simple: Merkle trees. The prover commits to each round's evaluation table by hashing it into a Merkle tree and publishing the root. When the verifier wants to spot-check specific points, the prover opens Merkle paths – logarithmic-sized authentication paths from leaf to root. The security rests entirely on collision resistance of the hash function. No group structure, no pairings, no discrete logs. Just hashing.

This is why FRI replaces elliptic curves with hash functions. A Merkle tree commitment to n evaluations costs $O(n)$ hashes to build and $O(\log n)$ hashes to open a single leaf. The total proof consists of Merkle roots (one per FRI round), Merkle paths (for the queried positions), and the final constant polynomial. The result is polylogarithmic: $O(\log^2 n)$ in total size, typically 50 to 200 kilobytes. Enormously larger than KZG's 48 bytes. But transparent. Plausibly post-quantum. And built from the simplest, most conservative cryptographic primitive we have: the hash function.

The tradeoff crystallizes the philosophical divide in zero-knowledge cryptography. KZG achieves its miracle of constant-size proofs by leveraging deep algebraic structure – bilinear pairings over elliptic curves, with all the trust assumptions and quantum vulnerabilities that entails. FRI achieves transparency and quantum plausibility by abandoning that structure entirely, accepting larger proofs as the price. Neither choice is wrong. Each is a coherent answer to a different question about what you are willing to assume and what you are willing to pay.

7.3.3 IPA / Bulletproofs (Inner Product Argument)

Built on the discrete logarithm problem without pairings. Bulletproofs use Pedersen commitments – a simpler construction than KZG that does not require bilinear maps. The key insight is an inner product argument: prove that the inner product of two committed vectors equals a claimed value, using a recursive halving protocol that produces $O(\log n)$ group elements.

IPA proofs are transparent (no trusted setup, just a random group element generator). Proof sizes are logarithmic – much smaller than FRI, but not constant like KZG. Verification, however, requires $O(n)$ work – linear in the statement size. This is the main drawback: the verifier is slow.

Halo (2019) proved that IPA-based schemes support recursion without pairings, using a technique called "nested amortization" that defers expensive verification across recursion steps. This was the

conceptual breakthrough that opened the door to transparent recursive proving. Halo2 powers Zcash's Orchard protocol and influenced the design of the Pasta curves (Pallas and Vesta).

The limitation: IPA is not post-quantum. It still relies on the discrete logarithm assumption. And the linear verification cost makes it impractical for on-chain verification without wrapping in a more succinct outer proof.

The inner product argument deserves to be understood on its own terms, because it is one of the most elegant constructions in modern cryptography. The problem it solves is this: you have committed to two vectors $\mathbf{a}$ and $\mathbf{b}$ of length n, and you claim their inner product is some value c. You want to prove this claim without revealing the vectors. The naive approach would require the verifier to reconstruct the entire inner product – n multiplications, $n - 1$ additions, linear work. The inner product argument reduces this to logarithmic work.

The technique is recursive halving. Split both vectors into their left and right halves: $\mathbf{a} = (\mathbf{a}_L, \mathbf{a}_R)$ and $\mathbf{b} = (\mathbf{b}_L, \mathbf{b}_R)$. Compute two "cross-terms": $L = \text{Commit}(\mathbf{a}_L, \mathbf{b}_R)$ and $R = \text{Commit}(\mathbf{a}_R, \mathbf{b}_L)$. Publish L and R. The verifier sends a random challenge x. Both parties compute new vectors: $\mathbf{a}' = x \cdot \mathbf{a}_L + x^{-1} \cdot \mathbf{a}_R$ and $\mathbf{b}' = x^{-1} \cdot \mathbf{b}_L + x \cdot \mathbf{b}_R$. The new vectors have half the length. The new inner product claim can be derived from the old one plus the cross-terms. Repeat.

Each halving adds exactly two group elements (L and R) to the proof. For vectors of length $n = 2^{20}$ – roughly a million entries – the protocol runs for 20 rounds, producing 40 group elements. At 32 bytes per element on a 256-bit curve, that is 1,280 bytes. A proof that two million-entry vectors have a specific dot product, in 1.3 kilobytes. Not constant like KZG. But logarithmic, transparent, and no trusted setup.

This is why Halo was a watershed. Before Halo, recursive proof composition required pairings – you needed the bilinear map to verify a KZG proof inside a circuit, and that meant you needed pairing-friendly curves, which meant you needed a trusted setup. Halo demonstrated that IPA's logarithmic proofs could be verified *incrementally* across recursion steps, deferring the expensive linear-time final check. The key was nested amortization: instead of verifying the IPA proof fully at each recursion step (which would cost linear time and destroy the efficiency), Halo accumulated the verification work and deferred it to the end. This meant each recursion step added only constant overhead, and the full linear verification happened once, at the very end of the recursion chain.

The result was the first recursive proof system with no trusted setup and no pairings. It required a cycle of curves – Pallas and Vesta, the "Pasta" curves, each one's scalar field being the other's base field – but no ceremony, no toxic waste, no trust assumptions beyond the hardness of the discrete logarithm. Zcash adopted Halo2 for its Orchard shielded pool, replacing the Sprout and Sapling ceremonies with transparent recursion. The ceremony was over. The mathematics was enough.

7.3.4 Lattice / Ajtai Commitments

Built on Module-SIS. The Ajtai commitment scheme works over a cyclotomic ring $R_q = \mathbb{F}_q[X]/(\Phi(X))$. To commit to a vector Z, compute $\mathrm{Com}(Z) = M \cdot Z$ where M is a public random matrix over the ring. The binding property reduces to the Module-SIS problem: forging a commitment requires finding a short vector in a lattice.

Lattice commitments are transparent (the matrix M is generated from public randomness) and post-quantum (Module-SIS resists quantum attacks). Proof sizes are logarithmic – $O(\log n)$ ring elements, concretely 50 to 60 kilobytes for current parameterizations.

The key algebraic property: Ajtai commitments are *module-homomorphic* over the ring. For a ring element ρ and a commitment $\mathrm{Com}(Z)$, you get $\rho \cdot \mathrm{Com}(Z) = \mathrm{Com}(\rho \cdot Z)$. This is strictly richer than scalar homomorphism and is what enables lattice-based folding. The challenges used in folding are ring elements from a carefully chosen "strong sampling set" with small coefficients, which controls the norm growth of folded witnesses.

Lattice commitments power Greyhound, LaBRADOR, LatticeFold, LatticeFold+, Neo, and Symphony. They are the youngest family and the least battle-tested in production, but they are the only family that simultaneously offers post-quantum security, algebraic structure sufficient for folding, and transparent setup.

The geometry of the short vector problem rewards intuition. Imagine a forest of trees planted in a perfectly regular grid – rows and columns spaced exactly one meter apart, stretching to the horizon in every direction. You are dropped at a random point in this forest. Finding the nearest tree is trivial: you can see the grid, feel its regularity, walk to the closest intersection. Now imagine the same forest, but the grid has been distorted. The trees are no longer evenly spaced. The rows curve. The columns tilt. The local pattern near any tree looks regular, but the global structure is scrambled by a "bad" basis – a set of directions that are long, nearly parallel, and unhelpful for navigation. You can still see trees around you, but finding the *nearest* tree – the closest lattice point to your position – has become exponentially hard. You wander among trees that look close but are not closest. The geometry of the lattice hides the answer in plain sight.

This is the Closest Vector Problem, and its computational cousin the Shortest Vector Problem, on which Module-SIS rests. The matrix M in the Ajtai commitment defines the lattice. The "short vector" condition – finding z such that $M \cdot z = 0$ and each component of z is bounded by a parameter β – is the lattice analogue of finding that nearest tree. The commitment to a message m uses randomness r: commit $= M \cdot [m; r]$. Binding holds because finding two different (m, r) pairs that produce the same commitment requires finding a short difference vector – a short element in the kernel of M. If the lattice

is properly parameterized (high dimension, appropriate modulus), no efficient algorithm can find such a vector.

Why do lattices resist quantum computers? The answer is structural. Shor's algorithm exploits *periodicity*. The discrete logarithm problem has a hidden periodic structure: given g^x, the function $f(a, b) = g^a \cdot h^b$ is periodic with period related to x. Shor's quantum Fourier transform finds this period efficiently. Lattice problems have no such periodicity. The Closest Vector Problem is not periodic – it is geometric. The difficulty comes from the high dimensionality and the scrambled basis, not from any hidden algebraic cycle. Grover's algorithm provides a generic quadratic speedup for unstructured search (turning 2^{128} into 2^{64}), but lattice algorithms are not brute-force searches. The best lattice algorithms (BKZ and its variants) operate by finding short vectors in projected sublattices, and no quantum algorithm significantly accelerates this process. The lattice estimator – the standard tool for selecting parameters – accounts for Grover's quadratic speedup and still produces comfortable security margins. Neo's 127-bit post-quantum security, for instance, already incorporates the best known quantum attacks.

The Module-SIS formulation adds one more layer. Instead of vectors over a field, you work with vectors over a polynomial ring $R_q = \mathbb{F}_q[X]/(\Phi(X))$. Each "entry" in the vector is itself a polynomial – a ring element with d coefficients. This means a vector of length κ over the ring actually contains $\kappa \cdot d$ field elements, giving the lattice its high dimension. The module structure is what provides the algebraic homomorphism: because the ring has multiplication, the commitment scheme inherits module-homomorphic properties that a plain integer lattice would not provide. It is this marriage of lattice hardness and ring algebra that makes the entire folding program possible.

7.3.5 The Four Families at a Glance

Property	KZG	FRI	IPA/Bulletproofs	Lattice/Ajtai
Proof size	$O(1)$, ~48 bytes	$O(\log^2 n)$, ~50-200 KB	$O(\log n)$, ~1-5 KB	$O(\log n)$, ~50-60 KB
Verification	$O(1)$ pairings	$O(\log^2 n)$ hashes	$O(n)$ group ops	$O(\sqrt{n})$ or $O(\log n)$
Trusted setup	Yes (SRS)	No	No	No
Post-quantum	No	Plausible	No	Yes (MSIS)

Property	KZG	FRI	IPA/Bulletproofs	Lattice/Ajtai
Homomorphic	Additive	No	Additive	Module-homomorphic
Algebraic structure	Rich (pairings)	Minimal (hashes)	Moderate (DLOG)	Rich (ring ops)

7.4 The Trilemma – And Its Dissolution

The original paper presented a "cryptographic primitives trilemma": a claim that any commitment scheme can achieve at most two of three desirable properties.

1. **Algebraic functionality** – the homomorphic structure needed for folding, composition, and efficient recursive proving.
2. **Post-quantum security** – resilience against quantum computers running Shor's and Grover's algorithms.
3. **Succinctness** – small proofs and fast verification.

The trilemma positioned the four families like this:

- **KZG** achieves algebraic functionality and succinctness but lacks post-quantum security.
- **FRI** achieves post-quantum security but lacks algebraic functionality (no homomorphism, so no folding) and offers only moderate succinctness (large proofs).
- **IPA** achieves moderate algebraic functionality but lacks both post-quantum security and full succinctness (linear verification).
- **Lattice** achieves algebraic functionality and post-quantum security. Succinctness is the remaining gap.

This framing was useful as a historical snapshot. As a statement of permanent truth, it is increasingly wrong.

The lattice revolution – Greyhound (2024), LatticeFold (2024), LatticeFold+ (2025), Neo (2025), Symphony (2026) – has been systematically closing the succinctness gap. Greyhound demonstrated 50-kilobyte proofs with sublinear verification. LaBRADOR achieved 58-kilobyte proofs for large constraint systems. Symphony's high-arity folding can compress a final proof via a compact SNARK that, if in-

stantiated with a pairing-based scheme, produces constant-size output – and if instantiated with a lattice-based scheme, remains fully post-quantum.

The trilemma is better understood as a *spectrum* that is being actively compressed. The engineering challenge is real (lattice proofs are still 1000x larger than KZG), but the trajectory is clear: lattice schemes are approaching practical competitiveness, and the gap shrinks with each generation. What looked like a permanent constraint on the geometry of the design space is turning out to be an artifact of our current engineering, not a law of mathematical nature.

To see the trilemma clearly, state the three properties any polynomial commitment scheme would ideally achieve:

1. **Small proofs, constant size.** The proof should not grow with the size of the polynomial. Ideally, one group element or a fixed number of bytes, regardless of degree.
2. **Fast verification, constant time.** The verifier's work should not depend on the polynomial's complexity. Ideally, a single algebraic check.
3. **No trusted setup, transparent.** The scheme should require no ceremony, no toxic waste, no trust assumptions beyond the hardness of a mathematical problem.

No known scheme achieves all three. KZG achieves the first two but requires a trusted setup ceremony. FRI achieves the third (transparent) and offers reasonable verification (polylogarithmic), but its proofs are polylogarithmic rather than constant – orders of magnitude larger. IPA achieves the third (transparent) with logarithmic proofs (impressively small), but its verification is linear – the verifier must do work proportional to the polynomial's degree. Lattice commitments achieve the third (transparent) with logarithmic proofs, but verification is sublinear rather than constant.

The question that should keep a mathematician awake at night is: *is this trilemma fundamental?* Is there a theorem – an impossibility result, an information-theoretic lower bound – proving that no commitment scheme can simultaneously achieve constant-size proofs, constant-time verification, and transparency?

The answer, as of 2026, is no. No one has proven that the ideal PCS is impossible. The barriers are engineering barriers, not mathematical barriers. The bilinear pairing that gives KZG its constant-size miracle is a specific algebraic structure tied to elliptic curves, and elliptic curves require structured reference strings to exploit pairings. But nothing in information theory says that constant-size polynomial commitments *require* pairings. Nothing says that transparency *requires* large proofs. The ideal scheme – transparent, constant-size, constant-verification, post-quantum – remains the field's holy grail. It may not exist. But its impossibility has not been proven, and the gap between what lattice schemes achieve today and what that grail demands shrinks with every new construction. The trilemma may be less a law of nature than a confession of our current ignorance.

7.5 Small Fields

If the commitment scheme is the "which family" decision, the finite field is the "which numbers" decision. And this choice – seemingly a detail buried deep in the mathematics – turns out to determine nearly everything about performance.

A finite field is a set of numbers equipped with addition and multiplication that "wrap around" at a fixed boundary, called the *modulus* or *prime*. Every value in a zero-knowledge circuit is an element of some finite field. Every arithmetic operation is performed modulo the field's prime. The choice of prime cascades upward through every layer of the system. It is one of those decisions that seems technical and narrow when you make it, and then turns out to have been the most important decision you made.

7.5.1 The Old World: BN254 and BLS12-381

For most of the 2010s, the ZK world standardized on two large primes:

BN254 (Barreto-Naehrig curve, 254-bit prime). This was the first widely deployed pairing-friendly curve. Ethereum embedded BN254 pairing operations as EVM precompiles in 2017, making it the de facto standard for on-chain Groth16 verification. Every deployed Groth16 verifier on Ethereum – including those used by the major rollups – runs over BN254.

BLS12-381 (Barreto-Lynn-Scott curve, ~253-bit prime). Introduced later with a higher security margin and better pairing efficiency. Used by Zcash, Filecoin, Midnight, and the Ethereum KZG ceremony (EIP-4844).

Both are enormous primes – 254 and 253 bits respectively. Arithmetic on 254-bit numbers requires multiple machine words on any existing processor. A single multiplication takes several CPU instructions and cannot exploit the native 32-bit or 64-bit arithmetic units that modern hardware is optimized for.

7.5.2 The Security Erosion Problem

BN254 was originally believed to provide 128-bit security – meaning an attacker would need roughly 2^{128} operations to break the discrete logarithm. But advances in the Tower Number Field Sieve (Tower NFS) [Kim and Barbulescu, "Extended Tower Number Field Sieve," Mathematics of Computation, 2016; Guillevic, "Comparing the pairing efficiency over composite-order and prime-order elliptic curves," ACNS 2013] have revised this estimate downward to approximately 100 bits. This does not mean BN254 is broken – 2^{100} operations is still astronomically expensive – but it means the security margin is significantly thinner than designed.

This matters because BN254 is embedded in Ethereum's EVM precompiles. Changing the precompiled curves requires a hard fork. Every Groth16 verifier on Ethereum depends on BN254. The security erosion is not academic – it affects the most widely deployed zero-knowledge infrastructure in the world.

BLS12-381 is not affected by Tower NFS and retains its 128-bit security estimate. But it is also not immune to future cryptanalytic advances. And neither curve survives a quantum computer.

7.5.3 The New World: BabyBear, M31, Goldilocks

Starting around 2022, a radical idea took hold: *use much smaller primes.*

BabyBear ($p = 2^{31} - 2^{27} + 1$, a 31-bit prime). Fits in a single 32-bit machine word. Arithmetic is native on every modern CPU and GPU. Used by RISC Zero and Plonky3.

Mersenne-31 / M31 ($p = 2^{31} - 1$, a 31-bit Mersenne prime). The simplest possible arithmetic – reduction modulo a Mersenne prime is a single addition. Used by StarkWare's Stwo and Circle STARKs.

Goldilocks ($p = 2^{64} - 2^{32} + 1$, a 64-bit prime). Fits in a single 64-bit machine word. Has high 2-adicity (2^{32} divides $p - 1$), enabling efficient FFTs with domain sizes up to 2^{32}. Used by Polygon's Plonky2 and by Neo/Nightstream.

The performance impact is not incremental. It is a factor of 100. Arithmetic on 31-bit numbers is roughly 100 times faster than arithmetic on 254-bit numbers [Haboeck, Levit, and Papini, "Circle STARKs," ePrint 2024/278; confirmed by SP1 Hypercube benchmarks, Succinct Labs, 2025]. This is not algorithmic improvement – it is the raw physics of computer hardware. A 31-bit multiply is one CPU instruction. A 254-bit multiply is an entire subroutine involving carry propagation, multi-limb multiplication, and modular reduction.

This single insight – that smaller fields make faster provers – catalyzed the performance explosion in zero-knowledge proving. Circle STARKs over M31 (Stwo) achieve throughputs that were unimaginable with BN254-based systems. Plonky2 over Goldilocks enabled the first practical recursive STARKs.

But smaller fields introduce a subtlety that Penrose would appreciate for its geometric elegance. A single 31-bit field element provides only 31 bits of security against certain attacks. To achieve 128-bit security, systems use *extension fields*. An extension field is built by the same trick as complex numbers: you take a small field and add extra "dimensions" to your arithmetic, and the security grows with the dimension. The cost is slightly more expensive arithmetic per operation – but each operation now works in a larger, more secure space – enlarging $\mathbb{F}_p$ to $\mathbb{F}_{p^k}$ for some small k. In Stwo, the extension degree is 4, giving effectively 124 bits. In Neo, the extension is $\mathbb{F}_{q^2}$ over Goldilocks, giving 128 bits. The extension adds complexity but the arithmetic is still vastly cheaper than native 254-bit operations.

7.5.4 Why This Choice Is a One-Way Door

The field choice is perhaps the most consequential "one-way door" decision in zero-knowledge system design. It cascades through every layer:

- The field determines which commitment schemes are efficient (pairing-friendly fields for KZG, STARK-friendly fields for FRI, cyclotomic fields for lattice schemes).
- The commitment scheme determines which proof systems work (KZG enables Groth16/PLONK, FRI enables STARKs, Ajtai enables lattice folding).
- The proof system determines the arithmetization format (PLONK gates, AIR, CCS).
- The arithmetization determines which programs can be efficiently proved.

Changing the field after deployment means rewriting the compiler, the prover, the verifier, the standard library, and every circuit. It is not a parameter change. It is a complete system redesign.

7.6 The Quantum Threat Horizon

The question is not whether quantum computers will break DLP-based cryptography. The question is when.

7.6.1 What Shor's Algorithm Does

Shor's algorithm, published in 1994, solves the discrete logarithm problem in polynomial time on a quantum computer. Given g and $h = g^x$, it finds x using roughly $O(n^3)$ quantum gates, where n is the bit length of the group order. For BLS12-381, this requires approximately 2,500 logical qubits. Each logical qubit requires thousands of physical qubits for error correction, meaning the actual hardware requirement is on the order of millions of physical qubits – roughly three orders of magnitude beyond current devices, which have demonstrated only a few thousand physical qubits.

When a cryptographically relevant quantum computer (CRQC) exists, the consequences are immediate and total:

- Every KZG commitment ever published becomes forgeable.
- Every Groth16 proof ever verified becomes suspect.
- Every elliptic curve signature (including BLS signatures and ECDSA) breaks.
- Every Pedersen commitment loses its binding property.
- Every system built on pairing-based cryptography fails simultaneously.

The failure mode is not gradual degradation. It is a cliff. One day the lock holds. The next day every lock of that type, everywhere, opens at once.

7.6.2 The Timeline

Estimates for when a CRQC will exist vary widely:

- **Optimistic (Forrester, 2024):** "Q-Day" by 2030.
- **Conservative (academic consensus):** 2032-2035.
- **NIST IR 8547 (November 2024):** Recommends that all federal agencies deprecate pre-quantum cryptographic algorithms by 2035.

NIST's position is the most policy-relevant. In August 2024, NIST published three post-quantum cryptography standards: FIPS 203 (ML-KEM, a key encapsulation mechanism based on Module-LWE), FIPS 204 (ML-DSA, a digital signature based on Module-LWE), and FIPS 205 (SLH-DSA, a stateless hash-based signature). These are not draft standards or proposals – they are finalized, mandatory standards for federal systems.

The IR 8547 guidance document sets a deadline: by 2035, federal systems must have migrated away from RSA, ECDSA, and all DLP-based cryptography. The reasoning is straightforward: if a CRQC arrives in 2035 and a migration takes 5-10 years, you need to start migrating now.

7.6.3 The HNDL Threat

The most insidious quantum threat is not future code-breaking but present data collection. "Harvest Now, Decrypt Later" (HNDL) describes the strategy of recording encrypted communications today for decryption by future quantum computers. A Federal Reserve discussion paper (FEDS 2025-093) explicitly identified HNDL as a risk to financial infrastructure.

For zero-knowledge systems, the HNDL analogue is this: an adversary records all on-chain data – commitments, proofs, public inputs – today, waiting for a quantum computer to extract the underlying secrets. In a system like Zcash or Midnight, where commitments hide transaction amounts and sender/receiver identities, a future CRQC could retroactively de-anonymize the entire transaction history.

The concern is not theoretical. The blockchain is a permanent, public record. Nothing posted to Ethereum or any other blockchain can be deleted. Every BLS12-381 commitment, every BN254 proof, every elliptic curve public key is preserved forever, waiting. The data is patient. A quantum computer needs only to be built once.

7.6.4 Deployed Systems at Risk

Any zero-knowledge system deployed today that relies on DLP-based cryptography faces a choice:

1. **Accept the expiration date.** Acknowledge that the system's security guarantees have a finite lifespan and plan accordingly. This is the pragmatic approach for systems that do not require long-term privacy (e.g., rollups where the transactions are already publicly visible).
2. **Migrate proactively.** Begin transitioning to post-quantum primitives before a CRQC exists. This is the only option for systems that provide long-term privacy guarantees (e.g., shielded transactions, confidential identity systems).
3. **Ignore the problem.** Hope that quantum computers take longer than expected, or that "crypto-agility" will allow a rapid migration when the time comes. This is the most common approach – and the most dangerous, because "crypto-agility" in practice means "complete architectural redesign."

7.7 Lattice-Based Proving

Against this backdrop – the ticking clock, the harvest-now threat, the cliff edge – a research program has been steadily building an alternative: lattice-based zero-knowledge proof systems that provide post-quantum security without sacrificing the algebraic structure needed for efficient proving.

The progression has been fast, compressing a decade of typical cryptographic development into two years.

7.7.1 Stage 1: Greyhound (2024)

The first demonstration that lattice-based SNARKs could be practical, not just theoretical. Greyhound achieved approximately 50-kilobyte proofs with sublinear (square root of N) verification, built entirely on Module-SIS. It was a standalone SNARK, not a folding scheme – a proof of concept that lattice proofs could fit in the same order of magnitude as STARK proofs.

7.7.2 Stage 2: LatticeFold (2024)

The conceptual breakthrough. Dan Boneh and Binyi Chen adapted Nova-style folding to work over cyclotomic rings with Ajtai commitments. The key insight: Ajtai's module homomorphism is the lattice analogue of Pedersen's additive homomorphism, and it is sufficient to enable the random-linear-combination technique that makes folding work.

LatticeFold introduced three composable reductions – Π_{CCS} (constraint satisfaction to evaluation claims), Π_{RLC} (random linear combination), and Π_{DEC} (decomposition to control norm growth) – that together form a complete folding scheme for CCS (Customizable Constraint Systems).

But LatticeFold had a critical limitation. It was restricted to power-of-two cyclotomic polynomials of the form $X^d + 1$. Over the Goldilocks field, this polynomial splits completely into degree-1 factors, meaning each NTT slot has only 64-bit security – insufficient for the 128-bit target.

7.7.3 Stage 3: LatticeFold+ (2025)

A comprehensive improvement: 5 to 10 times faster prover, simpler verification circuit, shorter folding proofs, and a new purely algebraic range proof replacing LaBRADOR's more complex approach. LatticeFold+ identified three concrete parameterizations, including one over Goldilocks with the 81st cyclotomic polynomial $\Phi_{81} = X^{54} + X^{27} + 1$. This polynomial does not split into linear factors over Goldilocks; instead, it factors into degree-2 irreducibles, giving the extension field $K = \mathbb{F}_{q^2}$ with 128-bit NTT slots.

LatticeFold+ also introduced a general composition theorem: if one reduction is "phi-restricted" with restricted knowledge soundness, and another is "phi-relaxed" knowledge sound, their composition is knowledge sound. This provided the formal foundation for chaining reductions.

7.7.4 Stage 4: Neo (2025)

Neo overcame LatticeFold's power-of-two limitation directly. Its central innovation is the *rotation matrix encoding* – the "bar transform" – which represents ring elements as rotation matrices in a commutative subring of the matrix ring. The map sends a ring element a in R_q to a $d \times d$ matrix $\mathrm{rot}(a)$, and this map is a ring isomorphism. The payoff: the matrix commitment scheme becomes S-homomorphic: $\mathrm{rot}(\rho) \cdot \mathrm{Com}(Z) = \mathrm{Com}(\mathrm{rot}(\rho) \cdot Z)$.

Neo works natively over Goldilocks with Φ_{81}, giving $d = 54$, $\kappa = 16$, $m = 2^{24}$, and 127-bit post-quantum security (verified via the standard lattice estimator). The guard condition $(k+1) \cdot T \cdot (b-1) = 2{,}808 < 4{,}096 = B$ ensures that norm growth remains bounded across arbitrarily many folding steps.

7.7.5 Stage 5: Symphony (2026)

The most ambitious design. Symphony pushes folding to high arity – folding 1,024 or more instances in a single step, rather than the standard two. This eliminates the need for recursive IVC (which requires embedding hash verification inside the SNARK circuit, a major overhead). Symphony folds poly(λ) NP statements into two committed linear statements in one shot, then proves these with a compact SNARK. Symphony also introduces approximate range proofs (replacing the exact norm proofs of LatticeFold+), reducing verification complexity further. Its concrete instantiation can handle 2^{32} R1CS constraints – over four billion – in a single batch.

If Symphony's compact SNARK is instantiated with a pairing-based scheme (Groth16), the final proof is constant-size. If instantiated with a lattice-based scheme, the entire pipeline is post-quantum. This modularity is the architectural insight: separate the bulk proving (which must be post-quantum) from the final compression (which can optionally use classical tools for maximum succinctness).

7.7.6 The Key Algebraic Insight

The entire lattice folding line rests on a single algebraic fact that deserves to be stated plainly, because it is the kind of fact that sounds narrow but turns out to govern everything.

An Ajtai commitment over the ring R_q is *module-homomorphic*. This means that for any challenge element ρ drawn from a strong sampling set with small coefficients, the equation $\rho \cdot \mathrm{Com}(Z) = \mathrm{Com}(\rho \cdot Z)$ holds. This is the lattice analogue of the scalar homomorphism that makes Nova-style folding work over elliptic curves.

But the lattice version is richer. The challenge ρ is not a scalar but a *ring element* – equivalently, a $d \times d$ rotation matrix. This richer structure enables three things simultaneously:

1. **Folding with norm control.** Challenges from the strong sampling set $\mathcal{C}$ have small coefficients (in $\{-2, -1, 0, 1, 2\}$ for Neo), so the folded witness grows slowly in norm.
2. **Sum-check compatibility.** Ring evaluation claims can be verified via the sum-check protocol over the base field, connecting the commitment layer to the constraint satisfaction layer.
3. **Decomposition.** The accumulated norm can be reduced back to the base bound b via bit-decomposition, enabling unbounded recursion without norm blowup.

This algebraic trifecta – homomorphism, sum-check compatibility, and decomposition – is what makes lattice-based folding possible. It is the "deep craft" of Layer 6: not a single clever trick, but an interlocking set of algebraic properties that together provide something no other family offers. The geometry of the lattice (its distances, its short vectors, its algebraic symmetries) does the work that pairings do in the elliptic curve world, but without the quantum vulnerability.

7.8 Case Study: Midnight

To see how Layer 6 choices play out in a real system, consider Midnight – a privacy-focused blockchain built by Input Output Global (IOG), the company behind Cardano. Midnight makes every choice from the pairing-based, pre-quantum playbook. The consequences cascade through every layer.

7.8.1 The Stack

Scalar field: BLS12-381, with a ~253-bit prime modulus r. Every value in Midnight's zero-knowledge circuits – inputs, outputs, intermediate computations, token balances – is an element of $\mathbb{F}_r$.

Commitment scheme: KZG, implied by the choice of BLS12-381 (a pairing-friendly curve). The wallet SDK caches BLS parameters locally (a structured reference string from a trusted setup ceremony). Proof size is constant. Verification is a single pairing check.

Embedded curve: Jubjub, a twisted Edwards curve whose order divides r. Jubjub lives "inside" BLS12-381's scalar field, enabling efficient elliptic curve operations (point addition, scalar multiplication, hash-to-curve) within zero-knowledge circuits without the overhead of non-native field arithmetic.

Hash functions: Poseidon-family algebraic hashes, represented at the ZKIR level as opaque opcodes (`transient_hash`, `persistent_hash`, `hash_to_curve`). Algebraic hash functions are dramatically more efficient inside ZK circuits than traditional hash functions like SHA-256, because their operations (field additions and multiplications) are native to the circuit's arithmetic.

Token model: UTXO-based shielded tokens (similar to Zcash Sapling). A coin is a triple (nonce, color, value) committed to a global Merkle tree via `persistent_hash`. Nullifiers prevent double-spending. Pedersen commitments on Jubjub hide transaction values.

7.8.2 What Midnight Gets

Midnight occupies the "high algebraic functionality + high succinctness" corner of the design space. The pairing enables constant-size KZG proofs. Jubjub enables rich in-circuit elliptic curve operations (key derivation, Pedersen commitments, hash-to-curve) with native efficiency. The PLONK-like proof system compiles from a purpose-built language (Compact) through a 24-opcode instruction set (ZKIR). The standard library provides Merkle trees, shielded token circuits, and a full Zswap protocol for private token transfers.

The result is maximum algebraic functionality. Every cryptographic primitive – hashing, commitment, key derivation, signature verification – operates natively within the circuit's arithmetic. Nothing requires emulation or non-native field arithmetic.

7.8.3 What Midnight Gives Up

Post-quantum resilience: none. Every component of Midnight's cryptographic stack depends on either the discrete logarithm problem (Jubjub key derivation, Pedersen commitments) or pairing-based assumptions (KZG proof verification). Shor's algorithm breaks all of it.

The vulnerability assessment is total:

Component	Assumption	Post-Quantum Status
Proof verification (KZG)	q-SDH on BLS12-381	Broken by Shor
Jubjub key derivation	ECDLP on Jubjub	Broken by Shor
Pedersen commitments	DLP on Jubjub	Broken by Shor (binding fails)
In-circuit hashing	CRHF (Poseidon)	Weakened but likely survivable
Merkle tree roots	CRHF	Likely survivable
Nullifiers	PRF	Likely survivable

The proof system is the deepest vulnerability. Even if Midnight replaced its Pedersen commitments with hash-based constructions, and even if it switched from Jubjub key derivation to a lattice-based signature scheme, the proof verification mechanism is fundamentally tied to the BLS12-381 pairing. Changing it would require replacing KZG with a different commitment scheme, which would require changing the field, which would require rewriting the compiler, the standard library, the prover, the verifier, and the wallet SDK.

The one-way-door property is in full force. Midnight's choice of BLS12-381 is not a parameter that can be updated. It is the foundation on which every other component is built. A post-quantum migration for Midnight would not be an upgrade. It would be a new system.

7.8.4 Midnight vs. Neo: Opposite Corners

The contrast with Neo/Nightstream makes the tradeoffs vivid:

Dimension	Midnight	Neo/Nightstream
Field	BLS12-381, ~253 bits	Goldilocks, 64 bits

Dimension	Midnight	Neo/Nightstream
Ring	N/A (field-based)	$\mathbb{F}_q[X]/(\Phi_{81})$, degree 54
Commitment	KZG (pairing)	Ajtai (lattice)
Hash	Poseidon (algebraic)	Ring-SIS (lattice)
Proof size	$O(1)$ curve points	$O(\log n)$ ring elements
PQ secure	No	Yes (127-bit)
EC in-circuit	Yes (Jubjub, native)	No (would need circuit emulation)
Trusted setup	Yes (powers-of-tau)	No

Neo trades Midnight's in-circuit elliptic curve operations and constant-size proofs for post-quantum security, transparent setup, and a simpler recursive architecture (no curve cycles needed). Neither system dominates on every dimension. The question is which tradeoffs matter more for your threat model and time horizon.

For a system deployed today that needs maximum on-chain efficiency and whose privacy guarantees are measured in years (not decades), Midnight's choices are defensible. For a system that needs to protect sensitive data for 15 or more years, or that must comply with NIST's 2035 deprecation timeline, Neo's choices are the only viable path.

7.9 The Cascade Effect

The deeper lesson of Layer 6 is that it is not really a "layer" at all. It is a foundation. Every choice made here propagates upward through the entire stack with the force of mathematical necessity.

Consider Neo's parameter cascade:

```
Field: Goldilocks (q = 2^64 - 2^32 + 1)
  --> Ring: Phi_81, degree d = 54
    --> Commitment: kappa = 16 rows, m = 2^24 columns
     --> Folding: b = 2 (base), k = 12 (decomposition depth), B = 4096 (norm bound)
        --> Challenge: T = 216 (expansion factor), |C| ~ 2^125
          --> Security: 127-bit MSIS
            --> Guard: (k+1) * T * (b-1) = 2,808 < 4,096 = B
```

Change any parameter and everything downstream shifts. Use a different prime and the cyclotomic factorization changes, which changes the extension field, which changes the security level, which changes the commitment parameters, which changes the folding parameters. This is not optional coupling – it is algebraic necessity. The parameters are not chosen independently. They are derived from each other, each one a consequence of the ones above it, the way the shape of a crystal is a consequence of the geometry of its atoms.

The same cascade operates in the pairing world. BLS12-381's prime determines the Jubjub embedding. Jubjub determines which in-circuit operations are efficient. The pairing determines which commitment scheme works. The commitment scheme determines the proof system. The proof system determines the arithmetization.

And so "crypto-agility" – the ability to swap cryptographic primitives without redesigning the system – is largely a fiction for zero-knowledge systems. You cannot change the field without changing everything. The choice at Layer 6 is a one-way door, and once you walk through it, you are committed.

To make this concrete, here is the decision tree that every zero-knowledge system architect walks, whether they realize it or not.

If you choose BabyBear (31-bit prime, $p = 2^{31} - 2^{27} + 1$): You get SIMD-friendly arithmetic – four field multiplications packed into a single 128-bit SIMD instruction, eight into a 256-bit AVX2 register. Your natural commitment scheme is FRI, because BabyBear has a multiplicative subgroup of order 2^{27}, large enough for practical NTT domains. Your constraint format is AIR (Algebraic Intermediate Representation) or CCS, depending on your proof system. Your setup is transparent – no ceremony, no trust. Your proofs are large (50 to 200 kilobytes) but your prover is *fast*, because every arithmetic operation is a single machine instruction. You achieve 128-bit security via a degree-4 extension field (four BabyBear elements per extended element, giving ~124 bits). This is the path chosen by RISC Zero and Plonky3. It optimizes for prover throughput at the cost of proof size.

If you choose Goldilocks (64-bit prime, $p = 2^{64} - 2^{32} + 1$): You get native 64-bit arithmetic – one multiplication per CPU instruction, no multi-limb overhead. Your natural commitment scheme is FRI (exploiting 2-adicity of 2^{32} for large NTT domains) or lattice-based Ajtai commitments (using the 81st cyclotomic polynomial for post-quantum security). Your constraint format is CCS or R1CS. Your setup is transparent in either case. If you choose FRI, your proofs are large but your prover leverages GPU-friendly 64-bit arithmetic. If you choose Ajtai, you get post-quantum security and folding capability, with proofs in the 50 to 60 kilobyte range. This is the path chosen by Plonky2 (FRI) and Neo/Nightstream (Ajtai). It balances prover speed, proof size, and – if lattice-based – quantum resilience.

If you choose BLS12-381 (254-bit pairing-friendly curve): You get the full power of bilinear pairings – KZG commitments with constant-size proofs (48 bytes), constant-time verification (one pairing check),

and the richest algebraic structure available. Your constraint format is PLONKish gates or R1CS. Your setup requires a trusted ceremony (powers-of-tau). Your proofs are the smallest in existence. But your arithmetic is the most expensive: a single 254-bit multiplication costs a multi-limb subroutine that is 100 times slower than BabyBear's native operation. And you inherit an expiration date: Shor's algorithm will break every pairing-based proof when a cryptographically relevant quantum computer arrives. This is the path chosen by Midnight, Zcash (pre-Orchard), every Ethereum rollup's final verification layer, and the EIP-4844 blob scheme. It optimizes for proof succinctness and verifier efficiency at the cost of prover performance and quantum resilience.

If you choose Mersenne-31 ($p = 2^{31} - 1$): You get the simplest possible modular reduction – subtraction of the carry bit, because $2^{31} \equiv 1 \pmod{p}$. Your commitment scheme is FRI, adapted via Circle STARKs to work with M31's multiplicative group structure (which lacks large 2-adic subgroups but has a circle group of order 2^{31}). Your prover is the fastest in existence for STARK-based systems, because M31 arithmetic is cheaper than any other field. Your proofs are transparent and plausibly post-quantum. This is StarkWare's Stwo path – maximum prover throughput, hash-based security, no algebraic frills.

Each path is internally consistent. Each forecloses the others. You cannot start down the BabyBear path and switch to KZG midstream – the field does not support pairings. You cannot start with BLS12-381 and add post-quantum security – the algebraic structure that gives you constant-size proofs is the same structure that Shor's algorithm destroys. The decision tree is not a menu. It is a set of branching tunnels, and once you enter one, the others seal behind you.

7.10 Algebraic vs. Traditional Hash Functions

One more Layer 6 choice deserves attention, because it illustrates how deeply the primitive selection affects practical performance.

Traditional hash functions (SHA-256, BLAKE3, Keccak) are designed for speed on general-purpose hardware. Their internal operations – bitwise rotations, XOR, addition with carry – are cheap on CPUs but extremely expensive inside zero-knowledge circuits, because the circuit's native operations are field additions and multiplications. Proving a single SHA-256 computation inside a SNARK requires tens of thousands of constraints.

Algebraic hash functions (Poseidon, Poseidon2, Rescue, Griffin) are designed for the opposite environment. Their internal operations are field multiplications and exponentiations – exactly the operations that are native to zero-knowledge circuits. A Poseidon hash inside a circuit costs hundreds of constraints instead of tens of thousands.

The performance difference is 100x or more. This is why every system that does significant hashing inside circuits (Merkle tree verification, Fiat-Shamir challenges, commitment randomness) either uses algebraic hashes or pays an enormous performance penalty.

But algebraic hashes are newer and less studied than SHA-256 or BLAKE3. Their security rests on assumptions about the difficulty of algebraic attacks (Grobner basis computations, interpolation attacks) that have not endured decades of cryptanalysis. Poseidon, in particular, has seen several parameter revisions in response to improved attacks.

The choice between algebraic and traditional hash functions is itself a Layer 6 decision that cascades upward. Midnight uses Poseidon-family hashes (maximizing in-circuit efficiency at the cost of less mature security analysis). STARK-based systems can use either, but algebraic hashes dramatically reduce the size of the verification circuit when recursion or wrapping is needed.

7.11 The Structural Advantage of Lattices

One insight from the lattice revolution deserves emphasis because it is easy to miss amid the parameter details: **lattice-based schemes are architecturally simpler** than their pairing-based predecessors, not just quantum-resistant.

Pre-quantum recursive proof systems (exemplified by Zexe) require:

- **Cycles of elliptic curves.** To verify a proof inside another proof, the verifier's field arithmetic must be efficient in the prover's circuit. This requires two curves whose scalar fields are each other's base fields – a "cycle." Finding such cycles constrains parameter choices severely, and arithmetic on the second curve is typically 2x or more expensive.
- **Non-native field arithmetic.** When the proof system operates over one field but the verifier checks computations in another, every operation must be emulated using multi-precision arithmetic inside the circuit. This is a major source of overhead.
- **Multiple structured reference strings.** Each curve in the cycle needs its own trusted setup, doubling the ceremony burden.

Lattice-based folding eliminates all three requirements. Neo operates over a single ring R_q. The rotation matrix encoding makes everything native to one algebraic structure. Recursion via folding requires no curve cycles and no non-native arithmetic. The setup is transparent (public random matrix).

This simplification is not cosmetic. Fewer moving parts mean fewer places for bugs, fewer parameters to choose and validate, fewer assumptions to audit. The lattice path is not only quantum-resistant – it is *simpler*. And in cryptographic engineering, simplicity is not a luxury. It is a security property.

7.12 Maturity and Readiness

As of early 2026, the picture looks like this:

Deployed and battle-tested: KZG (BN254 and BLS12-381), FRI/STARK (Goldilocks, BabyBear, M31), IPA/Bulletproofs (Pasta curves). These power every production ZK system – Ethereum rollups, Zcash, Midnight, Starknet.

Peer-reviewed and prototyped: LatticeFold (ASIACRYPT 2025, presentation by Boneh and Chen), LatticeFold+ (CRYPTO 2025). Neo has an active implementation in Rust (the Nightstream repository, 15 crates). Concrete benchmarks are emerging but sparse.

Proposed and promising: Symphony (ePrint 2025/1905, no implementation yet). The high-arity folding concept is validated theoretically but awaits engineering.

Standards in place: NIST FIPS 203/204/205 (August 2024) standardize lattice-based key encapsulation and signatures. No standard yet exists for lattice-based zero-knowledge proof systems, but the parameter selection methodology (the lattice estimator) is well established.

The adoption trajectory suggests lattice-based proof systems will move from research prototypes to production-ready systems in 2026-2027, with Neo/Nightstream among the first to target production deployment.

7.13 The One-Way Door

Layer 6 is unlike any other layer in the stack. Layers above it can be upgraded, swapped, and optimized. A rollup can change its sequencer, rewrite its compiler, switch arithmetization formats, even adopt a new proof system – all without changing the cryptographic foundations. But the foundations themselves are effectively permanent.

For architects making this one-way decision, the following rubric distills the tradeoffs:

Field	Size	PQ Status	Commitment Options	Sweet Spot
BN254	254-bit	Quantum-vulnerable	KZG (cheapest EVM verification)	Legacy Ethereum rollups; Groth16 wrapper

Field	Size	PQ Status	Commitment Options	Sweet Spot
BLS12-381	253-bit	Quantum-vulnerable	KZG (higher security margin)	Privacy systems needing pairings (Midnight, Zcash)
BabyBear	31-bit	Hash-PQ; lattice-PQ	FRI, Ajtai	Maximum prover speed; RISC-V zkVMs (SP1, RISC Zero)
Mersenne-31	31-bit	Hash-PQ	FRI (Circle STARK)	Fastest arithmetic; Stwo/Starknet ecosystem
Goldilocks	64-bit	Hash-PQ; lattice-PQ	FRI, Ajtai	Balance of speed and precision; Neo/Nightstream

The finite field determines the commitment scheme. The commitment scheme determines the proof system family. The hardness assumption determines the security lifespan. These choices are made once, at the beginning, and they propagate upward through every component with the inexorability of mathematical structure.

This is why the quantum threat is not a problem that can be deferred until quantum computers actually exist. A system deployed in 2026 with BN254 foundations will still be running in 2036. If a CRQC arrives in 2035, that system will have spent its final years accumulating a public record of commitments and proofs that can be retroactively broken. The HNDL threat means the privacy guarantees were never real – they were deferred revelations, secrets written in ink that merely required a light that had not yet been invented.

The lattice revolution is a construction project, not an academic exercise. It is building new foundations that can support the same architectural weight as pairing-based cryptography – folding, recursion, efficient composition – without the quantum expiration date. The trilemma that seemed permanent is being dissolved not by discovering new mathematics but by engineering better constructions from mathematics that has existed for decades.

The laws of physics do not change. But our understanding of which mathematical problems are hard does change, and a quantum computer represents a discontinuous shift in that understanding. The systems that survive will be the ones whose foundations were chosen with that shift in mind.

The physical laws are set. The field is chosen, the commitment scheme determined, the hardness assumption staked. Everything from here upward – the proof system, the arithmetization, the language, the setup – inherits the possibilities and constraints of this foundation. But none of it matters until someone checks the proof. Layer 7 is where the mathematics meets its audience: on a blockchain, in a smart contract, through a governance structure that can override everything we have built. The next chapter examines the verdict – and the uncomfortable truth that the audience's judgment is only as trustworthy as the institution that seats them.

Chapter 8: Layer 7 – The Verdict

"The audience can be deceived – or worse, someone can replace the audience entirely."

8.1 The Social Layer

Every layer of the zero-knowledge stack we have examined so far – the ceremony, the language, the witness, the arithmetization, the proof system, the cryptographic primitives – converges on a single moment: a piece of software reads a proof and says *yes* or *no*.

That piece of software is the audience. We established this in Chapter 1: the verifier is the audience, the entity that watches the trick and renders its verdict. On Ethereum, the audience is a smart contract. On Midnight, it is a node. On a private enterprise chain, it might be a service running in a data center. Whatever form it takes, the verifier is the point where all the private magic becomes a public verdict.

And here is the uncomfortable truth that most explanations of zero-knowledge proofs prefer to gloss over: the audience can be replaced. Not by breaking the cryptography. Not by forging a proof. By something much simpler.

By changing the software.

If three people on a governance multisig can upgrade the verifier contract to one that accepts every proof – or no proofs, or only their proofs – then the 128-bit security of the proof system, the million-dollar

ceremony, the carefully audited circuits, all of it becomes decorative. The math does not protect you from the admin key.

This chapter is about what happens after the proof is generated. It is about gas costs, data availability, implementation bugs, governance attacks, and the social structures that determine whether the cryptographic guarantees from Layers 1 through 6 actually reach the people they are supposed to protect.

Layer 7 is where cryptography meets politics. And politics, as a rule, wins.

Layer 7 carries four distinct responsibilities, and this chapter treats each in turn. First: the *economics* of rendering a verdict — what does verification cost, and who pays? Second: *implementation vulnerabilities* that can corrupt the verdict — Fiat-Shamir transcript bugs that enable proof forgery. Third: *governance structures* that can override the verdict — multisig attacks, upgrade mechanisms, and the social layer above the math. Fourth: *aggregation and data availability infrastructure* that sits between the prover and the verifier — SHARP, blob economics, and the emerging DA marketplace. These four concerns are operationally convergent — they all determine whether the audience's verdict is trustworthy — but they are logically distinct. A system can have perfect verification economics and catastrophic governance. Separating the concerns makes the trust analysis sharper.

8.2 The Price of a Verdict

Let us start with money, because money clarifies.

A Groth16 proof verification on Ethereum uses the BN254 elliptic curve pairing precompiles introduced in the Byzantium hard fork (2017) and made cheaper by the Istanbul upgrade's EIP-1108 (2019). The gas cost breaks down as follows:

Component	Gas Cost
Pairing check (4 pairings via EIP-1108)	181,000
Calldata (256-byte proof)	4,096
EVM scaffolding	~1,600
Per public input	~7,160 each
Total (fixed, no public inputs)	**~207,700**

The formula is roughly $(181 + 6L) \times 1{,}000$ gas for L public inputs. At typical Ethereum gas prices and ETH valuations, this works out to somewhere between fifty cents and two dollars per verification. Call it a dollar.

One dollar. To check a proof that summarizes thousands, or millions, of computations. That is the economic engine of the entire zero-knowledge rollup industry. It is also worth noting: the cost of *rendering a verdict* on an arbitrarily complex computation is effectively fixed. The computation can be ten steps or ten billion. The verdict costs the same.

But notice what that dollar buys. It buys a *Groth16* verification. Groth16 requires a trusted setup (Layer 1), uses elliptic curve pairings on BN254 (Layer 6), and produces the smallest proofs in the field – three group elements that fit in a tweet. The cheapness of the verdict is not free. It is subsidized by decisions made five layers below.

What about STARKs? The paper being revised presents STARK verification as expensive – two to five million gas – and contrasts this with SNARK cheapness. This framing was arguably accurate in 2022. It is misleading in 2026. The reason is simple: nobody posts raw STARKs to Ethereum.

The actual production pipeline looks like this:

1. Generate a STARK proof (transparent, no trusted setup, large – hundreds of kilobytes).
2. Recursively compress the STARK through multiple rounds.
3. Wrap the final compressed STARK inside a Groth16 proof.
4. Post the Groth16 proof to Ethereum.

Starknet's SHARP (Shared Prover) does this. Succinct's SP1 does this. Polygon's CDK does this. The verifier contract on Ethereum sees Groth16 in every case. The "STARK path" and the "SNARK path" converge at the courthouse door.

The actual cost differential between the two approaches, then, is not the 10-25x that a naive comparison of raw STARK versus Groth16 verification would suggest. It is closer to 2x – the overhead of the wrapping step, amortized across many proofs. The inner proof system matters enormously for prover economics (speed, hardware requirements, parallelizability), but the on-chain verification cost is nearly identical.

There is a throughput ceiling too. An Ethereum block has a 30-million-gas limit (raised from the historical 15 million, with a 45-million effective target under various proposals). At 207,700 gas per Groth16 verification, you can fit roughly 150 to 225 verifications per block. That sounds like a lot, until you realize that each verification corresponds to a batch of rollup transactions. If Ethereum hosts 50 rollups and each wants to verify once per block, they consume less than a quarter of the block's capacity. But if we want real-time proving (verification every L1 slot), with hundreds of rollups and bridges, the verification gas budget starts to matter.

FFLONK, an alternative to Groth16, costs roughly 236,000 gas per verification – slightly more, but with the advantage of a universal trusted setup (one ceremony works for all circuits, unlike Groth16's per-circuit setup). The gas difference is marginal. The governance and operational difference – not needing a new ceremony for each circuit – is substantial.

8.2.1 The Verification-Data Seesaw

Before March 2024, the dominant cost of running a ZK rollup on Ethereum was not verification. It was data availability. Posting transaction data (or state diffs) as calldata cost roughly 16 gas per byte. A typical rollup batch might include hundreds of kilobytes of data, costing millions of gas – dwarfing the ~200,000 gas for the proof check.

EIP-4844, deployed in the Dencun upgrade on March 13, 2024, changed this calculus fundamentally. It introduced "blob transactions" – a new data type designed specifically for rollup data. Each blob contains 4,096 field elements of 32 bytes (~128 KB), with a target of 3 blobs per block and a maximum of 6. Critically, blobs have their own fee market, separate from Ethereum's execution gas market, operating under a blob-specific EIP-1559 mechanism.

The result: rollup data costs dropped by 10-100x overnight. Blob fees settled near zero because demand was well below the 3-blob target – as of mid-2024, only about 34% of Ethereum blocks contained any blobs at all, and the average was 1.33 blob transactions per block.

But Ethereum did not stop at EIP-4844. Two subsequent upgrades expanded DA capacity further:

- **Pectra** (May 2025): Doubled blob targets from 3 to 6, and maximum from 6 to approximately 9.
- **Fusaka** (December 2025): Introduced PeerDAS (Peer Data Availability Sampling), implementing a distributed sampling scheme that raised the blob target to 14 and maximum to 21 – an 8x increase in DA capacity over the original EIP-4844 specification.

The seesaw has tipped. With blob fees near zero and DA capacity expanding rapidly, the ~200,000 gas verification cost has become the *dominant* L1 settlement expense for many ZK rollups. This inversion matters because it changes what is worth optimizing. Before EIP-4844, the rational investment was in compression (minimizing data). After EIP-4844, the rational investment is in proof aggregation (amortizing verification across more transactions per batch) and in cheaper verification schemes.

8.2.2 Beyond Ethereum: The DA Marketplace

Ethereum is not the only source of data availability. A marketplace has emerged:

Celestia charges roughly $0.07 per megabyte for data availability, compared to Ethereum's blob cost of roughly $3.83 per megabyte (when blobs are priced above the floor). Celestia achieves this by being a

purpose-built DA layer – it provides data ordering and availability guarantees without executing any transactions. The intellectual lineage traces directly to Mustafa Al-Bassam's LazyLedger (2019), which proposed a blockchain that does nothing but guarantee data is available and ordered, leaving execution to sovereign rollups that interpret their own transaction rules.

EigenDA V2 targets 100 megabytes per second of throughput – roughly two orders of magnitude more than Ethereum's native DA capacity. It achieves this by leveraging Ethereum's security through restaking (EigenLayer), where validators stake ETH to back DA guarantees.

Avail offers a third alternative, with its own DAS-based light client verification model.

The choice between these DA layers is not purely technical. A rollup that uses Celestia for DA instead of Ethereum blobs trades Ethereum's full consensus security for lower costs. This is a Layer 7 governance decision with Layer 6 security implications: the data availability guarantee is only as strong as the weakest link in the DA provider's consensus mechanism.

8.2.3 Data Availability

The term "data availability" is one of those phrases that sounds self-explanatory and is not. It does not mean "the data exists somewhere." It does not mean "the data is stored on a server." It means something specific and testable: if you send a transaction to a rollup, can any participant in the world reconstruct the rollup's complete state using only publicly available data?

If yes, the rollup has data availability. Anyone can verify that the rollup operator is honest by replaying all transactions from genesis and checking that the claimed state matches the computed state. If no – if some of the data is withheld, stored only on the operator's private servers, or available only to a privileged set of participants – then the operator could cheat and nobody would know. The operator could include a transaction that steals every user's funds, prove that the resulting state transition is "valid" (because the ZK proof only proves that *some* valid transition occurred), and nobody could challenge it because nobody can see the inputs.

This is the critical subtlety that connects data availability to zero-knowledge proofs. A ZK proof proves that a state transition was computed correctly. It proves that if you start from state S and apply transactions T, you arrive at state S'. What it does *not* prove – what it *cannot* prove, by design – is what state S actually was. The proof attests to the correctness of the computation, not the availability of the inputs. If the operator claims the starting state was S but actually started from a fabricated state S_fake, the ZK proof will happily prove that the transition from S_fake was computed correctly. Without DA, nobody can verify the starting point.

Data availability is the anchor. The ZK proof is the chain. Without the anchor, the chain secures nothing.

The three DA strategies represent different points on the cost-security tradeoff:

Ethereum calldata is the oldest and most expensive approach. Transaction data is posted directly as calldata in Ethereum L1 transactions, stored permanently by every full node, and protected by Ethereum's full consensus security. The cost is high – 16 gas per byte of calldata – but the guarantee is absolute: if Ethereum's consensus is secure, the data is available. This was the only option before March 2024, and it made rollup operations expensive enough that most of the early rollup economics were dominated by DA costs rather than verification costs.

Ethereum blobs (EIP-4844 and successors) are the middle ground. Blob data is posted to Ethereum and protected by Ethereum's consensus during a pruning window (currently approximately 18 days), after which nodes may discard it. The data is available long enough for any challenge period to complete, and it is significantly cheaper than calldata because blobs have their own fee market and do not compete with execution gas. This is the default choice for most production rollups in 2026.

External DA layers (Celestia, EigenDA, Avail) are the cheapest option with a different trust model. The data is posted to a separate blockchain or protocol that specializes in data ordering and availability guarantees. The cost can be 10-100x lower than Ethereum blobs. The tradeoff is that the DA guarantee depends on the external protocol's consensus and validator set, not Ethereum's. A rollup using Celestia for DA inherits Celestia's security assumptions. If Celestia's validator set colludes or fails, the rollup's data may become unavailable even though Ethereum itself is functioning correctly.

The choice of DA strategy is, in practice, one of the most consequential governance decisions a rollup team makes. It determines the rollup's operating cost, its security model, its relationship to Ethereum's consensus, and its vulnerability to the DA-saturation attacks discussed later in this chapter. It is a Layer 7 decision with implications that cascade through every layer below.

8.3 When the Transcript Lies: Fiat-Shamir Vulnerabilities

Chapter 6 introduced the Fiat-Shamir transform as the mechanism that seals Layer 5 proofs into non-interactive certificates, and flagged the binding requirement: the hash must include every public value the verifier would have seen. This section examines what happens when that requirement is violated in production – and why the consequences are uniquely catastrophic at Layer 7, where on-chain verifiers are permissionless and exploitation is automated.

The Fiat-Shamir heuristic is the mechanism that converts an interactive proof (where the verifier asks random questions in real time) into a non-interactive one (where the prover simulates the verifier's

questions using a hash function). Every non-interactive ZK proof deployed on a blockchain uses Fiat-Shamir. It is the invisible thread that holds the entire verification model together.

And it is the single most dangerous implementation surface in the entire stack.

8.3.1 Frozen Heart (2022)

In April 2022, Trail of Bits disclosed a vulnerability class they called "Frozen Heart" (a backronym: Forging Of Zero kNowledge proofs). The core error was simple. Devastatingly simple. Multiple independent implementations of ZK proof systems omitted public inputs from the Fiat-Shamir hash computation.

The implementations affected were not obscure academic prototypes. They were production-grade libraries used by real projects:

- **Dusk Network** (PLONK implementation)
- **Iden3/SnarkJS** (Groth16, used by Circom)
- **ConsenSys/gnark** (PLONK implementation)
- **ING Bank's zkrp** (Bulletproofs)
- **SECBIT Labs' ckb-zkp** (Groth16)
- **Adjoint Inc.'s bulletproofs** (Bulletproofs)

Six implementations. Three different proof systems. Four different organizations. All made the same mistake: they left the public inputs out of the hash that generates the verifier's challenges.

The consequence is total soundness failure. A malicious prover can forge proofs for *arbitrary false statements*. Not with some small probability. With certainty. The "proof" passes verification because the challenges are no longer bound to the specific statement being proved. The sealed certificate attests to nothing. It is wax without an impression.

The rule that was violated is not subtle: the Fiat-Shamir hash must include *all* public values from the ZK statement and *all* public values computed during the proof. Every commitment, every public input, every piece of data that the verifier would have seen in the interactive version must go into the hash. Omit any of it, and the binding between challenge and statement dissolves.

8.3.2 The Last Challenge Attack (2024)

The Last Challenge Attack, discovered during an audit of Linea's PLONK verifier in the gnark library, is a more surgical variant of the same disease. In KZG-based proof systems, the verifier often batches multiple polynomial evaluations using a random "batching challenge" derived via Fiat-Shamir. The Last Challenge Attack exploits the case where this batching challenge is computed from a *truncated* transcript – one that excludes the evaluation proofs themselves.

The attack is elegant in the way that a perfectly executed heist is elegant. The malicious prover:

1. Sets arbitrary (false) public inputs and proof components.
2. Computes the batching challenge from the truncated transcript.
3. Solves a linear system for the missing evaluation proofs.
4. The vulnerable verifier accepts the forged proof with probability 1.

Not "with high probability." With certainty. The forged proof is deterministically constructed to pass verification. The audience has been compromised not by force but by omission – a single value left out of a hash, and the entire edifice of mathematical certainty collapses.

The gnark advisory (GHSA-7p92-x423-vwj6) confirmed the vulnerability. The fix was straightforward: compute the batching challenge only *after* all evaluation proofs are included in the transcript. But the vulnerability existed in a production-quality library used by multiple rollup teams.

8.3.3 Solana ZK ElGamal (2025)

The pattern repeated in 2025 on Solana, where the ZK ElGamal implementation was found to have a Fiat-Shamir transcript that omitted the prover's challenge from the hash computation. Same class of error. Same catastrophic consequence. The same lesson, unlearned for the third time.

8.3.4 The Pattern

These are not isolated incidents. They are symptoms of a structural problem: the Fiat-Shamir heuristic is easy to describe ("hash everything the verifier would see") and remarkably easy to get wrong in implementation. The specification says "include all public values." The implementation omits one, because it seemed redundant, or because it made the code cleaner, or because the developer did not understand *why* it needed to be there.

For on-chain verifiers, this class of vulnerability is uniquely dangerous. An on-chain verifier is permissionless – anyone can submit a proof. If the verifier's Fiat-Shamir transcript is incomplete, exploitation is automated and instantaneous. There is no human in the loop to notice that something looks wrong. The forged proof passes the smart contract's checks, the state transition is accepted, and the attacker drains whatever value the rollup is protecting.

Every on-chain SNARK verifier – Groth16, PLONK, FFLONK, any KZG-based scheme – must be audited specifically and primarily for Fiat-Shamir transcript completeness. This should be the first check in any security review, not an item buried in a general audit report.

8.4 Governance: The Achilles Heel

If Fiat-Shamir bugs are the most exploited *implementation* vulnerability in zero-knowledge systems, governance is the most exploited *architectural* vulnerability. And unlike Fiat-Shamir bugs, governance vulnerabilities cannot be fixed with better code review. They are features, not bugs.

Here the story changes genre. Until now, we have been watching a technical narrative – mathematicians and engineers building increasingly sophisticated proof systems. Now the camera pulls back, and the audience discovers it has been watching a different show than it thought. The threats at Layer 7 are not mathematical. They are human.

8.4.1 The Beanstalk Flash Loan Attack ($182M, April 2022)

Beanstalk was a permissionless stablecoin protocol – a DeFi project built on the idea that an algorithmic stablecoin (called Bean) could maintain its dollar peg through a credit-based system of debt, deposits, and incentive cycles. The protocol had attracted over $100 million in total value locked. It had a community. It had audits. It had a governance mechanism that allowed holders of the protocol's internal Stalk token to propose and vote on changes to the system's parameters, its contracts, its entire economic logic.

The mathematics were sound. The smart contracts were audited. The governance mechanism was functioning exactly as designed.

That last sentence is the important one. Remember it.

Beanstalk's governance had one feature that seemed reasonable at the time: the emergency commit threshold. If a proposal attracted more than two-thirds of total Stalk voting power, it could be executed immediately – no waiting period, no time lock, no multi-day deliberation window. The designers' reasoning was practical: if a supermajority of stakeholders agreed on something, why force them to wait? Speed was a feature. In a fast-moving DeFi market, the ability to respond quickly to exploits or market conditions was considered an advantage.

On April 17, 2022, at approximately 12:24 UTC, someone demonstrated why speed is also a weapon.

The attacker – whose identity remains unknown to this day – began by taking flash loans from three decentralized lending protocols: Aave, Uniswap V2, and SushiSwap. A flash loan is a peculiar instrument unique to programmable blockchains: it allows you to borrow any amount of money, provided you repay it within the same transaction. If you cannot repay, the entire transaction reverts as if it never happened. The borrowing cost is essentially zero – just gas fees and a small protocol fee. There is no

credit check. There is no collateral. There is no application form. You simply ask for the money, use it, and return it, all within a single atomic operation that takes seconds.

On this day, the attacker borrowed approximately $1 billion in assets. One billion dollars, for thirteen seconds.

With the borrowed capital, the attacker swapped into Beanstalk's liquidity pools, acquiring enough of the protocol's Stalk and Seed tokens to control over 67% of total governance voting power. This was not a theoretical majority. It was an absolute supermajority – enough to clear the emergency commit threshold.

Then came the proposals. BIP-18 was the payload: a governance proposal whose code, when executed, would transfer all of Beanstalk's protocol reserves – every Bean, every LP token, every asset in the Silo – to a wallet controlled by the attacker. The code was not hidden. It was right there on the blockchain, readable by anyone who looked. But governance proposals are submitted and voted upon, not scrutinized line by line in the seconds between submission and execution, and nobody was watching for a proposal backed by a billion dollars of borrowed voting power.

BIP-19 was the other proposal: a donation of $250,000 to the Ukraine war relief wallet. Whether this was misdirection, moral compensation, ironic commentary, or simply a way to make the governance transaction look routine is a question the attacker left permanently unanswered. It remains one of the small, unsettling details that elevate this from a theft to a performance.

The attacker voted on BIP-18 with the borrowed supermajority. The emergency commit threshold was cleared. The governance system did what governance systems do: it executed the will of the majority. The protocol's reserves flowed from the Silo to the attacker's address. The attacker unwound the liquidity positions, converted the assets, repaid the flash loans to Aave, Uniswap, and SushiSwap – in full, with fees – and pocketed the difference.

Thirteen seconds. Borrow, vote, execute, extract, repay. The entire heist was a single atomic transaction on Ethereum. If any step had failed – if the flash loan had been too small, if the voting power had been insufficient, if the repayment had come up short – every step would have reverted, and the blockchain would have recorded nothing. But no step failed. The transaction succeeded. The money was gone.

Total protocol loss: $182 million in value destroyed. Net extraction by the attacker: approximately $77 million in non-Bean assets – the portion that had real market value independent of the now-collapsed protocol. The Bean stablecoin depegged immediately and never recovered. The protocol's entire treasury was emptied in a single block.

The root cause was not a code vulnerability. No contract was exploited in the traditional sense. No buffer was overflowed. No access control was bypassed. No reentrancy was triggered. The governance

mechanism worked exactly as designed. It accepted a vote from a stakeholder with a supermajority. It executed the proposal that the supermajority approved. It transferred the funds that the proposal specified. Every line of code behaved correctly.

The system was not broken. The system was *used*.

The lesson lands differently depending on who you are. If you are a protocol designer, the lesson is about time locks and minimum voting periods and the danger of emergency execution without delay. If you are a governance theorist, the lesson is about the difference between ownership and rental – the attacker did not own the voting power; he rented it for the cost of a flash loan fee. If you are building a ZK rollup with token-weighted governance over an upgradeable verifier contract, the lesson is existential: governance that can be rented by the hour is governance that can be captured in seconds. The flash loan is the instrument. The vulnerability is the assumption – the assumption that token holders are stakeholders, that voting power reflects long-term commitment, that the people who hold the keys today will hold them tomorrow. Flash loans dissolve that assumption into nothing. For the thirteen seconds that matter, anyone with gas money is a supermajority stakeholder.

8.4.2 The Tornado Cash Governance Attack (May 2023)

Tornado Cash was a privacy protocol built on zero-knowledge proofs – the very technology this book describes. It used ZK proofs to break the on-chain link between depositors and withdrawers. You deposit ETH into a pool, receive a cryptographic note, and later withdraw from the pool using a ZK proof that demonstrates you possess a valid note without revealing which deposit was yours. The cryptography was elegant, well-audited, and provably sound. The protocol's privacy guarantees were genuine. Its governance was controlled by a DAO with TORN token voting, and the governance was not.

The attack, when it came in May 2023, unfolded like a stage magic trick – not the kind where a rabbit appears from a hat, but the kind where the audience watches the magician's right hand while the left hand replaces the entire stage.

To understand the trick, you need to understand two pieces of Ethereum infrastructure that most users never think about.

The first is `CREATE2`. On Ethereum, when you deploy a contract, it gets an address. Normally, this address is derived from the deployer's address and a nonce (a sequential counter), so it is effectively unpredictable. `CREATE2`, introduced in EIP-1014, changes the formula: the new contract's address is derived from the deployer's address, a chosen salt, and the *hash of the bytecode being deployed*. This means you can calculate a contract's address before deploying it. More importantly – and this is the key to the trick – if you deploy an intermediary factory contract that itself uses `CREATE` (the old opcode),

and that factory deploys a child contract, and then you destroy both the factory and the child via `selfdestruct`, and then you redeploy the factory at its original `CREATE2` address, the factory's nonce resets to zero, and it can deploy a *completely different* child contract at the *same address* where the original child lived. The address is reused. The code is not.

The second is the proxy pattern. Tornado Cash's governance system, like many DAO governance contracts, used a proxy architecture (EIP-1967/UUPS). In a proxy pattern, there is a permanent proxy contract at a fixed address that users interact with. This proxy does not contain the actual governance logic. Instead, it contains a pointer – a storage slot at a specific, standardized location – that holds the address of an *implementation* contract. When you call a function on the proxy, the proxy uses `delegatecall` to forward your call to whatever implementation contract the pointer currently references. The proxy's storage is used, but the implementation's code runs. This means whoever can change the pointer controls what code executes when anyone interacts with the governance system. Change the pointer, and you change the governance – silently, without deploying a new visible contract, without changing the address that everyone knows and trusts.

Now the trick.

The attacker submitted a governance proposal to the Tornado Cash DAO. The proposal looked benign. Its description claimed it was identical to Proposal 16, a previously approved and uncontroversial proposal that penalized certain relayers for cheating. The voters did what voters do in a DAO with dozens of proposals per month: they read the description, saw it matched something familiar, and voted yes. They did not decompile and audit the proposal's bytecode. Why would they? The description said it was the same proposal. Reviewing raw EVM bytecode is not a skill most governance participants possess, and the social norm in DAO governance is to review descriptions, not opcodes.

The vote passed. The proposal was approved by the DAO's governance process, with legitimate TORN token holders casting legitimate votes through the legitimate governance interface. Democracy had spoken.

Then the floor opened.

The proposal contract that the voters had approved contained a hidden capability: `selfdestruct`. This EVM opcode does exactly what its name suggests – it destroys the contract at a given address, wiping its bytecode from the blockchain state and sending any remaining ETH balance to a specified recipient. After the vote passed and the proposal was executed, the attacker triggered `selfdestruct` on the proposal contract. The code that the voters had approved ceased to exist on the blockchain.

Then the attacker redeployed. Using the `CREATE2` intermediary trick described above, the attacker deployed entirely new bytecode at the same address where the original proposal contract had lived.

The Tornado Cash governance system still held a reference to that address. It still trusted that address. But the code living there was now completely different from what the voters had approved.

The new code did one thing: it gave the attacker the ability to mint TORN governance tokens to themselves – 10,000 TORN per iteration, repeatable, until the attacker held 1.2 million votes. The entire legitimate DAO held roughly 700,000 votes. The attacker now controlled a permanent, unchallengeable supermajority.

The misdirection was total. The malicious code was not present during the vote. It did not exist when the voters examined the proposal. The voters approved code A. The attacker destroyed code A and deployed code B at the same address. The governance system, still pointing at that address, treated code B as if it had the full authority of the vote that approved code A. The signed letter's text changed after the seal was broken – and the seal still looked intact.

Impact: complete control over Tornado Cash's governance. The attacker could drain locked tokens, modify protocol parameters, brick the router contract, or do anything else the governance system was authorized to do. Approximately $2.17 million was stolen directly. The TORN token price dropped 36% as the market priced in the total capture of the protocol's decision-making apparatus. A privacy protocol whose zero-knowledge cryptography was unbroken – whose mathematical guarantees remained perfectly sound – was nevertheless fully compromised, because the human layer that governed it was exploitable through misdirection and code replacement.

The root cause was two vulnerabilities woven together: a social one and a technical one. The social vulnerability was that voters verified the proposal's *description* but not its *code*. This is normal human behavior. It is also, in hindsight, a systemic weakness of every DAO that presents proposals as human-readable summaries rather than requiring formal verification of the underlying bytecode. The technical vulnerability was the `selfdestruct + CREATE2` pattern, which allowed post-approval code replacement at a trusted address – a capability that the governance system had no mechanism to detect or prevent.

Neither vulnerability alone would have been sufficient. Together, they allowed an attacker to go beyond exploiting the governance – to *become* the governance. The Beanstalk attacker rented governance power for thirteen seconds. The Tornado Cash attacker did something structurally worse: he permanently replaced the governance with himself. Beanstalk was a heist. Tornado Cash was a coup.

8.4.3 ZK Rollup Governance Risk

Both attacks targeted governance mechanisms that controlled upgradeable contracts. And ZK rollup verifier contracts are almost always deployed behind upgradeable proxy patterns – the same patterns catalogued in the 2023 survey by Meisami and Bodell, which documented EIP-1967 (OpenZeppelin transparent proxy), EIP-1822 (UUPS), EIP-2535 (Diamonds), and Beacon proxies.

The proxy pattern introduces its own attack surface beyond governance: storage layout corruption when state variables are reordered across upgrades, function selector collisions between proxy admin and implementation functions, and the fundamental risk that `delegatecall` means all storage operations in the implementation affect the proxy's storage.

But the deepest risk is simpler than any of these. Whoever controls the proxy admin controls the verifier. If the governance mechanism that controls the proxy admin is vulnerable to flash loans (Beanstalk-style) or code replacement (Tornado Cash-style), then the entire rollup's security reduces to the security of its governance mechanism.

The cryptography could be perfect. The ceremony could have had a million participants. The circuits could be formally verified. None of it matters if an attacker can replace the verifier contract with one that returns `true` for every proof. Six layers of mathematical elegance, and the seventh is a multisig.

8.4.4 L2Beat's Stages Framework

L2Beat, the independent rollup monitoring organization, has formalized the maturity of rollup decentralization into three stages:

Stage 0 – Full Training Wheels: The rollup is effectively run by its operators. It must have source-available software for state reconstruction from L1 data, and it must have *some* proof system to qualify. But governance can override everything. Most rollup deployments begin here.

Stage 1 – Limited Training Wheels: The proof system is fully functional. Fraud proof submission (for optimistic rollups) or verification (for ZK rollups) is permissionless. Users can exit without operator coordination through forced inclusion or escape hatches. A Security Council may override the proof system for bug fixes, but with constraints – for example, a 6-of-8 multisig with a 7-day delay.

Stage 2 – No Training Wheels: The rollup is fully managed by smart contracts. The proof system is permissionless. Users get at least 30 days' notice for unwanted upgrades. The Security Council is restricted to adjudicating on-chain-provable soundness errors only. Users are fully protected from governance attacks.

As of early 2026, most major ZK rollups are at Stage 0 or Stage 1. Achieving Stage 2 requires either formally verified verifier contracts (so bugs are unlikely enough that upgrade capability can be removed), multiple independent implementations that cross-check each other, or bounded upgrade windows with mandatory exit periods of 30 or more days.

The tension is real and irreducible. ZK verifier contracts are among the most complex smart contracts ever deployed. They implement pairing checks, polynomial evaluations, and Fiat-Shamir transcript verification in a language (Solidity, or Yul) that was not designed for this kind of arithmetic. The probability of bugs is non-trivial. But the ability to fix bugs via governance is the same ability that allows governance to introduce them.

Stage 2 is where the cryptographic guarantees from Layers 1 through 6 actually bind. Below Stage 2, they are advisory. A Stage 0 rollup with 256-bit proof security is, from the user's perspective, only as secure as the governance multisig's operational security.

8.5 Proof Aggregation: The Missing Layer

Between the prover (who generates proofs) and the on-chain verifier (who checks them), a significant infrastructure layer has emerged that the seven-layer model does not account for: proof aggregation services.

SHARP (Shared Prover), built by StarkWare for Starknet, is the original aggregation service. Multiple applications submit their execution traces to SHARP, which generates a single STARK proof covering all of them, then wraps that proof in Groth16 for on-chain verification. The verification gas cost is amortized across all participating applications.

Aligned Layer, launched on mainnet with over $11 billion in restaked ETH (via EigenLayer), provides verification-as-a-service. Rollups and applications submit proofs to Aligned Layer, which batches and verifies them, posting the aggregated result to Ethereum.

NEBRA, live since August 2024, provides proof aggregation with a focus on universal verification – supporting multiple proof systems (Groth16, PLONK, STARK) within a single aggregation layer.

The economic logic is straightforward. If a single Groth16 verification costs ~200,000 gas, and an aggregation service can batch 100 proofs into a single on-chain verification, the per-proof verification cost drops from ~$1 to ~$0.01. At sufficient volume, aggregation makes proof verification nearly free.

But aggregation introduces a new trust assumption. Users must trust that the aggregation service correctly includes their proof in the batch, and that the aggregated proof faithfully represents all constituent

proofs. If the aggregation service is centralized (as SHARP is for Starknet), this is a single point of failure at Layer 7 that can undermine the decentralization guarantees of the underlying proof system.

8.6 Case Study: Midnight and the Three-Token Architecture

Midnight, developed by IOG (Input Output Global), provides an instructive case study for Layer 7 because it makes different architectural choices than the Ethereum rollup model. Where Ethereum rollups post proofs to a general-purpose L1 and rely on upgradeable verifier contracts, Midnight integrates verification into its consensus layer and uses a novel three-token economic model that directly shapes the verification experience.

8.6.1 The Verification Pipeline

Midnight uses a split execution model. Smart contracts are written in Compact, a domain-specific ZK language, but they never execute on-chain in the traditional sense:

1. The user's SDK executes the Compact circuit locally, computing the new state.
2. A proof server generates a ZK proof that the state transition is valid.
3. The SDK packages the proof, fee inputs, and state delta into a transaction.
4. Every Midnight node verifies the proof against the circuit's verifier keys, which were deployed on-chain when the contract was created.
5. If valid, the blockchain updates the contract's ledger state.

The critical difference from the Ethereum rollup model: verification is not performed by a specialized verifier contract that can be upgraded via governance. It is performed by every node as part of consensus. The verifier keys are stored on-chain at the contract address and are immutable – once deployed, a contract's verification logic cannot be changed. A new contract must be deployed for logic changes.

This is a strong answer to the governance-as-attack-surface problem. You cannot upgrade what is immutable. But it creates a different problem: what happens when there is a bug? The answer is migration – deploy a corrected contract and convince users to move to it. This is slower and messier than a governance upgrade, but it cannot be exploited by an attacker with admin keys. The tradeoff is explicit: Midnight accepts the inconvenience of immutability in exchange for immunity to the Beanstalk-style attack.

8.6.2 Three Tokens, Three Privacy Levels

Midnight's three-token model is not an arbitrary design choice. Each token represents a different point on the privacy-transparency spectrum, and together they create an economic system where verification costs, privacy guarantees, and governance rights are separate and independently tunable.

Night is the unshielded native token. It is fully transparent – all operations are publicly visible on the ledger, stored as UTXOs with public-key authentication. Night serves two purposes: staking (and therefore governance) and backing for DUST generation. Every Night token registered for dust generation produces DUST over time at a deterministic rate.

Shielded tokens are ZK-private custom tokens. Any Compact contract can mint shielded tokens with unique type identifiers ("colors"). Balances and transaction details are hidden via zero-knowledge proofs. Shielded tokens use UTXOs with Pedersen commitments – only the key holder can view or modify wallet state. All shielded token transfers go through contracts, not direct peer-to-peer.

DUST is the fee token. All transaction fees are denominated in DUST, but DUST is not mined or minted in the traditional sense. It is a time-dependent scalar computed from Night holdings. A user registers Night UTXOs for dust generation, and DUST accumulates over time according to a deterministic formula with parameters for rate, maximum cap, and creation time.

This creates a novel fee model with direct implications for verification economics:

- **No fee market**: Fees are computed deterministically, not bid. There is no gas auction.
- **Rate-limited spam**: Transaction throughput is naturally limited by DUST regeneration rate relative to Night holdings. An attacker who wants to spam the network must hold (or acquire) Night tokens and wait for DUST to regenerate.
- **Staking alignment**: Only Night holders can generate DUST. Transaction capability is tied to network participation.
- **Time-gated recovery**: A user who has spent all their DUST must wait for regeneration before transacting again. This is a natural circuit breaker against denial-of-service attacks.

8.6.3 Disclosure Rules and Compiler-Enforced Privacy

In Compact, everything is private by default. Values only appear on-chain when the developer explicitly calls `disclose()`. The Compact compiler enforces this through static disclosure analysis at compile time – privacy is a compiler guarantee, not developer discipline.

What must be disclosed includes contract ledger state (any `export ledger` field), counter increments and decrements, state transition deltas (so validators can verify the new state), nullifiers (to prevent double-spending), and hash commitments (stored for future verification). What stays private includes

secret keys, witness values (circuit private inputs), shielded balances, vote choices (only aggregate tallies change on-chain), authorization preimages, and computation logic (ZK proof hides the execution path).

This compiler-enforced privacy boundary is a significant Layer 7 innovation. In the Ethereum model, what is public and what is private depends on the developer's care in managing calldata, events, and storage. In Midnight, the compiler draws the line, and crossing it requires an explicit annotation that is visible in code review.

8.6.4 Private Governance

Midnight's DAO governance pattern demonstrates what private on-chain governance can look like:

- **Anonymous identity**: Voters prove membership via hash commitments, never revealing their real identity.
- **Weighted voting**: Contract-state token balances serve as anonymous voting weights via ZK circuit reads.
- **Per-proposal nullifier domains**: Each proposal has its own nullifier space, preventing double-voting while allowing participation across multiple proposals.
- **Vote privacy**: The voter's choice remains private – only the aggregate tally changes on-chain.
- **Irreversible state machines**: Proposal status is encoded as counter increments (0 unused, 1 open, 2 approved, 3 executed), and counters are monotonically increasing, so state transitions are irreversible by construction.

The multi-signature treasury adds M-of-N threshold approval with propose/approve/execute circuits, where signer identity is verified via hash commitment and double-vote prevention uses per-(proposal, signer) nullifiers.

Every design choice is a direct architectural response to the attacks we just witnessed. Anonymous weighted voting means an attacker cannot flash-loan governance tokens and vote – they would need to know the secret key that corresponds to a registered hash commitment, and flash-loaning tokens does not give them that. Irreversible state machines mean a proposal cannot be rolled back after execution. Per-proposal nullifier isolation means vote manipulation in one proposal cannot leak to another.

The Beanstalk attacker borrowed a billion dollars of voting power for the duration of a single transaction. Against Midnight's architecture, that borrowing would be useless. You cannot vote with a key you do not possess.

8.6.5 The Gaps

Midnight's approach is not without its own Layer 7 vulnerabilities:

- **Protocol upgrade governance**: The documentation does not describe how consensus-level parameters (fee rates, dust generation parameters, consensus rules) are governed. This is the most significant gap. Immutable contracts solve contract-level governance attacks, but someone must still govern the protocol itself.
- **Oracle centralization**: All oracle patterns in the current documentation use single-party authorization via hash commitment. There is no multi-oracle or threshold-oracle pattern. A single compromised oracle can feed false data to every contract that depends on it.
- **Fixed participant sets**: Current governance contracts hardcode 2-3 participant slots. Production governance with dynamic participant sets would require Merkle-tree-based registration, which has been demonstrated (in the lending pool pattern) but not yet integrated into governance contracts.
- **No emergency procedures**: There is no documented kill switch, pause mechanism, or emergency parameter override for deployed contracts. Immutability is a feature for preventing governance attacks, but it is a liability when a critical bug is discovered.

8.7 The Deepest Symmetry

There is a symmetry in the seven-layer model that becomes visible only at Layer 7, and it concerns the nature of trust.

At Layer 1, the setup ceremony, security rests on a social claim: "At least one of N participants honestly destroyed their toxic waste." This is not a mathematical statement. It is a statement about human behavior. You trust the ceremony because you trust that at least one person, out of thousands, did the right thing.

At Layer 7, the verifier deployment, security rests on a parallel social claim: "The governance mechanism that controls the verifier will not be captured by an adversary." This is not a mathematical statement either. It is a statement about institutional design, incentive alignment, and ultimately human judgment.

The layers in between – the language, the witness, the arithmetization, the proof system, the primitives – are mathematical. They provide computational guarantees that hold against any polynomial-time

adversary. They are the part of the trick that actually works by the laws of mathematics, not by the conventions of human society. But they are sandwiched between two layers of social trust.

None of this represents a failure of the model. It is a description of reality. Zero-knowledge proofs do not eliminate trust. They *compress* it. Instead of trusting a bank with your financial data every day, you trust that a ceremony was run honestly once and that governance will not go rogue in the future. These are weaker assumptions than trusting a single counterparty for every transaction. But they are assumptions nonetheless.

The honest framing is not "trustless." It is "trust-minimized." And the remaining trust assumptions – ceremony integrity at the bottom, governance integrity at the top – are worth stating explicitly so that readers can evaluate whether the trust reduction justifies the complexity.

Feynman, who had a gift for puncturing pretension, would probably say something like this: "You have built a beautiful machine that converts social trust into mathematical certainty and back into social trust again. The mathematical part in the middle is genuinely impressive. But do not pretend the social parts at the ends do not exist."

He would be right. And the fact that he would be right is itself the deepest insight Layer 7 has to offer. The magic trick is real. The mathematics works. But the trick is performed for an audience, and the audience is governed by people, and people are not mathematical objects. The security of the whole system is a chain, and the endpoints of that chain are anchored in human soil.

8.8 Pricing Attacks

The relationship between verification costs and data availability costs creates exploitable seams. A 2025 study by Chaliasos et al. identified two novel attack classes that exploit mismatches in how rollups price their three cost dimensions: L2 execution, L1 data availability, and L1 settlement/proving.

8.8.1 DA-Saturation Attacks

An attacker floods an L2 with data-heavy, compute-light transactions – essentially random calldata followed by a STOP opcode. Each such transaction is cheap in L2 gas (the computation is trivial) but expensive in L1 DA cost (it fills blob space with incompressible junk). The study found that sustained denial-of-service on Linea cost as little as 0.87 ETH per hour, and on Optimism roughly 2 ETH per 30 minutes.

The effects cascade: the L2 is congested, finality delays increase by 1.45x to 2.73x compared to direct L1 blob stuffing, and the rollup operator hemorrhages money because the fees collected from the spam transactions do not cover the L1 DA costs. All major rollups studied were found susceptible. Four bug bounties, each worth tens of thousands of dollars, were paid.

8.8.2 Prover-Killer Attacks

These exploit the mismatch between EVM gas metering and ZK proving costs. Not all EVM opcodes are equally expensive to prove in zero knowledge. The study measured "cycles per gas" ratios – how many proving cycles are needed per unit of gas cost:

Opcode / Precompile	EVM Gas	Proving Cycles/Gas	Attack Leverage
JUMPDEST	1	1,039.79	Very High
MODEXP	(varies)	2,961.72	Extreme
BN_PAIRING	45,000+	1,642.15	High
SHA256	60+	Moderate	Moderate

A MODEXP attack – filling blocks with maximum-cost modular exponentiation operations – delayed finality by 94x (over 8 hours) and cost the rollup operator $42.26 per attack block. The rollup's proving system crashed after 10,266 seconds.

8.8.3 The Concrete Scenarios

These are not theoretical concerns. They are playbooks.

DA-saturation in practice. An attacker constructs transactions that contain the maximum possible calldata – random bytes, incompressible by design – followed by a STOP opcode. The EVM execution cost is negligible: STOP costs 0 gas, and the transaction is technically valid. But the data must be

posted to L1 for data availability, and incompressible random data consumes maximum blob space. The attacker submits these transactions continuously, filling every blob slot in every Ethereum block with garbage.

The immediate effect: blob fees spike. Ethereum's blob fee market operates under EIP-1559 dynamics – when blobs are consistently full, the base fee increases exponentially. Under sustained saturation, blob fees can increase by 100x or more within minutes. Every legitimate rollup that needs to post data to Ethereum sees its operating costs spike proportionally. The attacker pays blob fees too, of course, but the attacker's cost is the cost of the attack. Every other rollup's cost is collateral damage.

The secondary effect is subtler and worse: rollup operators, facing unexpectedly high L1 costs, must choose between posting data at a loss (subsidizing operations from their treasury), delaying batch submissions (increasing finality time for users), or raising L2 fees (driving users to competitors). The attacker does not need to sustain the attack indefinitely. A few hours of blob saturation can cause lasting reputational and economic damage to rollups that depend on predictable L1 costs.

The defense is multidimensional fee pricing on the L2 side: a separate DA fee component that adjusts dynamically based on actual L1 blob costs, passed through to users in real time rather than absorbed by the operator. Several rollups have implemented this in response to the Chaliasos findings, but the adjustment is inherently reactive – the fee increases *after* the attack begins, which means the operator absorbs losses during the lag period.

Prover-killer in practice. An attacker submits transactions that are cheap in EVM gas but catastrophically expensive to prove in zero knowledge. The canonical example is MODEXP – modular exponentiation with maximum-size inputs. The EVM prices MODEXP based on the size of the operands and a formula that was calibrated for native CPU execution, not for ZK circuit execution. A MODEXP operation with 256-byte base, exponent, and modulus costs roughly 200 gas in the EVM. Proving that same operation inside a ZK circuit requires the prover to decompose the modular exponentiation into field arithmetic over the proving system's native field, which involves thousands of multiplication gates per limb, per exponentiation step. The ratio of proving cost to EVM gas cost – the "cycles per gas" metric – reaches nearly 3,000 for MODEXP. A single MODEXP transaction can consume as much proving capacity as 3,000 normal transactions.

The attacker does not need exotic tools. They submit valid transactions – MODEXP calls with legitimate inputs that any EVM will execute without complaint. The transactions pass all validation checks. They pay standard gas fees. They are included in blocks by the sequencer because the sequencer has no reason to reject a valid, fee-paying transaction. But when those blocks reach the prover, the prover must generate a ZK proof of the entire block's execution, including the MODEXP operations. A single block

stuffed with MODEXP calls can take the prover 10,000 seconds to prove – over 2.7 hours for a block that the sequencer produced in seconds.

If the attacker sustains this for multiple blocks, the prover falls behind. Proving latency grows. Finality – the time before a rollup batch is verified on L1 – stretches from minutes to hours. The rollup is technically still functioning, but its security guarantee degrades: until the proof is posted and verified, the rollup's state transitions are unproven claims, not verified facts. A sustained prover-killer attack can push a ZK rollup into a state where it behaves, from the user's perspective, more like an optimistic rollup – running on trust rather than proof, hoping nothing goes wrong during the gap.

The defense requires the sequencer to price transactions based on their *proving cost*, not just their EVM gas cost. This is the multidimensional pricing problem: EVM gas, DA cost, and proving cost are three independent resources, and a single gas price cannot accurately reflect all three. Until rollups implement proving-cost-aware fee markets, the prover-killer attack remains viable against any ZK rollup that prices transactions using only EVM gas metering.

The root cause is fundamental: current rollup fee mechanisms use a single-dimensional gas price that bundles L2 execution, L1 DA, and proving costs into one number. When these three resources have different scarcity profiles, the bundled price necessarily misprices at least one of them. The fix is multidimensional pricing – separate base fees for each resource type, each following its own EIP-1559-style adjustment mechanism.

8.9 Who Verifies the Verifier?

The verifier smart contract itself can have bugs. The FOOM Club exploit targeted a misconfigured snarkjs deployment where the verification key parameter delta_2 was set equal to gamma_2, weakening the Groth16 verification equation. The proof system was not broken – the *deployment configuration* was wrong.

The vulnerability is a supply-chain problem. The verifier contract depends on:

1. The proof system specification (mathematical, usually correct).
2. The reference implementation (code, sometimes buggy – see Frozen Heart).
3. The deployment configuration (operational, frequently wrong).
4. The Ethereum precompiles (hardware/protocol, generally reliable but not immune to bugs).
5. The compiler that compiled the verifier contract (Solidity, Vyper, or Yul – each with their own bug history).

An analogy to the XZ Utils supply-chain attack (CVE-2024-3094) is apt. In that case, a sophisticated attacker spent years contributing to an open-source compression library, gained maintainer trust, and inserted a backdoor. The same attack vector applies to ZK verifier libraries: snarkjs, gnark, arkworks, and halo2 are open-source projects maintained by small teams. A compromised maintainer could introduce a subtle verification bypass that passes all existing tests.

Verifier ossification – the strategy of deploying a verifier contract and making it permanently immutable, treating it like a protocol-level constant rather than upgradeable software – is one defense. But it requires very high confidence in the verifier's correctness, because bugs in an ossified verifier cannot be fixed without deploying an entirely new contract and migrating all dependent applications.

The tradeoff between immutable and upgradeable verifiers is one of the sharpest architectural decisions at Layer 7:

Property	Immutable Verifier	Upgradeable Verifier
Bug patching	Impossible without contract migration	Possible via governance vote or multisig
Governance capture	Immune — no upgrade path to exploit	Vulnerable — Beanstalk/Tornado Cash-style attacks
Regulatory compliance	Fixed at deploy time; cannot adapt	Adaptable to changing requirements
User trust model	Trust the code (audit once, rely forever)	Trust the governance (ongoing vigilance)
L2Beat Stage	Stage 2 candidate (if verifier is correct)	Stage 0-1 (governance can override proofs)
Quantum migration	Requires full system replacement	Can upgrade to PQ verifier via governance
Example	Midnight (immutable verifier keys)	Most Ethereum ZK rollups (proxy pattern)

Neither choice dominates. Immutable verifiers maximize cryptographic integrity at the cost of operational flexibility. Upgradeable verifiers maximize adaptability at the cost of governance risk. The choice reflects a system's threat model: does it fear bugs more, or governance capture more?

8.10 On-Chain Verification in 2026

The state of on-chain verification as of early 2026:

Verification costs have stabilized. Groth16 on BN254 remains the dominant on-chain proof format, at roughly 200,000-250,000 gas per verification. The verification cost floor is set by the pairing precompile gas schedule, which is a protocol parameter that changes only through Ethereum governance (EIPs and hard forks).

Data availability is abundant and cheap. Three Ethereum upgrades in two years (Dencun, Pectra, Fusaka) have expanded DA capacity by roughly 16x. Alternative DA layers (Celestia, EigenDA, Avail) provide even cheaper options at the cost of different security assumptions.

Governance maturity lags. Most ZK rollups remain at Stage 0 or Stage 1 of L2Beat's framework. The path to Stage 2 – where governance can no longer override the proof system – requires either formally verified verifier contracts, multi-prover architectures, or long mandatory exit windows. No major ZK rollup has achieved Stage 2 as of this writing.

Fiat-Shamir security is improving through hard experience. The Frozen Heart disclosure, the Last Challenge Attack, and the Solana ZK ElGamal bug have established Fiat-Shamir transcript completeness as a first-order security property. Audit firms now check for it specifically. But new implementations continue to be written, and the pattern will recur until proof system libraries converge on a small number of battle-tested implementations.

Proof aggregation is maturing. SHARP, Aligned Layer, and NEBRA demonstrate that aggregation can reduce per-proof verification costs by 10-100x. But aggregation services are themselves centralization points that need their own governance and security analysis.

The net picture: Layer 7 is the layer where the mathematical elegance of Layers 1 through 6 collides with the messy realities of software deployment, economic incentives, governance design, and human judgment. The cryptography is strong. The implementations are getting stronger. But the governance – the social layer that determines who can change the software that checks the math – remains the binding constraint on the security that zero-knowledge proofs can actually deliver to end users.

Until the governance matures to Stage 2 – until the smart contracts that verify proofs are either immutable or governed by mechanisms that provably resist capture – the verdict remains provisional. The audience is competent. The math checks out. But the audience serves at the pleasure of a committee that can replace it at any time.

Layer 7 is the last layer. The seven-layer tour – from setup ceremony to on-chain verdict – is complete. But zero-knowledge proofs do not operate in isolation. They belong to a family of privacy-enhancing

technologies – MPC, FHE, differential privacy, TEEs – and understanding ZKPs without understanding their siblings leads to architectures that reach for the right mathematics and solve the wrong problem. Before we synthesize the seven layers in Part III, we map the family.

Chapter 9: Privacy-Enhancing Technologies

"Privacy is not a feature you bolt on. It is a property of the architecture – and if the architecture does not have it, no amount of cryptography will give it to you."

This chapter makes a single argument: zero-knowledge proofs are necessary but not sufficient for privacy. A system that deploys ZKPs alone will leak through side channels, metadata, composition boundaries, and the gaps between what the proof covers and what the system exposes. The architect who reaches for ZKPs without understanding MPC, FHE, and differential privacy will build a house with a vault door on the front and a screen door on the back. Each technology in the PET family answers a different question. Understanding which question you are actually asking – and which tool answers it – is the prerequisite for any privacy architecture that works in practice.

We examine four technologies (ZKP, MPC, FHE, differential privacy), classify their security guarantees into three tiers (information-theoretic, computational, heuristic), study their composition patterns through five real-world deployments, and end with a decision matrix for system architects choosing between them.

9.1 The Four Pillars

Zero-knowledge proofs are one member of a family. The family is called privacy-enhancing technologies – PETs – and understanding ZKPs without understanding their siblings is like watching a magician and concluding that all performance is sleight of hand. It is not. Some performers sing. Some dance. Some vanish entirely. And the best shows combine all of them.

There are four major PET categories that matter for system architects making decisions in 2026. Each answers a different question. Each has a different relationship to the stage.

Zero-Knowledge Proofs (ZKPs) answer: *How do I prove a statement about my private data without revealing the data itself?*

Think of ZKPs as the magician's core trick: the audience sees the result but not the method. "My balance exceeds the minimum." "I am over 18." "This computation was performed correctly." The key insight, often missed, is that ZKPs are a tool for *selective disclosure*, not blanket privacy. The proof reveals the *truth of the statement* – which is itself information. Proving "my balance exceeds $1 million" tells you something about the balance, even though the exact figure stays hidden. The magician chooses which cards to reveal. That choice is itself a disclosure.

If ZKPs are the magician's core trick – proving truth without revealing method – then the three siblings each perform a different kind of magic.

Secure Multi-Party Computation (MPC) answers: *How can multiple parties jointly compute a function on their combined data without any party revealing its input to the others?*

Picture three rival magicians who want to know whose trick is most popular – but none will reveal their ticket sales to the others. MPC is the protocol that lets them compute the answer as if a trusted accountant had all the books, without any such accountant existing. The inputs stay private. Only the agreed-upon output is revealed.

MPC is not a single protocol. It is a family, and the family members have very different properties:

Protocol Family	Trust Model	Security Type	Best For
Shamir secret sharing	Honest majority (>50% honest)	Information-theoretic	Statistical computations, few parties

Protocol Family	Trust Model	Security Type	Best For
SPDZ (dishonest majority)	Any number of corruptions	Computational	Adversarial settings, financial computation
Garbled circuits	Two parties, semi-honest or malicious	Computational	Two-party computation

The distinction between honest-majority and dishonest-majority protocols matters enormously. Shamir-based MPC with honest majority achieves *information-theoretic* security – it remains secure even against an adversary with unlimited computational power, including quantum computers. SPDZ provides security against any number of corruptions, but relies on computational hardness assumptions.

To understand what MPC actually does, consider the problem that started the field. In 1982, Andrew Yao posed what is now called the Millionaires' Problem: two millionaires want to determine who is richer without either revealing their net worth. They are standing at a cocktail party. Neither will say a number. Neither trusts the other to be honest. And no accountant is available whom both would trust with the truth. How do they find out?

Here is the trick. Alice has a net worth of, say, $7 million. Bob has $5 million. Alice encodes her wealth into an encrypted lookup table – a garbled circuit – that represents the comparison function "is Alice's input greater than Bob's input?" She hands the garbled table to Bob. Bob, using a sub-protocol called oblivious transfer, obtains the encryption key corresponding to his own input ($5 million) without Alice learning which key he selected. Bob evaluates the garbled circuit with his key and obtains a single bit of output: "Alice is richer." He announces the result. Neither party learned the other's number. The function was computed. The inputs stayed private.

The garbled circuit deserves a moment of its own, because it is one of the most counterintuitive constructions in all of cryptography. Alice takes the computation she wants to perform and compiles it into a Boolean circuit – AND gates, OR gates, NOT gates, the same primitives that make up a physical processor. She then encrypts every wire in the circuit with random labels: each wire gets two labels, one for "0" and one for "1," and Alice encrypts each gate's truth table so that only the correct pair of input labels decrypts to the correct output label. The result is a garbled mess – a table of ciphertexts that encodes the computation but reveals nothing about it. Bob can evaluate the garbled circuit gate by

gate, decrypting one entry per gate, following the circuit from input to output. He sees the computation unfold, but the labels are random strings. He learns the output and nothing else. Alice never sees Bob's input. Bob never sees Alice's circuit internals. The computation happens in a kind of cryptographic fog, visible only at the endpoints.

Scale this from cocktail-party curiosity to industrial infrastructure and the applications multiply. Private auctions: bidders submit encrypted bids to an MPC protocol that determines the winner and the clearing price without revealing any losing bid. The auction house learns who won and at what price. It never learns what the losers were willing to pay – information that, in traditional auctions, the house can exploit in future rounds. Dark pool matching in finance: two investment banks want to match buy and sell orders for the same security without revealing their order books to each other or to the market. MPC lets them compute the intersection of their orders – the trades that both sides want – without exposing the orders that did not match. The matched trades execute. The unmatched orders remain invisible.

Private set intersection, or PSI, is the simplest and most widely deployed MPC primitive. Two parties each hold a set of items. They want to learn which items appear in both sets – and nothing else. When COVID-19 contact tracing required matching infected individuals against location databases, PSI offered a path that did not require a central authority to hold everyone's location history. Google and Apple's Exposure Notification framework used a related technique: devices broadcast rotating pseudonymous identifiers, and the matching computation happens locally on each device. The computation is distributed. The data never congregates.

The cost of MPC is communication, not computation. Each gate in the circuit requires the parties to exchange encrypted values. For garbled circuits, this means transmitting roughly four ciphertexts per gate. A circuit with a billion gates requires transmitting billions of ciphertexts – tens of gigabytes over the network. The computation itself is fast. The network is the bottleneck. This is why MPC shines for problems where the function is simple but the privacy requirement is absolute, and struggles for problems where the function is complex and latency matters. The magician works quickly. The postal service does not.

MPC is the ensemble performance: multiple magicians, each holding one card of a shared secret, jointly computing a result that none of them could produce alone. The audience sees the answer. No single performer ever sees the full hand.

Fully Homomorphic Encryption (FHE) answers: *How can I outsource computation on my data without the computing party learning anything about the data?*

Craig Gentry, who invented FHE, gave it the perfect image: performing surgery on a patient inside a sealed glovebox. The surgeon's hands are inside the gloves, manipulating the patient, but the glovebox prevents any direct contact. The surgeon can work – she can cut, stitch, probe – but she never touches

the patient directly, and nothing from the operating field crosses the barrier. It is a vivid image from a different domain – not our magician's stage but a laboratory – and we borrow it here because Gentry's metaphor has become inseparable from the concept itself.

You encrypt your data. You send the ciphertext to a cloud provider. The cloud provider performs operations on the ciphertext. You decrypt the result. The cloud provider learns nothing – not the data, not the result, not even which operations were meaningful. The gloves never come off.

But the glovebox is thick, and the gloves are stiff. Current FHE computations are 10,000x to 1,000,000x slower than their plaintext equivalents. Ciphertexts accumulate noise with each operation, requiring periodic "bootstrapping" that is computationally expensive. Not all operations are equally efficient – additions and multiplications are the native operations, while comparisons and divisions are far more costly. The surgeon can work, but she works very, very slowly.

To understand why the gloves are so stiff, you need to see what an FHE computation actually looks like from the inside. The dominant schemes – BFV, BGV, and CKKS – all share a common mathematical structure. A plaintext value (say, the number 42) is encoded as a polynomial in a ring, typically $\mathbb{Z}_q[X]/(X^n + 1)$ for $n = 2^{13}$ or 2^{14} and q a large modulus, perhaps 200 to 800 bits long. Encryption adds a carefully sampled noise term to this polynomial. The ciphertext is a pair of ring elements, each thousands of bits wide. Where a plaintext integer fits in 64 bits, its ciphertext might occupy 32 kilobytes. The glovebox is thick because the encoding is thick.

Addition through the glovebox is relatively gentle. You add two ciphertexts component-wise, and the noise terms add as well. The noise grows, but only linearly. Encrypted addition is perhaps 100x slower than plaintext addition – expensive, but not prohibitively so.

Multiplication is where the cost explodes. When you multiply two ciphertexts, the underlying polynomial multiplication produces a result with noise that is roughly the *product* of the two input noise levels, not the sum. One multiplication might double the noise budget. Two multiplications might quadruple it. After a dozen consecutive multiplications without intervention, the noise overwhelms the signal – the ciphertext becomes a random-looking polynomial that decrypts to garbage. The noise is not a flaw in the design. It *is* the security. Without noise, the lattice-based hardness assumptions that make FHE secure would not hold. The glovebox is thick because thinning it would make it transparent.

Gentry's breakthrough – the idea that launched FHE from theoretical impossibility to practical research program – was bootstrapping. The concept is recursive and almost paradoxical: to clean the noise from a ciphertext, you decrypt it *homomorphically*. That is, you take the noisy ciphertext, encrypt the decryption key under a fresh public key, and then run the decryption algorithm as a circuit *inside the encryption*. The output is a fresh ciphertext encrypting the same plaintext, but with reset noise – as if you had just encrypted the value for the first time. The surgeon, working through the glovebox, performs

a second surgery on the glovebox itself, replacing the foggy glass with a clean pane, all without ever removing her hands.

The cost of this cleaning step is enormous. A single bootstrapping operation might take 10 to 100 milliseconds on modern hardware – which sounds fast until you realize that the plaintext operation it replaces (a single multiplication) takes about one nanosecond. The ratio is 10 million to one. And bootstrapping must be performed after every few multiplications to keep the noise below the fatal threshold. The 10,000x to 1,000,000x slowdown is not a single penalty applied once. It is the accumulated cost of performing every arithmetic operation on bloated polynomial ciphertexts and periodically cleaning the noise through a decryption-inside-encryption cycle that is itself a complex computation.

Recent optimizations have attacked every link in this chain. TFHE (Torus FHE) reduces the bootstrapping cost for Boolean circuits by working over a different algebraic structure. GPU acceleration parallelizes the polynomial ring arithmetic. Hybrid schemes use leveled FHE (no bootstrapping) for shallow circuits and switch to bootstrapping only when depth demands it. The overhead is shrinking – from a million-fold five years ago to ten-thousand-fold today – but the fundamental structure remains: encrypted computation is expensive because the noise that provides security must be managed, and managing it costs orders of magnitude more than the computation itself.

And here is the connection that brings us back to our magician. FHE lets you compute on encrypted data, but how do you know the computation was performed *correctly*? The cloud provider claims it evaluated your function honestly. But the output is encrypted – you cannot inspect the intermediate steps. The provider could have computed a different function, or computed the right function incorrectly, and you would not know until you decrypted the result and found it nonsensical. Verifiable FHE – zkFHE – addresses this by having the computing party produce a zero-knowledge proof that the homomorphic operations were performed according to specification. The surgeon operates through the glovebox, and a camera inside the glovebox records the procedure for the review board. The patient stays sealed. The surgery is verified. This is where FHE and ZKPs converge, and it is one of the most active research frontiers in applied cryptography.

The field is improving rapidly. Zama, the leading FHE company, achieved a $1 billion valuation in June 2025 and launched the Confidential Blockchain Protocol testnet in July 2025. Their roadmap projects hundreds of transactions per second with GPU acceleration. But a 10,000x overhead, even if it shrinks to 1,000x, means FHE is practical only for computations where the privacy guarantee is worth the performance cost. The glovebox is for surgery you cannot perform any other way.

FHE is the trick performed inside a sealed box: the computation happens on encrypted data, and even the magician who executes the computation never sees the plaintext. The box opens to reveal only the result.

Differential Privacy (DP) answers: *How can I release statistical insights about a dataset while guaranteeing that no individual's record can be reverse-engineered from the output?*

If the other PETs are stage tricks – precise, targeted, visible to the audience – differential privacy is fog. It blurs the picture just enough that no individual face can be identified, while the overall scene remains recognizable. Apple uses it for iOS telemetry (since 2016, with $\varepsilon = 2$ per day for most data types). Google deployed RAPPOR (Randomized Aggregatable Privacy-Preserving Ordinal Response) for Chrome usage monitoring. The US Census Bureau used it for the 2020 Census – the first-ever deployment at national scale, motivated by the Dinur-Nissim database reconstruction theorem, which proved that releasing too many exact statistics about a dataset inevitably leaks individual records.

DP works by adding carefully calibrated noise to query results. The noise is large enough to mask any individual's contribution but small enough to preserve the statistical utility of the aggregate. The privacy guarantee is parameterized by epsilon (lower epsilon = more privacy = more noise = less accuracy), and composability is formalized by a composition theorem: sequential queries consume a "privacy budget," and once the budget is exhausted, no more queries can be safely answered. The fog has a finite supply. Use it wisely.

The epsilon parameter deserves a longer look, because it is where the mathematics of differential privacy meets the politics of data collection. Think of epsilon as controlling the blur radius on a photograph. At $\varepsilon = 0.1$, you are looking at a Monet painting – the water lilies are recognizable as water lilies, but individual petals dissolve into impression. The aggregate is preserved. The particular is lost. At $\varepsilon = 1$, you are looking at a photograph taken through frosted glass – shapes and proportions are clear, but faces are unreadable. At $\varepsilon = 10$, you are looking at a photograph with a slight smudge – almost everything is visible, and a determined adversary with auxiliary information might identify individuals. At $\varepsilon = \infty$, there is no blur at all. You are looking at the raw data.

The art of differential privacy is choosing the blur. Too much noise (low epsilon) and the data is useless – a census that cannot distinguish New York from Nebraska serves no one. Too little noise (high epsilon) and the privacy guarantee is hollow – a medical database that lets researchers reconstruct individual diagnoses is not private in any meaningful sense. The Dinur-Nissim theorem makes the stakes precise: for any dataset of n individuals, if you answer more than $O(n)$ counting queries with accuracy better than $1/\sqrt{n}$, you can reconstruct the entire dataset. The blur is not optional. Without it, the data eventually gives up everyone's secrets.

What does this look like in practice? Apple's deployment adds noise locally, on each device, before data is transmitted – a technique called local differential privacy. When your iPhone wants to report which emoji you use most frequently, it does not send "thumbs up." It sends "thumbs up" with probability $(e^{\varepsilon})/(e^{\varepsilon} + 1)$ and a random emoji with probability $1/(e^{\varepsilon} + 1)$. Any individual report is plausibly random.

But aggregate millions of reports, and the noise cancels out, revealing the population-level distribution. Apple uses $\varepsilon = 2$ per day for most data types and $\varepsilon = 8$ for some health-related queries. Google's RAPPOR uses a similar local model with a two-stage randomization that provides both plausible deniability for individual responses and high accuracy for aggregate statistics. You never see any of this. Your phone adds the noise silently, the aggregation server receives randomized data, and the statistical team extracts population trends from the collective fog.

The composition problem is the silent killer of differential privacy deployments. Each query against a dataset consumes a portion of the privacy budget. If you query a medical database once with $\varepsilon = 1$, you get a strong privacy guarantee. If you query it twice, the effective epsilon is (at most) 2 – weaker, but still meaningful. If you query it a thousand times, each with $\varepsilon = 1$, the effective epsilon is (at most) 1000 – and at that point, the privacy guarantee is essentially worthless. Advanced composition theorems (Dwork, Rothblum, and Vadhan, 2010) give tighter bounds: k queries with epsilon each compose to roughly $\varepsilon \cdot \sqrt{k}$ rather than $\varepsilon \cdot k$. But the fundamental truth remains: privacy budgets are finite, and every query spends them. A dataset that has been queried ten thousand times is not the same, from a privacy standpoint, as one that has been queried ten times. The fog dissipates with each question asked. The Census Bureau's TopDown Algorithm was designed with this in mind: the total privacy budget was fixed before any queries were defined, and the noise allocation was optimized across all geographic levels simultaneously, from national aggregates down to census blocks. The budget was spent once, carefully, and then the books were closed.

9.2 Three Kinds of Security

These four technologies provide fundamentally different *types* of security guarantees, and conflating them leads to bad architecture decisions. This is the part where precision matters more than analogy.

Information-theoretic security means the guarantee holds even against an adversary with unlimited computational power. No mathematical breakthrough, no quantum computer, no advance in algorithms can break it. MPC with honest majority (e.g., Shamir secret sharing where more than half the participants are honest) achieves this. The security follows from information theory, not computational hardness. The caveats: you need an honest majority, and the communication cost scales with the number of parties and the complexity of the computation.

Computational security means the guarantee holds against adversaries bounded to polynomial-time computation. It rests on assumptions: "discrete logarithms are hard," "the Learning With Errors problem is hard," "the Ring-LWE problem is hard." ZKPs and FHE provide computational security. If the un-

derlying hardness assumption falls – as discrete logarithm will fall to Shor's algorithm on a sufficiently large quantum computer – the security evaporates retroactively. Every proof ever generated under that assumption becomes suspect. The lock does not weaken. It ceases to exist.

Heuristic security means the guarantee rests on practical observations rather than formal proof. Trusted Execution Environments (TEEs) like Intel SGX, AMD SEV, and ARM CCA provide heuristic security. The hardware manufacturer attests that the enclave is isolated. But SGX has been broken by Spectre, Meltdown, Foreshadow, Plundervolt, SGAxe, and AEPIC Leak. AMD SEV has shown vulnerabilities (SEVered, CipherLeaks). Intel SGX was deprecated on consumer processors in 2021. The 2025 attacks Battering RAM (~50 euros) and Wiretap (~$1,000) demonstrated physical attacks at commodity prices. TEE security is real in practice against most adversaries, but it lacks the mathematical foundation of cryptographic privacy and carries an expiration date set by the next side-channel attack. It is a stage built from plywood rather than steel: functional, but not what you want for the long run.

The metaphor of plywood is generous. To understand TEEs concretely, picture a room within a room. Your CPU – the physical chip on the motherboard – creates an isolated memory region called an enclave. Code and data inside the enclave are encrypted in RAM. The operating system cannot read the enclave's memory. The hypervisor cannot read it. Even a system administrator with root access and physical possession of the machine cannot read it. The CPU itself enforces the boundary: any attempt to access enclave memory from outside the enclave returns encrypted garbage. Intel SGX (Software Guard Extensions) pioneered this architecture. ARM TrustZone implements a similar concept at the processor level, splitting the chip into a "secure world" and a "normal world" with hardware-enforced isolation between them.

The promise is strong: you can run sensitive computation on an untrusted machine, and the machine's owner cannot observe or tamper with it. Cloud computing without trusting the cloud. The allure is obvious. The history is cautionary.

Foreshadow (August 2018) broke SGX isolation through a speculative execution attack. The CPU's branch predictor, trying to execute instructions ahead of time for performance, would speculatively read enclave memory and leave traces in the L1 cache. An attacker could measure cache timing to reconstruct the enclave's secrets. The attack required no physical access – it could be performed by a process running alongside the enclave on the same machine. Intel patched it with microcode updates, but the patch reduced performance and the fundamental vulnerability – that speculative execution can leak secrets across security boundaries – proved to be architectural, not incidental.

AEPIC Leak (August 2022) was worse. It exploited a bug in Intel's Advanced Programmable Interrupt Controller to read stale data from the enclave's memory hierarchy. Unlike Foreshadow, which required careful cache timing, AEPIC Leak provided architecturally guaranteed data leakage – the CPU would

hand you the enclave's data directly if you asked the right hardware register. No timing side channel, no statistical analysis. A clean read.

Downfall (August 2023) exploited the Gather instruction, which loads data from scattered memory locations into a vector register. The vulnerability allowed an attacker to read data from other security domains – including SGX enclaves – by observing the contents of internal CPU buffers during Gather operations. Intel's mitigation involved disabling the optimization that made Gather fast, resulting in up to 50% performance degradation for workloads that depended on it.

Intel quietly deprecated SGX on consumer processors (12th generation and later) beginning in 2021. The feature remains available on server-class Xeon processors, where it is marketed for cloud confidential computing. But the deprecation on consumer chips tells a story: Intel concluded that the attack surface was too large and the performance cost of mitigations too high for a feature intended to run on every laptop. The room within a room is still available – but only in the data center, where the threat model is different and the economic calculus favors the convenience of hardware isolation despite its known fragility.

For the system architect, this taxonomy matters because it determines *what you are actually trusting*. If your system uses MPC with honest majority for the core computation and ZKPs for the verifiable output, you have information-theoretic privacy for the computation and computational privacy for the proof. If you then run the whole thing inside a TEE, the TEE adds performance (fast) and convenience (no complex protocol choreography) but does not strengthen the privacy beyond what the cryptography already provides – and may weaken it if the TEE is compromised.

The magician's guarantee depends on which lock protects the trick. Information-theoretic security is a lock that cannot be picked, even with infinite time. Computational security is a lock that cannot be picked in practice – but a quantum locksmith might change the calculus. Heuristic security is a lock that has never been picked, without proof that it cannot be.

9.3 Composability: When One PET Is Not Enough

The real power of PETs emerges when they are composed. No single instrument plays the whole symphony. Consider a realistic healthcare scenario – and notice how each PET enters at the moment its particular strength is needed:

1. **Step 1 (MPC)**: Five hospitals jointly compute aggregate statistics on a rare disease using their combined patient records. Each hospital contributes its data to an MPC protocol. No hospital sees any other hospital's records. The output is aggregate statistics: prevalence rates, treatment outcomes, demographic distributions.
2. **Step 2 (Differential Privacy)**: Before the aggregate statistics leave the MPC computation, differential privacy noise is added. This ensures that even the aggregate output cannot be used to infer individual patient records. The privacy budget (epsilon) is tracked across queries.
3. **Step 3 (FHE)**: The differentially private aggregate statistics are encrypted under FHE. An AI firm trains a predictive model on the encrypted data. The AI firm never sees the plaintext statistics. The hospitals never see the AI firm's model architecture (which may be proprietary). The glovebox, again.
4. **Step 4 (ZKP)**: The AI firm produces a zero-knowledge proof that the trained model meets accuracy and fairness criteria specified in a regulatory standard, without revealing the model's weights or the training data. A regulator verifies the proof and certifies the model for clinical use. The magician performs. The audience – in this case, the regulator – verifies.

Each step uses the PET best suited to its specific trust problem: MPC for multi-party data aggregation, DP for statistical disclosure control, FHE for outsourced computation on sensitive data, and ZKP for verifiable compliance without disclosure.

But the transitions between steps are not trivial. The MPC-to-FHE handoff requires either the hospitals to encrypt the MPC output under the AI firm's FHE public key (which means they see the plaintext), or a protocol that converts MPC secret shares directly to FHE ciphertexts (an active research frontier). The FHE-to-ZKP handoff requires verifiable FHE – proving in zero knowledge that an FHE computation was performed correctly – which is emerging but not yet production-ready.

The composability lesson: PETs compose in theory. In practice, each composition point requires protocol engineering that is often harder than the individual PET deployments. The system architect must understand not just what each PET does, but how they hand off to each other. The orchestra sounds beautiful when everyone enters on cue. Getting the cues right is the hard part.

Three composition patterns deserve particular attention, because they recur across domains and will likely define the privacy architecture of the next decade.

ZKP + MPC: Verified Inputs to Joint Computation. The healthcare scenario above assumes each hospital contributes honest data to the MPC protocol. But what if a hospital submits fabricated records – inflating its patient count to increase its share of research funding, or omitting records to conceal a malpractice pattern? MPC computes correctly on whatever inputs it receives. It does not verify that the inputs are truthful. This is where ZKPs enter: each participant produces a zero-knowledge proof that its MPC input satisfies agreed-upon constraints – the records come from a certified database, the patient count matches a signed attestation from the hospital's electronic health record system, the data format conforms to the protocol specification. The MPC protocol verifies these proofs before accepting the inputs. The joint computation proceeds on data that is both private *and* certified. No party reveals its data. Every party proves its data is legitimate. The magician does not merely perform behind a curtain – she presents her credentials before stepping onto the stage.

This pattern – ZKP-verified inputs to MPC – appears in private auctions (prove your bid is backed by sufficient funds without revealing the bid amount), in private voting (prove you are an eligible voter without revealing your identity), and in collaborative machine learning (prove your training data meets quality thresholds without revealing the data itself). In each case, MPC provides the privacy during computation, and ZKPs provide the integrity of the inputs. The two PETs are not redundant. They address orthogonal trust problems. Privacy without integrity is a system that computes correctly on lies. Integrity without privacy is a system that reveals everything it verifies.

ZKP + FHE: Verifiable Encrypted Computation. This is the zkFHE frontier mentioned earlier, and it deserves a structural explanation. The problem: a cloud provider performs FHE computation on your encrypted data and returns an encrypted result. You decrypt and get an answer. But did the provider actually compute the function you requested? Or did it compute a cheaper approximation, or a different function entirely, or simply return a random ciphertext? FHE guarantees confidentiality – the provider cannot see your data. It does not guarantee integrity – the provider can lie about what it computed. ZKPs close this gap. The provider produces a zero-knowledge proof that the sequence of homomorphic operations it performed on the ciphertext corresponds exactly to the function specification. The proof is verified against the input ciphertext, the output ciphertext, and the function description. If it checks out, you know the computation was honest. If it does not, you know to reject the result and find another provider.

The difficulty is that proving FHE computations in zero knowledge is very expensive. Each homomorphic operation involves polynomial arithmetic over large rings, and the ZKP circuit must encode all of this arithmetic faithfully. Current zkFHE prototypes achieve verification for small circuits – a few

hundred multiplication gates – and the proving overhead adds another order of magnitude atop FHE's already steep costs. But the research trajectory is clear, and the incentive is enormous: anyone who wants to outsource computation on sensitive data to an untrusted cloud needs both confidentiality (FHE) and integrity (ZKP). Neither alone is sufficient.

The Privacy Stack. The key insight about PET composition is architectural: do not think of PETs as individual tools to be selected. Think of them as layers in a protocol stack, analogous to the network stack that separates TCP from IP from Ethernet. At the bottom, differential privacy provides statistical-level guarantees for aggregate data releases – the coarsest and cheapest form of privacy, suitable for telemetry and census-scale statistics. Above it, MPC provides computation-level privacy for multi-party protocols – stronger than DP (it protects individual inputs, not just statistical aggregates), but more expensive and limited to specific interaction patterns. Above that, FHE provides data-level privacy for outsourced computation – stronger still (the computing party learns nothing at all), but with the highest performance cost. And at the top, ZKPs provide verification-level privacy – the ability to prove properties of private data or private computation without revealing the underlying secrets.

Each layer addresses a different threat. Each has a different cost. And like network layers, they compose vertically: a system might use DP for its public-facing analytics dashboard, MPC for its inter-institutional data sharing, FHE for its cloud-based model training, and ZKPs for its compliance proofs – all within the same architecture, each operating at its appropriate level of the stack. The system architect who treats PET selection as a single choice ("we will use ZKPs") is making the same mistake as the network engineer who treats protocol selection as a single choice ("we will use TCP"). The answer is almost always a stack, not a single layer.

9.4 Real-World Deployments: Five Case Studies

9.4.1 1. Decentriq and the Swiss National Bank

In a federal pilot project beginning in 2021-2022, data clean room technology enabled encrypted collaboration between the Swiss National Bank, SIX (Switzerland's financial market infrastructure provider), and Zurich Cantonal Bank. The goal was cybersecurity threat detection: analyzing patterns of suspicious financial activity across institutions without any institution revealing its transaction data to the others.

The architecture used MPC-style computation within Decentriq's confidential computing platform, combining software-level privacy guarantees with hardware TEEs. The result demonstrated that finan-

cial regulators can gain systemic risk visibility without requiring banks to share raw transaction data – a significant precedent for privacy-preserving financial regulation. The regulator sees the pattern. The banks keep the data. Everyone sleeps better.

9.4.2 2. DTCC and the Canton Network

The Depository Trust and Clearing Corporation (DTCC), which processes virtually all US securities transactions, partnered with Digital Asset's Canton Network in December 2025 to tokenize US Treasuries on a permissioned blockchain with privacy-preserving settlement. The architecture uses the Canton protocol's built-in privacy model, where participants see only the portions of the ledger relevant to them.

This deployment matters because of who is adopting. DTCC is not a startup experimenting with privacy. It is the backbone of US securities infrastructure, processing trillions of dollars annually. When DTCC chooses a privacy-preserving architecture, it signals that privacy is not a nice-to-have feature but a regulatory and competitive necessity. The largest financial plumbing system in the world has decided it needs these tools. Pay attention.

9.4.3 3. Partisia and Toppan Edge: Digital Student IDs

Toppan Edge and Partisia announced joint development of privacy-preserving digital student IDs in 2025, with a proof-of-concept conducted at the Okinawa Institute of Science and Technology from June to September 2025. The system combines facial recognition for identity verification, decentralized identifiers (DIDs) for credential management, smartphone NFC for physical access, and MPC via Partisia's blockchain for privacy-preserving identity verification.

The key innovation: the student's biometric data is never stored in a single location or revealed to a single party. MPC ensures that identity verification can be performed without any single server holding the student's facial template. The platform is targeted for students enrolling from April 2026. Your face opens the door, but no one holds a copy of your face.

9.4.4 4. Privacy Pools: Pragmatic On-Chain Privacy

Privacy Pools, co-authored by Vitalik Buterin and implemented by 0xbow, launched on Ethereum mainnet on April 1, 2025. Buterin was one of the first users, depositing 1 ETH.

The design addresses the fundamental tension between on-chain privacy and regulatory compliance – a tension that destroyed Tornado Cash and haunts every privacy protocol. Users deposit funds into

a pool and can later withdraw them, breaking the link between deposit and withdrawal addresses (similar to Tornado Cash). But Privacy Pools add a compliance layer: an Association Set Provider (ASP) screens deposits for connections to sanctioned or illicit addresses, and the zero-knowledge proof used for withdrawal includes a proof that the user's funds are drawn from a compliant "association set."

The result is "pragmatic privacy" – transaction privacy for legitimate users, with a built-in compliance mechanism that prevents sanctioned funds from mixing with clean funds. As of early 2026, Privacy Pools has processed over $6 million in volume across more than 1,500 users. The broader ecosystem includes more than 35 teams pursuing approximately 13 distinct approaches to private transfers on Ethereum. The magician proves she is not cheating – and the regulator is satisfied.

9.4.5 5. Apple, Google, and the US Census Bureau: Differential Privacy at Scale

The largest PET deployments in the world are not blockchain systems. They are not even close. They are differential privacy systems serving billions of users:

- **Apple** introduced DP in iOS 10 (2016) for emoji usage statistics, Safari search queries, HealthKit data, and keyboard autocorrect improvements. Each device adds local noise before transmitting data, with $\varepsilon = 2$ per day for most data types.
- **Google** deployed RAPPOR for Chrome settings monitoring, adding randomized responses to usage data before aggregation.
- **US Census Bureau** used the TopDown Algorithm for the 2020 Census, adding calibrated noise to census statistics at every geographic level. The decision was motivated by a concrete threat: the Dinur-Nissim reconstruction theorem proved that releasing too many exact statistics from a dataset eventually allows full reconstruction of individual records.

These deployments demonstrate that differential privacy is the only PET to have achieved planetary-scale adoption. ZKPs, MPC, and FHE remain orders of magnitude smaller in deployment footprint. For the system architect, this suggests that DP should be the first tool considered for statistical data release, with the other PETs reserved for use cases that require computation on raw data or verifiable individual claims. The fog machine is the most popular tool in the privacy toolkit. The magic wand is catching up.

9.5 Privacy Architectures for Smart Contracts: Kachina and Zexe

Two academic systems – Kachina and Zexe – represent the theoretical foundations for how private smart contracts can be deployed on blockchains. They take complementary approaches, and understanding both illuminates the design space that every privacy-focused blockchain must work within.

9.5.1 Kachina: Privacy as a Parameter

Kachina, developed by Kerber, Kiayias, and Kohlweiss at the University of Edinburgh and IOHK, provides a UC-secure (universally composable) framework for privacy-preserving smart contracts. Its key innovation is treating privacy as a *parameter*, not a binary choice. Think of it as a dimmer switch rather than an on/off toggle.

Contract state is split into *shared public* state (on-chain) and *individual private* state (per party, off-chain). Users prove in zero knowledge that their state transitions are valid given some private state and input. The critical architectural insight is the *state oracle transcript*: instead of proving full state transitions (which would require locking shared state), users capture oracle queries and responses as partial transcripts. These transcripts are partial functions over state, enabling concurrent transactions to succeed even when state changes between proof creation and proof submission.

The privacy leakage is formally captured by a *leakage function* Lambda that specifies exactly what information each transaction reveals. Lambda can be tuned from "full leakage" (equivalent to Ethereum, where everything is visible) to "near-zero leakage" (equivalent to Zerocash, where only nullifiers and commitments are visible). This parameterization means the same framework can model both transparent and private contracts, and everything in between. The magician decides, contract by contract, how much of the trick to reveal.

Proving complexity is $O(|T_\rho| + |T_\sigma|)$ – proportional to the *transcript lengths*, not the full state size. This matters for scalability: a contract with millions of state entries can support private transactions that only touch a few entries, and the proving cost reflects only the entries accessed.

9.5.2 Zexe: Function Privacy

Zexe, developed by Bowe, Chiesa, Green, Miers, Mishra, and Wu (2018), takes a UTXO-based approach with a stronger privacy guarantee: not only are the transaction data hidden, but the *function being computed* is hidden as well. An observer cannot distinguish a token transfer from a governance vote from a swap – all transactions look identical on-chain. The audience sees identical envelopes. Every envelope looks the same. The contents are unknowable.

The architecture uses a "records nano-kernel" (RNK) – a minimalist shared execution environment where records have birth and death predicates. Transactions consume old records and create new ones by satisfying these predicates in zero knowledge. The on-chain footprint is constant: 968 bytes for a 2-input/2-output transaction, regardless of the complexity of the off-chain computation.

Zexe uses recursive proof composition (bounded depth 2, not full recursion) with a BLS-12 curve for inner SNARKs and a Cocks-Pinch curve for outer composition. Proof generation takes roughly one minute plus computation-dependent time. Verification takes tens of milliseconds.

9.5.3 Comparison

Property	Kachina	Zexe
State model	Account-based (state machine)	UTXO-based (records)
Data privacy	Parameterizable (Lambda function)	Full
Function privacy	No (function identity visible)	Yes (all transactions indistinguishable)
Concurrency	State oracle transcripts	UTXO model (naturally concurrent)
On-chain cost	O(transcript length)	Constant (968 bytes)
Proving cost	O(transcript length)	~1 minute + computation
Security model	UC-secure ($\mathcal{F}_{\text{nizk}}$, $\mathcal{G}_{\text{ledger}}$ hybrid)	Simulation-based

Midnight's architecture follows the Kachina model most closely – parameterizable disclosure via `disclose()`, account-based state, and compiler-enforced privacy boundaries. Aztec's design follows the Zexe model more closely – UTXO-based notes, client-side proving, and a Private Execution Environment (PXE) that handles proof generation.

For the system architect choosing between these approaches, the key question is: do you need function privacy? If yes (all transactions must be indistinguishable), the Zexe/UTXO model is the natural choice. If no (you can tolerate revealing which function was called, as long as the arguments are private), the Kachina/account model offers simpler programming and easier state management.

9.6 The Regulatory Intersection

Privacy-enhancing technologies do not operate in a regulatory vacuum. For the system architect building in 2026, two regulatory developments demand attention – and both, in different ways, are pulling the same direction as the technology.

9.6.1 GDPR and the Blockchain Immutability Paradox

The European Data Protection Board (EDPB) adopted Guidelines 02/2025 during its April 2025 plenary, providing the most authoritative guidance to date on GDPR compliance for blockchain systems. The guidelines address a fundamental tension: blockchain's immutability directly conflicts with Article 17 of GDPR – the right to erasure. If personal data is stored on-chain, it cannot be deleted. The regulation says it must be deletable. Something has to give.

The EDPB's answer is unambiguous: do not store personal data on-chain. The recommended architecture is "off-chain storage and hashing" – store personally identifiable information (PII) in a mutable off-chain database, and store only a cryptographic hash on-chain. If a data subject exercises their right to erasure, delete the off-chain data and the cryptographic keys linking it to the hash. The on-chain hash becomes an orphaned, meaningless string of characters. The commitment remains. The secret it referenced is gone.

ZKPs play a natural role in this architecture. Instead of storing "Alice is 25 years old and lives in Berlin" on-chain, store a hash of Alice's credential and allow Alice to generate ZK proofs about properties of that credential: "I am over 18" (for age-gated access), "I am a resident of the EU" (for jurisdictional compliance), "I am not on a sanctions list" (for regulatory compliance). The on-chain system never learns Alice's age, address, or identity. It learns only the truth of the specific claims she chooses to prove. The magician reveals exactly what the audience needs to see. No more.

This pattern is called zKYC (zero-knowledge Know Your Customer), and it is rapidly gaining traction. Galactica Network, zyphe, and hyli implement zKYC systems that enable selective disclosure for regulatory compliance. The promise: compliance without surveillance.

The tension between GDPR's right to erasure and blockchain's immutability is worth dwelling on, because it illustrates a deeper architectural principle. The naive response is to declare that blockchains and GDPR are incompatible – that you cannot have an append-only ledger and a right to delete. But the off-chain-storage-with-on-chain-hash pattern resolves the tension elegantly, and the resolution is instructive. The hash on-chain is not personal data. It is a commitment – a mathematical fingerprint that proves a piece of data existed at a particular time, without revealing what the data was. Delete

the off-chain data, destroy the linking keys, and the hash is cryptographically orphaned. It sits on the blockchain forever, a meaningless 32-byte string, pointing to nothing. The right to erasure is satisfied not by deleting the blockchain entry but by severing the link between the entry and the person it once referenced. The commitment survives. The secret is gone. The regulation is satisfied. This is not a workaround. It is good architecture – the kind of architecture that PETs make possible.

The zKYC pattern makes this concrete. Consider a bar that needs to verify a customer is over 21. Today, the customer shows a driver's license, and the bartender sees the customer's full name, date of birth, address, driver's license number, organ donor status, and photograph. The bartender needs exactly one bit of information: is this person at least 21? Instead, the customer reveals a dozen pieces of personally identifiable information to a stranger. With zKYC, the customer holds a verifiable credential in a digital wallet – issued and signed by the government – and generates a zero-knowledge proof: "the date of birth in my credential, when compared to today's date, yields an age of at least 21." The bartender's verification terminal checks the proof and the government's signature. It learns exactly one fact: the customer is old enough. The name, the address, the license number, the photograph – none of it crosses the bar. The trick reveals only what the audience needs to see.

9.6.2 eIDAS 2.0 and the European Digital Identity Wallet

The European Union's eIDAS 2.0 regulation, effective from 2024, mandates that all EU member states offer citizens a European Digital Identity Wallet by 2026. The wallet must support verifiable credentials and selective disclosure – proving specific attributes (citizenship, age, professional qualifications) without revealing the entire identity document.

ZKPs are a natural technical foundation for selective disclosure in identity wallets. The Architecture and Reference Framework (ARF) for eIDAS 2.0 envisions a credential ecosystem where issuers (governments, universities, professional bodies) issue cryptographically signed credentials, holders store them in their wallets, and verifiers check proofs of specific attributes without seeing the full credential.

The scale of this mandate is easy to understate. By late 2026, every EU member state must provide a digital identity wallet to every citizen who requests one. That is a potential user base of 450 million people. The wallet must interoperate across borders – a Spanish wallet must be accepted by a German verifier, a French credential must be verifiable in Italy. And the wallet must support selective disclosure by design, not as an afterthought. When a Belgian student presents her wallet to a Portuguese university, the university should be able to verify her degree and her citizenship without learning her tax ID, her medical history, or her home address. The credential is a bundle of attributes. The wallet discloses only the attributes the verifier needs. The rest stays sealed.

This is not a theoretical design. The EU's Large Scale Pilots (LSPs) – POTENTIAL, EU Digital Identity Wallet Consortium, NOBID, and DC4EU – have been testing these architectures since 2023, with real users, real credentials, and real cross-border verification. The technical challenge is not the ZKP itself (the cryptography is well understood) but the credential format, the revocation mechanism, the issuer trust framework, and the user interface that makes selective disclosure comprehensible to a non-technical user. The magician's trick is elegant. The stage production – lighting, sound, audience management – is where the engineering budget goes.

The regulatory pull is significant: eIDAS 2.0 creates a legal mandate for the privacy properties that ZKPs can provide. For the first time, a major regulatory framework is not just permitting but *requiring* selective disclosure. This transforms ZKPs from a voluntary privacy choice to a compliance necessity for any service that needs to verify European identities. The law now demands the trick.

9.6.3 Non-Compliance Is Expensive

GDPR violations can result in fines of up to 4% of global annual revenue or 20 million euros, whichever is greater. For a technology company with $10 billion in revenue, a GDPR violation could cost $400 million. This makes privacy architecture decisions directly material to business risk.

The implication for system architects: the choice of PET is not merely a technical decision. It is a risk management decision with quantifiable financial exposure. An architecture that stores personal data on a public blockchain is not just a privacy risk – it is a potential nine-figure liability.

9.7 The Decision Matrix

For the system architect who needs to choose a PET (or a combination of PETs), the decision depends on four questions:

1. What is the trust model? - If you need privacy against computationally unbounded adversaries (including future quantum computers): MPC with honest majority provides information-theoretic security. - If you trust computational hardness assumptions: ZKPs and FHE provide computational security. - If you trust hardware manufacturers: TEEs provide heuristic security with high performance.

2. Who has the data? - If the data holder needs to prove a property: ZKP (selective disclosure). - If multiple parties need to compute on their combined data: MPC (collaborative computation). - If the data needs to be processed by an untrusted third party: FHE (encrypted outsourcing). - If aggregate statistics need to be released from a dataset: DP (statistical disclosure control).

3. What performance is acceptable? - ZKP proof generation: seconds to minutes. Verification: milliseconds. - MPC: communication rounds proportional to circuit depth. Latency is the bottleneck. - FHE: 10,000-1,000,000x overhead over plaintext. Improving rapidly, but still orders of magnitude slower. - DP: negligible overhead (adding noise to query results is cheap). - TEE: near-native performance (<5% overhead for many workloads).

4. What regulatory regime applies? - GDPR/eIDAS 2.0: Selective disclosure (ZKP), off-chain storage with on-chain hashing, right-to-erasure compatibility. - Financial regulation (AML/KYC): zKYC (ZKP for compliance proofs), Privacy Pools (ZKP for provenance), MPC for inter-institutional analysis. - Healthcare (HIPAA, EU Clinical Trials Regulation): MPC for multi-site computation, DP for statistical releases, FHE for outsourced AI model training.

No single PET answers all four questions. The art is in composition – and the engineering is in the handoffs between them.

9.8 Open Problems

Three capabilities sit at the frontier of PET research and will likely shape the next generation of privacy architectures:

Verifiable FHE: Proving in zero knowledge that an FHE computation was performed correctly. This closes the loop in the healthcare scenario: the AI firm not only computes on encrypted data but also proves that it computed *correctly* on the encrypted data. The surgeon not only operates through the glovebox – she provides a certificate that the operation was performed to specification. The zkFHE project and SherLOCKED prototype (using RISC Zero's Bonsai zkVM) are early implementations.

Collaborative/threshold proving: Distributing ZK proof generation across multiple servers using MPC, so that no single server sees the full witness. The work by Ozdemir and Boneh (USENIX Security 2022) and subsequent improvements in 2024 demonstrate that proof generation itself can be privacy-preserving. This creates a fifth proving model – between client-side (private but expensive) and delegated (cheap but witness-exposing) – that combines the privacy of the former with the performance of the latter. The magician's backstage preparation is distributed across multiple locked rooms. No single stagehand sees the whole act.

Private Information Retrieval (PIR): Querying a database without revealing which record you are accessing. A client-side prover in a private rollup (like Aztec) needs to retrieve encrypted notes from the network without revealing which notes belong to them. Recent advances at EUROCRYPT 2026

achieved information-theoretic PIR with sublinear server time and quasilinear space, moving PIR from theoretical curiosity toward practical deployment for billion-entry databases.

9.9 The Incomplete Stack

A thread runs through this chapter that is worth stating plainly.

Privacy is not a feature that you add to a system after it is built. It is a property of the architecture, present from the first design document or absent forever. You cannot retrofit privacy onto a transparent blockchain any more than you can retrofit soundproofing onto a glass house. Or, to stay with our metaphor: you cannot add trapdoors to a stage after the audience is already seated.

The four PETs – ZKPs, MPC, FHE, and DP – are not competing technologies. They are complementary tools in a single toolkit. The magician's wand, the glovebox, the fog machine, and the locked vault where multiple parties contribute secrets they never share. Each excels at a different trust problem. Each fails at problems the others solve well. The system architect who understands all four, and who understands how they compose, has a genuine advantage over one who knows only ZKPs and treats every privacy problem as a nail to be hit with the zero-knowledge hammer.

The regulatory environment is, for the first time, pulling in the same direction as the technology. GDPR, eIDAS 2.0, and the global trend toward data sovereignty create legal mandates for exactly the capabilities that PETs provide. The question is no longer "should we use privacy-enhancing technologies?" but "which ones, in what combination, and how do we prove to regulators that they work?"

That last part – proving to regulators that the privacy technology works – is, fittingly, itself a zero-knowledge problem. And we have the tools to solve it.

Part III: Synthesis and the Road Ahead

The trick has been performed seven times, each time revealing a deeper layer of the mechanism. Now we step back from the stage. What does the whole show look like from the back of the theater? Where is the art heading? And does the magic hold up outside the theater, in the harsh light of commerce, regulation, and the passage of time?

Chapter 10: The Synthesis – Three Paths, Not Two

10.1 The Binary That Broke

For three years, the zero-knowledge community organized its world along a single axis: SNARK or STARK. Trusted setup or transparent. Small proofs or big ones. Algebraic elegance or hash-based brute force. Every conference talk, every investor pitch, every architectural decision began with this fork in the road.

That binary is dead.

It did not die because one side won. It died because the winning move turned out to be using both sides simultaneously – and then a third path appeared that neither side had anticipated. This is the story of how a two-path map became a three-path map, and why the seven-layer model from earlier in this book must bend to accommodate what actually happened.

10.2 The Map Redrawn

Chapter 1 presented seven layers as a stack – neat, linear, one resting on the next. Nine chapters of evidence say otherwise. The seven layers are a directed acyclic graph with fourteen causal edges, and we owe you the honest picture before proceeding.

```
                  ┌──────────────────────────────┐
                  │  THE SEVEN-LAYER CAUSAL WEB  │
                  │  (14 directed design-time edges)  │
                  └──────────────────────────────┘


   ┌─────────┐         ┌─────────┐         ┌─────────┐
   │ Layer 1 │◄────────│ Layer 2 │◄────────│ Layer 3 │
   │  Setup  │         │Language │         │ Witness │
   └────┬────┘         └────┬────┘         └────┬────┘
        │                   │ ▲                 │
        │                   │ │                 │
        ▼                   ▼ │                 ▼
   ┌─────────┐         ┌─────────┐         ┌─────────┐
   │         │         │ Layer 4 │◄────────│         │
   │         │         │  Arith  │- - - - ►│         │
   │         │         └────┬────┘         │         │
   │ Layer 5 │◄─────────────┘              │ Layer 7 │
   │  Proof  │◄──────────────┐             │ Verdict │
   │ System  │         ┌────┴────┐         │         │
   │         │◄────────│ Layer 6 │◄────────│         │
   │         │────────►│Primitive│────────►│         │
   └─────────┘         └─────────┘         └─────────┘


   ── ► Upward chain (6 edges): 6→5→4→3→2→1 (design flows up)
   ◄ ── Downward pressure (8 edges): 7→6, 7→5, 4→2, 3≡4,
        6→7, 1→5, 5→7, 2→3
```

Reading the diagram. The upward chain (six edges) is what the book has followed: field choice (Layer 6) determines commitment scheme (Layer 5), which shapes arithmetization (Layer 4), which constrains

witness layout (Layer 3), which influences ISA design (Layer 2), which determines setup scope (Layer 1). If this were the whole story, the stack metaphor would suffice.

It is not the whole story. Eight downward and cross-cutting edges complicate the picture. Ethereum gas economics (Layer 7) force BN254 pairings (Layer 6) and STARK-to-SNARK wrapping (Layer 5). Cairo's constraint system (Layer 4) dictated its ISA design (Layer 2). Jolt fused witness generation (Layer 3) into arithmetization (Layer 4). The graph has no cycles – design-time constraints are asymmetric – but it has width: multiple independent paths connect the same pair of nodes, and a single parameter change propagates through the web along all of them simultaneously.

The three paths below are routes through this DAG, not floors in a building. Each path makes different choices at each node, and the edges explain why those choices are coupled.

10.3 Path One: The Hybrid STARK-to-SNARK Pipeline

The dominant production pattern in 2026 is not SNARK. It is not STARK. It is STARK *wrapped in* SNARK.

Here is how the trick works. A prover generates a STARK proof – transparent, no trusted setup, post-quantum by construction, but large (50-200 KB) and expensive to verify on-chain (1-5 million gas on Ethereum). This STARK proof is then compressed through recursive aggregation and finally wrapped in a Groth16 SNARK proof – tiny (192 bytes), cheap to verify on-chain (~250-300K gas), but requiring a trusted setup ceremony (the Powers-of-Tau).

Think of it as a two-act show. In the first act, the magician performs behind a glass wall – everything is transparent, no trapdoors, no hidden compartments. In the second act, the performance is photographed, and the photograph is sealed in a tamper-proof envelope small enough to slip under a door. The audience in the theater (the blockchain) only sees the envelope. But anyone who opens it can verify that it faithfully captures the transparent performance.

Every major production system does this or plans to. SP1 Hypercube generates multilinear STARK proofs over the BabyBear field, recursively compresses them, then wraps the result in Groth16 over BN254 for Ethereum verification. Stwo generates Circle STARK proofs over the Mersenne-31 field, aggregates them via SHARP, then wraps to Groth16 via Herodotus for Ethereum settlement. RISC Zero, Airbender, ZisK, Pico Prism – all follow the same pattern. Even StarkWare, the company that built its identity on transparent proving, wraps to Groth16 for Ethereum L1 because the gas economics demand it.

The wrapping pipeline is not a Layer 5 phenomenon. It pierces three layers simultaneously: Layer 5 (the proof system switches from STARK to SNARK), Layer 6 (the field transitions from BabyBear or M31 to BN254), and Layer 7 (the verification target shifts from prover-internal consistency to EVM smart contract). This vertical shaft through the stack is invisible in the original framing, which treats STARKs and SNARKs as competing alternatives occupying the same layer.

Why did this happen? Economics. Pure and simple. A raw STARK verification costs 1-5 million gas on Ethereum – roughly $5-$25 at typical gas prices. A Groth16 verification costs ~250K gas – roughly $0.50-$1.00. When you amortize this across thousands of transactions per batch, the difference is enormous. The inner STARK gives you transparency and post-quantum readiness. The outer SNARK gives you on-chain affordability. The combination gives you both.

The hybrid path has a structural weakness that deserves a name: the outer Groth16 wrapper is not post-quantum. Even though the inner STARK is quantum-resistant, the final on-chain verification depends on BN254 pairings, which Shor's algorithm breaks in polynomial time. The chain of trust is only as strong as its weakest link. Today's hybrid systems are transparent and fast on the inside, but they inherit quantum vulnerability from their verification wrapper. The glass wall is strong. The envelope has an expiration date.

10.4 Path Two: Pure Transparent

The Ethereum Foundation has staked a different position. For the L1 zkEVM – the project to prove every Ethereum block in zero knowledge – the mandate is explicit: no trusted setup, period. This is not a preference; it is a requirement. The reasoning is straightforward: Ethereum's base layer cannot depend on a trusted ceremony that quantum computers will eventually break. The stage must be made of glass, all the way down.

This mandate forces a different engineering path. Pure transparent systems use only hash-based commitments (FRI, Merkle trees) and avoid all pairing-based cryptography. The cost is larger proofs and more expensive on-chain verification. The benefit is that no ceremony is ever needed, no toxic waste exists, and the security assumptions survive quantum computing (assuming hash functions with sufficient output size).

The EF's December 2025 pivot is instructive. Having declared the speed race "effectively won" – four teams proved >99% of Ethereum blocks within the 12-second slot time – the Foundation shifted its 2026 targets to security: 100-bit provable security by May 2026, 128-bit by December 2026. The new primary

metric is energy consumption per proof (kWh), not raw speed. The Foundation explicitly rejects reliance on unproven mathematical conjectures (proximity gap assumptions) for production soundness.

This matters because it validates a specific engineering philosophy: formal provable security over empirical performance. SP1 Hypercube already eliminates proximity gap conjectures in its multilinear STARK. The EF's security-first stance means that systems with strong formal guarantees will be preferred for the most security-critical application in the ecosystem – proving the base layer itself.

Brevis's Pico Prism occupies an interesting position on this path. Built on Plonky3 with hash-based FRI commitments, it achieves 99%+ real-time proving on 16 RTX 5090 GPUs while remaining fully transparent. For non-Ethereum-L1 applications that still need on-chain verification, it wraps to Groth16 – but the inner pipeline is transparent end to end.

10.5 Path Three: Post-Quantum Folding

The third path is the newest and least traveled. It abandons both pairing-based cryptography and hash-based commitments in favor of lattice-based constructions that provide additive homomorphism (enabling folding), post-quantum security, and increasingly competitive proof sizes.

The key systems are LatticeFold (Boneh & Chen, ASIACRYPT 2025), LatticeFold+ (CRYPTO 2025), Neo/SuperNeo (Nguyen & Setty, ePrint 2025/294), and Symphony (Chen, ePrint 2025/1905). These systems use Module-SIS commitments – lattice-based structures whose hardness NIST has validated through the FIPS 203/204 standardization process – to achieve folding over polynomial rings without relying on discrete logarithms or pairings.

The proof sizes are larger than Groth16 but rapidly improving: Greyhound (Nguyen & Seiler, CRYPTO 2024) achieves ~50 KB proofs with lattice-based commitments, and LaBRADOR (CRYPTO 2023) achieves ~58 KB. These are orders of magnitude larger than Groth16's 192 bytes but competitive with raw STARK proofs. The performance trajectory suggests that lattice-based schemes may reach practical production within 3-5 years.

Why does this path matter? Because it is the only path that survives quantum computing without any caveats. The hybrid path (Path One) is quantum-vulnerable at the verification wrapper. The pure transparent path (Path Two) relies on hash functions whose collision resistance degrades under quantum attack (SHA-256 drops from 128-bit classical to ~85-bit quantum collision resistance via the BHT algorithm). The lattice path relies on Module-LWE/SIS, which is believed to be quantum-resistant by design – the same assumption family that NIST chose for its post-quantum standards. This path does not merely survive the quantum era. It was built for it.

The post-quantum folding path has a structural advantage that maps directly onto the Layer 6 analysis from Chapter 7. That chapter presented a trilemma: algebraic functionality, post-quantum security, and succinctness – pick two. Lattice-based commitments provide additive homomorphism (enabling folding, which is a form of algebraic functionality), post-quantum security, and increasingly competitive succinctness. The trilemma is not being sidestepped; it is being actively compressed. Whether it can be fully dissolved remains an open question – KZG's $O(1)$ proof size with full homomorphism has no post-quantum match yet. But the gaps are narrowing with each paper.

10.6 The Three-Path Table

Path	Setup	Inner Primitive	Outer Verification	PQ Status	Production Status
Hybrid STARK-to-SNARK	STARK inner (transparent) + Groth16 outer (trusted)	Hash-based FRI / Merkle	BN254 pairing (~250K gas)	Inner: quantum-safe; Outer: quantum-vulnerable	Dominant production default
Pure Transparent	Transpar only	Hash-based FRI, no pairings	Large on-chain proof or alternative verification	Quantum-safe (with sufficient hash output)	Ethereum L1 mandate; advancing
Post-Quantu Folding	Transpar (lattice-based)	Module-SIS commitments	Lattice verification (higher cost)	Quantum-safe by design	Research frontier; 3-5 year horizon

10.7 The Causal Web: Why It Is a DAG, Not a Stack

The book has presented seven layers as floors in a building – stacked, with each resting on the one below. The evidence from Parts I and II tells a different story. The layers are not a stack; they are a directed acyclic graph (DAG) with bidirectional pressures. The building metaphor was useful for learning. Now we must complicate it.

The most consequential causal chain runs *upward* from Layer 6. Small-field primitives (BabyBear, M31) at Layer 6 enabled Circle STARKs and efficient multilinear proving at Layer 5, which enabled lookup-based and AIR-based arithmetization at Layer 4, which shaped shard-based witness generation at Layer 3, which favored RISC-V ISAs at Layer 2, and universal zkVMs amortized setup costs at Layer 1. The foundation shaped the building. That much the metaphor gets right.

But there are equally important *downward* pressures. The audience shapes the show:

Layer 7 forces Layer 6. Ethereum gas economics demand Groth16 verification, which requires BN254 pairings, which constrains the outer proof system. The verifier's economics shape the prover's cryptography. The audience's ticket price dictates the magician's technique.

Layer 7 forces Layer 5. STARK-to-SNARK wrapping exists because Ethereum's gas costs make raw STARK verification uneconomical. The verification layer forces a compression step that the proof system layer would not otherwise need.

Layer 2 constrains Layer 4. Cairo was designed so that its ISA minimizes arithmetization cost. The language was shaped by the constraint system, not the other way around. Layer 4 requirements propagated upward to shape Layer 2 design. The choreography was rewritten to fit the stage machinery.

Layer 3 collapses into Layer 4. In Jolt, witness generation *is* the arithmetization – every instruction is decomposed into lookups on subtables. There is no meaningful boundary between "generate the trace" and "encode the computation."

The pedagogical ordering (Layer 1 first, Layer 7 last) follows the *data flow*: setup before language, language before witness, witness before proof. This is how a user encounters the system. But the *engineering causality* is inverted: the field choice at Layer 6 determines the commitment scheme, which determines the polynomial representation, which determines the arithmetization, which shapes everything above it.

These four examples are not anomalies. They are the norm. Once you catalog every causal arrow in the system, the picture that emerges is not a tower but a web – and the web has a specific mathematical structure that is worth naming precisely.

10.7.1 The Shape of the Web

Roger Penrose, in *The Road to Reality*, draws a distinction between structures that are merely complicated and structures that are *irreducibly entangled*. A stack is complicated: many parts, one ordering. A DAG is entangled: many parts, many orderings, no cycles. The seven-layer model, once you draw all the arrows, is a DAG with at least fourteen directed edges and zero cycles. It has structure, but that structure is not linear.

Consider the full edge set. Layer 6 forces Layer 5 (field choice determines commitment scheme). Layer 5 forces Layer 4 (commitment scheme shapes arithmetization). Layer 4 shapes Layer 3 (constraint format determines witness layout). Layer 3 shapes Layer 2 (witness cost influences ISA design). Layer 2 shapes Layer 1 (ISA scope determines setup complexity). That is the upward chain – six edges, roughly linear, roughly matching the pedagogical ordering. If this were all, the stack metaphor would suffice.

But it is not all. Layer 7 forces Layer 6 (gas economics demand BN254). Layer 7 forces Layer 5 (verification cost forces wrapping). Layer 4 forces Layer 2 (constraint cost shapes ISA, as Cairo demonstrates). Layer 3 collapses into Layer 4 (Jolt's lookup singularity). Layer 6 forces Layer 7 (field size determines proof size, which determines verification cost). Layer 1 forces Layer 5 (setup type constrains which proof systems are available). Layer 5 forces Layer 7 (proof format determines verifier contract design). Layer 2 forces Layer 3 (ISA instruction count determines trace width).

That is fourteen edges among seven nodes. The graph has no cycles – you cannot follow arrows from any node back to itself – which is what makes it a DAG rather than a general directed graph. But the graph has *width*: multiple independent paths connect the same pair of nodes. Layer 6 reaches Layer 7 both directly (field determines proof size) and indirectly via Layer 5 (field determines proof system, which determines verifier). Layer 7 reaches Layer 5 both directly (gas cost forces wrapping) and indirectly via Layer 6 (gas cost forces BN254, which constrains proof system). These parallel paths are why changing a single parameter – say, the base field – propagates unpredictably through the stack. The change follows multiple routes, and those routes interfere with each other.

Penrose would recognize this as a feature, not a bug. Physical theories have the same structure: general relativity and quantum mechanics are not stacked but entangled, each constraining the other through multiple channels. The seven layers of zero-knowledge proofs exhibit the same irreducible entanglement. You cannot understand Layer 5 (the proof system) without simultaneously understanding Layer 6 (the field) and Layer 7 (the verifier). You cannot design Layer 2 (the ISA) without understanding Layer 4 (the constraint system). The system is not modular. It is coherent – every part is connected to every other part through at most two hops.

10.7.2 Why No Cycles?

The absence of cycles is not obvious and deserves explanation. Why can you not follow arrows from Layer 7 back to Layer 7?

The answer is that the arrows represent *design-time constraints*, not *runtime data flow*. At runtime, data flows in a rough circle: the user submits a transaction (Layer 2), which generates a witness (Layer 3), which is arithmetized (Layer 4), which is proved (Layer 5), which is verified (Layer 7), which triggers a state change that enables the next transaction (back to Layer 2). That loop is a cycle, and it is real. But the *design* constraints – which architectural choices force which other architectural choices – are acyclic. Choosing M31 at Layer 6 forces Circle STARKs at Layer 5, but choosing Circle STARKs at Layer 5 does not force M31 at Layer 6 (you could use any Mersenne prime). The arrows are asymmetric. The forcing goes one way.

This distinction – cyclic runtime flow, acyclic design constraints – explains why the seven-layer model is useful despite being wrong. The model captures the design-time DAG by projecting it onto a linear ordering. The projection loses information (it hides the downward and cross-cutting arrows) but preserves the acyclicity. A stack is the simplest DAG. The seven-layer stack is the simplest correct projection of the seven-layer web. It is a useful lie that points toward a more interesting truth.

A seven-layer model that acknowledges this bidirectionality is more accurate than one that implies simple top-down dependency. The layers are aspects of a single integrated system, not modules with clean interfaces. The magician, the stage, and the audience are not separable. They are one show.

10.8 Trust Decomposition: Seven Weaker Assumptions

Now we arrive at the heart of the matter.

Everything in this book has been building toward this section. The seven layers, the three paths, the proof core, the causal DAG – all of it converges on a single observation, and it is the most important thing this book has to say.

A traditional financial system requires trusting a single institution. The bank holds your money, knows your balance, controls your transactions, and you trust that it will behave honestly. You trust *one entity* with *everything*.

Zero-knowledge proofs do not eliminate trust. They do something different. They *decompose* it. They shatter a single monolithic trust assumption into seven independent, weaker assumptions. And because

the assumptions are independent, no single failure breaks the system. This is not trustlessness. It is something better: trust that is distributed, auditable, and replaceable.

Here are the seven assumptions, and each one is a thread you can pull:

Layer 1: At least one of N ceremony participants was honest (for trusted setups), or that hash functions are collision-resistant (for transparent setups). This is the trust in the stage itself – that the mathematical parameters were generated fairly.

Layer 2: The circuit was correctly written and audited. The under-constrained circuit epidemic catalogued in Chapter 3 remains the dominant vulnerability class, and a bug here lets the prover prove false statements. This is the trust in the choreography: that the script describes the trick accurately.

Layer 3: The hardware running the prover does not leak the witness through side channels. Timing attacks on Zcash's Groth16 prover leaked transaction amounts with $R = 0.57$ correlation. Cache timing attacks on ZK-friendly hash functions (Poseidon, Reinforced Concrete) have been demonstrated in cloud environments. This is the trust in the backstage – that no one can see behind the curtain through cracks in the wall.

Layer 4: The arithmetization correctly encodes the computation. If the translation from program to polynomial constraints is wrong, the proof system faithfully proves the wrong thing. This is the trust in the encoding – that the mathematical puzzle accurately represents the trick.

Layer 5: The proof system is sound – no efficient adversary can forge proofs. This depends on the Fiat-Shamir transform being correctly implemented (the Frozen Heart bug of 2022 affected three independent implementations simultaneously) and on the underlying interactive proof being sound. This is the trust in the seal – that the certificate cannot be forged.

Layer 6: The mathematical hardness assumptions hold. Discrete logarithms are hard (or lattice problems are hard, or hash preimages are hard). These are conjectures, not theorems. Tower NFS improvements already reduced BN254's estimated security from ~128 bits to ~100 bits. This is the trust in mathematics itself – the deepest assumption, and the one we have the least power to verify.

Layer 7: The governance structure will not override the cryptography. Most deployed ZK rollups are Stage 0 or Stage 1 on L2Beat's framework, meaning a multisig can override the proof system. The Beanstalk flash-loan governance attack ($182M, April 2022) and the Tornado Cash CREATE2 contract replacement (May 2023) demonstrate that governance can be exploited. This is the trust in the theater management – that the people who run the venue will not rig the show.

10.8.1 When Each Thread Snaps: Seven Failure Scenarios

The seven assumptions above are not hypothetical. Every one of them has failed, is failing, or will fail in a live system. Isaac Asimov once observed that the most exciting phrase in science is not "Eureka!" but "That's funny..." – the moment when an assumption you did not know you were making turns out to be wrong. In zero-knowledge systems, "That's funny..." is the sound of money disappearing. Here is what each failure looks like in practice, what breaks, and – critically – what does *not* break.

If Layer 1 fails (compromised setup): An attacker who knows the toxic waste from a trusted setup ceremony can forge proofs of arbitrary statements. They can mint tokens from nothing, approve transactions that never happened, fabricate state transitions whole cloth. The terrifying property of this failure is its *invisibility*. A forged Groth16 proof is indistinguishable from a legitimate one – both are 192 bytes, both pass the verifier, both look identical on-chain. No alarm fires. No anomaly appears in the logs. The counterfeiting is perfect by construction. This is why Zcash's Powers-of-Tau ceremony involved 87 independent participants across six continents – the assumption is that at least one of them destroyed their toxic waste. If all 87 were compromised (through coercion, incompetence, or a coordinated state-level attack), the entire shielded pool would be silently forgeable. Note what does *not* break: the circuit logic (Layer 2) is still correct, the witness privacy (Layer 3) is still intact, the math (Layer 6) still holds. The stage was rigged, but the trick's choreography was genuine.

If Layer 2 fails (buggy circuit): The proof system faithfully proves a false statement. This is the most common failure mode in production, and the canonical example is Tornado Cash. A single missing constraint in Tornado Cash's withdrawal circuit allowed an attacker to generate valid proofs for withdrawals from deposits that never existed. The circuit was supposed to verify that a nullifier corresponded to a real deposit commitment in the Merkle tree. A missing range check meant the prover could satisfy the constraints with fabricated values. The proof was *valid* – it passed the verifier – because the verifier only checks that the proof matches the circuit, and the circuit was wrong. The Zcash "InternalH" bug (CVE-2019-7167, February 2019) is the same species: a missing check in the Sapling circuit would have allowed unlimited counterfeiting of shielded ZEC. Found by a Zcash engineer during routine review, it was quietly patched before exploitation. The gap between "found by an auditor" and "found by an attacker" was a matter of months. Circom's under-constrained circuit epidemic – where developers use the `<--` assignment operator instead of the `<==` constraining operator – produces the same failure at industrial scale. The proof system is not broken. The program is broken. The seal is perfect; the document it seals is a forgery.

If Layer 3 fails (witness leakage): The zero-knowledge property evaporates. The proof is still valid, and the computation is still correct, but the secret inputs are exposed. The system proves the truth and

simultaneously reveals what it was supposed to hide. Timing side-channel attacks on Zcash's Groth16 prover demonstrated this concretely: by measuring how long proof generation took, an observer could infer the transaction amount with $R = 0.57$ correlation. The proof said "this transaction is valid" without revealing the amount – but the *time it took to generate the proof* leaked the amount through a side channel. In cloud proving environments (AWS, GCP), cache-timing attacks on ZK-friendly hash functions like Poseidon and Reinforced Concrete are even more direct: a co-located VM can observe memory access patterns and reconstruct the witness. This is a privacy catastrophe but not an integrity catastrophe. The computations are still correct. The proofs are still sound. But the magician's secrets are visible through cracks in the dressing room wall.

If Layer 4 fails (incorrect arithmetization): The constraint system does not faithfully represent the computation it claims to encode. This is subtler than a Layer 2 bug because the *program* may be correct – the error is in the *translation* from program to polynomial constraints. Consider a zkVM that claims to prove RISC-V execution. If the arithmetization incorrectly encodes the behavior of, say, the `slt` (set-less-than) instruction – treating signed comparison as unsigned, or failing to handle the overflow edge case at INT_MIN – then the zkVM produces valid proofs of executions that never happened. The program was correct. The proof system was sound. The encoding between them was wrong. This failure is especially dangerous in hand-rolled constraint systems (pre-zkVM era), where a developer manually translates each operation into R1CS or AIR constraints. SP1's formal verification of all 62 RISC-V opcodes against the Sail specification exists precisely to prevent this class of failure. The trust is not in the magician or the seal, but in the translator standing between them – and translators make mistakes.

If Layer 5 fails (broken proof system): An attacker can forge proofs without knowing the witness. The Frozen Heart vulnerability (2022) is the textbook case. Three independent implementations of the Fiat-Shamir transform – Bellman (Zcash), Gnark (ConsenSys), and an academic reference – all made the same mistake: they failed to bind the public inputs to the transcript hash. An attacker could take a valid proof for one statement and re-use it for a different statement. The proof said "I know a witness for X" but could be replayed to claim "I know a witness for Y." All three implementations were broken simultaneously, because all three misunderstood the same subtle requirement of the Fiat-Shamir transform. This is a soundness catastrophe. The seal can be forged. Valid-looking proofs can be manufactured for false statements. Unlike a Layer 2 failure (where the circuit is wrong but the proof system is honest), a Layer 5 failure means the proof system itself is compromised. Every proof it has ever generated becomes suspect.

If Layer 6 fails (broken hardness assumption): The mathematical foundation dissolves. This has not happened catastrophically yet, but it has happened incrementally, and incrementally is frightening

enough. Tower NFS improvements reduced BN254's estimated security from approximately 128 bits to approximately 100 bits – not a break, but an erosion. The 2023 lattice basis reduction advances by Ducas and van Woerden tightened the known attacks on NTRU lattices, and while they did not break any deployed scheme, they moved the boundary closer. A full break of BN254's discrete logarithm problem would compromise every Groth16 proof ever generated on that curve: past, present, and future. Every wrapped STARK-to-SNARK proof on Ethereum would be forgeable. Every ZK rollup using BN254 verification would lose its security guarantee retroactively. Shor's algorithm achieves exactly this for all pairing-based and elliptic-curve cryptography, given a sufficiently large quantum computer. The question is not *whether* this assumption will weaken but *when* and *how fast*. This is the failure that the post-quantum folding path (Path Three) exists to survive.

If Layer 7 fails (governance override): The cryptography is irrelevant because the humans in charge simply bypass it. This is the most prosaic failure and arguably the most dangerous, because it requires no mathematical sophistication – only social engineering, legal coercion, or economic incentive. The Beanstalk flash-loan governance attack ($182M, April 2022) demonstrated the economic version: an attacker borrowed enough governance tokens to pass a malicious proposal in a single transaction, draining the protocol's treasury. The cryptography was never touched. The proofs were never forged. The governance mechanism – the human layer above the math – was the attack surface. The Tornado Cash CREATE2 replacement (May 2023) demonstrated the supply-chain version: the deployer address used CREATE2 to replace the governance contract with a malicious one, granting the attacker control over all locked funds. Again, no cryptographic break was needed. Most deployed ZK rollups operate with upgrade multisigs that can push new verifier contracts, effectively overriding any proof system guarantee. If three of five multisig holders collude (or are coerced by a state actor), they can deploy a verifier that accepts all proofs, or no proofs, or only proofs from approved provers. The mathematical fortress has a human door, and the door has a human lock.

10.8.2 The Cascade Structure

Not all failures are created equal, and not all are independent. The seven assumptions form their own internal DAG of failure propagation.

A Layer 6 failure cascades into Layer 5 (if the hardness assumption breaks, the proof system built on it is unsound) and Layer 1 (if discrete logs are easy, trusted setup toxic waste can be reconstructed). But it does *not* cascade into Layer 2 (the circuit logic is still correct), Layer 3 (the witness generation is still private against non-quantum adversaries), or Layer 4 (the arithmetization is still faithful). A quantum computer that breaks BN254 makes Groth16 proofs forgeable, but it does not introduce bugs into Circom circuits. The choreography is fine. The seal is broken.

A Layer 2 failure is *contained*. A buggy circuit produces provably wrong results, but the proof system is still sound (it just proves the wrong thing), the setup is still valid, the hardware does not leak, and the math still holds. This containment is precisely what makes Layer 2 failures survivable: fix the circuit, redeploy the verifier, and the system recovers. The Zcash InternalH bug was patched in a single release. The stage machinery was fine; only the script needed rewriting.

A Layer 7 failure is the most isolated and the most devastating. It requires no interaction with any other layer. A governance override does not break the math, corrupt the circuit, or leak the witness. It simply ignores all of them. This isolation means that Layer 7 cannot be fixed by improving any other layer. Better proof systems, stronger fields, more rigorous audits – none of these matter if a three-of-five multisig can replace the verifier contract. The only defense is the same defense that human institutions have always relied on: constitutional constraints, time-locks, social consensus, and the slow, unglamorous work of governance design.

The cascade structure reveals a counterintuitive truth about the seven-layer model. The *deepest* failures (Layer 6: math breaks) are the most catastrophic in scope but the least likely in practice. The *shallowest* failures (Layer 2: buggy circuit, Layer 7: governance override) are the most common in practice but the most recoverable. The threat landscape is inverted: the risks you encounter most often are the risks you can fix most easily. This inversion is what makes the trust decomposition genuinely useful. A monolithic trust model (trust the bank) gives you one failure mode: total. A decomposed trust model gives you seven failure modes, most of which are partial and recoverable. The system degrades gracefully, and graceful degradation is the definition of resilient engineering.

Each of these is independently falsifiable, independently auditable, and independently improvable. Breaking one does not necessarily break the others (though some failures cascade – a quantum computer breaks Layers 1, 5, and 6 simultaneously for pairing-based systems). This decomposition is the

genuine value proposition of zero-knowledge proofs: not trustlessness, but trust minimization through distribution.

Remember the bank from Chapter 1? The institution that holds your money, knows your balance, and controls your transactions? With zero-knowledge proofs, you no longer trust one bank with everything. You trust that at least one ceremony participant was honest. You trust that the circuit was correctly written. You trust that the hardware does not leak. You trust that the math is hard. You trust that the governance will not go rogue. Seven assumptions instead of one. Each weaker. Each testable. Each replaceable.

That is not a marketing slogan. It is a structural transformation.

10.9 "Trustless" versus "Trust-Minimized"

The analysis concludes with a question that should be printed on the wall of every ZK team's office:

> If zero-knowledge proofs provide "trustless" computation, but the setup requires trusting ceremony participants (Layer 1), the program requires trusting the developer not to make constraint errors (Layer 2), the witness requires trusting the hardware not to leak side channels (Layer 3), the proof system requires trusting that mathematical hardness assumptions hold (Layer 6), and the verifier requires trusting that governance will not override the math (Layer 7) – then where, exactly, is the "trustless" part?

The answer: nowhere. "Trustless" is a word that flatters the technology and misleads the user. Zero-knowledge proofs do not eliminate trust. They minimize and distribute it. Instead of trusting one bank with your financial data, you trust that: (a) at least one ceremony participant was honest, (b) the circuit was correctly written and audited, (c) the hardware is not leaking, (d) discrete logarithms are hard, and (e) the governance multisig will not go rogue.

Each of these is a weaker assumption than trusting a single entity. The combination is far more resilient than any single point of trust. But they are assumptions nonetheless, and a responsible guide to zero-knowledge proofs should catalog them explicitly.

We stated this thesis in the opening pages of Chapter 1: trust decomposition, not trust elimination. Ten chapters later, the decomposition is precise. Seven assumptions instead of one. Fourteen causal edges instead of a monolith. Three architectural paths, each with different failure profiles. The word "trustless" obscures every one of these distinctions. The word "trust-minimized" preserves them.

The remaining chapters of this book will use “trust-minimized” rather than “trustless.” Not because it sounds better – it sounds worse, deliberately – but because it is accurate. And accuracy in naming things is where understanding begins.

Chapter 11: zkVMs – The Universal Stage

11.1 zkVMs Across the Stack

We need to talk about the thing that changed everything.

The zkVM is not a Layer 2 phenomenon. It is a technology that reaches into every layer of the stack – from the field choice at Layer 6, through the arithmetization at Layer 4, to the verification economics at Layer 7. If the seven-layer model is the map, the zkVM is the earthquake that reshaped the terrain.

Before zkVMs, every zero-knowledge application required building a custom stage: hand-crafting constraint systems, choosing a field, designing a witness format, writing a custom verifier. Each application was a bespoke production – a one-night show with its own scenery, its own props, its own choreography. The zkVM changed this by providing a **universal stage** that can host any trick. A developer writes ordinary Rust, compiles it to RISC-V, and the zkVM handles everything else: witness generation, arithmetization, proving, compression, and verification. The stage is universal; the performances are infinite.

The preceding layer-by-layer analysis reveals why this matters so deeply. The most urgent findings at nearly every layer are not isolated gaps but consequences of the same underlying shift. Polygon zkEVM's

shutdown at Layer 2, the Witness Gap's amplification at Layer 3, CCS and LogUp's emergence at Layer 4, folding's rise and the hybrid pipeline's dominance at Layer 5, the small-field revolution at Layer 6, STARK-to-SNARK wrapping at Layer 7 – these are the seismic effects of a single tectonic event: the zero-knowledge virtual machine reorganized the entire stack around itself.

11.2 The Landscape Table (March 2026)

The numbers tell the story concisely. Eight of ten major zkVMs now target RISC-V. Only Stwo (Cairo) and zkWASM (WebAssembly) hold out – and even StarkWare's ecosystem hedges via Kakarot's EVM-on-Stwo path. The Ethereum Foundation declared the speed race "effectively won" in December 2025 and pivoted to 128-bit provable security by end of 2026.

	SP1 Hypercube	RISC Zero	Jolt	Stwo	Airbender	ZisK	Pico Prism
Org	Succinct	RISC Zero	a16z	StarkWare	Matter Labs	SilentSig	Brevis
ISA	RISC-V (RV32IM)	RISC-V (RV32IM)	RISC-V (RV32I)	Cairo (custom)	RISC-V (RV32IM)	RISC-V 64	RISC-V (RV32IM)
Arithmetiz	Multilinear AIR + LogUp-GKR	AIR (DEEP-ALI)	R1CS + Lasso lookups	Circle AIR + LogUp	AIR (degree-2)	AIR / PIL	AIR (Plonky3)
Proof system	Multilinear STARK	FRI STARK	Spartan sum-check	Circle STARK	DEEP STARK	STARK (recursive)	STARK (Plonky3)
Field	BabyBear (31-bit)	BabyBear (31-bit)	BN254 (256-bit)	M31 (31-bit)	M31 (31-bit)	Goldilocks (64-bit)	BabyBear / M31
SNARK wrap	Groth16	Groth16	Planned	None (native)	Groth16	Groth16	Groth16

	SP1 Hypercube	RISC Zero	Jolt	Stwo	Airbender	ZisK	Pico Prism
Eth block	6.9 s / 16 GPU	44 s / cluster	N/A	N/A (Cairo)	35 s / 1 GPU	6.6 s / 24 GPU	6.9 s / 16 GPU
Maturity	Production	Production	Beta	Production	Production	Adv. testnet	Production

Glossary. *ISA* = instruction set architecture, the fundamental language a processor understands. *Arithmetization* = the method of translating computation into mathematical equations. *AIR* = Algebraic Intermediate Representation, encoding computation as polynomial constraints over an execution trace. *LogUp-GKR* = a sumcheck-based lookup argument. *Circle STARK* = a STARK adapted to work over the circle group of a Mersenne prime. *SNARK wrap* = compressing a large transparent proof into a small proof for cheap on-chain verification. *Field* = a set of numbers with special arithmetic properties; "31-bit" and "64-bit" refer to element size, with smaller fields enabling faster operations on modern hardware.

Three systems dropped from this table deserve brief mention. Nexus 3.0 abandoned Nova-based folding for a Stwo backend after observing a 1000x speed penalty from classical folding – a telling result for the practical viability of folding in production zkVMs. Valida uses a custom stack-based ISA designed from scratch for ZK proving, with no independent large-scale benchmarks. zkWASM is the only remaining PLONKish/KZG outlier, targeting WebAssembly rather than RISC-V.

For architects choosing a zkVM, the landscape table above describes *what exists*. The rubric below describes *how to choose*:

Goal	Recommended	Rationale
General-purpose execution	SP1 Hypercube, RISC Zero	RISC-V, mature tooling, broadest Rust ecosystem support
Maximum throughput	SP1 Hypercube, Airbender	Best Ethereum block proving benchmarks (6.9s, 21.8M cycles/sec)

Goal	Recommended	Rationale
ZK-native efficiency	Stwo (Cairo)	Purpose-built ISA eliminates translation overhead; Starknet-native
Post-quantum trajectory	Neo/SuperNeo (watch)	Lattice-based, CCS-native, 127-bit PQ security; 3-5 year horizon
Lookup-heavy design	Jolt	Lasso decomposable lookups; sumcheck-native; avoids NTT bottleneck
Minimum trusted setup	Stwo, Pico Prism	Fully transparent (hash-based FRI); no ceremony required
Formal verification	SP1	62 RISC-V opcodes verified against Sail spec; strongest correctness guarantees

11.3 Three zkVMs Through Seven Layers

The best way to understand why the seven-layer model bends under zkVM pressure is to trace three representative systems through all seven layers. Watch how they cross the same territory by different routes.

11.3.1 SP1 Hypercube: The General-Purpose Champion

Layer 1 (Setup). Hybrid – transparent inner loop (hash-based Poseidon2 over BabyBear), trusted outer wrap (Groth16 over BN254). The KZG ceremony is for the wrapper only. This demonstrates that the trusted-or-transparent choice is actually "both."

Layer 2 (ISA). RISC-V (RV32IM). Developers write ordinary Rust; the compiler handles the rest. SP1 implements 39+ RISC-V instructions, with all 62 core opcodes formally verified against the official RISC-V Sail specification.

Layer 3 (Witness). Shard-based execution traces with continuations. Each shard is an independent proving unit; "shared challenges" enforce consistency at shard boundaries. This is where the Witness Gap lives – SP1's witness generation is CPU-bound while its proving is GPU-accelerated, meaning witness generation consumes an estimated 60-70% of total proving time.

Layer 4 (Arithmetization). Multi-table AIR with LogUp-GKR cross-table lookups over multilinear polynomials. Each RISC-V instruction type has its own constraint table ("chip"). Precompiles (SHA-256, Keccak, secp256k1) are independent STARK tables connected via LogUp.

Layer 5 (Proof). Four-stage pipeline: core STARK proof per shard, recursive compression, shrink (field transition), Groth16 wrap. Proves 99.7% of Ethereum L1 blocks in under 12 seconds on 16 RTX 5090 GPUs.

Layer 6 (Primitives). BabyBear field (31-bit), Poseidon2 hash, Jagged PCS (commits only to occupied trace rows, eliminating padding waste), FRI-based commitment.

Layer 7 (Verifier). Groth16 on-chain verification at approximately 250-300K gas on any EVM chain. Live on Ethereum mainnet via the Succinct Prover Network.

What makes SP1 architecturally distinctive. SP1 Hypercube is an exercise in *factoring a monolithic problem into independent pieces and then reassembling them under a single algebraic umbrella*. The system's defining architectural idea is not any single layer but the interaction between three design choices that reinforce each other. First, the multi-table AIR architecture assigns each RISC-V opcode its own constraint table – a "chip" – so that the constraint degree and column count for a SHA-256 precompile need not compromise the constraint shape for a simple register-to-register add. Second, LogUp-GKR cross-table lookups bind these independent chips together using sumcheck over multilinear extensions, which avoids the quadratic blowup that a naive permutation argument would impose as chip count grows. Third, sharding with continuation challenges means that execution traces of arbitrary length can be sliced into fixed-size proving units, each provable in parallel on a separate GPU, with algebraic consistency enforced by shared random challenges drawn after the shards are committed.

The result is a system that scales *horizontally* in two independent dimensions: more instruction types (more chips) and longer executions (more shards). Adding a new precompile – say, a BLS12-381 pairing for Ethereum validator operations – requires only a new chip table and new LogUp entries; existing chips and the recursive compression pipeline remain unchanged. This modular extensibility explains why SP1 has accumulated over 39 instruction types and counting, whereas architectures with monolithic trace layouts face painful refactoring when adding even one new opcode.

The Jagged PCS matters just as much. Standard polynomial commitment schemes commit to a fixed-size domain, which means that a chip with 10,000 occupied rows in a shard of capacity 2^{20} wastes commitment work on a million empty rows. Jagged PCS commits only to the non-trivial portion, eliminating padding overhead. In practice, most shards have a few "hot" chips (ALU, memory) and many "cold" chips (uncommon opcodes). Without Jagged PCS, the cold chips would dominate commitment cost despite contributing almost nothing to the computation. With it, proving cost tracks actual computation, not worst-case table size.

SP1's four-stage pipeline – core STARK per shard, recursive compression, field-transition shrink, Groth16 wrap – is also a study in staged trust assumptions. The inner stages are fully transparent, hash-based, and post-quantum resilient. Only the final wrap introduces a trusted setup and classical-security assumption. If and when a post-quantum on-chain verifier becomes practical, SP1 can drop the Groth16 stage and expose the transparent inner proof directly. The architecture is built to survive the quantum transition, not merely to endure it.

11.3.2 Stwo/Cairo: The ZK-Native ISA Champion

Layer 1. Fully transparent (hash-based). For Ethereum L1 settlement, a Groth16 wrapper exists via Herodotus, but the primary pipeline remains STARK-native.

Layer 2. Cairo – a custom ISA designed to minimize arithmetization cost. Here the model's top-down flow inverts. The ISA *is* the constraint system, by design. Layer 4 requirements shaped Layer 2. The choreography was written to serve the stage machinery, not the other way around.

Layer 3. Cairo VM execution producing columnar M31 traces. The field choice is baked into the VM itself, not merely the proving step – a deeper coupling than in RISC-V zkVMs.

Layer 4. Flat AIR with one component per instruction, LogUp cross-component lookups, mixed-degree constraints. Circle STARKs use the circle group over M31 (where $p+1 = 2^{31}$ is a power of two) to enable FFTs over a field that lacks large multiplicative subgroups.

Layer 5. Circle STARK with SHARP aggregation. Approximately 100x faster than its predecessor Stone. Live on Starknet mainnet since November 2025.

Layer 6. M31 field (31-bit). M31 arithmetic is approximately 125x faster than the 252-bit Stark field used by Stone. The field choice *created* the need for Circle STARKs – the strongest example of Layer 6 forcing a Layer 5 invention.

Layer 7. Native STARK verification on Starknet (no wrapping needed). For Ethereum L1: SHARP-aggregated Groth16.

What makes Stwo architecturally distinctive. Stwo is what happens when you design the entire stack backward from a single mathematical insight: *the group of points on the circle* $x^2+y^2 = 1$ *over a Mersenne prime has order* $p + 1$*, and when* $p = 2^{31} - 1$*, that order is exactly* 2^{31} *– a power of two*. This accident of number theory is the seed from which the entire Stwo architecture grows.

Conventional STARKs require a field with large multiplicative subgroups for FFT-based polynomial evaluation. The Mersenne prime $M31 = 2^{31} - 1$ has no such subgroups – its multiplicative group has order $2^{31} - 2$, which factors badly. A naive approach would disqualify M31 from STARK construction entirely. Circle STARKs solve this by abandoning the multiplicative group in favor of the circle group, where the "FFT" becomes a circle-group analog (the Circle Number Theoretic Transform). The evaluation domain is the set of points on the unit circle over $\mathbb{F}_{M31}$, not the powers of a generator in $\mathbb{F}^*_{M31}$. This is not a minor algebraic substitution; it requires rethinking polynomial commitments, coset structures, and FRI queries from the ground up.

The payoff is a 125x speedup. M31 arithmetic – 31-bit integer addition and multiplication with a single modular reduction – maps directly onto 32-bit CPU and GPU instructions with no multi-precision overhead. A single SIMD lane processes one field element. Compare this with the 252-bit Stark field used by Stone, where each field multiplication requires multiple 64-bit limb operations and carry propagation. The measured 125x speedup over Stone is not a software optimization; it is a consequence of matching the algebraic structure to the hardware word size.

Cairo's role is just as distinctive. Where RISC-V zkVMs treat the ISA and the constraint system as separate concerns – the ISA defines computation, the arithmetization encodes it – Cairo *collapses the two*. A Cairo instruction is simultaneously a machine operation and a set of polynomial constraints. The compiler does not translate programs into constraints; it emits programs that *are* constraints. This eliminates the "arithmetization tax" that RISC-V zkVMs pay: the overhead of encoding a general-purpose instruction (designed for silicon hardware) into an algebraic form (designed for polynomial provers). Gassmann et al.'s finding that standard LLVM optimizations yield 40% improvement on RISC-V zkVMs – because LLVM optimizes for caches and branch predictors that do not exist in ZK execution – quantifies exactly the tax that Cairo avoids.

The SHARP (Shared Prover) aggregation layer adds a dimension absent from SP1 and Jolt: *amortization across applications*. SHARP batches proofs from multiple independent Starknet applications into a

single recursive proof, so that the fixed cost of Ethereum L1 verification is shared among all applications that submit proofs in the same batch window. This is economic aggregation, not just cryptographic recursion. A small contract with ten transactions per hour pays a fraction of the L1 verification cost that it would bear alone. The theater shares its rent among all the acts on stage.

The trade-off is ecosystem lock-in. Cairo is not Rust, not C, not any language with a pre-existing developer community of millions. Every Cairo developer is a developer that StarkWare's ecosystem must recruit and train. The Kakarot project – an EVM interpreter written in Cairo, running on Stwo – is the architectural hedge: it lets Ethereum developers write Solidity while Cairo and Stwo handle the proving underneath. Whether this bridge is sturdy enough to carry mainstream adoption is the open strategic question.

11.3.3 Jolt: The Lookup Singularity Pioneer

Layer 1. Trusted (Hyrax/Pedersen commitments), with transparent alternatives planned (Basefold in Jolt-b).

Layer 2. RISC-V (RV32I). Same as SP1, but the divergence begins at Layer 4.

Layer 3. Every instruction is decomposed into lookups on small subtables. Witness generation *is* the arithmetization – no meaningful boundary exists between Layers 3 and 4. This is the first crack in the seven-layer model. The backstage preparation and the encoding of the trick are the same act.

Layer 4. Lookup-based arithmetization via Lasso. Instead of encoding each operation as constraints, the system verifies every step against pre-approved entries in a comprehensive reference table. A thin R1CS wrapper (~60 constraints per cycle) handles control flow via Spartan. This is a genuinely distinct paradigm from AIR, PLONKish, or CCS.

Layer 5. Sumcheck-based proving. No recursion in production; no STARK-to-SNARK wrapping pipeline. Approximately 6x faster than RISC Zero on initial benchmarks, but the gap has narrowed with GPU acceleration.

Layer 6. BN254 scalar field (256-bit). 256-bit field operations are approximately 100x more expensive per operation than 31-bit, but Jolt compensates by performing far fewer operations per CPU step.

Layer 7. No production on-chain verifier. This is Jolt's most significant gap relative to SP1 and Stwo. The trick is performed brilliantly, but the theater has no box office.

What makes Jolt architecturally distinctive. Jolt is the most radical of the three systems, because it asks a question that the other two never consider: *what if we stopped writing constraints entirely?* SP1 and Stwo both encode computation as polynomial constraints – they differ in how they organize and

evaluate those constraints, but both accept the premise that proving a computation means constraining it. Jolt rejects this premise. In Jolt, proving a computation means *looking it up*.

The core insight, which Jolt inherits from the Lasso lookup argument, is that any function on small inputs can be represented as a table. A 32-bit addition is a function from two 16-bit operands to a 17-bit result – a table with 2^{32} entries. Proving that "a + b = c" does not require writing a constraint that the prover satisfies; it requires showing that the triple (a, b, c) appears in the addition table. The prover's obligation shifts from "solve this system of equations" to "demonstrate membership in this pre-computed set." This is not a small change in formalism. It is a different epistemology of computation.

The practical problem is that 2^{32}-entry tables are too large to materialize. Lasso solves this by decomposing large lookups into combinations of small subtable lookups, using the algebraic structure of the operations themselves. Addition decomposes by limbs; bitwise operations decompose by individual bits; shifts decompose by position. Each subtable is small enough to commit to directly – typically 2^{16} entries or fewer. The Lasso sumcheck protocol then proves that the decomposed lookups are consistent with the original instruction. The result is a system where the per-instruction proving cost scales with the *decomposability* of the instruction, not with the number of constraints needed to describe it.

This decomposition is why Jolt's Layer 3 and Layer 4 merge into a single act. In SP1, the witness (Layer 3) is an execution trace – a table of register states and memory values at each cycle – and the arithmetization (Layer 4) is a set of polynomial constraints that the trace must satisfy. These are conceptually and computationally distinct stages. In Jolt, the "witness" for each instruction *is* the lookup decomposition: the set of subtable indices and values that reconstruct the instruction's behavior. Generating the witness and performing the arithmetization are the same computation, executed in a single pass. There is no moment where "the trace is ready and now we constrain it." The trace is the constraint.

The choice of BN254 as the base field is worth examining. SP1 and Stwo chose 31-bit fields for raw throughput, accepting hash-based commitments and transparent proofs. Jolt chose a 256-bit elliptic-curve field, accepting 100x slower per-operation arithmetic, because BN254 enables Hyrax commitments – a multi-scalar-multiplication-based polynomial commitment scheme with *no trusted setup* and *logarithmic* verification time. Hyrax commitments over BN254 are the reason Jolt avoids both a ceremony (unlike Groth16 wrappers) and hash-chain verification (unlike FRI). The field choice is not a performance concession; it is a commitment-scheme selection that happens to be expensive.

Jolt's current absence of a production on-chain verifier is not merely an engineering gap waiting to be filled. It reflects a deeper tension: the sumcheck-based proving paradigm produces proofs whose verification cost does not compress as neatly into the constant-size Groth16 format that Ethereum L1 expects. The planned Jolt-b variant, which replaces Hyrax with Basefold (a hash-based, transparent commitment scheme), would enable a more conventional STARK-to-SNARK wrapping pipeline. But

this replacement changes Jolt's security model – from discrete-log hardness to hash collision resistance – and alters the system's identity. Whether Jolt can ship a verifier without becoming a different system is the question that will determine whether the lookup singularity remains a research landmark or becomes a production architecture.

11.4 The "Proof Core" Triad

The comparative analysis reveals a pattern that the seven-layer model obscures: Layers 4, 5, and 6 form a tightly coupled triad – the **proof core** – in every production zkVM. These three layers are not three choices. They are one choice with three manifestations.

In Stwo: M31 (Layer 6) forces Circle groups (Layer 5) forces Circle AIR (Layer 4).

In SP1: BabyBear (Layer 6) enables multilinear PCS (Layer 6) enables LogUp-GKR (Layer 4/5) enables Jagged multi-table AIR (Layer 4).

In Jolt: BN254 (Layer 6) enables Hyrax commitment (Layer 6) makes sumcheck natural (Layer 5) makes Lasso lookups natural (Layer 4).

In each case, the field choice *determines* the commitment scheme, which *determines* the polynomial representation, which *determines* the arithmetization. The proof core is the inseparable nucleus of {field, commitment scheme, polynomial representation} that straddles Layers 4, 5, and 6. Acknowledging this concept does not require restructuring the seven-layer model, but it does require acknowledging that the model's clean layer boundaries are pedagogical simplifications, not architectural truths.

To see why this coupling is not merely incidental but structurally inevitable, consider what happens when you try to swap one element of the triad while holding the others fixed. Suppose you wanted to keep M31 arithmetic (for speed) but use Hyrax commitments (for transparent elliptic-curve proofs without a trusted setup). Hyrax requires multi-scalar multiplication over an elliptic curve whose scalar field matches the proving field. No standard curve has a 31-bit scalar field – the smallest curves used in practice have 254-bit or 256-bit scalar fields. You would need to embed M31 elements into a vastly larger field for every commitment operation, destroying the throughput advantage that motivated the M31 choice. The triad resists substitution because its elements are not independent components connected by interfaces; they are facets of a single algebraic structure viewed from different angles.

The same rigidity appears in the other direction. Suppose you wanted to keep BN254 (for Hyrax compatibility) but switch to AIR-based arithmetization (for the modular chip architecture that makes SP1 extensible). AIR evaluation requires FFTs over the proving field, and BN254's scalar field has a multiplicative subgroup of order 2^{28} – adequate but not generous. More critically, BN254 field operations are

100x slower than BabyBear operations, so the FFT-intensive AIR evaluation that runs in milliseconds over BabyBear would take seconds over BN254. The AIR paradigm's viability depends on cheap field arithmetic; cheap field arithmetic depends on small fields; small fields depend on hash-based commitments. Pull one thread and the entire fabric moves.

This structural coupling explains an otherwise puzzling empirical observation: despite the enormous design space of possible {field, commitment, arithmetization} combinations, production systems cluster around a small number of triads. The March 2026 landscape shows essentially three: {small field, hash-based FRI, AIR} (SP1, Stwo, Airbender, RISC Zero, Pico Prism), {small field, hash-based FRI, AIR + lookup} (ZisK, OpenVM), and {large field, MSM-based, sumcheck + lookup} (Jolt). The clustering is not a failure of imagination. It is the proof core exerting its gravitational pull: only certain combinations are self-consistent, and the self-consistent combinations are few.

The decisive fork is at Layer 6 (field choice). SP1 and Stwo both chose small fields (31-bit) optimized for hardware throughput, accepting extension fields and hash-based commitments. Jolt chose a 256-bit field, accepting higher per-operation cost for native elliptic curve compatibility. This single parameter cascades through every other layer. The choice is not primarily about arithmetic speed – it is about which mathematical universe the entire proof system will inhabit. A 31-bit field lives in the universe of hash trees, Merkle commitments, and transparent proofs. A 256-bit elliptic-curve field lives in the universe of pairings, discrete logarithms, and structured reference strings. These universes have different physics, and a system that enters one cannot easily visit the other.

The second fork is at Layer 4 (arithmetization). SP1 and Stwo both use AIR-based constraint systems. Jolt abandons AIR entirely in favor of lookup-based arithmetization. AIR systems *describe* computation as polynomial constraints; Jolt *tabulates* computation as lookup entries. The distinction is deeper than syntax. In an AIR system, the prover must *solve* the constraint system – find a witness that satisfies all polynomial equations simultaneously. The constraint system is a specification that the witness must match. In a lookup system, the prover must *demonstrate membership* – show that each instruction's input-output behavior appears in a pre-computed table. The table is not a specification to be satisfied but a reference to be consulted. One paradigm asks "does this answer satisfy the question?" The other asks "is this answer in the book?"

The practical consequence is that AIR systems scale with constraint complexity (more constraints per instruction means more work per step), while lookup systems scale with table size and decomposition depth (more subtables means more sumcheck rounds). For simple arithmetic – adds, multiplies, shifts – the lookup approach can be dramatically cheaper, because the subtables are small and the decomposition is clean. For complex operations – hash functions, elliptic-curve arithmetic – the lookup approach

faces a combinatorial explosion in table size, which is why even Jolt uses a thin R1CS layer for control flow rather than attempting to tabulate branching logic.

Where the model breaks. Cairo shows Layer 4 shaping Layer 2 (bidirectional dependency). Jolt shows Layers 3 and 4 collapsing into one. The STARK-to-SNARK wrapping pipeline is not a Layer 5 choice; it pierces Layers 5, 6, and 7 as a single vertical shaft. The seven layers are better understood as seven *aspects* of a single integrated system, not seven *modules* with clean interfaces. The proof core triad is the strongest evidence for this view: three layers that the model presents as independent choices are in fact a single crystalline structure, as rigid and as beautiful as any lattice in mathematics. Change one axis and the crystal shatters. The map is useful. The territory is more interesting.

11.5 Performance: The Cost Collapse

The performance trajectory of zkVMs over the past 24 months is one of the steepest cost collapses in applied cryptography. It deserves to be stated plainly.

Real-time Ethereum proving is solved. Four independent teams proved over 99% of mainnet blocks within the 12-second slot time by late 2025:

System	Hardware	Avg Block Time	% Blocks < 12s
SP1 Hypercube	16x RTX 5090	6.9 s	99.7%
ZisK	24x RTX 5090	6.6 s	99.7%
Pico Prism	16x RTX 5090	6.9 s	99%+
OpenVM 2.0	16x RTX 5090	< 12 s (p99)	~99%+

Airbender proves a block in 35 seconds on a *single* H100 GPU – the fastest single-GPU result, at 21.8 million cycles per second at the base STARK layer.

Cost trajectory. The 2,000-fold cost collapse described in Chapter 6 continued to accelerate: within 2025 alone, a further 45x reduction (from $1.69 to four cents per block). Roughly 10x per year, driven by algorithmic improvements, GPU optimization, and competition. A real-time proving cluster runs $60,000-$100,000: sixteen RTX 5090 GPUs (~$32K), dual-socket server, 512 GB RAM. For less than the price of a suburban house, you can prove every Ethereum block in real time.

The Witness Gap grows with acceleration. As GPU provers drove cryptographic proving time down 10-50x, witness generation – still CPU-bound – became the dominant bottleneck. The proportional shift described in Chapter 4 is now the defining structural constraint of zkVM performance: witness generation in a zkVM equals full VM emulation, which resists the parallelism that NTT and MSM exploit so effectively. The magician's backstage preparation now takes longer than sealing the proof.

Active optimization research is attacking this gap from multiple directions. ZKPOG achieved up to 52x speedup by moving witness generation onto GPUs. OpenVM 2.0's SWIRL prover includes an ahead-of-time compiler executing at 3.8 GHz, eliminating JIT overhead. Nexus 3.0 runs the program twice – first to gather memory statistics, then to produce an optimized trace.

The EF security pivot (December 2025). The Ethereum Foundation declared the speed race won and shifted focus: - May 2026 target: 100-bit provable security across all zkEVM teams - December 2026 target: 128-bit provable security, sub-300 KB proofs - New primary metric: energy per proof (kWh), replacing raw speed - The EF rejects unproven conjectures (proximity gap assumptions) for production soundness

This pivot validates systems with strong formal security guarantees. It also shifts the competitive axis from "who can prove fastest" to "who can prove most securely" – precisely where lattice-based and tensor-code approaches hold structural advantages. The race for speed is over. The race for rigor has begun.

11.6 RISC-V Convergence

The numbers are unambiguous: eight of ten major zkVMs target RISC-V. The original three-philosophy taxonomy – EVM-Compatible, ZK-Native ISA, General-Purpose ISA – is accurate as a historical classification, but the market has rendered its verdict. RISC-V won the general-purpose category decisively. Even EVM-focused projects now build RISC-V backends and layer EVM compatibility on top.

Why RISC-V? Three reasons converge.

First, RISC-V's register-transfer architecture maps cleanly onto tabular execution traces, which are the native input format for AIR and lookup-based arithmetization. A RISC-V instruction reads source registers, performs an operation, and writes a destination register – exactly one row in a trace table.

Second, RISC-V's compiler ecosystem is decades deep. Any Rust, C, or C++ program can be compiled to RISC-V using standard LLVM toolchains. This means millions of existing programs become provable without modification. The universal stage accepts any act, because any act can be translated into its language.

Third, RISC-V is open and royalty-free. Unlike ARM (proprietary) or x86 (legacy-encumbered), RISC-V has no licensing costs and no vendor lock-in. For an open-source ecosystem, this matters.

The holdouts are instructive. Cairo (Stwo) is a ZK-native ISA designed to minimize arithmetization cost – the ISA *is* the constraint system. This gives Cairo a structural efficiency advantage: the compiler optimization study (Gassmann et al., 2025) found that standard LLVM optimizations yield over 40% improvement on RISC-V zkVMs because they target hardware features (caches, branch predictors) absent in ZK contexts. Cairo avoids this overhead by design. Whether that advantage justifies a smaller developer ecosystem is the strategic question StarkWare has answered with "yes" for Starknet and "maybe not" for broader adoption (hence Kakarot's EVM-on-Stwo path).

zkWASM (Delphinus Lab) targets WebAssembly, offering the broadest language support of any zkVM (any language that compiles to WASM). Its PLONKish/KZG architecture is a generational outlier – it uses pairing-based commitments rather than hash-based – and it has not demonstrated Ethereum block proving. zkWASM appears to be a niche play for web-native applications rather than a contender for the mainstream proving market.

Chapter 12: Midnight – The Privacy Theater

Every magic show needs a theater – a physical space designed so that the audience sees exactly what the magician intends and nothing more. The lighting, the curtains, the sight lines, the trapdoors: all are engineered to control information flow. What the audience sees is a choice, not an accident. What the audience does not see is an architecture, not an oversight.

Midnight is such a theater. Every architectural choice – from the language to the proof system to the token model – serves a single design goal: the audience learns the truth of a claim, and nothing else. The question that animates this chapter is whether the theater's engineering matches its ambition.

12.1 Midnight as Test Case

Most ZK systems use zero-knowledge proofs as an optimization – a way to compress computation for cheaper on-chain verification. The magician's sealed certificate is a convenience. Midnight is different. On Midnight, ZK proofs are not an optimization layer bolted onto an existing execution model. They *are* the execution model. Every state transition is proven in zero knowledge. Every contract deployment, every circuit call, every token transfer passes through a local proof server before reaching the chain. The trick is not incidental to the show. The trick *is* the show.

This makes Midnight an unusually complete test of whether the seven-layer model actually maps to a working system. With 473 pages of verified documentation across five reference documents – Developer Guide (191pp), Compact Language Reference (75pp), ZKIR Specification (60pp), MidnightJS SDK Reference (87pp), and Wallet SDK Reference (60pp) – Midnight provides concrete evidence at every layer. Where the earlier chapters' examples are necessarily abstract, Midnight supplies specific opcodes, measured latencies, deployed contracts, and compiler error messages.

The analysis that follows draws on findings from seven dedicated layer analysts. Every claim references specific Midnight documentation or measured behavior.

12.2 Midnight at a Glance

Midnight is a Cardano sidechain that executes privacy-preserving smart contracts written in **Compact**, a TypeScript-inspired domain-specific language. The architecture follows a pipeline:

```
Compact source --> compactc compiler --> ZKIR circuits + TypeScript bindings + proving keys
                                                |                   |                |
                                        Proof server       DApp frontend   On-chain verifier
                                     (localhost:6300)    (browser/node)  (blockchain node)
```

The compiler produces three artifacts from a single `.compact` file: ZKIR circuit descriptions in JSON, TypeScript API bindings for the DApp frontend, and cryptographic proving/verifier key pairs. This three-part output reflects the fundamental architecture of privacy-preserving computation: what can be proven (ZKIR), what runs privately (TypeScript witnesses), and what makes proofs possible (keys).

Midnight's token model has three layers. **NIGHT** is the governance and staking token, always unshielded (transparent). **DUST** is the fee token, a public (unshielded) token per the wallet SDK, generated from staking NIGHT over time, with a balance computed from generation parameters. **Custom tokens**

can be either shielded (encrypted, spent via ZK proofs and nullifiers) or unshielded (transparent, spent via BIP-340 Schnorr signatures), at the developer's choice per UTXO.

12.3 Full Seven-Layer Mapping

12.3.1 Layer 1: BLS12-381 and the Trusted Ceremony

The book's Layer 1 asks: trusted or transparent? Midnight answers: **trusted, with a universal SRS**.

Midnight uses the BLS12-381 elliptic curve with a PLONK-family proof system (Halo 2). The Developer Guide's architecture diagram (p.8) shows a dedicated `midnight-trusted-setup` repository for running a Powers-of-Tau ceremony. The SRS is universal – one ceremony serves all circuits – but the compiler generates per-circuit proving and verifier keys derived from that SRS. For a simple counter contract, the compiler produces an `increment.prover` key (13.7 KB) and an `increment.verifier` key (1.3 KB).

Midnight's cryptographic history reveals pragmatic trade-offs that echo the theme from Chapter 2. The system originally used Pluto-Eris curves for recursive proof composition but switched back to BLS12-381 for mainnet, citing faster proof generation, wider ecosystem compatibility, and support for higher transaction volumes. Theoretical optimality yielded to engineering reality – a pattern we have seen repeatedly throughout this book.

Post-quantum implication: BLS12-381 provides approximately 128-bit classical security but zero post-quantum security. Shor's algorithm breaks the discrete logarithm and pairing assumptions simultaneously. The stage is strong today. Its quantum shelf life is finite.

12.3.2 Layer 2: Compact as Fourth-Philosophy DSL

The three-philosophy taxonomy from earlier chapters – EVM-Compatible, ZK-Native ISA, General-Purpose ISA – does not accommodate Compact. Compact represents a **fourth philosophy**: the application-specific DSL.

Compact does not prove a processor. It compiles a domain-specific smart contract language directly to chain-specific ZKIR, with the compiler enforcing application-level invariants that no ISA-level approach can express. Its closest relatives are Leo (Aleo) and Mina's o1js.

The critical differentiator is **disclosure analysis**. The Compact compiler includes a `track-witness-data` analysis pass that traces witness values through all program paths and rejects any program where private data might reach public surfaces without explicit `disclose()` calls. This is a hard compile-time error,

not a warning. The Developer Guide documents that a first attempt at private voting was rejected with 11 disclosure errors, each tracing the path from witness to ledger operation.

No other ZK language prevents accidental privacy leaks at compile time. In Circom, Noir, and Cairo, a disclosure mistake produces a privacy leak, not a compiler error. Compact eliminates an entire vulnerability class – accidental disclosure – at the language level. The stage manager locks the doors to prevent accidental reveals. The magician must explicitly ask for the key.

12.3.3 Layer 3: The disclose() Boundary

Witnesses in Midnight are arbitrary JavaScript functions that run off-chain. The `disclose()` operator is the sole gateway from the witness world to the circuit world – the single controlled opening in the curtain between backstage and audience.

Every ZKIR circuit has two transcript channels: the `publicTranscript` (ledger operations visible to the verifier) and `privateTranscriptOutputs` (witness values visible only to the prover). Without `disclose()`, witness values remain entirely invisible to the circuit and the chain. The ZKIR checker enforces transcript integrity with specific diagnostic error messages – tampering with either transcript causes immediate rejection.

What does this boundary feel like to the developer who must work inside it every day? Consider a programmer writing a private voting contract. She writes her witness function in TypeScript – ordinary, familiar, unexotic code that fetches voter eligibility from a local database and computes a ballot. Every variable in that function is invisible to the chain by default. She could write a hundred lines of witness logic, and none of it would leave her machine. Then she reaches the moment of commitment: she needs the circuit to know which candidate received the vote, without revealing who cast it. She writes `disclose(candidateId)`. That single call is the seam in the curtain, the controlled opening where one piece of information – and only that piece – crosses from the private rehearsal room into the public theater. The compiler has already analyzed every path through her code; if she accidentally wrote `disclose(voterId)` three functions earlier, the build would have failed with a traced error showing exactly how the private value reached the public surface. The developer experience is not one of navigating cryptographic abstractions. It is one of writing normal code inside a system that has opinions – strong, enforced, non-negotiable opinions – about what leaves the room. Asimov imagined robots governed by laws they could not violate. The Compact developer works inside a compiler governed by disclosure laws it will not bend. The fiction writer's dream of the incorruptible guardian is, in this narrow domain, an engineering reality.

The theater analogy sharpens here. In a well-designed theater, the lighting grid is the real enforcer of what the audience sees. A spotlight operator who accidentally swings a beam toward the wings will reveal the stagehands, the props not yet in play, the illusion's scaffolding. Midnight's disclosure analysis is the lighting grid: it does not merely suggest where the light should fall – it physically prevents the spots from swinging toward the wings. The developer sets the cues. The compiler locks the grid. And when the show runs, only what was meant to be seen is seen.

Side-channel gap: None of the five reference documents address timing attacks, cache attacks, or metadata leakage through the indexer's GraphQL API. Proof generation dominates transaction time (~18-20 seconds), providing natural but unintentional timing padding. The curtain is thick, but no one has tested whether light leaks through the seams. A theater may control its spotlights perfectly and still betray its secrets through the sound of trapdoors opening, the vibration of machinery beneath the stage, the draft of air from a hidden passage. Midnight's formal privacy model covers what the proofs reveal. It does not yet cover what the infrastructure whispers.

12.3.4 Layer 4: ZKIR as High-Level Constraint IR

Midnight's ZKIR is a typed instruction-level intermediate representation with 24 base instructions organized into 8 categories: arithmetic (`add`, `mul`, `neg`), constraints (`assert`, `constrain_eq`, `constrain_bits`, `constrain_to_boolean`), comparison (`test_eq`, `less_than`), control flow (`cond_select`, `copy`), type encoding, cryptographic operations (targeting the Jubjub curve embedded in BLS12-381), I/O, and division.

ZKIR sits above the mathematical constraint formalism but below the source language. In the taxonomy of R1CS, AIR, PLONKish, and CCS, ZKIR is most analogous to PLONKish but operates at a higher abstraction level – the developer sees typed operations with semantic meaning, not bare multiplication gates. This is arithmetic with a human face.

12.3.5 Layer 5: Halo 2 and the Four-Phase Pipeline

Midnight uses Halo 2 (UltraPlonk) over BLS12-381. The proof server runs locally at `localhost:6300`. The transaction lifecycle follows a four-phase pipeline: `callTx()` (execute circuit locally) to `proveTx()` (generate ZK proof, ~17-24 seconds) to `balanceTx()` (bind, sign, merge) to `submitTx()` (submit to chain, where the node verifies the proof).

Proof generation dominates latency: deploy ~17-28 seconds, circuit call ~17-24 seconds, balancing and submission sub-second. The magician rehearses for twenty seconds. The audience's verdict takes a heartbeat.

12.3.6 Layer 6: BLS12-381, Jubjub, and Poseidon

All ZKIR values are elements of the BLS12-381 scalar field ($\sim 2^{253}$). The Jubjub twisted Edwards curve, embedded natively in BLS12-381, enables in-circuit elliptic curve operations. Poseidon-based hashing via `persistent_hash` (two field elements for collision resistance) and `transient_hash` (single field element) provides ZK-friendly hash computation.

The large field (253 bits vs. 31-64 bits for STARK-friendly fields) is the primary performance cost – 10-100x slower per operation – but is necessary for the pairing and embedded curve that enable KZG commitments and constant-size proofs. Midnight pays for privacy with patience.

12.3.7 Layer 7: Three Tokens and the Verifier Key Lifecycle

Midnight's three-token model (NIGHT/DUST/custom) and per-UTXO privacy choice create a deployment architecture distinct from Ethereum-style gas economics. DUST fees are paid in a public token generated by time-locked staking.

The economics deserve closer scrutiny, because they encode a philosophy. On Ethereum, gas is purchased with ETH on the open market – every fee payment is a visible transaction, a data point for chain analysts, a signal. On Midnight, DUST accrues silently from staked NIGHT over time, like interest accumulating in an account the holder never visits. A developer deploying a contract pays ~490 trillion SPECK per circuit call, but that DUST was not purchased in a transaction anyone can observe. It was generated by the passage of time and the act of staking. The economic consequence is that fee payment itself becomes a poor signal: an observer who watches DUST expenditures sees activity, but cannot easily link that activity back to a market purchase, a wallet funding event, or an exchange withdrawal. The three-token architecture is not merely a governance convenience. It is a privacy mechanism at the economic layer – a recognition that in a system where computation is private, the payment for computation must be private too, or else the ticket stub betrays the show. Penrose once observed that the geometry of a space constrains what can happen within it. Midnight's token geometry – the separation of governance (NIGHT), fees (DUST), and application value (custom tokens) – constrains the information that economic activity can leak. The shape of the money shapes the privacy of the system. This is Layer 7 not as an afterthought but as architecture.

The SDK API includes functions for dynamically managing verifier keys on-chain (`submitInsertVerifierKeyTx`, `submitRemoveVerifierKeyTx`), raising governance questions that parallel the book's discussion of upgradeable proxy contracts. The theater management retains the ability to change the locks – to swap out the verifier that guards a contract's stage door, to retire a circuit and replace it with another. In a traditional theater, this is the producer's prerogative: the show can be recast, the set redesigned,

the script rewritten between seasons. But in a privacy theater, changing the verifier key changes the terms under which secrets were originally committed. A user who shielded tokens under one verifier's rules may find those rules altered by a governance action she never approved. The trapdoor that was locked for the performer's protection can be unlocked by the theater's owner. This tension – between upgradeable infrastructure and the immutability that privacy demands – is not resolved in Midnight's current design. It is, at most, acknowledged.

12.4 Where Midnight Validates the Model

The seven-layer decomposition maps cleanly to Midnight's architecture in five places:

1. Layer 1 maps to `midnight-trusted-setup`. The ceremony produces the SRS; the compiler derives per-circuit keys. The capex/opex framing applies directly: one-time ceremony cost amortized across all contracts, with per-transaction proof costs of ~18 seconds and ~490 trillion SPECK.

2. Layer 2 maps to Compact. The language choice determines what developers can express and what mistakes they cannot make. Disclosure analysis validates the argument that language design has security implications beyond expressiveness.

3. Layer 4 maps to ZKIR. The 24-opcode instruction set is a concrete instance of the arithmetization layer, sitting above PLONKish constraints and below the source language.

4. Layer 5 maps to the proof server. The four-phase pipeline instantiates the proof generation and verification layer with measured latencies and a clear component boundary.

5. Layer 7 maps to the three-token model. The verifier key deployment, fee economics, and governance structure are concrete instances of deployment concerns.

12.5 Where Midnight Challenges the Model

Three aspects of Midnight's architecture do not fit cleanly into seven layers:

1. The compiler spans Layers 2, 3, and 4 simultaneously. The Compact compiler's nanopass architecture takes source code (Layer 2), performs disclosure analysis (a Layer 3 concern – who sees the witness?), and emits ZKIR (Layer 4 arithmetization) in a single continuous pipeline of 26 intermediate languages. There is no clean boundary where "language" ends and "arithmetization" begins. The magician's script, the backstage preparation, and the mathematical encoding blur into a single creative act.

2. The SDK is a cross-layer orchestrator. The four-phase transaction pipeline spans Layer 3 (witness construction in `callTx`), Layer 5 (proof generation in `proveTx`), Layer 6 (BIP-340 signatures in `balanceTx`), and Layer 7 (on-chain submission in `submitTx`). The SDK is not "at" any single layer; it is the glue connecting all of them.

3. Privacy is not a layer – it is a cross-cutting concern. The book treats privacy primarily as a Layer 3 issue (witness secrecy). In Midnight, privacy decisions propagate through every layer: BLS12-381's pairing structure enables shielded commitments (Layer 1/6), disclosure analysis enforces privacy at the language level (Layer 2), the two-transcript model separates public and private data (Layer 3/4), the proof server keeps witnesses local (Layer 5), and the shielded/unshielded UTXO model determines what the verifier sees (Layer 7). Privacy is not a room in the theater. It is the architecture of the theater itself.

The following table crystallizes what the Midnight case study proves and what it leaves unresolved:

Dimension	Midnight Validates	Midnight Does Not Solve
Compiler-enforced privacy	Disclosure analysis catches 11 error types at compile time	Side-channel leakage (timing), metadata via indexer/network layer
Seven-layer decomposition	Clean mapping at 5 of 7 layers	Compact compiler spans L2–L4; SDK spans L3–L7; layers not cleanly separable
Privacy as architecture	Every layer serves privacy – UTXO model, local proving, shielded state	Cross-contract token transfers between DApps remain unsupported
Trust decomposition	Three-token model (Night/Shielded/DUST) is genuinely novel	Governance key management and upgrade path remain centralized
Post-quantum readiness	Architecture acknowledged as non-PQ	No migration path to lattice-based commitments without full redesign
Developer experience	Compact ZKIR proof pipeline is coherent end-to-end	17-28s proof times create UX barrier; no GPU acceleration

12.6 The Privacy Theater Analogy

The magician metaphor that has sustained this book finds its fullest expression in Midnight. But to see the analogy in its full depth, we must think not as the audience watching the trick but as the architect who designed the theater – the one who decided where the walls would go, where the sight lines would converge, where the trapdoors would open, and which doors would lock from the inside.

The **open stage** (unshielded) faces the audience directly. Performers (transactions) are fully visible. The audience (validators) can see every movement, verify every step. NIGHT tokens live here – governance requires transparency. The audience sees the magician's face. The lighting is full and even, the equivalent of a bare bulb in a police interrogation room. Nothing hides. Nothing can.

The **curtained stage** (shielded) is separated by a one-way mirror. The audience cannot see the performers, but they can hear the music (the ZK proof) and verify that it follows the score (the ZKIR circuit). Shielded tokens live here. The performer's identity (UTXO owner) and movements (transaction values) are hidden, but the proof guarantees the performance was legitimate. The trick happens in the dark, and the sealed certificate emerges into the light. The sight lines have been engineered so that from every seat in the house, the audience sees only the proof – a single sheet of paper slid under the curtain. The geometry of the theater makes curiosity futile. There is simply no angle from which the backstage is visible.

The **stage manager** (Compact compiler) enforces the rules. A performer cannot accidentally step from behind the curtain onto the open stage – the compiler's disclosure analysis physically prevents the transition unless the performer explicitly calls `disclose()`. This is not a best-practice recommendation posted backstage; it is a locked door that requires a key. The stage manager does not trust the performers to remember the blocking. She has bolted the doors, wired the lighting grid to fixed positions, and removed the handles from the wrong side. The performers can improvise within their space. They cannot improvise their way into the audience's view.

The **proof server** is the rehearsal room. All practice (witness computation) happens here, in private, at localhost:6300. Only the final performance (the ZK proof) reaches the theater. The audience never sees the rehearsal. The rehearsal room has no windows, no microphones, no cameras. It exists on the performer's own machine, in a process that never opens a network socket to anything but the local proof server. The separation is not a policy. It is a wall.

The **trapdoors** are the governance mechanisms – the verifier key management functions, the upgrade paths, the administrative controls that the SDK exposes. Every theater has trapdoors, and every trapdoor is a dual-use technology: it enables the performer to appear and disappear as the trick requires, but it also enables the theater owner to access spaces the audience was promised would remain sealed. In

Midnight's current architecture, the trapdoors exist. They are documented. Whether they are adequately governed is a question the documentation does not answer.

DUST is the ticket price. Every performance requires a ticket, generated by staking NIGHT tokens over time. The ticket price is uniform for same-type performances (~490 trillion SPECK per circuit call), providing some metadata privacy – you cannot tell what happened behind the curtain by looking at the ticket stub. The tickets are not sold at a box office window where a clerk might remember your face. They materialize in your wallet through the silent mechanics of staking – as if the theater rewarded loyal patrons by slipping tickets under their doors in the night.

Penrose argued that the geometry of spacetime is not a backdrop against which physics happens but the thing that *is* physics – that curvature and matter are the same story told in two languages. Midnight's privacy theater operates on a similar principle. The privacy is not a feature applied to a blockchain. It is the shape of the blockchain itself. The curve choice, the language design, the transcript separation, the token model, the proof locality – these are not independent decisions that happen to support privacy. They are the curvature of Midnight's operational space, and privacy is what that curvature produces. To remove privacy from Midnight would not be to disable a feature. It would be to flatten the geometry, and the theater would cease to be a theater at all.

12.7 Five Lessons for ZK System Design

12.7.1 Lesson 1: Privacy as Cross-Cutting Concern

The book treats privacy primarily as a Layer 3 phenomenon. Midnight demonstrates that privacy is an architectural decision at every layer: curve selection (L1), language design (L2), witness boundaries (L3), transcript separation (L4), proof locality (L5), commitment schemes (L6), and UTXO encryption (L7). A reader applying the seven-layer model to Midnight would need to trace privacy through all seven layers to understand the system's guarantees.

The OSI network model offers a precedent: security is not a layer but a property that each layer must independently maintain. ZK privacy deserves the same treatment.

12.7.2 Lesson 2: The "Compiler Protects You" Philosophy

The under-constrained vulnerability epidemic described in Chapter 3 is the dominant failure mode in ZK systems. Compact's disclosure analysis addresses an equally severe class – accidental disclosure – at the language level. But Compact's protection has limits: it prevents accidental leakage, but it cannot prevent a developer from choosing to `disclose()` too much, storing secrets in ledger state, or making application-logic errors. The locked door keeps you from stumbling through. It does not stop you from handing someone the key.

Compiler enforcement raises the floor of security but does not guarantee correctness. The comparison between Circom's manual constraint authoring (where under-constrained bugs thrive) and Compact's automatic constraint generation (where disclosure bugs are caught) illustrates the Layer 2 security spectrum concretely.

12.7.3 Lesson 3: The Three-Token Economic Model

The book's Layer 7 focuses on Ethereum-style gas economics. Midnight's three-token model represents a fundamentally different architecture: fees are paid in a shielded token generated by time-locked staking, not purchased on the open market. This means fee payment itself has privacy properties – you cannot determine a user's transaction volume by observing fee purchases. Layer 7 economics are not just about gas costs but about the information leakage of the fee mechanism itself. Even the ticket stub can be a clue, and Midnight's design tries to minimize what it reveals.

12.7.4 Lesson 4: The Application-Specific DSL as Fourth Philosophy

Compact demonstrates that a domain-specific ZK language can achieve properties impossible for general-purpose approaches: compiler-enforced privacy boundaries, first-class blockchain state, integrated token operations, and a unified DApp development pipeline. The trade-off is vendor lock-in – Compact contracts cannot run on any chain except Midnight. The three-philosophy taxonomy should be expanded to include this fourth philosophy.

12.7.5 Lesson 5: ZKIR as Concrete Layer 4

The book's Layer 4 discussion of R1CS, AIR, PLONKish, and CCS is necessarily abstract. ZKIR provides a concrete example that sits above these mathematical abstractions. Its 24 typed instructions with semantic meaning (not just "multiplication gate" but `persistent_hash` and `private_input`) show that production arithmetization layers carry more structure than the mathematical formalism suggests. The puzzle has names for its pieces.

12.7.6 Maturity Assessment

Midnight is best characterized as a late-stage testnet / early mainnet system. The proof system works, the compiler catches real privacy bugs, and the devnet supports end-to-end contract deployment and execution. However, cross-contract token transfers fail with SDK errors, the > and <= operators have a documented compiler bug, and deployment latency (dominated by proof generation, as detailed in the Layer 5 section above) indicates room for proving optimization. On the L2Beat Stages framework, Midnight would sit at approximately Stage 0-1: operational with ZK proofs providing validity guarantees, but with governance mechanisms retaining significant centralized control.

The theater is built. The rehearsals are underway. The opening night has not yet arrived.

Midnight is one theater. The zero-knowledge ecosystem has built dozens more – each with different stages, different audiences, different trust bargains. The next chapter surveys six market segments where the mathematics meets money, and asks the question that every technology must eventually answer: who is buying tickets, and what do they think they are paying for?

Chapter 13: The Market Landscape

The magician has performed. The audience has verified. The mathematics works. But a magic show that no one attends is just a person alone in a room doing card tricks. The most elegant proof system in the world, deployed on the most secure stage, verified by the most rigorous audience, means nothing if no one pays for a ticket.

The question that determines whether zero-knowledge proofs reshape industries or remain a cryptographer's curiosity is not "does the math work?" but "does anyone pay for it?" And the question that determines whether the money is well-spent is sharper still: *where is trust actually being minimized – and where is it merely being moved?*

This chapter follows the money. But it watches where the trust goes.

13.1 ZK Rollups: The Proving Grounds (Production)

Zero-knowledge rollups are where the thesis of this chapter becomes immediately testable. The mechanism is straightforward: execute transactions off-chain, generate a ZK proof of correct execution, and post the proof plus compressed state data to Ethereum for verification. Ethereum-grade security with 10-100x lower transaction costs. At Layer 7, the audience can verify every proof. The math is sound. The trick works.

But look at the governance. Every major rollup in production today operates at Stage 0 or Stage 1 – meaning a security council, a multisig, or a governance committee can override the proof system. The audience can check the math, but a committee can still replace the audience. Trust has been minimized at the cryptographic layer and preserved, nearly intact, at the institutional layer. This is not a criticism. It is a fact about where we are, and it should be stated plainly.

The market has consolidated rapidly, and one of its most expensive lessons has already been paid. Polygon zkEVM, once the flagship example of EVM-compatible ZK rollups (Philosophy A in the original taxonomy), was shut down in 2025/2026 after approximately $250 million in investment. The core team, led by co-founder Jordi Baylina, spun off to found ZisK. A quarter-billion dollars bought a lesson: in a field moving this fast, the first-mover advantage is often a first-mover trap.

The current production leaders tell a more encouraging story – but the trust question follows them onto every stage:

Scroll ($748M TVL as of early 2026) – the largest zkEVM by total value locked. Scroll uses a halo2-based proof system with KZG commitments, generating PLONKish proofs that are verified directly on Ethereum L1. Seven hundred forty-eight million dollars locked means users trust the math with real money. Whether they have examined the governance is a different question.

Linea ($2B TVL) – ConsenSys's zkEVM, targeting full EVM equivalence. Linea uses a custom prover with Fiat-Shamir-based PLONK and posts compressed proofs to Ethereum. Two billion dollars locked. That is not a pilot program. It is also not a trustless one.

Starknet – powered by Stwo, the Circle STARK prover that went live on mainnet in November 2025. Starknet stands apart: it uses the Cairo ISA rather than EVM compatibility, and its STARK proofs are verified natively on L1 without Groth16 wrapping (accepting larger proof sizes in exchange for transparency). The only major rollup that performs the entire trick on a glass stage – no trusted setup, no opaque wrapping. The trust tradeoff is different here: you trust the transparency of the mathematics, and you pay for it in proof size.

ZKsync Era – Matter Labs' zkEVM, powered by the Airbender prover. ZKsync achieved 21.8 million cycles per second on a single H100 GPU and deployed via the Atlas upgrade. The 2026 roadmap targets formal verification and adoption as a "universal standard" for ZK proving.

The economics have improved sharply. The 2,000-fold cost collapse described in Chapter 6 means that at current rates, continuously proving every Ethereum block costs roughly $102,000 per year – less than the GPU cluster required to do it. Post-Pectra (May 2025) and Fusaka (December 2025), data availability costs have also dropped, with blob capacity increasing 8x via PeerDAS. The trick has become cheaper than the theater's electricity bill. But cost is not the same as trust. A cheap proof verified by a governance council you cannot remove is still a proof verified by a governance council you cannot remove.

Trust relocated from: L1 execution validators **to:** proof system soundness + governance multisig. **Net:** genuine minimization for computation integrity; governance remains the binding constraint.

13.2 ZK Coprocessors: Off-Chain Computation, On-Chain Verification (Growth)

ZK coprocessors represent a category that the original seven-layer framework entirely missed – and the trust dynamics are subtle enough to deserve careful attention. A coprocessor allows a smart contract to verifiably read and compute over historical blockchain data without the gas cost of executing that computation on-chain. The contract sends a query; the coprocessor executes it off-chain, generates a ZK proof of correct execution, and returns the result with the proof for on-chain verification.

The magician, in this case, is not performing for a general audience. She is performing for a smart contract – an audience that cannot reason, only verify. The proof guarantees that the computation was done correctly. But the computation happened somewhere: on specific hardware, operated by a specific entity, with access to the query inputs. Trust has not been eliminated. It has been moved off-chain.

Axiom raised $20 million in Series A funding and operates the leading ZK coprocessor platform. Axiom's coprocessor can access any historical Ethereum block header, account state, storage slot, transaction, or receipt, generate a computation over that data, and deliver a verified result to a smart contract in a single callback.

Brevis operates both a ZK coprocessor and the Pico Prism zkVM. Brevis's ProverNet launched mainnet beta in December 2025, enabling decentralized proof generation for coprocessor queries.

Lagrange provides ZK coprocessing for cross-chain state proofs and data availability verification.

The coprocessor market matters because it extends ZK proofs beyond transaction execution into data analytics. A DeFi protocol can use a coprocessor to compute time-weighted average prices over 30 days of on-chain data, prove the computation is correct, and use the result in a lending decision – all without trusting an oracle. The sealed certificate now covers not just "the computation was correct" but "the data was real." That is a genuine advance. But who watches the prover's hardware? The proof says the computation was faithful. It does not say the machine was honest about what it was asked to compute. The trust has shifted, not vanished.

Trust relocated from: on-chain computation (expensive, public) **to:** off-chain proving (cheap, private) + data availability assumptions. **Net:** cost reduction with new trust in prover correctness and DA layer.

13.3 ZKML: Provable Machine Learning (Research)

Here the trust question becomes genuinely uncomfortable. Zero-knowledge proofs for machine learning inference – proving that a specific model produced a specific output on a specific input without revealing the model's weights – can prove that a neural network ran faithfully. They cannot prove the neural network should have run at all.

Read that distinction carefully. ZKML proves the inference was correct *given the model*. It says nothing about whether the model is fair, whether it was trained on representative data, whether its architecture is appropriate for the task, or whether deploying it is wise. You can prove the magician performed the trick exactly as rehearsed. You cannot prove the trick was worth performing. Trust in the computation is not trust in the model's correctness, and conflating the two is the central danger of this category.

The fundamental technical difficulty is that neural network operations (matrix multiplications, activation functions, normalization) map poorly onto the finite field arithmetic that ZK proof systems require.

Lagrange's DeepProve is the current leader, claiming 700x faster ZK proofs for ML inference compared to previous approaches. DeepProve achieves this through specialized arithmetization for neural network operations, avoiding the general-purpose overhead of running ML models inside a generic zkVM.

EZKL provides an open-source toolkit for generating ZK proofs of neural network inference, targeting the halo2 proof system. EZKL converts ONNX models into ZK circuits.

To appreciate the difficulty, consider what proving a single inference of a modest neural network (say, a 10-layer transformer with 100 million parameters) actually requires. Each matrix multiplication involves billions of field multiplications. Activation functions like ReLU require comparison operations that are expensive in finite fields. Layer normalization involves division and square roots, both

of which must be decomposed into constraint-friendly operations. The overhead tax from Chapter 5 – already 10,000-50,000x for general computation – can be even steeper for ML workloads, because neural network arithmetic is optimized for floating-point hardware that has no analogue in finite field circuits. DeepProve's 700x improvement is impressive precisely because the starting point was so far from practical.

The ZKML market is pre-revenue and research-heavy, but the applications are clear: verifiable AI inference (prove that a content moderation decision was made by a specific model), model privacy (prove model performance without revealing weights), and auditable AI (regulatory compliance for model decisions). In a world increasingly shaped by opaque AI systems, the ability to prove properties of a model without revealing the model itself may become one of the most valuable applications of zero-knowledge proofs. But only if the field resists the temptation to let "proven inference" stand in for "trustworthy AI." The proof is one layer. The model is another. And no amount of cryptographic elegance in the first layer compensates for negligence in the second.

Trust relocated from: model operator's self-attestation **to:** proof of inference correctness. **Net:** promising but pre-revenue; trust in model integrity itself remains unaddressed.

13.4 ZK Identity (Growth / Regulatory Mandate)

This is the clearest win. In every other market segment, trust is being moved, delegated, or repackaged. In identity, the magician's trick genuinely replaces a gatekeeper. Before ZK identity, proving you were over 18 required handing your driver's license to a stranger – trusting them not to memorize your address, your full name, your date of birth. The trust was total, and it was demanded by a bouncer. ZK-based selective disclosure replaces that entire interaction with mathematics: proving attributes about yourself (age over 18, citizenship of a specific country, possession of a valid credential) without revealing the underlying data. Trust shifted from institutions to proofs. Not trust minimized. Trust *replaced*.

World (formerly Worldcoin) uses iris-scanning orbs to generate unique identity commitments, with ZK proofs enabling "proof of personhood" without revealing biometric data. World's approach is controversial (biometric data collection) but large-scale (millions of enrollments). The magician proves she is human. The audience learns nothing else. But note the uncomfortable residue: the orb that scans your iris is a piece of hardware operated by a company. The proof is trustless. The enrollment is not. Even in the clearest case, trust has a way of hiding in the infrastructure.

EU eIDAS 2.0 mandates digital identity wallets for all EU citizens by 2026, with selective disclosure as a core requirement. ZK proofs are the leading cryptographic mechanism for implementing the selective disclosure specified in the regulation.

Humanity Protocol ($1.1 billion valuation) focuses on palm-vein-based biometric identity with ZK proofs for privacy-preserving verification.

The ZK identity market is projected to reach $7.4 billion, driven by regulatory mandates (eIDAS 2.0, GDPR's tension with blockchain transparency) and institutional demand for verifiable credentials. When the law requires selective disclosure and billions of citizens need identity wallets, the market does not need to be created. It has been mandated. And of all the audiences the magician now performs for, this one – individual human beings trying to prove who they are without surrendering who they are – is the one that matters most.

Trust relocated from: credential presenter (showing full document) **to:** credential issuer (signing the claim) + enrollment hardware. **Net:** genuine minimization for disclosure; enrollment creates new trust surface.

13.5 Proving-as-a-Service: The Prover Market (Production)

Here the privacy tradeoff from Chapter 4 returns with full force. Generating ZK proofs is computationally expensive but highly parallelizable, so a natural market has emerged: delegate your proving to someone with better hardware. The efficiency gain is real. The trust consequence is this: the person who proves for you sees your data.

Read that again. The architecture that protects your data the most – client-side proving, where your secrets never leave your device – requires hardware that most people do not own. The architecture that works on any device – delegated proving – requires trusting the prover with your secrets. Chapter 4 called this out as the economic structure of privacy in 2026. Here it becomes a business model.

Succinct operates the leading prover network. By early 2026, the Succinct Network had generated over 6 million proofs on mainnet, secured over $4 billion in value, and launched the $PROVE token. The network's SP1 Hypercube zkVM powers multiple rollup deployments.

RISC Zero Boundless launched its open proof marketplace in September 2025. Boundless processed 542.7 trillion cycles by December 2025, then forced migration from its centralized prover service to the decentralized marketplace. This migration – shutting down the centralized option to force adoption of the decentralized alternative – is a notable governance decision. Trust in a centralized prover was replaced by trust in a marketplace mechanism. Whether that is an improvement depends on what you think markets are.

ZkCloud (formerly Gevulot) provides production-ready proving infrastructure with its Firestarter platform.

Aligned Layer takes a different approach: it uses EigenLayer restaking ($11 billion+ restaked ETH) to provide proof verification as a service, allowing any application to submit proofs for verification without deploying its own verifier contract.

The proving market is transitioning from an infrastructure cost (borne by rollup operators) to a tradeable service (priced by market dynamics). The $PROVE token represents the first attempt to create a liquid market for computational integrity. You can now buy and sell the ability to generate mathematical truth. But the trust has been delegated to prover hardware, and the privacy question – who sees the witness? – has no market solution. It has only engineering solutions, each with its own trust assumptions. TEEs, MPC-based proving, client-side hardware acceleration: each moves the trust, none eliminates it.

The economics of proving-as-a-service deserve to be spelled out, because they determine who actually pays for mathematical truth – and at what margin. The cost structure has three layers: hardware (GPU clusters, typically H100 or A100 machines running at $2-3 per GPU-hour on cloud providers, less for

owned hardware with amortized capex), energy (a single H100 draws roughly 700W under proving load; at industrial electricity rates of $0.05-0.08/kWh, that is $0.035-0.056 per GPU-hour), and engineering (the prover software itself, maintained by teams of 10-50 engineers at salaries that dwarf the hardware costs). The first two are commodity inputs. The third is the moat.

Who pays? Today, the answer is straightforward: rollup operators pay. A ZK rollup that generates proofs for every batch of transactions absorbs proving costs as an operational expense, recouped through user transaction fees. At current rates, proving costs for a major rollup run $200,000-$500,000 per year – significant, but a small fraction of the sequencer revenue for any rollup with meaningful throughput. The user pays indirectly, through transaction fees that embed a proving surcharge. That surcharge has been falling: the 2,000-fold cost collapse described in Chapter 6 means the proving component of a typical L2 transaction fee has dropped from dollars to fractions of a cent. The user, in most cases, does not notice.

But as ZK proofs extend beyond rollups into coprocessors, ZKML, identity, and enterprise applications, the payment model diversifies. A DeFi protocol using a ZK coprocessor to compute a verified TWAP pays per query – perhaps $0.10-$1.00 per coprocessor call, depending on the computation's complexity. An enterprise using ZK proofs for compliance verification pays per attestation – perhaps $5-50 per KYC proof, still far cheaper than the manual compliance process it replaces. A ZKML application proving model inference pays per inference – and here the costs are still high enough to be prohibitive for all but the most valuable use cases, because the computational overhead of proving neural network operations remains steep.

The margin structure is revealing. Hardware and energy costs are transparent and declining. The margin on proving-as-a-service therefore comes from three sources: software differentiation (a faster prover generates more proofs per GPU-hour, extracting more revenue from the same hardware), network effects (a larger prover network attracts more proof requests, spreading fixed costs across more revenue), and trust premiums (a prover network with a longer track record and more stake at risk can charge more than a new entrant, because the customer is paying not just for computation but for reliability). The first source rewards engineering excellence. The second rewards early movers. The third – and this is the one that connects to our thesis – rewards *visible trustworthiness*. The margin on proving-as-a-service is, in part, a margin on trust. The customer pays more to a prover they trust more. Trust has not been eliminated. It has been priced.

Current gross margins for proving services are estimated at 40-60% for operators with owned hardware, dropping to 15-25% for operators renting cloud GPU capacity. As the market matures and competition intensifies, margins will compress toward the cloud-rental floor – unless operators differentiate on software speed, privacy guarantees, or the economic security of their staking mechanisms. The $PROVE

token's role in this dynamic is worth watching: it attempts to align prover incentives with network reliability through slashing (stake at risk if proofs are invalid or late), which is an economic mechanism for trust. You do not trust the prover's goodwill. You trust the prover's financial exposure. Whether that is trust-minimized or merely trust-financialized is a question the market has not yet answered.

The long-term equilibrium may resemble cloud computing: a few large operators (Succinct, RISC Zero, ZkCloud) competing on price and performance, with specialized boutique provers serving niche markets (enterprise privacy, ZKML, identity). The commodity layer – raw proof generation – will be a low-margin, high-volume business. The value layer – proof orchestration, privacy-preserving proving, domain-specific optimization – will be a higher-margin, lower-volume business. In both layers, the customer pays for computation and receives a proof. But in the value layer, the customer also pays for the *conditions* under which the proof was generated: was the witness kept private? Was the hardware audited? Was the proving pipeline formally verified? These conditions are trust assumptions, and they carry a price. The proving market does not sell trustlessness. It sells *degrees of trust*, at different price points, to customers with different risk tolerances. This is an honest market. It is not the market the cypherpunks imagined.

Trust relocated from: self-hosted proving infrastructure **to:** proving marketplace operators + hardware availability. **Net:** cost efficiency with new dependencies on prover liveness and correctness.

13.6 Enterprise Pilots (Pilot)

The magician has left the theater. She now performs in boardrooms, data centers, and regulatory offices. The trick is the same. The audience has changed – and this audience does not care about trustlessness. It cares about compliance.

Deutsche Bank, UBS, and Goldman Sachs have all conducted pilots on ZKsync, exploring tokenized assets with ZK-verified compliance. These pilots use ZK proofs to demonstrate regulatory compliance (KYC/AML verification) without revealing the underlying customer data to settlement counterparties. The bank proves its customer is compliant. The counterparty learns nothing about who the customer is. This is not trust-minimization in the way a cypherpunk would recognize it. It is trust in compliance frameworks, verified by cryptographic proofs but governed by regulatory requirements. The magician performs for regulators now, and the trick she performs is: "we followed the rules, and here is a proof."

The Deutsche Bank pilot reveals what enterprise adoption actually looks like from the inside. Deutsche Bank's Project Guardian participation – coordinated through the Monetary Authority of Singapore – tested whether institutional-grade foreign exchange and government bond transactions could settle

on-chain with ZK-verified KYC attestations replacing the traditional correspondent banking chain. The specific mechanism: Deutsche Bank's compliance team verifies a client's KYC status through conventional channels, then issues a ZK credential – a proof that the client passed all required AML/CFT checks, without encoding *which* checks, *what* documents, or *whose* name. The counterparty's smart contract verifies the proof on-chain and permits settlement. The transaction completes in minutes rather than the T+2 standard. The compliance data never crosses an institutional boundary.

Understand what has changed and what has not. The settlement speed improved. The data exposure shrank. The compliance *cost* dropped, because the correspondent banking chain – each intermediary performing redundant KYC on the same client – collapsed into a single proof verified by mathematics. But the trust did not disappear. It migrated. Before: you trusted the correspondent bank's compliance department. After: you trust Deutsche Bank's compliance department *and* the correctness of their ZK credential issuance system *and* the soundness of the proof system *and* the smart contract that verifies it. The number of trust assumptions increased. Their *visibility* increased too – and visibility is not nothing. A trust assumption you can examine is better than one you cannot. But it is still a trust assumption.

The pilot's economics are instructive. Correspondent banking fees for cross-border institutional transactions typically run 0.1-0.3% of notional value. For a $100 million FX trade, that is $100,000-$300,000 in intermediary costs. The ZK-verified on-chain settlement reduced this to gas fees plus proving costs – roughly $50-200 per transaction at current rates. The savings are not marginal. They are structural. And they create an incentive so large that the technology's imperfections become, from an institutional perspective, tolerable. Banks do not adopt ZK proofs because they believe in trustlessness. They adopt ZK proofs because the alternative costs a quarter of a million dollars per transaction and takes two days. Trust-moved is acceptable when the cost savings are three orders of magnitude.

DTCC (the Depository Trust & Clearing Corporation) partnered with the Canton Network for tokenized Treasury securities in December 2025, using ZK proofs for privacy-preserving settlement. The DTCC clears approximately $2.5 quadrillion in securities annually. If even a single-digit percentage of that volume migrates to ZK-verified on-chain settlement within a decade, the proving infrastructure required would dwarf every other ZK application combined.

The Canton Network partnership is structurally different. Canton is a privacy-first blockchain designed by Digital Asset, built on the Daml smart contract language, where every transaction is visible only to its direct participants. ZK proofs enter not as a replacement for Canton's native privacy (which is achieved through data partitioning) but as a *bridge* between partitions: proving to a regulator that aggregate settlement volumes across private partitions satisfy capital adequacy requirements, without revealing the individual transactions. The regulator sees a proof. The proof says: "the sum of all positions held by

Institution X nets to Y, and Y satisfies threshold Z." The regulator learns Y and whether it exceeds Z. The regulator learns nothing about the individual positions that produced Y.

This is trust architecture of a very specific kind. The DTCC is not minimizing trust in itself – it *is* the trusted institution, and it intends to remain so. It is minimizing the *data exposure* required to maintain that trust. The regulator still trusts the DTCC. But the regulator no longer needs to see every transaction to verify compliance. The proof substitutes for the data. The magician has not replaced the theater manager. She has given the theater manager a way to check the books without reading every page.

Partisia Blockchain operates in a different corner of the enterprise market, one that reveals how ZK proofs interact with multiparty computation in government applications. Partisia's collaboration with the Danish government on digital student credentials – part of Denmark's broader digitization initiative – uses ZK proofs to enable students to prove enrollment status, degree completion, or GPA thresholds to employers and other institutions without revealing their full academic transcript. A student proves she graduated with honors. The employer learns that fact and nothing else – not the specific courses, not the grades in individual subjects, not the institution's internal student ID number.

The trust analysis here differs from the banking cases in a way that matters. In the Deutsche Bank pilot, the institution issuing the credential (the bank) is the same institution the counterparty already trusts. The ZK proof is an efficiency gain, not a trust transformation. In the Partisia student credential system, the institution issuing the credential (the university) and the institution verifying it (the employer) have no pre-existing trust relationship. The ZK proof is not making an existing trust relationship more efficient. It is *creating a new trust pathway* – one that runs through mathematics rather than through phone calls to a registrar's office. This is closer to genuine trust minimization. The employer trusts the proof, not the university's willingness to answer the phone.

But even here, the trust has not vanished. It has been moved to the credential issuance step. The ZK proof guarantees that the credential was issued by the university's signing key. It does not guarantee that the university's signing key was not compromised, that the university's records were accurate, or that the student did not commit academic fraud that went undetected. The mathematics is honest about the student's credential. It says nothing about whether the credential is honest about the student. Trust in the proof is not trust in the underlying reality. This distinction – between proving a document is authentic and proving a document is *true* – is the gap that no amount of cryptography can close. The magician can prove the card was in the deck. She cannot prove the deck was not rigged.

The regulatory dimension of enterprise adoption inverts the usual narrative. In every other context we have examined, ZK proofs are adopted *despite* regulatory uncertainty – builders move fast, regulators catch up later. In the enterprise space, the dynamic is reversed. EU eIDAS 2.0, the Markets in Crypto-Assets Regulation (MiCA), Basel III's treatment of tokenized assets, and the SEC's evolving stance on

digital securities all create compliance obligations that ZK proofs can satisfy. The regulation arrives first. The technology follows. This means enterprise ZK adoption is not driven by technological enthusiasm. It is driven by legal necessity. When a regulation says "you must verify compliance without sharing customer data across jurisdictions," the list of technologies that satisfy that requirement is very short. ZK proofs are at the top of it.

Privacy Pools (0xbow) launched on Ethereum mainnet in April 2025, enabling users to prove the provenance of their funds – that they did not originate from sanctioned addresses – without revealing their full transaction history. This is a direct response to the regulatory challenges highlighted by Tornado Cash sanctions. Privacy Pools is the most philosophically interesting enterprise application: it uses ZK proofs to prove innocence without proving identity. The trust assumption is not in a bank or a regulator but in the mathematical definition of "sanctioned address." That definition is still set by a government. The mathematics enforces the policy. It does not choose the policy.

Trust relocated from: centralized audit/compliance processes **to:** ZK-verified attestations + institutional key management. **Net:** mixed – cryptographic integrity gains offset by institutional adoption risk and key ceremony complexity.

13.7 Market Sizing

The zero-knowledge proof market is growing fast from a small base:

Year	Market Size	Source
2024	$1.28 billion (total ZKP market)	Grand View Research [42]
2025	$1.54 billion (total ZKP market)	Grand View Research [42]
2025	$97 million (proving services only)	Chorus One
2030	$1.34 billion (proving services only)	Chorus One
2033	$7.59 billion (total ZKP market)	Grand View Research [42]

Note on sources: The total ZKP market figures ($1.28B, $1.54B, $7.59B) are from Grand View Research's "Zero-Knowledge Proof Market Size Report" (2025), which covers all ZKP segments at a 22.1% CAGR. The proving-services sub-market figures ($97M, $1.34B) are from Chorus One's "The Economics of ZK-Proving: Market Size and Future Projections"

(2025), which covers the narrower ZK proving services segment at a higher CAGR. Earlier versions of this table conflated the two sources.

The Grand View Research figures cover the full ZK ecosystem: proof generation hardware and services, ZK rollup infrastructure, ZK identity systems, ZKML tooling, and enterprise licensing. The Chorus One figures cover the narrower proving-services sub-market. The largest segment of the total market is ZK rollups (by transaction fee revenue), followed by proving-as-a-service (by infrastructure investment), and ZK identity (by projected end-market size).

The cost trajectory suggests that ZK proving is following a classic deflationary technology curve – the same pattern that drove computing from mainframes to smartphones. As costs approach zero, the binding constraint shifts from "can we afford to prove this?" to "what else can we prove?" This shift is what drives the expansion from rollups (proving transaction execution) into coprocessors (proving data queries), ZKML (proving model inference), and identity (proving personal attributes).

But the market numbers, by themselves, tell you nothing about trust. A $7.59 billion market in which trust has merely been moved from one institution to another is not the same as a $97 million market in which trust has genuinely been minimized. The thesis of this chapter – that the market reveals where trust is actually being minimized and where it is merely being relocated – should make you read these projections with a specific question: in each segment, what trust assumption remains that the buyer does not examine?

The enterprise market is the largest long-term opportunity but the slowest to materialize. Financial institutions move on regulatory timelines (years, not months), require extensive compliance review, and demand vendor stability. The fact that major banks are conducting ZK pilots – rather than dismissing the technology – suggests the institutional adoption curve has begun, but the revenue impact will take 3-5 years to manifest at scale.

Six venues. Six audiences. In two of them – rollups and proving-as-a-service – the technology is production-grade and the trust reduction is measurable: cryptographic proofs replace re-execution, and the economics are favorable. In two – identity and enterprise compliance – the trust minimization is genuine but the adoption path depends on regulatory mandates (eIDAS 2.0, DTCC pilots) rather than pure market forces. In two – coprocessors and ZKML – the technology works but the trust story is incomplete: new dependencies on data availability, model integrity, and prover correctness introduce assumptions that are not yet well-tested at scale. The market is honest about what it sells. The question is whether buyers are honest about what they are buying.

The market is real, growing, and diversifying beyond its blockchain origins. The magician now performs in six different venues – rollups, coprocessors, ML inference, identity wallets, proving marketplaces, and boardrooms – for six different audiences with six different trust requirements. In some venues, trust is

being genuinely minimized. In others, it is being moved to a new location and given a new name. The numbers tell us where the technology is being adopted. They do not tell us where it is honest. For that, we need to ask different questions – not “how big is the market?” but “what remains unsolved?” Those questions are the subject of our final chapter.

The market is growing. The technology works. The money is real. But beneath every market segment, open questions persist – questions about governance, quantum vulnerability, privacy guarantees, and whether the trust minimization marketed to buyers matches the trust decomposition the mathematics actually delivers. The final chapter names these questions, assesses which are solvable and which may be permanent, and draws the line between what zero-knowledge proofs have achieved and what remains to be built.

Chapter 14: Open Questions and the Road Ahead

14.1 The Seven Questions That Remain Open

Richard Feynman had a test for understanding. If you cannot explain something simply, you do not understand it. If you can explain it simply and then find the place where your explanation breaks down, you have found the frontier.

We have spent thirteen chapters explaining zero-knowledge proofs – layer by layer, system by system, from the stage to the audience. Now we arrive at the places where the explanation breaks down. These are not rhetorical questions. They are research problems with measurable progress and no current solution. Each one is an open door. Consider this an invitation to walk through.

14.1.1 Question 1: Can witness generation be made fully parallel on GPUs?

Witness generation involves dependency chains – each intermediate value may depend on previous ones in a directed acyclic graph that resists naive parallelism. When cryptographic proving moves to GPUs (achieving 10-50x speedups via NTT and MSM parallelism), witness generation remains CPU-bound, creating the Witness Gap. ZKPOG (2025) demonstrated 22.8x speedups by restructuring witness

computation for GPU execution, but the general problem – making arbitrary computation graphs GPU-friendly – remains open.

The Witness Gap grows with every proving speedup, as Chapter 4 documented: accelerate the cryptographic step and the CPU-bound witness share swells to dominate total latency. The asymptotic state of zkVM performance may be entirely witness-bound. The backstage preparation may always take longer than the performance. Or perhaps not – but no one has proven otherwise.

The concrete stakes are worth spelling out. If witness generation stays fundamentally sequential, then proving costs plateau regardless of how many GPUs you throw at the problem. You could have a warehouse of A100s computing NTTs in perfect lockstep, and the entire pipeline would still wait on a single CPU thread chasing a dependency chain through the execution trace. The fastest GPU prover in the world is only as fast as the slowest CPU witness generator feeding it. This is Amdahl's Law wearing a cryptographic costume.

ZKPOG's approach – restructuring the witness computation's dependency graph so that independent sub-computations can be dispatched to GPU threads – works when the graph has enough independent width. Many real circuits do. A Merkle tree hash has natural parallelism at each level. An elliptic curve multi-scalar multiplication decomposes into independent scalar-point products. But general computation does not cooperate this neatly. A loop where iteration n depends on the result of iteration $n - 1$ is inherently sequential, and no amount of clever scheduling changes that.

This creates a mismatch between two kinds of parallelism. Polynomial arithmetic – NTTs, MSMs, commitment evaluations – exhibits *regular* parallelism: the same operation applied uniformly to millions of independent data points. GPUs were designed for exactly this. Witness generation exhibits *irregular* parallelism: the computation graph's shape depends on the specific program, the specific input, and the specific execution path taken. GPUs were emphatically not designed for this. Regular parallelism maps to SIMT execution. Irregular parallelism maps to task-graph scheduling, which is the domain of CPU thread pools, not GPU warps.

The dependency graph approach points toward a middle path: analyze the witness computation ahead of time, identify the parallel width at each stage, and schedule GPU work only where the width justifies the dispatch overhead. But this requires a compiler that understands both the circuit semantics and the GPU memory hierarchy – a compiler that does not yet exist in production. The gap between "this is possible in principle" and "this ships in a proving pipeline" is measured in years of compiler engineering.

What makes this question thrilling rather than depressing is that the bottleneck is *not* fundamental. Sequential witness generation is a property of current architectures, not a property of the underlying mathematics. The witness is a function of the public input and the private input. Nothing about that

function requires sequential evaluation. The sequentiality comes from how we *describe* the computation (as a step-by-step trace), not from the computation itself. Whoever figures out how to describe the same computation in a form that exposes its inherent parallelism – without changing the circuit it proves – will unlock the next order-of-magnitude proving speedup. The NTT revolution happened when people stopped thinking of polynomials as coefficient lists and started thinking of them as evaluation vectors. The witness revolution will happen when someone finds the equivalent change of representation for execution traces.

Executive Risk: Proving costs plateau regardless of GPU investment. Hardware budgets may hit diminishing returns before witness generation is parallelized. **Timeline:** 2-4 years for resolution.

14.1.2 Question 2: What is the proven lower bound on post-quantum proof size?

Chapter 7 presented a trilemma: algebraic functionality, post-quantum security, and succinctness – pick two. But is this a fundamental limitation or a reflection of current constructions? No paper proves that $O(1)$-size post-quantum proofs are impossible. The lattice-based schemes are achieving 50-100 KB proofs with post-quantum security, approaching practical territory. But whether there exists a post-quantum polynomial commitment scheme with $O(1)$ proof size and $O(1)$ verification time – matching KZG's properties under lattice assumptions – is unknown.

The ideal PCS problem comes into view: find a commitment scheme that is transparent (no trusted setup), produces constant-size proofs, enables constant-time verification, and relies only on post-quantum assumptions. If such a scheme exists, it would collapse the three-path framework of Chapter 10 into a single path. If it provably cannot exist, the three paths are permanent. Either answer would reshape the field. Neither answer exists today.

To appreciate the gap, consider the numbers side by side. A KZG polynomial commitment produces a proof that is a single elliptic curve point: 48 bytes. A Groth16 proof – the entire argument of knowledge, not just the commitment – fits in 192 bytes. These are absurdly small. They are small because the pairing structure of elliptic curves allows the verifier to check a polynomial identity with a single bilinear map evaluation. No redundancy. No repetition. Just one algebraic equation that either holds or doesn't.

Now look at the post-quantum world. The best lattice-based proofs run 50-100 KB. Hash-based proofs (STARKs) run larger still – hundreds of kilobytes to low megabytes before recursion compresses them. The gap is three orders of magnitude. A KZG proof fits in a tweet. A lattice proof fills a small PDF. Why?

The reason is structural, not incidental. Pairing-based schemes exploit a specific algebraic trick: the pairing $e(g, h)$ lets you check multiplicative relations between committed values without revealing the values themselves. This trick compresses verification into a constant number of pairing evaluations, re-

gardless of the polynomial's degree. Lattice-based schemes have no analogous trick. Their security rests on the hardness of finding short vectors in high-dimensional lattices – a problem that is (probably) hard even for quantum computers, but that does not offer the same algebraic leverage. Verification requires checking that a vector is short, and the proof that a vector is short inherently requires transmitting information proportional to the vector's dimension.

The Wee-Wu line of results suggests that this gap may have deep roots. Their work on compact functional encryption and related primitives shows that certain cryptographic tasks require specific algebraic structure (pairings, or their lattice analogues) to achieve compactness, and that this structure is hard to instantiate from lattice assumptions without blowup. The barrier is not a proof of impossibility – nobody has shown that $O(1)$-size post-quantum proofs *cannot* exist. But the barrier results suggest that if such proofs exist, they will require fundamentally new algebraic ideas, not incremental improvements to existing lattice techniques.

What would a proof of impossibility mean? It would mean the ZK field permanently bifurcates: fast-and-small proofs (pairing-based, quantum-vulnerable) versus large-and-safe proofs (lattice-based, quantum-resistant), with no bridge between them. Every system would need to choose a lane. The STARK-to-SNARK wrapping strategy described in Chapter 10 – use hash-based proofs for soundness, then compress the proof with a pairing-based scheme for on-chain verification – would be the permanent architecture, not a temporary compromise. The three paths would harden into the three permanent roads.

Conversely, if someone constructs a post-quantum scheme with $O(1)$ proof size, it would be the most important result in applied cryptography since Groth16. It would mean the entire field's current architecture – the wrapping, the recursion, the careful balancing of proof size against verification cost – is a temporary artifact of our incomplete understanding, not a fundamental constraint. Every ZK engineer is implicitly betting on which answer is correct. The field has placed most of its chips on "the gap is permanent" and built its infrastructure accordingly. If that bet is wrong, the infrastructure will need to be rebuilt. If that bet is right, the infrastructure is the final form. Either way, someone needs to settle the question. The mathematics is patient, but the engineering decisions are being made now.

Executive Risk: Permanent bifurcation of the ZK ecosystem into pre-quantum and post-quantum paths with no migration bridge. Systems deployed today on pairing-based foundations face mandatory replacement, not upgrade. **Timeline:** 5-10 years for theoretical resolution; practical deployment 3-5 years after.

14.1.3 Question 3: When will Stage 2 bind?

L2Beat's Stages framework defines when ZK rollup governance can override cryptographic guarantees: Stage 0 (centrally controlled), Stage 1 (limited governance overrides with 7-day exit windows), Stage 2 (fully decentralized with 30-day exit windows and no governance override).

As of early 2026, most ZK rollups are Stage 0 or Stage 1. This means governance can override the proof system. A Stage 0 rollup could have 256-bit security and it would not matter if one person can push a verifier upgrade. The cryptographic guarantees of Layers 1-6 do not actually bind until a system reaches Stage 2. The magic trick is perfect, but the theater manager can rewrite the ending.

When will Stage 2 happen? Nobody knows, because Stage 2 requires resolving deep engineering challenges (bug-free verifier contracts, escape hatches that work in practice, proof submission that cannot be censored) and deep governance challenges (who decides when a system is ready, and what happens when a critical bug is found post-lock). The path forward likely involves multiple independent verifier implementations that cross-check each other, formal verification of the verifier contract's core logic, and mandatory exit windows long enough for users to withdraw if they disagree with a governance decision. Each of these is technically feasible; none has been fully achieved. This is the question where mathematics ends and human institutions begin.

Consider what Stage 2 actually looks like in practice. Picture a ZK rollup where no single entity – not the founding team, not a multisig, not a governance token majority – can push a verifier upgrade without triggering a mandatory 30-day exit window. During that window, every user can inspect the proposed change, compare the old and new verifier contracts, and withdraw their funds to L1 if they disagree. Multiple independent verifier implementations – written by different teams, in different languages, tested against different fuzzing suites – cross-check each other's results. If any implementation disagrees with the others, the system halts and the dispute resolution mechanism activates. The verifier contract itself has been formally verified: not just audited by humans who might miss an edge case, but proven correct by a theorem prover that has checked every execution path.

That is the goal. Now consider the tensions that make it hard.

The first tension is between immutability and patchability. An immutable verifier contract cannot be captured by governance – but it also cannot be patched when a critical bug is discovered. And critical bugs *will* be discovered. The history of smart contract security is unambiguous on this point: every contract of sufficient complexity has bugs, and the bugs are found on timelines measured in months to years, not days. A Stage 2 system that locks its verifier contract is betting that the contract is bug-free. That bet has never been won at scale. A Stage 2 system that retains upgrade capability, even with timelocks, is betting that the governance process cannot be captured. That bet has been lost repeatedly in

traditional finance. The equilibrium, if it exists, involves a narrow corridor: upgrades are possible, but only through a process so slow and so transparent that capture is impractical.

The second tension is between safety and liveness. A system with 30-day exit windows is safe – users can always leave. But if a critical vulnerability is discovered and exploited, 30 days is an eternity. An attacker who finds a soundness bug can drain the rollup in a single transaction. The users' 30-day exit window is worthless if the funds have already been stolen. This suggests that Stage 2 needs not just exit windows but also emergency circuit-breakers – but a circuit-breaker is a governance override, which is exactly what Stage 2 is supposed to eliminate. The circle closes on itself.

The third tension is economic. Multiple independent verifier implementations are expensive. Formal verification is expensive. Long exit windows impose opportunity costs on users. Who pays? In a Stage 0 system, the founding team pays for everything and recoups the cost through token appreciation or sequencer fees. In a Stage 2 system, the costs must be borne by the protocol itself, which means they are ultimately borne by users through fees or inflation. The economic question is whether the security premium of Stage 2 is worth the cost – and the answer depends on how much value the rollup secures. A rollup holding $10 billion has a very different cost-benefit calculus than a rollup holding $10 million.

The honest answer may be that Stage 2 is not a destination but a spectrum. Different systems will reach different points on that spectrum depending on their value secured, their user base's risk tolerance, and the maturity of their verifier contract. The first true Stage 2 system will likely be a rollup that has been in production for years, has undergone multiple independent audits and formal verification efforts, and secures enough value to justify the engineering cost. It will be boring. It will be slow. It will be the most important thing in the ecosystem. The question is not *whether* it will happen, but whether the incentives align for it to happen before a catastrophic failure forces it to happen. The field should prefer the former.

Executive Risk: Governance override negates all cryptographic guarantees. Over $20B in TVL sits in systems where a multisig can replace the verifier. This is the single largest unresolved risk in ZK infrastructure. **Timeline:** 1-3 years for first Stage 2 rollups.

14.1.4 Question 4: When will "trustless" become real?

The trust decomposition in Chapter 10 identified seven independent assumptions underlying ZK systems. Today, every deployed system requires trusting all seven simultaneously. The trajectory points toward progressive reduction:

- Under-constrained circuits (Layer 2): formal verification tools (Picus, ZKAP, Coda) are reducing but not eliminating the vulnerability class catalogued in Chapter 3; the tools catch many but not all.

- Fiat-Shamir bugs (Layer 5): standardization of transcript protocols and automated fuzzing (ARGUZZ found 11 bugs across 6 zkVMs) are addressing the implementation gap.
- Governance override (Layer 7): Stage 2 maturation will eventually remove governance as an attack surface.
- Quantum vulnerability (Layer 6): lattice-based and hash-based primitives provide migration paths.

But "trustless" – meaning zero residual trust assumptions – requires proving that hardware does not leak (impossible without information-theoretic security), proving that all software is correct (requiring formal verification of the entire stack), and proving that mathematical hardness assumptions hold (which is inherently impossible – hardness is a conjecture, not a theorem).

The honest trajectory: trust will continue to decrease, asymptotically approaching but never reaching zero. "Trust-minimized, and getting better every day" is the accurate description. Like Zeno's arrow, the gap closes by half with each advance. Zero is not the destination. The asymptote is the destination, and it is a good one.

But the Zeno metaphor deserves to be pushed harder, because it reveals something the optimistic narrative glosses over. Each layer's trust assumption is shrinking independently. Layer 2 gets safer as formal verification improves. Layer 5 gets safer as transcript standardization matures. Layer 6 gets safer as lattice assumptions are studied. Layer 7 gets safer as governance decentralizes. Viewed individually, each trend is encouraging. But the system's overall trustworthiness is not the average of its layers – it is the *conjunction*. All seven must hold simultaneously. If any single layer fails, the system fails.

This is the Zeno's paradox of trust in its precise form. Suppose each layer independently has a 99% chance of holding its trust assumption. Seven independent layers at 99% each give a system-level confidence of $0.99^7 \approx 93.2\%$. Improve each layer to 99.9% and the system reaches 99.3%. Improve to 99.99% and you get 99.93%. The system-level confidence always trails the weakest layer, and the gap between layer-level confidence and system-level confidence grows with the number of layers. Adding layers – which is what happens as the stack matures and new trust assumptions are identified – makes the individual layers stronger but the conjunction weaker.

This is not a reason for despair. It is a reason for precision. The trajectory toward trust-minimization is real, but the conjunction effect means that "almost trustless" is a category error. A system with six perfect layers and one compromised layer is not "mostly trustless" – it is compromised. The chain is as strong as its weakest link, and a seven-link chain has seven opportunities for weakness.

The practical implication is that the field needs to track system-level trust, not layer-level trust. A dashboard showing "Layer 2: formally verified, Layer 5: standardized transcripts, Layer 6: post-quantum ready, Layer 7: Stage 1.5" tells you about components. It does not tell you about the system. The system-

level question is: what is the probability that *all seven* hold simultaneously for *this specific deployment* with *this specific configuration*? Nobody can answer that question today. Developing the methodology to answer it – a kind of actuarial science for cryptographic trust – is itself an open research problem.

The deepest version of this question is philosophical, not technical. Is there a fundamental lower bound on residual trust, the way there is a fundamental lower bound on measurement uncertainty in quantum mechanics? Can we prove that *any* computational system, no matter how carefully designed, must retain some irreducible trust assumption? The answer is almost certainly yes – you always trust that the laws of physics haven't changed, that your hardware is computing what you think it's computing, that the mathematical axioms underlying the proofs are consistent. But articulating exactly where that floor sits, and how close current systems are to reaching it, would transform "trust-minimized" from a vague aspiration into a measurable engineering target. The person who defines that floor will have done for cryptographic trust what Shannon did for information capacity: turned an intuition into a theorem with a number attached to it.

Executive Risk: Marketing claims exceed technical reality. Organizations deploying 'trustless' systems inherit seven trust assumptions they may not have assessed. Due diligence must enumerate all seven. **Timeline:** Ongoing.

14.1.5 Question 5: How do streaming witness approaches interact with folding?

Nair et al. (2025) demonstrated streaming witness generation that uses space proportional to trace width (registers) rather than trace length (steps). This is essential for proving long computations without requiring hundreds of gigabytes of RAM. But folding schemes (Nova, HyperNova, LatticeFold) accumulate state across folding steps – the accumulated instance grows with each fold. How streaming witnesses interact with accumulating folding state is an open architectural question.

This matters because folding and streaming attack different bottlenecks: folding reduces prover time by avoiding redundant computation; streaming reduces prover memory by generating witness chunks on-the-fly. An architecture that combines both would be strictly superior to either alone, but the interaction is non-trivial. Two good ideas that have not yet learned to dance together.

The architectural tension runs deeper than a scheduling problem. It is a fundamental conflict between two strategies for managing state.

Folding *accumulates* state. Each folding step takes the current accumulated instance and a new instance and produces a new accumulated instance that absorbs both. In Nova, this means the error vector $\mathbf{E}$ grows – or more precisely, the relaxed R1CS instance carries forward information about all previous folding steps. In LatticeFold, the accumulated witness includes norm-bounded vectors whose bounds

increase with each fold. The accumulator is the memory of the system. It remembers everything that has been folded into it. That memory is what makes folding powerful: the verifier checks a single accumulated instance at the end, rather than checking each step individually. But that memory has a cost. The accumulated state must be stored, updated, and eventually opened. It cannot be discarded mid-computation.

Streaming *discards* state. The entire point of streaming witness generation is to produce a chunk of the witness, use it, and then forget it. The prover's memory footprint stays proportional to the width of the computation (how many registers the program uses) rather than its length (how many steps the program takes). A streaming prover for a billion-step computation uses the same memory as a streaming prover for a million-step computation. That is the magic. But it works only if the prover can forget the past. The moment you require the prover to retain information about earlier chunks – which is exactly what folding demands – the streaming property degrades.

The crux is this: folding needs the prover to remember the accumulated error from all previous steps. Streaming needs the prover to forget the witness from all previous steps. These are not the same thing. The accumulated error is a compact summary (a single relaxed instance), while the witness is a massive expansion (the full execution trace). In principle, you can stream the witness while retaining only the accumulated instance. In practice, the folding step itself requires access to both the accumulated witness *and* the new witness chunk simultaneously, because the folding combiner must compute cross-terms between old and new. Those cross-terms are the technical barrier.

One possible resolution: a folding scheme where the cross-terms can be computed incrementally, using only the compact accumulated instance and the current witness chunk, without ever materializing the full accumulated witness. This would require the cross-term computation to be decomposable into a streaming-friendly form – which is a non-trivial algebraic constraint on the folding scheme's structure. Nova's cross-term involves an inner product between the accumulated witness and a matrix applied to the new witness. That inner product can, in principle, be computed in a streaming fashion if the matrix-vector product is streamed. But the accumulated witness itself is not compact; it is as large as the original witness. The circularity is apparent.

A different resolution: a layered architecture where streaming operates within each folding step and folding operates across steps. Generate the witness for step k using streaming (small memory), fold it into the accumulator (small memory for the accumulator, large memory transiently for the fold), then discard the witness for step k and move to step $k+1$. The peak memory is determined by the single-step witness size plus the accumulator size, not by the total trace length. This is closer to what practical implementations will likely achieve, but it requires that each folding step can be completed before the

next witness chunk is generated – which imposes a sequential dependency between folding and witness generation that limits parallelism.

Nobody has built an architecture that resolves all three constraints simultaneously: streaming memory, folding time-savings, and parallel execution. The system that does will dominate the proving market for the next generation of zkVMs. It is the kind of problem where the first correct solution will seem obvious in retrospect and impossible in prospect – which is the surest sign that it is worth working on.

Executive Risk: Memory requirements constrain prover hardware to high-end data center configurations, limiting decentralization and increasing vendor lock-in. **Timeline:** 2-3 years for streaming witness architectures to mature.

14.1.6 Question 6: Can constant-time ZK proving be made practical?

Side-channel attacks on ZK implementations – timing attacks on Zcash's Groth16 prover ($R = 0.57$ correlation with transaction amounts), cache timing leakages in Poseidon/Reinforced Concrete/Tip5, electromagnetic emanation from field arithmetic – demonstrate that "zero-knowledge" is a mathematical property, not an implementation guarantee. The proof reveals nothing in theory. The hardware whispers everything in practice.

Constant-time implementations exist (Monero's Bulletproofs prover achieves $R = 0.04$ correlation) but impose significant performance costs. GPU proving complicates this further: GPUs execute in SIMT (Single Instruction, Multiple Thread) mode, where thread divergence is observable. Constant-time code creates artificial thread divergence, reducing GPU utilization. The tension between side-channel resistance and hardware throughput has no known resolution. The backstage walls need to be soundproofed, but soundproofing slows down the preparation.

The GPU problem is worth understanding in detail, because it illustrates why this is not simply a matter of writing more careful code.

A GPU does not execute threads independently. It executes them in *warps* – groups of 32 threads (on NVIDIA hardware) that share a single instruction pointer. When all 32 threads in a warp take the same branch, the warp executes at full speed. When threads diverge – some taking the `if` branch, others taking the `else` branch – the warp must execute *both* branches serially, masking out the threads that shouldn't participate in each branch. This is thread divergence, and it is the fundamental mechanism by which GPUs trade flexibility for throughput.

Constant-time code, by definition, must execute the same instructions regardless of the input. In a CPU, this means taking both branches and selecting the result with a conditional move – a small overhead, typically 2x or less. In a GPU, this means *forcing* all 32 threads in a warp to execute both branches even

when all 32 threads would naturally take the same branch. The constant-time requirement converts natural coherence (all threads agree) into artificial divergence (all threads must pretend to disagree). On a CPU, constant-time code is slower by a constant factor. On a GPU, constant-time code destroys the execution model's fundamental assumption.

The numbers are stark. A well-optimized GPU NTT can achieve 80-90% of theoretical throughput because the computation is uniform: every thread does the same butterfly operation on different data. A constant-time GPU implementation of the same NTT – one that pads all branches to equal length and eliminates data-dependent memory access patterns – drops to 30-50% throughput. The field arithmetic itself (modular addition, modular multiplication) can be made constant-time without much overhead, because it is naturally branchless. But the *control flow* of the proving algorithm – which polynomials to evaluate, which commitment rounds to perform, how to handle edge cases in the MSM bucket accumulation – is where the branches live, and where constant-time enforcement hurts.

This creates a stark engineering tradeoff. On one side: constant-time code on CPU. Viable, proven (Monero does it), but slow. A CPU prover is 10-50x slower than a GPU prover. On the other side: variable-time code on GPU. Fast, but the timing variations leak information about the witness through observable channels. The proving time itself becomes a side channel. An attacker who can measure how long your proof took can infer something about your private input – exactly the information the zero-knowledge property is supposed to hide.

No current system offers both. No current system even articulates a clear path to offering both. The approaches being explored – oblivious RAM for memory access patterns, garbled circuits for branching logic, hardware enclaves for execution isolation – each solve part of the problem while introducing new trust assumptions. Oblivious RAM has logarithmic overhead that compounds badly at GPU scale. Garbled circuits are designed for two-party computation, not single-prover execution. Hardware enclaves (SGX, TrustZone) are the very kind of trusted hardware that ZK proofs were supposed to make unnecessary.

The resolution, when it comes, will likely involve a new hardware abstraction: a proving accelerator designed from the ground up for constant-time cryptographic computation, where the SIMT model is replaced by a model that treats uniform execution time as a feature rather than a constraint. Such hardware does not exist. Designing it is an open problem at the intersection of chip architecture, cryptographic engineering, and side-channel analysis. The person who designs it will need to understand all three. That is a rare combination, which is why the problem remains open.

Executive Risk: Privacy leaks through side channels in production. Proving time correlates with transaction characteristics (demonstrated $R = 0.57$ in Zcash). Systems marketed as 'private' may leak information to network observers. **Timeline:** 3-5 years for constant-time provers to become standard.

14.1.7 Question 7: Is "seven" the right number of layers?

The evidence suggests that the seven-layer model is pedagogically useful but architecturally approximate. Layers 3 and 4 collapse in Jolt. Layers 4, 5, and 6 form an inseparable "proof core." STARK-to-SNARK wrapping pierces Layers 5, 6, and 7. Data availability and proof aggregation are substantial infrastructure layers not represented. Privacy is a cross-cutting concern, not a single-layer property.

The model might be better as four macro-layers: (1) Trust Setup, (2) Programming Model, (3) Proof Core {witness + arithmetization + proof system + primitives}, (4) On-Chain Settlement {verification + data availability + governance}. Or it might be better as nine layers, adding data availability and proof aggregation explicitly. Or seven might be exactly right for the pedagogical purpose it serves, with explicit acknowledgment that production systems do not respect the boundaries.

The question is not academic. The number of layers determines how people think about the system, which determines which design decisions they consider independent (and thus can optimize separately) versus coupled (and must co-design). Getting the decomposition wrong leads to suboptimal architectures. Getting it right enables the field to progress faster. The map shapes the territory.

The practical cost of a wrong decomposition is already visible. Consider Layers 4 and 5 in this book's model – the arithmetization layer and the proof system layer. The model presents them as separate, and this separation is conceptually clean: one translates the program into equations, the other proves the equations are satisfied. Different teams can work on each. Different papers can advance each. Different benchmarks can measure each. The separation enables a division of labor that has served the field well.

But in practice, Layers 4 and 5 are deeply coupled. The arithmetization's structure determines which proof system can operate on it efficiently. R1CS works naturally with Groth16 and Nova. AIR works naturally with STARKs. CCS works naturally with HyperNova. Choosing an arithmetization *is* choosing a family of proof systems, and choosing a proof system constrains the arithmetization. Teams that optimize Layer 4 independently of Layer 5 – designing a beautiful arithmetization and then discovering that no efficient proof system can consume it – have wasted months of work. This has happened. It will happen again whenever the model's boundaries do not match reality's boundaries.

The same coupling appears between Layers 3 and 4. Jolt demonstrated this forcefully: in Jolt's architecture, the witness *is* the arithmetization. There is no separate "witness generation" step followed by a "constraint compilation" step. The lookup table that defines the instruction semantics simultaneously serves as the witness and the constraint. Trying to optimize "witness generation" and "arithmetization" separately in Jolt is like trying to optimize a coin's heads and tails independently. They are the same object viewed from two angles.

The four-macro-layer model – Trust Setup, Programming Model, Proof Core, On-Chain Settlement – has the virtue of grouping the coupled components together. The "Proof Core" macro-layer acknowledges that witness generation, arithmetization, the proof system, and the underlying primitives are a single coupled system that must be co-designed. But it loses granularity. It tells the proof-core team "this is all yours" without helping them understand the internal structure of their subsystem.

The nine-layer model – adding data availability and proof aggregation as explicit layers – has the virtue of representing real infrastructure that the seven-layer model ignores. Data availability is not a detail. It is a trust assumption as fundamental as any in the stack: if the data is not available, the proof is unverifiable in practice, regardless of its mathematical soundness. Proof aggregation – the recursive composition of proofs into proofs-of-proofs – is the mechanism that makes ZK rollups economically viable, and it introduces its own trust assumptions (the aggregation circuit must be correct, the recursive verifier must be sound). Omitting these from the model means omitting trust assumptions from the analysis, which defeats the model's purpose.

The right answer may not be a fixed number at all. It may be that the useful decomposition depends on the question you are asking. If you are asking "how does trust flow through a ZK system," seven layers (or nine, or four) provide different but complementary views. If you are asking "how should engineering teams be organized," the coupling structure matters more than the layer count. If you are asking "what can go wrong," the answer is a threat model, not a layer diagram, and the threat model cuts across every decomposition.

The OSI model survived not because seven was the right number of layers for networking – it was widely criticized for being both too many and too few, depending on context – but because it gave a generation of engineers a shared vocabulary. This book's seven-layer model will succeed or fail on the same terms. If it gives ZK engineers a shared vocabulary for discussing trust assumptions, it will have served its purpose even if no production system has exactly seven layers. If a better decomposition emerges – one that captures the couplings, represents the missing layers, and scales from pedagogy to production – it should replace this one without ceremony. The goal was never to be right about the number. The goal was to be useful about the structure.

Executive Risk: Low. This is a pedagogical question, not a deployment risk. The layer model is a communication tool; systems work regardless of how we categorize their components.

14.2 The Three Frontiers

The ZK field is transitioning through three sequential frontiers, each building on the previous:

14.2.1 Frontier 1: Performance (2023-2025) – Largely Crossed

The performance frontier asked: can we prove computation fast enough and cheaply enough for production use? The answer, as of late 2025, is yes. Real-time Ethereum proving is achieved. The cost curve traced in Chapters 1 and 6 has flattened at pennies per block. The EF declared the speed race won. The remaining performance work is optimization (energy efficiency, memory reduction, witness acceleration), not breakthrough. The magician can now perform in real time. That question is settled.

Evidence this frontier is active: - SP1 Hypercube: 6.9s Ethereum block proving on 16 GPUs (Dec 2025) - Airbender: 21.8M RISC-V cycles/sec on single H100 (2025) - Proving cost: $80 → $0.04 in 24 months (2,000x reduction) - EF declared speed race 'effectively won,' pivoted to security (Dec 2025)

14.2.2 Frontier 2: Security (2026-2028) – Currently Active

The security frontier asks: can we prove that our proofs are actually sound, with quantifiable security guarantees? The EF's 2026 targets (100-bit by May, 128-bit by December) define this frontier precisely. It includes formal verification of zkVM implementations, provable soundness without conjectured assumptions (proximity gaps), post-quantum readiness, and protection against Fiat-Shamir vulnerabilities.

This frontier is harder than performance because security is a property you prove, not a metric you optimize. You cannot benchmark soundness the way you benchmark throughput. The tools (Picus, ZKAP, ARGUZZ, soundcalc) are emerging but immature. The field's first formal verification efforts (SP1's RISC-V Sail specification verification, Jolt's ACL2 lookup semantics verification) are promising but cover only fragments of the full stack. Proving that the trick works is easy. Proving that it *cannot* be faked is the real challenge.

Evidence this frontier is active: - EF 2026 targets: 100-bit provable security by May, 128-bit by December - SP1: formal verification of 62 RISC-V opcodes against Sail specification - Arguzz (Hochrainer, Wustholz, Christakis, 2025): found 11 bugs across 6 major zkVMs - soundcalc: automated soundness margin calculator for proof system parameters - Picus, ZKAP: emerging static analysis tools for constraint under-specification

14.2.3 Frontier 3: Privacy (2027+) – Approaching

The privacy frontier asks: can we build systems where users genuinely control their data, where the "zero-knowledge" property holds not just mathematically but in practice? This frontier includes constant-time implementations, metadata privacy (timing, transaction structure, network-level information), and application-layer privacy design (compiler-enforced disclosure boundaries, selective disclosure for regulatory compliance).

Midnight represents an early attempt at this frontier. Its disclosure analysis is the most sophisticated compiler-level privacy enforcement in any ZK system. But the documentation's silence on side channels, the indexer's metadata leakage, and the SDK-level timing correlations demonstrate how far the field has to go.

The privacy frontier is the most difficult of the three because it requires solving problems across all seven layers simultaneously. Performance is primarily a Layers 3-5 problem. Security spans Layers 2-6. Privacy, as Midnight demonstrates, is a property of the entire stack. The theater must be lightproof at every joint.

Evidence this frontier is active: - Compact disclosure analysis: 11 compile-time privacy errors caught in single voting contract - Monero Bulletproofs: constant-time implementation ($R = 0.04$ timing correlation) - eIDAS 2.0: EU mandate for selective disclosure in digital identity wallets (2026) - Midnight testnet: end-to-end private smart contract execution with local proving - Zcash timing vulnerability patched; field now aware of side-channel class

14.3 Convergence

The zero-knowledge proof ecosystem in March 2026 is in a state that would have been difficult to predict three years ago. Real-time proving of arbitrary computation is solved. Costs are negligible. Seven production zkVMs compete on a standardized benchmark (Ethereum block proving). The Ethereum Foundation has shifted its primary metric from speed to security. Lattice-based constructions are approaching practical viability for post-quantum proving. Enterprise financial institutions are conducting pilots. A $97 million market is projected to grow to $7.59 billion in seven years.

The seven-layer model – for all its imperfections, its outdated binary, its missing primitives – got the fundamental insight right: zero-knowledge proof systems are stacks, not monoliths. Understanding them requires understanding each layer's contribution and, critically, the interactions between layers. The model bends where layers couple (the proof core) and breaks where layers collapse (Jolt's

witness-is-arithmetization), but it serves its pedagogical purpose: giving a structured way to think about a technology that is simultaneously simpler than it looks (the core math is elegant) and more complex than it looks (the engineering is a web of coupled decisions).

The seven open questions remain open. The ideal PCS has not been found. Stage 2 governance has not bound. "Trustless" has not become real. But the trajectory is unambiguous: the trust assumptions are getting weaker, the proofs are getting cheaper, the security is getting stronger, and the privacy is getting more principled.

Trust-minimized, not trustless – and getting better every day.

14.4 Coda

The magician walks on stage. She faces an audience of strangers – people who have no reason to trust her and every reason to doubt. She makes a claim that sounds impossible.

But the audience is no longer what it once was. Over thirteen chapters, the stage has been built, inspected, and rebuilt. The choreography has been written in languages that prevent the worst mistakes. The backstage recording has been compressed into a mathematical certificate that no forger can reproduce. The seal rests on problems that have resisted every attack for decades, and increasingly on problems designed to resist quantum computers too. The verdict is rendered by software that is slowly, painfully, being placed beyond the reach of any committee or key-holder.

The trick still requires trust. It always will.

But the trust has been decomposed, distributed, and minimized until what remains is not faith in a person or an institution but confidence in a conjunction of mathematical facts – each independently testable, each independently replaceable, each weaker than trusting any single entity with everything.

Think about what that means for a real person.

Consider Alice. She is a small-business owner applying for a loan. In the world before this technology, the bank demands everything: tax returns, account balances, transaction histories, personal identification, credit reports. The bank sees all of Alice's financial life, stores it on servers that get breached with depressing regularity, and shares it with partners and regulators and data brokers in ways Alice never consented to and cannot control.

Now consider Alice in the world this technology is building. She generates a zero-knowledge proof that her income exceeds the lending threshold. She proves her identity is verified and her funds are not sanctioned. She proves her credit score falls within an acceptable range. The bank receives three proofs and a loan application. It learns that Alice qualifies. It learns nothing else – not her exact income, not her

account numbers, not her transaction history, not the names of her customers. The sealed certificates say: she qualifies. The mathematics guarantees it. The bank verifies in milliseconds.

Alice controls her data. The bank gets the answer it needs. The regulator can verify that the process was followed correctly. And no database holds Alice's financial life story, waiting to be breached.

That is what seven layers of mathematics make possible. Not trustlessness – trust-minimization. Not perfection – progress. Not magic – engineering that looks like magic until you understand every layer, at which point it looks like something better: a system where telling the truth is easier than lying, and where proving a fact does not require surrendering the context that makes it private.

The magician performs. The audience verifies. And between them, seven layers of mathematics ensure that the proof reveals nothing but the truth.

Complete Bibliography

14.4.1 Chapter 1: The Promise

1. Clarke, Arthur C. *Profiles of the Future*. Harper & Row, 1962.
2. Goldwasser, Shafi, Silvio Micali, and Charles Rackoff. "The Knowledge Complexity of Interactive Proof Systems." *SIAM Journal on Computing* 18(1): 186–208, 1989.
3. Chaliasos, Stefanos, et al. "SoK: What Don't We Know? Understanding Security Vulnerabilities in SNARKs." *USENIX Security Symposium*, 2024.

14.4.2 Chapter 2: Layer 1 – Building the Stage

4. Kate, Aniket, Gregory M. Zaverucha, and Ian Goldberg. "Constant-Size Commitments to Polynomials and Their Applications." *ASIACRYPT 2010*.
5. Bowe, Sean, Ariel Gabizon, and Ian Miers. "Scalable Multi-party Computation for zk-SNARK Parameters in the Random Beacon Model." ePrint 2017/1050.
6. Groth, Jens. "On the Size of Pairing-Based Non-interactive Arguments." *EUROCRYPT 2016*.
7. Gabizon, Ariel, Zachary J. Williamson, and Oana Ciobotaru. "PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge." ePrint 2019/953.
8. Ben-Sasson, Eli, et al. "Scalable, Transparent, and Post-Quantum Secure Computational Integrity." ePrint 2018/046.
9. Bunz, Benedikt, et al. "Bulletproofs: Short Proofs for Confidential Transactions and More." *IEEE S&P 2018*.
10. Wang, Faxing, Shaanan Cohney, and Joseph Bonneau. "SoK: Trusted Setups for Powers-of-Tau Strings." *FC 2025*. ePrint 2025/064.
11. Boneh, Dan and Binyi Chen. "LatticeFold+: Faster, Simpler, Shorter Lattice-Based Folding for Succinct Proof Systems." *CRYPTO 2025*. ePrint 2025/247.

14.4.3 Chapters 3-5

12. Pailoor, Shankara, et al. "Automated Detection of Under-Constrained Circuits in Zero-Knowledge Proofs." *PLDI 2023*. ePrint 2023/512. (See also ref [47].)
13. Wen, Hongbo, et al. "Practical Security Analysis of Zero-Knowledge Proof Circuits." *USENIX Security 2024*. ePrint 2023/190. (See also ref [48].)
14. Setty, Srinath, Justin Thaler, and Riad Wahby. "Customizable Constraint Systems for Succinct Arguments." ePrint 2023/552.
15. Setty, Srinath, Justin Thaler, and Riad Wahby. "Unlocking the Lookup Singularity with Lasso." ePrint 2023/1216.
16. Arun, Arasu, Srinath Setty, and Justin Thaler. "Jolt: SNARKs for Virtual Machines via Lookups." ePrint 2023/1217.

14.4.4 Chapter 6: Layer 5 – The Sealed Certificate

17. Kothapalli, Abhiram, Srinath Setty, and Ioanna Tzialla. "Nova: Recursive Zero-Knowledge Arguments from Folding Schemes." *CRYPTO 2022*.
18. Kothapalli, Abhiram and Srinath Setty. "HyperNova: Recursive Arguments for Customizable Constraint Systems." *CRYPTO 2024*. ePrint 2023/573.
19. Bunz, Benedikt and Binyi Chen. "ProtoStar: Generic Efficient Accumulation/Folding for Special Sound Protocols." *ASIACRYPT 2023*. ePrint 2023/620.
20. Boneh, Dan and Binyi Chen. "LatticeFold: A Lattice-based Folding Scheme and its Applications to Succinct Proof Systems." *ASIACRYPT 2025*. ePrint 2024/257.
21. Nguyen, Wilson and Srinath Setty. "Neo: Lattice-based Folding Scheme for CCS over Small Fields and Pay-per-bit Commitments." ePrint 2025/294.
22. Trail of Bits. "Frozen Heart: Forgery of Zero Knowledge Proofs." Blog post, April 2022.
23. Haboeck, Ulrich, David Levit, and Shahar Papini. "Circle STARKs." ePrint 2024/278.
24. LFDT-Nightstream. "Nightstream: Lattice-Based Folding Implementation." GitHub repository, 2025. https://github.com/LFDT-Nightstream/Nightstream

14.4.5 Chapter 7: Layer 6 – The Deep Craft

24. Shor, Peter W. "Algorithms for Quantum Computation: Discrete Logarithms and Factoring." *FOCS 1994*.
25. NIST. FIPS 203, 204, 205. August 2024.
26. NIST. "Transition to Post-Quantum Cryptography Standards (IR 8547)." November 2024.

14.4.6 Chapter 8: Layer 7 – The Verdict

27. L2Beat. "Stages Framework for L2 Maturity." https://l2beat.com/stages. Accessed March 2026.
28. Chaliasos, Stefanos, et al. "Unaligned Incentives: Pricing Attacks Against Blockchain Rollups." arXiv 2509.17126, 2025.

14.4.7 Chapter 9: Privacy-Enhancing Technologies

29. Gentry, Craig. "Fully Homomorphic Encryption Using Ideal Lattices." *STOC 2009*.
30. Kerber, Thomas, Aggelos Kiayias, and Markulf Kohlweiss. "Kachina – Foundations of Private Smart Contracts." *IEEE CSF 2021*.
31. Buterin, Vitalik, et al. "Blockchain Privacy and Regulatory Compliance: Towards a Practical Equilibrium." *Blockchain: Research and Applications*, 2023. SSRN 4563364.

14.4.8 Chapters 10-11: Synthesis and zkVMs

32. Gassmann, Thomas, et al. "Evaluating Compiler Optimization Impacts on zkVM Performance." arXiv 2508.17518, 2026.
33. Ozdemir, Alex, Fraser Brown, and Riad Wahby. "CirC: Compiler Infrastructure for Proof Systems, Software Verification, and More." *IEEE S&P 2022*. ePrint 2020/1586.
34. Liu, Junrui, et al. "Certifying Zero-Knowledge Circuits with Refinement Types (Coda)." *IEEE S&P 2024*. ePrint 2023/547.
35. Maller, Mary, Sean Bowe, Markulf Kohlweiss, and Sarah Meiklejohn. "Sonic: Zero-Knowledge SNARKs from Linear-Size Universal and Updatable Structured Reference Strings." CCS 2019. ePrint 2019/099.
36. Groth, Jens, Markulf Kohlweiss, Mary Maller, Sarah Meiklejohn, and Ian Miers. "Updatable and Universal Common Reference Strings with Applications to zk-SNARKs." *CRYPTO 2018*. ePrint 2018/280.
37. Kohlweiss, Markulf, Mary Maller, Janno Siim, and Mikhail Volkhov. "Snarky Ceremonies." *ASIACRYPT 2021*. ePrint 2021/219.
38. Kim, Taechan and Razvan Barbulescu. "Extended Tower Number Field Sieve: A New Complexity for the Medium Prime Case." *CRYPTO 2016* (LNCS 9814, pp. 543–571). ePrint 2015/1027.
39. Guillevic, Aurore. "Comparing the Pairing Efficiency over Composite-Order and Prime-Order Elliptic Curves." *ACNS 2013*. ePrint 2013/218.
40. Succinct Labs. "SP1 Hypercube: Proving Ethereum in Real-Time." Blog post, May 2025. https://blog.succinct.xyz/sp1-hypercube/
41. ZKsync. "Airbender: GPU-Accelerated RISC-V Proving." Product announcement, June 2025. https://www.zksync.io/airbender
42. Kadianakis, George. "Shipping an L1 zkEVM #2: The Security Foundations." Ethereum Foundation Blog, December 2025. https://blog.ethereum.org/2025/12/18/zkevm-security-foundations

43. Chen, Binyi. "Symphony: Scalable SNARKs in the Random Oracle Model from Lattice-Based High-Arity Folding." ePrint 2025/1905.

14.4.9 Chapter 12: Midnight

39. Midnight Network. "Compact Language Reference." 2025. https://docs.midnight.network/develop/reference/
40. Midnight Network. "ZKIR Intermediate Representation Reference." 2025. https://docs.midnight.network/develop
41. Midnight Network. "Developer Guide." 2025. https://docs.midnight.network/

14.4.10 Chapter 13: The Market Landscape

42. Grand View Research. "Zero-Knowledge Proof Market Size, Share & Trends Analysis Report." Report GVR-4-68040-808-5, 2025. https://www.grandviewresearch.com/industry-analysis/zero-knowledge-proof-market-report
43. CastleLabs. "ZK Proofs: Is Privacy Cheap Enough to Be Mainstream?" 2025. https://research.castlelabs.io
44. Ethproofs. "ZK Proving Cost Tracker." Ethereum Foundation, 2025. https://ethproofs.org
45. Klich, Rafal (Chorus One). "The Economics of ZK-Proving: Market Size and Future Projections." Research report, March 2025.
46. DTCC. "DTCC and Digital Asset Tokenize US Treasuries on Canton Network." Press release, December 2025. https://www.dtcc.com/digital-assets/tokenization
47. Tools for Humanity (World). "World Whitepaper." 2024. https://whitepaper.world.org/
48. European Union. "Regulation (EU) 2024/1183 – European Digital Identity Framework (eIDAS 2.0)." *Official Journal of the European Union*, 2024.

14.4.11 Chapter 14: Open Questions

45. Nair, Vineet, Justin Thaler, and Michael Zhu. "Proving CPU Executions in Small Space." ePrint 2025/611.
46. Ozdemir, Alex, Evan Laufer, and Dan Boneh. "Volatile and Persistent Memory for zkSNARKs via Algebraic Interactive Proofs." *IEEE S&P 2025*. ePrint 2024/979.
47. Pailoor, Shankara, et al. "Automated Detection of Under-Constrained Circuits in Zero-Knowledge Proofs." *PLDI 2023*. ePrint 2023/512.
48. Wen, Hongbo, et al. "Practical Security Analysis of Zero-Knowledge Proof Circuits." *USENIX Security 2024*. ePrint 2023/190.
49. Hochrainer, Christoph, Valentin Wustholz, and Maria Christakis. "Arguzz: Testing zkVMs for Soundness and Completeness Bugs." arXiv 2509.10819, 2025.

50. Wee, Hoeteck and David J. Wu. “Lattice-Based Functional Commitments: Fast Verification and Cryptanalysis.” *ASIACRYPT 2023*. ePrint 2024/028.
51. Mascelli, Jillian and Megan Rodden. “ ‘Harvest Now Decrypt Later’: Examining Post-Quantum Cryptography and the Data Privacy Risks for Distributed Ledger Networks.” FEDS 2025-093, Federal Reserve Board, 2025.
52. NIST. “Module-Lattice-Based Digital Signature Standard (FIPS 204).” August 2024.